



Doble Grado en Ingeniería Informática y Matemáticas

Francisco José Cortijo Bon

cb@decsai.ugr.es

Curso 2022 - 2023



Departamento de Ciencias de la Computación e Inteligencia Artificial

Universidad de Granada

"Lo que tenemos que aprender a hacer, lo aprendemos haciéndolo".
Aristóteles



"In theory, there is no difference between theory and practice. But, in practice, there is".
Jan L. A. van de Snepscheut



"The gap between theory and practice is not as wide in theory as it is in practice".



"Theory is when you know something, but it doesn't work. Practice is when something works, but you don't know why. Programmers combine theory and practice: Nothing works and they don't know why".



Fundamentos de Programación

Guión de Prácticas

Curso 2022/2023

Autor: **Francisco José Cortijo Bon** (cb@decsai.ugr.es)

Basados en el guión original de: **Juan Carlos Cubero** (JC.Cubero@decsai.ugr.es)

Departamento de Ciencias de la Computación e Inteligencia Artificial
Universidad de Granada.

Índice del Guión de Prácticas

Sobre el guión de prácticas	1
Instalación de Orwell Dev C++ en nuestra casa	3
Tabla resumen de accesos directos usados en Orwell Dev C++	8
SESIÓN 1	9
El Entorno de Programación. Compilación de Programas	9
SESIÓN 2	21
Problemas básicos (entrada / cálculos / salida) (I)	21
SESIÓN 3	23
Problemas básicos (entrada / cálculos / salida) (II)	23
SESIÓN 4	25
Problemas básicos (entrada / cálculos / salida) (III)	25
Redireccionando las entradas de cin y las salidas a cout	26
SESIÓN 5	29
Estructuras condicionales	29
Depuración	32
SESIÓN 6	39
Estructuras repetitivas (I)	39
SESIÓN 7	41
Estructuras Repetitivas (II)	41
SESIÓN 8	43
Funciones	43
Depuración de funciones	46
SESIÓN 9	49
Registros. Vectores (1)	49
SESIÓN 10	53
Vectores (2). Matrices	53
SESIÓN 11	55
Clases (I): Conceptos básicos - Constructores	55

Inspección de objetos	57
SESIÓN 12	61
Clases (II): Constantes - Objetos con arrays - Secuencias	61
SESIÓN 13	63
Clases (III): Métodos y objetos - Objetos en objetos	63
SESIÓN 14	65
Clases (IV): Tablas	65
RELACIÓN DE PROBLEMAS I. Introducción a C++	RP-I.1
RELACIÓN DE PROBLEMAS II. Estructuras de Control	RP-II.1
RELACIÓN DE PROBLEMAS III. Funciones	RP-III.1
RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices	RP-IV.1
RELACIÓN DE PROBLEMAS V. Clases (Primera parte)	RP-V.1
RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)	RP-VI.1
RELACIÓN DE PROBLEMAS VII. Recursividad	RP-VII.1

Sobre el guión de prácticas

Este guión de prácticas contiene las actividades a realizar por el alumno. Gran parte de ellas las hará de manera autónoma. La resolución de los ejercicios de programación propuestos es una tarea fundamental para superar con éxito la asignatura ya que la programación es una tarea que requiere de un trabajo continuado, organizado y ordenado. Las habilidades, técnicas y destrezas que se requieren en la programación no se pueden adquirir leyendo o estudiando simplemente, sino que necesitan de un trabajo práctico continuado.

El guión está dividido en **sesiones**, que corresponden a cada semana lectiva. Las actividades no presenciales (las encontrará bajo el epígrafe *Actividades a realizar en casa*) de cada sesión deben realizarse antes de la correspondiente sesión de prácticas. Cada sesión se desarrolla en un periodo de 2 horas en la que el profesor interviene e interactúa de manera síncrona a la participación de los alumnos. Las sesiones están organizadas para, en base a los contenidos desarrollados en las clases de teoría, proporcionar un contenido eminentemente práctico. Así, el profesor puede explicar el entorno de programación, corregir los ejercicios propuestos, ampliar profundizar sobre contenidos teóricos sobre problemas concretos, etc.

La primera sesión presencial de prácticas (página 9) tendrá lugar durante la segunda semana de clases, por lo que el alumno deberá realizar con anterioridad las actividades encargadas.

Una actividad autónoma recurrente es la resolución de problemas, organizados en **Relaciones de Problemas**, que encontrará al final de este documento.

La organización de cada una de las sesiones de prácticas será, generalmente, la siguiente.

- El profesor propone con antelación suficiente la resolución de una serie de ejercicios de una relación de problemas (enumeradas en el epígrafe **Actividades a realizar en casa**). Estos ejercicios deberán resolverse en casa, y las soluciones se entregarán de acuerdo las indicaciones y plazos establecidos usando la plataforma **PRADO**.
- Los ejercicios más importantes serán corregidos en las sesiones de prácticas. Los alumnos deberán defenderlos individualmente (públicamente, explicándolo a sus compañeros). La calidad del trabajo y la defensa realizada forma parte de la nota final de la asignatura (10 % tal y como se explica en el método de evaluación de la asignatura).
- El profesor propondrá soluciones alternativas/mejoras a toda la clase y se debatirá sobre ellas, y en su caso, serán implementadas en la sesión activa para ser evaluadas.

Del conjunto de problemas que forman las relaciones de problemas seleccionamos para cada sesión una serie de problemas obligatorios y opcionales.

1. **Obligatorios**: Como su nombre indica hay que realizarlos y entregarlos para que la entrega sea considerada para su evaluación.

2. **Opcionales:** Son ejercicios que complementan (generalmente) ejercicios obligatorios y pueden ser una buena motivación para ampliar la funcionalidad de éstos. El alumno debería intentar resolver por su cuenta un alto porcentaje de éstos.

Para la realización de estas prácticas, se utilizará, preferentemente, el entorno de programación **Orwell Dev C++**. En la página 3 se encuentran las instrucciones para su instalación en nuestra casa. En cualquier caso, el alumno puede instalar y usar en su casa cualquier otro compilador, como por ejemplo Code::Blocks → <http://www.codeblocks.org/>

Es muy importante que la asignatura se lleve al día para poder realizar los ejercicios propuestos .



Instalación de Orwell Dev C++ en nuestra casa

El entorno de desarrollo que usaremos será Orwell Dev C++. Puede descargarse desde:



<http://sourceforge.net/projects/orwelldvcpp/>

En este momento están disponibles en la misma descarga las versiones 5.11 del entorno de trabajo y 4.9.2 del compilador:

- Dev-Cpp 5.11 (<http://orwelldvcpp.blogspot.com.es/>)
- TDM-GCC 4.9.2 (<https://jmeubank.github.io/tdm-gcc/>)

Cuando lo instalemos en nuestra casa, configurar las siguientes opciones del **editor** (en el menú principal: Herramientas | Opciones del Editor):

Herramientas -> Opciones del editor

-> Principal

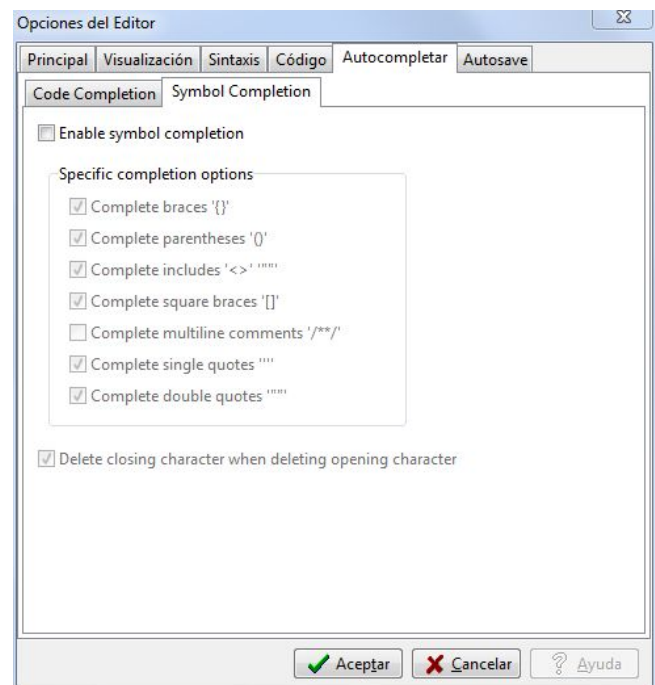
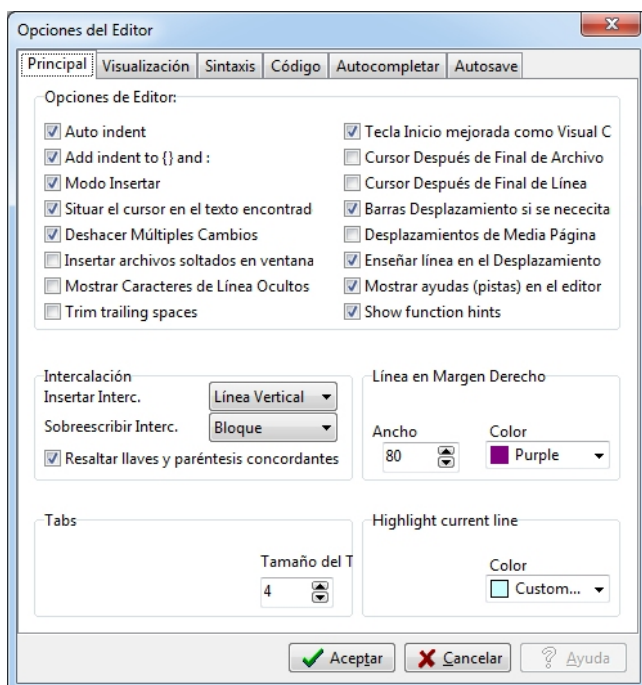
Desmarcar Tabuladores inteligentes

Tamaño del tabulador: 4

Línea en Margen Derecho --> Ancho: 80 Color: Purple

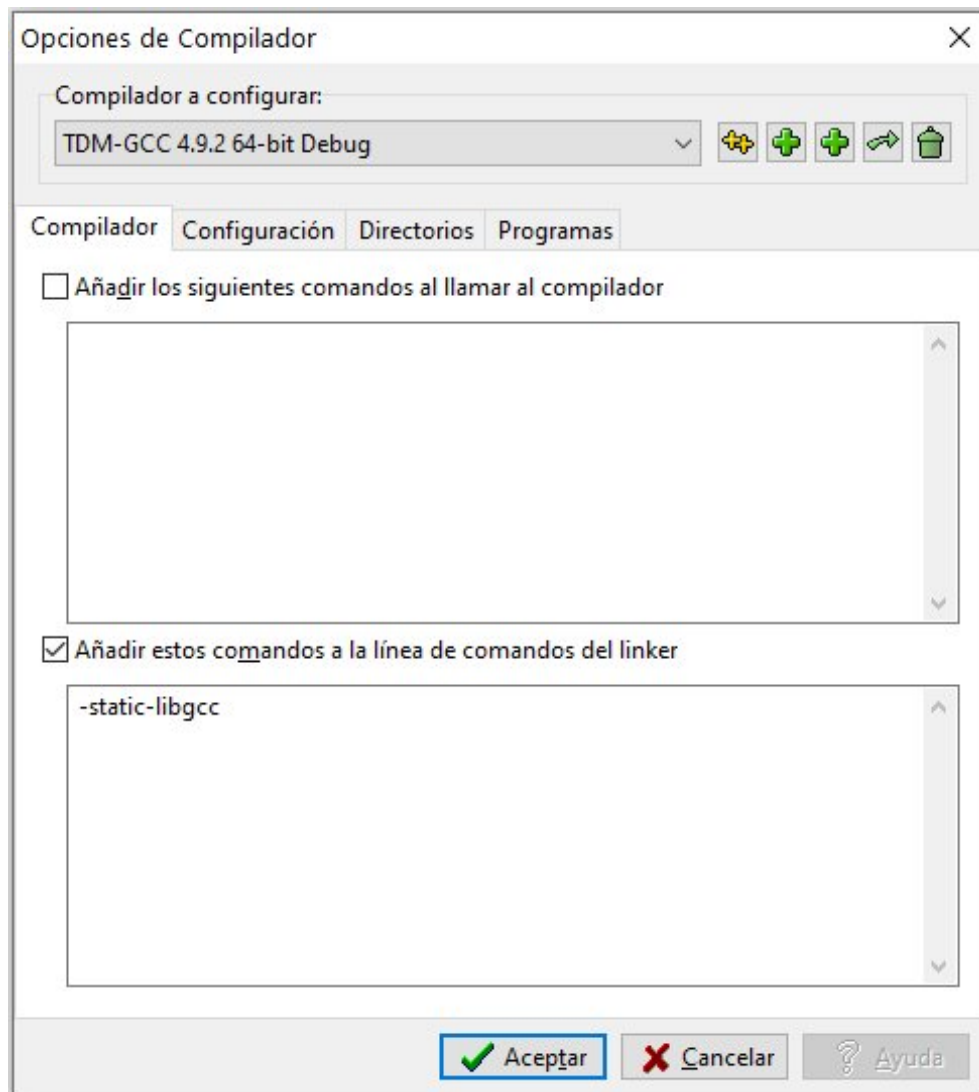
-> Autocompletar -> Symbol completion

Desmarcar Enable Symbol completion



Después, configurar las siguientes opciones del **compilador** (en el menú principal: Herramientas | Opciones del Compilador).

En primer lugar indicaremos que el compilador a configurar será el TDM-GCC 4.9.2 y usaremos la versión de 64 bits con el perfil de trabajo Debug (*depuración*):



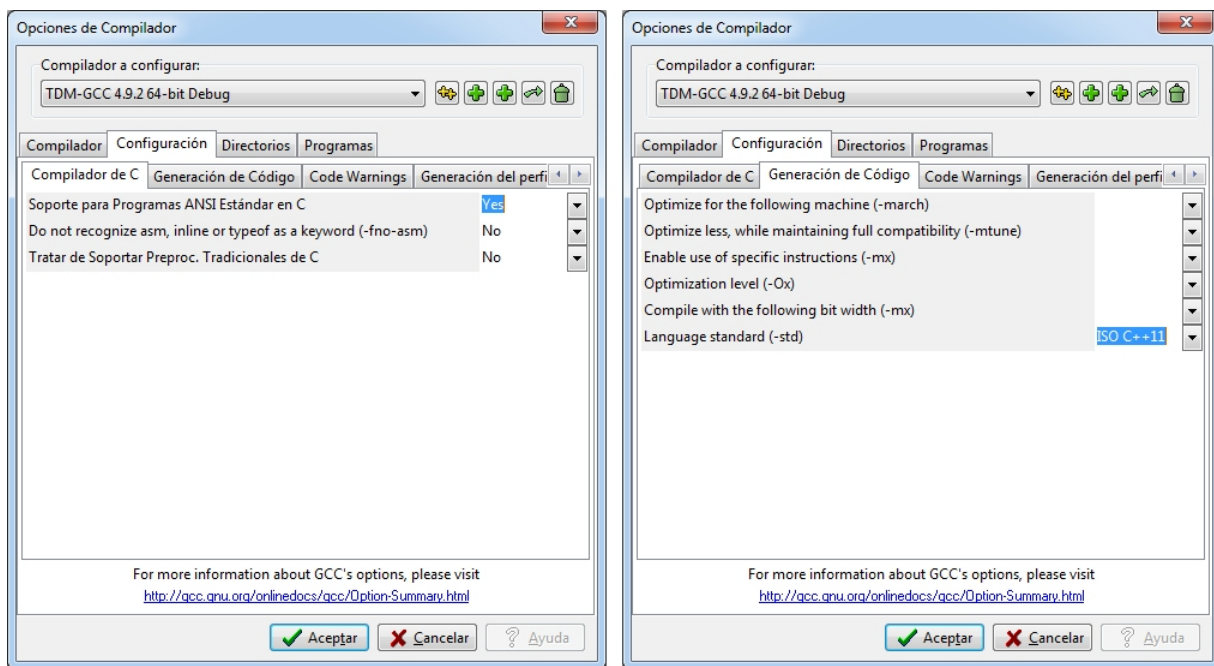
Debemos asegurarnos que se va a compilar correctamente código que sigue los estándares de programación para los lenguajes C y C++:

- ANSI C, en el caso de programar en C y
- ISO C++ 11, para C++.

Puede leer en Wikipedia (<https://es.wikipedia.org/wiki/C%2B%2B#C++11>) *En 2011 C++11 inauguró una nueva era en la historia de C++, iniciando un ciclo trienal de lanzamiento de nuevas versiones. A C++11 le siguió C++14 y luego C++17, que es la versión actual en 2019; C++20 se encuentra próximo a estandarizarse, y ya se está trabajando en la versión C++23. Los compiladores intentan adelantarse incorporando de manera experimental algunas novedades antes de los lanzamientos oficiales. Pero cada nueva versión de C++ incluye tal cantidad de agregados que los compiladores más adelantados no suelen terminar de incorporarlos hasta dos o tres años después del lanzamiento de esa versión.*

Herramientas -> Opciones del Compilador ->
Configuración -> Compilador de C
Soporte para Programas ANSI Estándar en C: Yes

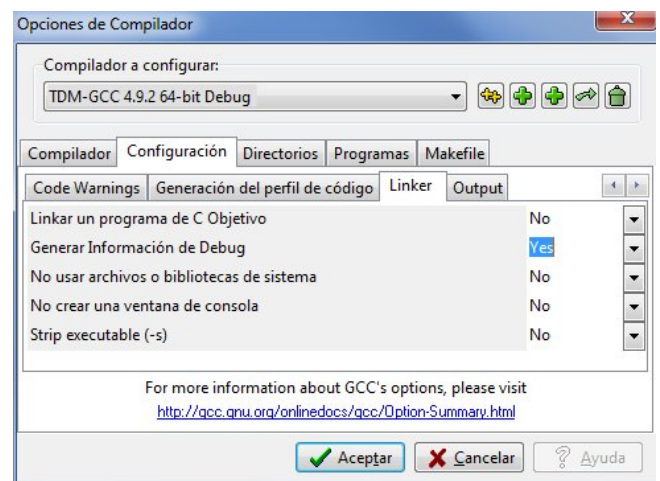
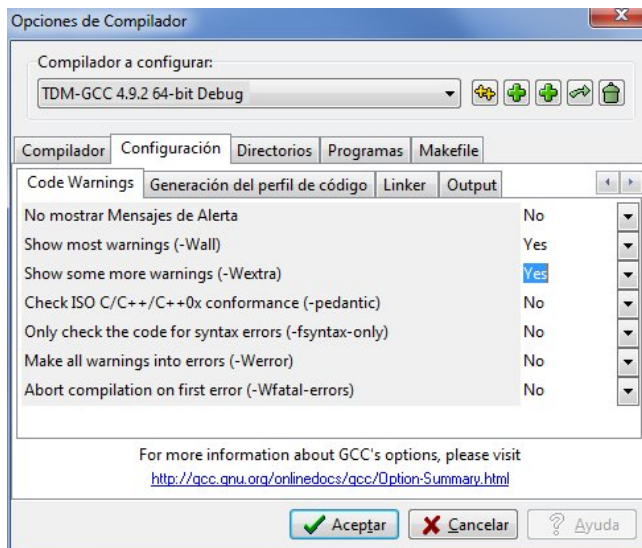
Herramientas -> Opciones del Compilador ->
Configuración -> Generación de Código
Language standard (-std): ISO C++ 11



Después se configurará qué tipo de avisos (*warnings*) queremos que se activen para el proceso de compilación (izquierda) e indicaremos que queremos generar información de depuración (*bebug*) para poder monitorizar la ejecución del programa (derecha):

Herramientas -> Opciones del Compilador ->
Configuración -> Code Warnings
Show most warnings: Yes
Show some more warnings: Yes

Herramientas -> Opciones del Compilador ->
Configuración -> Linker
Generar información de Debug: Yes



Aunque no es necesario un conocimiento detallado, debe saber que la opción:

```
-finput-charset=cp1252 -fexec-charset=cp850
```

indica al compilador que el código fuente está codificado con la página de códigos 1252 (de Windows) y que el ejecutable (la consola) será compatible con la 850 (ver apuntes de la asignatura).

De esta forma conseguimos mostrar los acentos y los caracteres especiales como la ñ en la consola que se abre al ejecutar nuestros programas.

Por otra parte, la opción

```
-Wl,--stack,21000000
```

indica al enlazador (*linker*) que utilice un tamaño de memoria para la pila de 21000000 bytes. Esto será necesario cuando usemos vectores con tamaños muy grandes (tema IV).

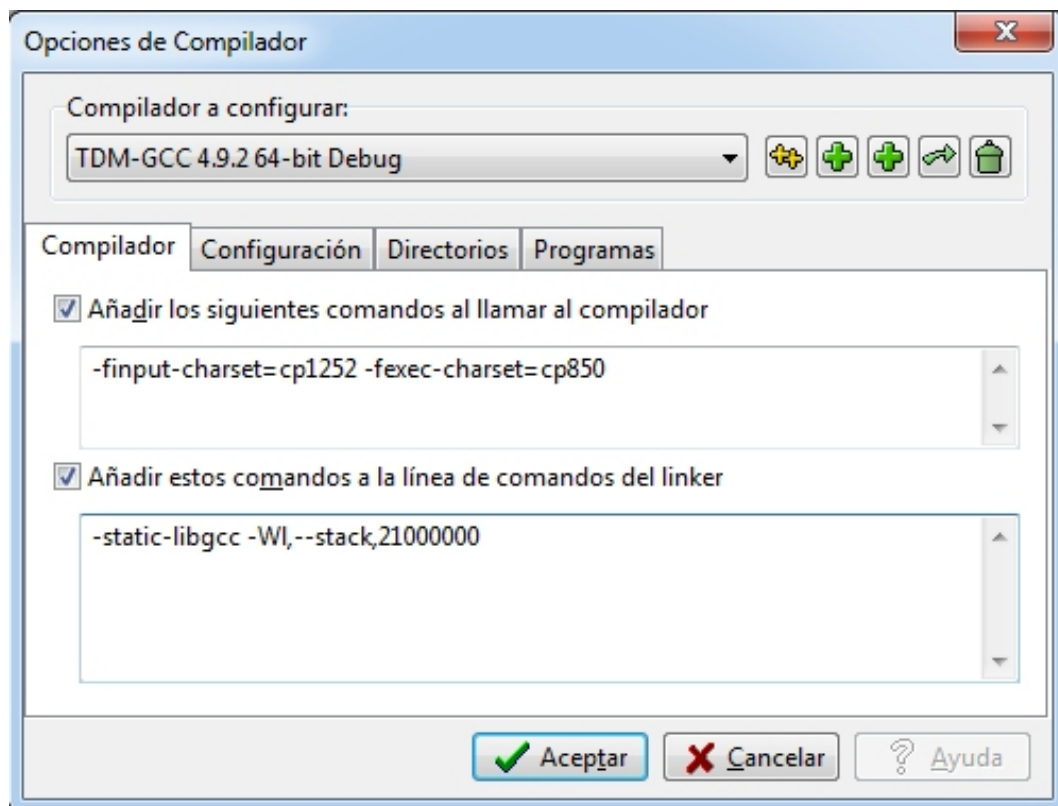


Tabla resumen de accesos directos usados en Orwell Dev C++

F9	Compilar
F10	Ejecutar
F11	Compilar y Ejecutar
F5	Depurar Empieza la depuración
F7	Siguiente paso Ejecución paso a paso sin entrar en los métodos o funciones
Shift F7	Avanzar paso a paso Ejecución paso a paso entrando en los métodos o funciones
Ctrl F7	Saltar paso Ejecución hasta el siguiente punto de ruptura
Shift F4	Ir a cursor Ejecuta sentencias hasta llegar a la posición actual del cursor

Sesión 1

El Entorno de Programación. Compilación de Programas

La primera sesión de prácticas se desarrolla en un periodo de tiempo en el que el alumno desconoce los aspectos más básicos de la programación. En las clases de teoría está familiarizándose con los tipos de datos básicos, expresiones, y entendiendo el mecanismo de entrada/salida.

Es posible, no obstante, empezar a programar usando un **modelo basado en ejemplos**. A partir de programas sencillos que el profesor le proporciona pueden proponerse otros problemas que se resuelven con una estrategia similar y que permiten investigar y aprender de manera autónoma nuevas funciones de la biblioteca estándar de C++. Los primeros ejercicios que se pueden resolver están basados en el modelo simple de tres etapas:

1. **Entrada:** lectura de los valores de los datos que intervienen en el programa.
2. **Cálculos:** obtener los valores de los datos en los que estamos interesados a partir de los datos de entrada (normalmente traduciendo una o más fórmulas/ecuaciones matemáticas a expresiones y asignaciones en C++).
3. **Salida:** presentación de los resultados obtenidos.

Para poder realizar un trabajo práctico real se requiere escribir el programa en un **editor** de texto y usar un **compilador** para generar el programa ejecutable. El proceso de compilación y generación de código ejecutable puede ser una tarea larga y compleja, especialmente si se trabaja en proyectos complejos en el que intervienen diferentes equipos de trabajo que pueden trabajar con diferentes herramientas de desarrollo. En esta asignatura, no obstante, trabajaremos en proyectos simples y emplearemos herramientas simples (se trata, no lo olvide, del primer contacto con la programación de ordenadores). Durante esta primera práctica podrá escribir y compilar sus programas con relativamente poco esfuerzo. Emplearemos un sencillo **IDE** (Integrated **D**evelopment **E**nvironment, ó entorno de desarrollo integrado) con este objetivo.

Como requisito previo el alumno debe haber resuelto sus obligaciones previas relativas a la obtención de las credenciales (cuenta y correo en la UGR) que le identifican como alumno de la UGR. No olvide que necesita disponer también de las credenciales para el acceso a los ordenadores de las aulas de prácticas de la ETSIT. No hace falta decir que es vital disponer de un ordenador para el estudio y la realización de las tareas de prácticas.

► **Objetivos**

Los objetivos de esta sesión de prácticas son:

1. Descargar e instalar en su ordenador personal el entorno de desarrollo **Dev C++**.
2. Configurar las características básicas de Dev C++.
3. Ser capaces de cargar un fichero .cpp en Dev C++ y grabar un fichero .cpp
4. Compilar un programa en C++ y entender qué es y qué consecuencias tiene.
5. Ejecutar un programa escrito en C++.
6. Leer, entender y actuar ante errores de compilación y ejecución.
7. Detectar (diseñando casos de prueba) los errores lógicos en los programas y actuar ante ellos.
8. Crear nuevos programas en C++.

► **Actividades a realizar en casa**

Actividad: Conseguir login y password.

Antes de empezar la primera sesión de prácticas el alumno debe disponer de una cuenta en la Universidad , tal y como se indica en

<https://csirc.ugr.es/estudiantes/acceso-servicios>

Con objeto de facilitar la comunicación electrónica y otros servicios, la Universidad de Granada pone a disposición de los estudiantes la Cuenta de Acceso a Servicios Informáticos o Cuenta de Correo Electrónico. Esta cuenta para los estudiantes es del tipo usuario@correo.ugr.es.

Esta cuenta le permite tener acceso con un nombre de usuario y una contraseña, a todos los servicios informáticos que oferta la Universidad de Granada como son:

- Correo electrónico (preguntas frecuentes).
- Conexión con CVI-UGR (Wi-Fi universitaria).
- Acceso externo a RedUGR (VPN).
- Acceso a PRADO.

...

La responsabilidad última de conservación y guarda de los correos corresponde al usuario.

Actividad: Instalación de Orwell Dev C++.

Durante la primera semana de clase, el alumno debería instalar en su casa el compilador Orwell Dev C++. Consultad la sección de instalación (página 3) de este guión.

Actividad: Resolución de Problemas.

Leer las actividades que se van a realizar durante las horas de prácticas en las aulas de ordenadores (ver página 12 de este guión).

Intentad resolver previamente los ejercicios enumerados a continuación. Escribid las soluciones en los ficheros .cpp indicados y guardarlos en un *pendrive* o en la nube.

1. (voltaje.cpp)

Crear un programa que pida un valor de intensidad y resistencia e imprima el voltaje correspondiente, según la *Ley de Ohm*:

$$\text{voltaje} = \text{intensidad} * \text{resistencia}$$

2. (circunferencia.cpp)

Cread un programa que nos pida la longitud del radio, calcule el área del círculo y la longitud de la circunferencia correspondientes, y nos muestre los resultados en pantalla. Recordad que:

$$\text{longitud circunferencia} = 2\pi r \quad \text{área círculo} = \pi r^2$$

Usad el literal 3.1416 a lo largo del código, cuando se necesite multiplicar por π .

Una vez hecho el programa, cambiad las apariciones de 3.1416 por 3.1415927, recompilad y ejecutad.

3. (salario.cpp)

Indique cuál sería el resultado de las siguientes operaciones:

```
int salario_base;
int salario_final;
int incremento;

salario_base = 1000;
salario_final = salario_base;
incremento = 200;

salario_final = salario_final + incremento;
salario_base = 3500;

cout << endl;
cout << "Salario base: " << salario_base << endl;
cout << "Salario final: " << salario_final << endl;
```

Responda razonadamente a la siguiente pregunta: ¿El hecho de realizar la asignación `salario_final = salario_base;` hace que ambas variables estén ligadas durante todo el programa y que cualquier modificación posterior de `salario_base` afecte a `salario_final`?

► **Actividades a realizar en las aulas de ordenadores**

Durante esta sesión de prácticas, el profesor guiará a los alumnos en el desarrollo de las siguientes actividades que se desarrollarán en el aula de prácticas:

Arranque del Sistema Operativo

Para poder arrancar el SO en las aulas de ordenadores, es necesario obtener el login y password indicados en las actividades a realizar en casa.

En la casilla etiquetada como Código, introduciremos `fp`. Al arrancar el SO, aparecerá una instalación básica de Windows con el compilador Orwell Dev C++. Todo lo que escribamos en la unidad `C:` se perderá al apagar el ordenador. Por ello, el alumno dispone de un directorio de trabajo en la unidad lógica `U:`, cuyos contenidos permanecerán durante todo el curso académico. En cualquier caso, es recomendable no saturar el espacio usado ya que, en caso contrario, el compilador no podrá funcionar.

El alumno deberá crear el directorio de trabajo para FP en la unidad `U:` (`U:\FP`).

Si durante la sesión se le indica que debe abrir algún fichero, éste puede encontrarse en **PRADO**. Descárguelo y copielo en la carpeta adecuada de `U:\FP`

Muy Importante. No emplearemos los ficheros que se encuentran en la unidad `H:`. Si usted decidiera hacerlo, sepa que esos ficheros están protegidos y no pueden modificarse. Por tanto, habrá que copiarlos a la unidad local `U:`, dónde sí podrán ser modificados.

Para acceder a la unidad `U:` desde nuestras casas, debemos usar cualquier programa de `ftp` que use el protocolo `ssh`, como por ejemplo `filezilla` o `winscp`. Instalamos este programa en nuestra casa y simplemente nos conectamos a `turing.ugr.es` con nuestras credenciales.

El primer programa

Copiando el código fuente

En la sección PRÁCTICA 1 de **PRADO** encontrará en la carpeta Archivos Sesión 1 el fichero `pitagoras.cpp`.

Cread una carpeta en `U:\FP` llamada `SESION_01` y copiar en ella el fichero `pitagoras.cpp`.

Desde el Explorador de Windows, entrad en la carpeta recién creada en vuestra cuenta:

`U:\FP\SESION_01`

y haced doble click sobre el fichero `pitagoras.cpp`. Debe aparecer una ventana como la de la figura 1:

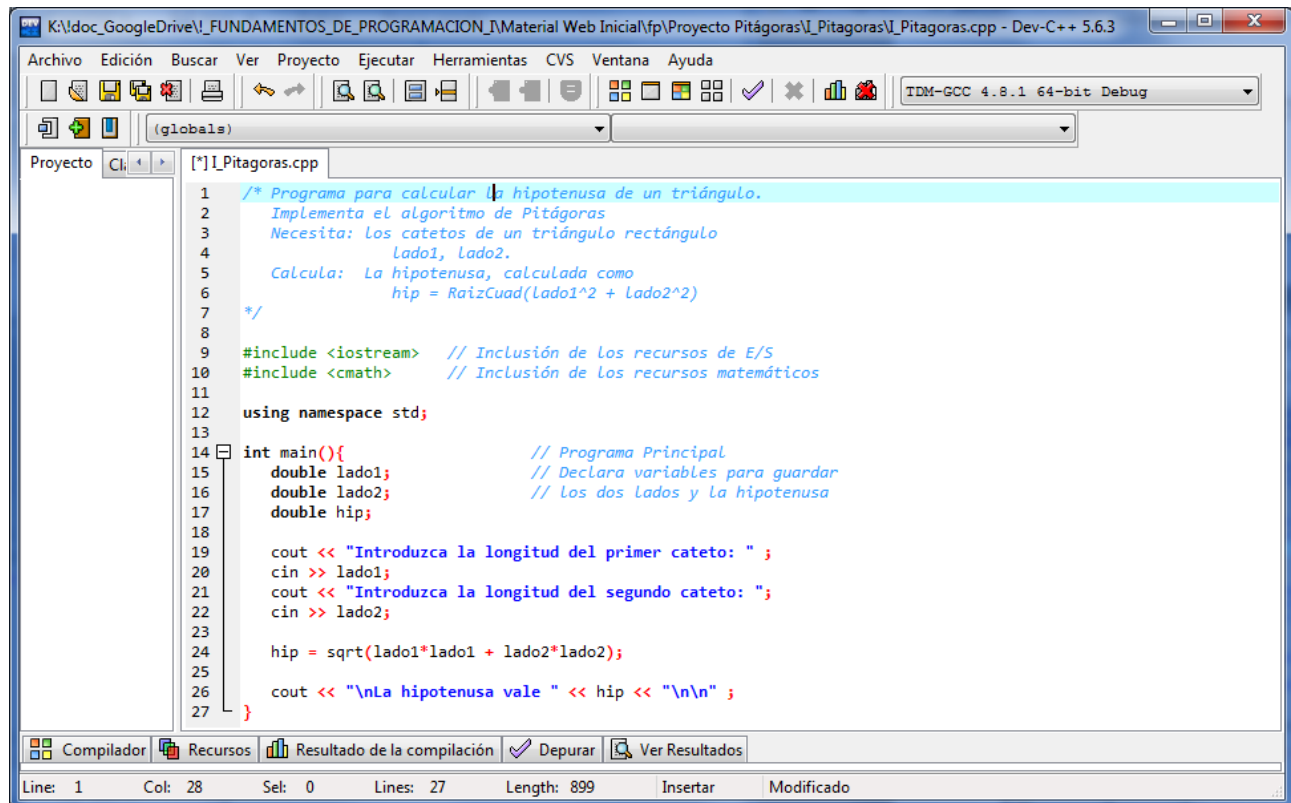


Figura 1: Programa que implementa el algoritmo de Pitágoras

Algunas consideraciones con respecto a la escritura de código en C++ (ver figura 2)

- Es bueno que, desde el principio se incluyan comentarios indicando el objetivo del programa y resaltando los aspectos más importantes de la implementación.
- Es muy importante una correcta tabulación de los programas. Por ahora, incluiremos todas las sentencias del programa principal con una tabulación. Sólo es necesario incluir la primera; el resto las pone automáticamente el entorno, al pasar a la siguiente línea.
- Para facilitar la lectura del código fuente, se deben usar espacios en blanco para separar las variables en la línea en la que van declaradas, así como antes y después del símbolo `=` en una sentencia de asignación. Dejad también un espacio en blanco antes y después de `<<` y `>>` en las órdenes que usan `cout` y `cin` respectivamente.

```
int main() {  
    double lado1;  
    double lado2;  
    double hip;  
  
    cout << "Introduzca la longitud del primer cateto: " ;  
    cin >> lado1;  
    cout << "Introduzca la longitud del segundo cateto: " ;  
    cin >> lado2;  
  
    hip = sqrt(lado1*lado1 - lado2*lado2);  
  
    cout << "\nLa hipotenusa vale " << hip << "\n\n" ;  
    system("pause");  
}
```

Declaraciones

Entradas datos

Computos

Salida Resultado

líneas en blanco

Comentarios separados visualmente del código

espacios en blanco

Figura 2: Escritura de código

No respetar las normas de escritura de código baja puntos en todos los exámenes y prácticas de la asignatura



Compilación

Una vez cargado el programa, pasamos a comprobar si las sentencias escritas son sintácticamente correctas, es decir, pasamos a *compilar* el programa. Para ello pulsamos F9, o bien sobre el icono .

Para que el proceso de compilación se realice de forma correcta y se obtenga el programa ejecutable, es necesario que el código fuente no contenga errores sintácticos. Si aparecen errores, es necesario volver a la fase de edición, guardar de nuevo el código fuente y repetir la fase de compilación.

Como resultado de la fase de compilación, en la parte de abajo del entorno debe aparecer un mensaje del tipo:

Compilation succeeded

Una vez compilado el programa, tendremos el fichero `pitagoras.exe`. Para ejecutarlo desde el entorno basta pulsar sobre F10. Si se quiere, ambos pasos (compilación y ejecución) pueden realizarse pulsando sobre F11. Debe aparecer una consola en la que se

estará ejecutando el programa. La ejecución del programa se detendrá en aquellos puntos del mismo donde se requiera la intervención del usuario, es decir, en las operaciones de entrada de datos a través del dispositivo estándar de entrada. En este ejemplo, sería en las dos operaciones `cin`. En el resto de los casos, la ejecución del programa continuará hasta el final. La introducción de datos mediante la sentencia `cin` se hace siempre de la misma manera; primero se introduce el valor que se desee y al terminar se pulsa la tecla RETURN. Introducid ahora los valores pedidos en el ejemplo de Pitágoras y comprobad la respuesta del programa.

Como hemos indicado anteriormente, en la fase de generación del ejecutable se ha creado un fichero en el Sistema que se llama igual que nuestro fichero pero sustituyendo la extensión `"cpp"` por `"exe"`, es decir, `pitagoras.exe`. Este fichero se encuentra en el mismo directorio que el del fichero `cpp`. Para mostrar que el fichero generado es independiente del entorno de programación, hacemos lo siguiente:

1. Cerramos Orwell Dev C++.
2. Abrid una ventana de Mi PC.
3. Situarse en la carpeta que contiene el ejecutable.
4. Haced doble click sobre el fichero `pitagoras.exe`.

Prueba del programa

Uno podría pensar que una vez que consigo un fichero ejecutable a partir de mi código fuente, el problema está terminado. Sin embargo esto no es así. Tras el proceso de compilado se requiere una fase de prueba. Dicha fase intenta probar que el algoritmo planteado resuelve el problema propuesto. Para llevar a cabo esta fase, es necesario ejecutar el programa y verificar que los resultados que obtiene son los esperados.

Ahora que podemos ver el resultado obtenido por el programa implementado, verifiquemos mediante el siguiente conjunto de pruebas que el programa funciona de forma correcta.

lado1	lado2	hip
3	4	5
1	5	5.099
2.7	4.3	5.077
1.25	2.75	3.02

Una vez que el algoritmo supera la fase de prueba, podemos considerar que se ha concluido con la fase inicial del desarrollo del software.

Introducción a la corrección de errores

Los errores de compilación

Ya hemos visto los pasos necesarios para construir un fichero ejecutable a partir del código fuente. El paso central de este proceso era la fase de compilación. En esta parte de este guión de prácticas aprenderemos a corregir los errores más comunes que impiden una compilación exitosa del fichero fuente.

Cargad el fichero `pitagoras.cpp`. Quitadle una `'u'` a alguna aparición de `cout`. Intentad compilar. Podemos observar que la compilación no se ha realizado con éxito. Cuando esto sucede, en la parte inferior de la ventana principal aparecen los errores que se han encontrado. Aparece una descripción del error, así como otra información, como el número de línea en la que se produjo. Los pasos que debemos seguir para la corrección son los siguientes:

1. Ir a la primera fila de la lista de errores.
2. **Leer el mensaje de error e intentar entenderlo.**
3. Hacer doble click sobre esa fila con el ratón. Esto nos posiciona sobre la línea en el fichero fuente donde el compilador detectó el error.
4. Comprobar la sintaxis de la sentencia que aparece en esa línea. Si se detecta el error, corregirlo. Si no se detecta el error mirar en la línea anterior, comprobar la sintaxis y repetir el proceso hasta encontrar el error.
5. Después de corregir el posible error, guardamos de nuevo el archivo y volvemos a compilar. Esto lo hacemos aunque aparezcan más errores en la ventana. La razón es que es posible que el resto de los errores sean consecuencia del primer error.
6. Si después de corregir el error aparecen nuevos errores, volver a repetir el proceso desde el paso 1.

A veces, el compilador no indica la línea exacta en la que se produce el error, sino alguna posterior. Para comprobarlo, haced lo siguiente:

- Comentad la línea de cabecera `#include <iostream>` desde el principio. El compilador no reconocerá las apariciones de `cin` o `cout`.
- Quitad un punto y coma al final de alguna sentencia. Dará el error en la línea siguiente.

Para familiarizarnos con los errores más frecuentes y su corrección vamos a realizar el siguiente proceso: a partir del código fuente del ejemplo `pitagoras.cpp`, iremos introduciendo deliberadamente errores para conocer los mensajes que nos aparecen. A continuación se muestran algunos errores posibles. No deben introducirse todos ellos a la vez, sino que han de probarse por separado.

1. Cambiad algún punto y coma por cualquier otro símbolo
2. Cambiad `double` por `dpuble`
3. Cambiad la línea `using namespace std;` por `using namespace STD;`
4. Poned en lugar de `iostream`, el nombre `iotream`.
5. Borrád alguno de los paréntesis de la declaración de la función `main`
6. Introducid algún identificador incorrecto, como por ejemplo `cour`
7. Usad una variable no declarada. Por ejemplo, en la definición de variables cambiad el nombre a la variable `lado1` por el identificador `lado11`.
8. Borrád alguna de las dobles comillas en una constante de cadena de caracteres, tanto las comillas iniciales como las finales.
9. Borrád alguna de las llaves que delimitan el inicio y final del programa.
10. Borrád la línea `using namespace std;` (basta con comentarla con `//`)
11. Cambiad un comentario iniciado con `//`, cambiando las barras anteriores por las siguientes `\\`
12. Cambiad la aparición de `<<` en `cout` por las flechas cambiadas, es decir, `>>`. Haced lo mismo con `cin`.
13. Suprimid todo el `main`. No hace falta borrar el código, basta con comentarlo.

Además de los errores, el compilador puede generar *avisos*. Estos se muestran como **Warning** en la misma ventana de la lista de errores. Estas advertencias indican que algún código puede generar problemas durante la ejecución. Por ejemplo, al usar una variable que todavía no tiene un valor asignado, al intentar asignar un entero *grande* a un entero *chico*, etc. Sin embargo, no son errores de compilación, por lo que es posible generar el programa ejecutable correspondiente.

Los errores lógicos y en tiempo de ejecución

Aunque el programa compile, esto no significa que sea correcto. Puede producirse una excepción durante la ejecución, de forma que el programa terminará bruscamente (típico error en Windows de *Exception Violation Address*) o, lo que es peor, dará una salida que no es correcta (error lógico).

Sobre el programa `pitagoras.cpp`, haced lo siguiente:

- Cambiad la sentencia

`sqrt(lado1*lado1 + lado2*lado2)` por:
`sqrt(lado1*lado2 + lado2*lado2)`

Ejecutad introduciendo los lados 2 y 3. El resultado no es correcto, pero no se produce ningún error de compilación ni en ejecución. Es un error lógico.

- Para mostrar un error de ejecución, declarad tres variables ENTERAS (**tipo int**) resultado, numerador y denominador. Asignadle cero a denominador y 7 a numerador. Asignadle a resultado la división de numerador entre denominador. Imprimid el resultado. Al ejecutar el programa, se produce una excepción o error de ejecución al intentar dividir un entero entre cero.

Creación de un programa nuevo

En esta sección vamos a empezar a crear nuestros propios programas. El primer ejemplo que vamos a implementar corresponde al ejercicio 1 sobre la Ley de Ohm, propuesto en la *Actividad: Resolución de Problemas* de esta sesión de prácticas.

Para crear un programa nuevo, abrimos Orwell Dev C++ y elegimos

Archivo->Nuevo Código Fuente (Ctrl-N)

Para cambiar el nombre asignado por defecto, seleccionamos Archivo -> Guardar Como. Seleccionamos la carpeta U:\FP\SESION_01 e introducimos el nombre del nuevo fichero fuente: voltaje (no es necesario escribir la extensión .cpp).

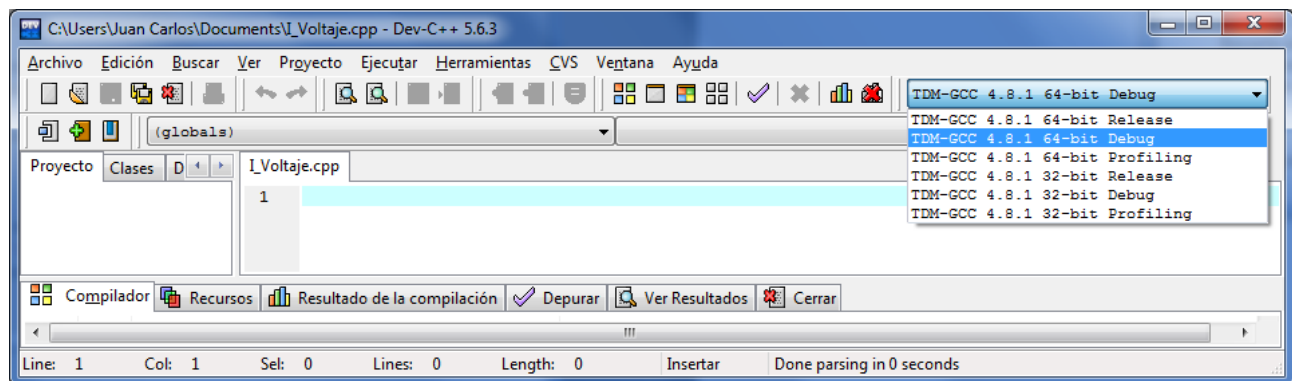


Figura 3: Creación de un programa nuevo

Confirmad que en la esquina superior derecha está seleccionada la opción de compilación

TDM-GCC ... Debug

Ya estamos en condiciones de resolver el problema pedido. Escribimos el código en la ventana de edición.

Este ejercicio sigue un esquema similar al del ejercicio que calculaba la longitud de la hipotenusa de un triángulo rectángulo cuando se conocen las longitudes de los dos catetos:

1. Entrada: leer (desde teclado) los valores de entrada (intensidad y resistencia),
2. Cálculos: Aplicar la fórmula que relaciona los datos de entrada con la salida (voltaje).
3. Salida: Mostrar el resultado.

Recordad que compilamos con F9 y ejecutamos con F10, o directamente ambas acciones con F11.

Nota. Cuando tenemos varias variables en el código, podemos empezar a escribir el nombre de alguna de ellas y antes de terminar, pulsar Ctr-Barra espaciadora. La ayuda nos mostrará los identificadores disponibles que empiecen por las letras tecleadas.

Sesión 2

Problemas básicos (entrada / cálculos / salida) (I)

► **Actividades a realizar en casa**

Actividad: Lectura de soluciones de ejercicios resueltos.

Lea la solución de los siguientes ejercicios, que podrá encontrar en PRADO (sección dedicada a la Sesión 2 de Prácticas).

- 2 (Ley de Ohm) I_Voltaje.cpp
- 4 VIDEO (Aproximaciones de π (1)) I_Aprox_PI.cpp
- 5 VIDEO (Aproximación de π (2)) I_Aprox_PI_ArcoSeno.cpp
- 6 VIDEO (Conversión de grados a radianes) I_Grados_a_Radianes_ctes.cpp
- 7 (Tarifa aérea) I_Tarifa_Aerea_KM.cpp
- 8 (Tarifa aérea con descuento) I_Tarifa_Aerea_KM_descuento.cpp
- 10 (Interés bancario) I_Interes.cpp
- 13 (Circunferencia -con constantes-) I_Circunferencia_ctes.cpp

► **Actividades a realizar en las aulas de ordenadores**

Realizar los ejercicios propuestos por el profesor.

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas y **comentadas**.

Sesión 3

Problemas básicos (entrada / cálculos / salida) (II)

► **Actividades a realizar en casa**

Actividad: Lectura de soluciones de ejercicios resueltos.

Lea la solución de los siguientes ejercicios, que podrá encontrar en PRADO (sección dedicada a la Sesión 3 de Prácticas).

19 VIDEO 1 - VIDEO 2 (Tiempo entre dos instantes) I_DiferenciaTiempo.cpp

26 VIDEO (Serparar digitos) I_PintaDigitos_int.cpp

Actividad: Resolución de problemas.

Resolved los ejercicios siguientes de la Relación de Problemas I (página RP-I.1).

- **Obligatorios:**

- 9 (Radar de tramo) I_RadarTramo.cpp

- 12 (Cambio monedas) I_Cambio.cpp

- 16 (Consumo de gasolina) I_ConsumoGasolina.cpp

- 18 (Ajustar horas, minutos y segundos) I_AjusteTiempo.cpp

- 20 (Ritmo y velocidad) I_RitmoVelocidad_a.cpp y I_RitmoVelocidad_b.cpp

- 24 (Gaussiana) I_Gaussiana.cpp

Recordad que hay que entregar las soluciones en **PRADO** siguiendo escrupulosamente las instrucciones, y en el plazo indicado en la convocatoria de la entrega.

- **Opcionales:**

- 11 (Incremento porcentual de salario) I_IncrementoSalario.cpp

- 15 (Conversión de unidades de distancia) I_ConversionUnidadesDistancia.cpp

- 22 (Saltos en intervalos) I_Desplazamiento_Intervalo.cpp

- 31 (Expresiones) I_Expresiones.cpp

► **Actividades a realizar en las aulas de ordenadores**

Durante esta sesión de prácticas se discutirá sobre las soluciones a las actividades propuestas.

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas y **comentadas**.

► **Normas detalladas**

Deberá entregar únicamente el fichero FP_sesion03.zip (**importante:** no .rar ni otro formato, sólo .zip), resultado de empaquetar todos los ficheros fuente (.cpp), uno por cada ejercicio resuelto.

Los nombres de los ficheros fuente deben ser los indicados en la lista de ejercicios propuestos (página 23).

La entrega se realizará en la plataforma web de la asignatura, en el periodo habilitado para ello (consulten **PRADO** o su correo electrónico).

Sesión 4

Problemas básicos (entrada / cálculos / salida) (III)

► **Actividades a realizar en casa**

Actividad: Lectura de soluciones de ejercicios resueltos.

Lea la solución de los siguientes ejercicios, que podrá encontrar en PRADO (sección dedicada a la Sesión 4 de Prácticas).

- 21 VIDEO (Lectura simple de intervalo) I_Intervalo.cpp
- 22 VIDEO (Saltos en intervalos) I_Desplazamiento_Intervalo.cpp
- 35 VIDEO (Mínimo número de bits) I_Min_Num_Bits.cpp
- 37 (Distancia Euclídea) I_DistanciaEuclidea.cpp

Actividad: Resolución de problemas.

Resuelva los ejercicios siguientes de la Relación de Problemas I (página **RP-I.1**).

Recuerde que antes del inicio de esta sesión en el aula de ordenadores hay que entregar las soluciones en **PRADO** siguiendo las instrucciones y en el plazo indicado en la convocatoria de la entrega.

- **Obligatorios:**

- 25 (Pasar de minúscula a mayúscula) I_Minuscula_a_Mayuscula.cpp
- 28 (Cifrado de caracteres) I_CifradoRotacion.cpp
- 36 (Truncar decimales) I_TruncaDecimales.cpp
- 32 (Expresiones lógicas simples) I_Expresiones_Logicas_Simples.cpp
- 40 (Expresiones lógicas) I_Expresiones_Logicas.cpp
- 41 (Líneas de índice) I_AnalizaLineaIndice.cpp
- 42 (Intercambia dígitos extremos) I_IntercambiaExtremos.cpp

- **Opcionales:**

- 33 (PVP) I_PVP_Automovil.cpp
- 38 (Distancia entre dos puntos) I_Haversine.cpp
- 39 (Área de un triángulo) I_AreaTriangulo.cpp

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas, **comentadas** y **modularizadas**.

► **Normas detalladas**

Deberá entregar únicamente el fichero `FP_sesion04.zip` (**importante:** no `.rar` ni otro formato, sólo `.zip`), resultado de empaquetar únicamente los ficheros fuente (`.cpp`), uno por cada ejercicio resuelto.

Los nombres de los ficheros fuente deben ser los indicados en la lista de ejercicios propuestos (página 25).

La entrega se realizará en PRADO, en el periodo habilitado para ello.

► **Actividades a realizar en las aulas de ordenadores**

Mientras el profesor corrige individualmente los ejercicios de algunos alumnos, realizar la actividad siguiente y trabajar sobre los ejercicios opcionales.

Durante el resto de la sesión se discutirá sobre las soluciones propuestas a los ejercicios de esta sesión.

Redireccionando las entradas de `cin` y las salidas a `cout`

Cada vez que se ejecuta una instrucción `cin >>`, se lee un dato desde el periférico de entrada por defecto (el teclado): el usuario escribe el valor del dato y confirma/termina pulsando la tecla ENTER.

Pueden introducirse los valores de varios datos (incluso de diferentes tipos) en una única instrucción de lectura. Tan sólo hay que escribir los valores de los datos separados por espacios en blanco y tabuladores (caracteres *separadores*). Para comprobarlo, copiad el fichero `Redireccion_cin.cpp` desde PRADO. Observad el código del programa y ejecutadlo. Para introducir los valores de los datos pedidos (un entero y dos caracteres) empezaremos por escribir los dígitos del entero, un número arbitrario de espacios en blanco y/o tabuladores, un carácter, un número arbitrario de espacios en blanco y/o tabuladores, otro carácter y finalmente, ENTER.

Para comprobar el correcto funcionamiento de nuestros programas, habitualmente tendremos que ejecutarlos en repetidas ocasiones usando distintos valores de entrada. Este proceso lo repetiremos hasta que no detectemos fallos. Para no tener que introducir los valores pedidos uno a uno, podemos recurrir a un simple *copiar-pegar*. Para comprobarlo, cread

un fichero *de texto* con un entero y dos caracteres. Separad estos tres datos con varios espacios en blanco, por ejemplo. Seleccionad con el ratón los tres y copiadlos al portapapeles (Click derecho-Copiar). Ejecutad el programa y cuando aparezca la consola del sistema haced click derecho sobre la ventana y seleccionad Editar-Pegar.

Otra alternativa es ejecutar el fichero .exe desde el sistema operativo y **redirigir** la entrada de datos desde fichero que contiene los datos. Para poder leer los datos del fichero, basta con ejecutar el programa .exe desde una consola del sistema y especificar que la entrada de datos será desde un fichero a través del símbolo de redireccionamiento < (no ha de confundirse con el token << que aparece en la instrucción de salida cout << de C++).

Para probarlo, descargad desde decsai el fichero

```
Redireccion_cin_datos_entrada.txt
```

y copiadlo en la misma carpeta en la que se ha descargado el programa Redireccion_cin.cpp. Abrimos dicha carpeta desde el explorador y seleccionamos con el click derecha del ratón Abrir Símbolo del Sistema¹

Una vez nos hemos situado en el directorio que contiene el ejecutable y el fichero de datos, introducimos la instrucción siguiente:

```
Redireccion_cin.exe < Redireccion_cin_datos_entrada.txt
```

Ahora, cada vez que se ejecute una instrucción cin >> en el programa, se leerá un valor de los presentes en el fichero de texto y no se empleará el teclado para la introducción de datos.

Hay que destacar que este redireccionamiento de la entrada lo estamos haciendo en la llamada al ejecutable desde la consola del sistema operativo².

También es posible redireccionar la salida para que cada vez que se ejecute una instrucción cout << en el programa, el texto se “envíe” a un fichero de texto en lugar de a la consola. En este caso cuando escribimos la instrucción siguiente:

```
programa.exe > resultado.txt
```

todo lo que se envía a cout en programa es enviado al fichero resultado.txt en lugar de ser mostrado en la consola (el programa se ejecuta *en silencio*).

Finalmente, es posible redireccionar la entrada y salida simultaneamente.

```
programa.exe < entrada.txt > salida.txt
```

¹En nuestra casa, o bien instalamos un programa que permita abrir una consola en el directorio actual, como por ejemplo *Open Command Prompt Shell Extension* disponible en <http://code.kliu.org/cmdopen/> o bien abrimos un símbolo del sistema (Inicio->Ejecutar->cmd) y vamos cambiando de directorio con la orden cd

²También pueden leerse datos de un fichero desde dentro del propio código fuente del programa, pero esto se verá en el segundo cuatrimestre

Sesión 5

Estructuras condicionales

► **Actividades a realizar en casa**

Actividad: Lectura de soluciones de ejercicios resueltos.

Lea la solución de los siguientes ejercicios resueltos, que podrá encontrar en PRADO (sección dedicada a la Sesión 5 de Prácticas).

- **VIDEO** Lea `II_Gaussiana_Comp.cpp` y `II_Gaussiana_Comp_ECS.cpp`. Se trata de dos ampliaciones del ejercicio **24** (Gaussiana) de la Relación de Problemas I (página **RP-I.1**). Incluye instrucciones condicionales en la **entrada de datos** para evitar realizar cálculos cuando los valores suministrados sean incorrectos (en este caso, el valor de la desviación típica σ).
- Los programas siguientes son de la Relación de Problemas II (página **RP-II.1**):
 - 1 **VIDEO** (Subir sueldo)
`II_Subir_Sueldo.cpp` y `II_Subir_Sueldo_Comp.cpp`
 - 2 (Ver si dos números se dividen) `II_SeDividen.cpp`
 - 3 (Números ordenados) `II_EstanOrdenados.cpp`
 - 6 **VIDEO 1** y **VIDEO 2** (Tiempo entre dos instantes -II-)
`II_DiferenciaTiempo.cpp` y `II_DiferenciaTiempo_Comp.cpp`
 - 7 **VIDEO** (Año bisiesto) `II_Bisiesto.cpp` y `II_Bisiesto_Comp.cpp`
 - 10 (Velocidad imputada) `II_Velocidad_Imputada.cpp`
 - 12 (Pertenencia a intervalo) `II_Pertenece_Intervalo.cpp`
 - 16 **VIDEO 1**, **VIDEO 2** y **VIDEO 3** (Parking)
`II_Parking_basico.cpp` y `II_Parking_detalle.cpp`
- Estudie el seminario sobre los **tipos de datos definidos por enumeración** (también explicados en los apuntes, sección 1.7.6 del Tema II) y lea la solución del siguiente ejercicio (disponible en PRADO, sección dedicada a la Sesión 4 de Prácticas):
 - 19 **VIDEO 1** y **VIDEO 2** (Números ordenados - enumeración)
`II_EstanOrdenados_enum.cpp` y
`II_EstanOrdenados_enum_Ampliacion.cpp`

Actividad: Resolución de problemas.

Ampliar los ejercicios de la Relación de Problemas I (página **RP-I.1**) que se enumeran a continuación. Debe incluir las instrucciones condicionales adecuadas en la **entrada de datos** para evitar realizar cálculos cuando los valores suministrados por el usuario sean incorrectos, y en ese caso debe mostrar un mensaje de **error** adecuado antes de finalizar.

Todos los ejercicios indicados a continuación son **obligatorios**.

- 25 (Pasar de minúscula a mayúscula, con comprobación)
II_Minuscula_a_Mayuscula_Comp.cpp
- 36 (Truncar decimales, con comprobación) II_TruncaDecimales_Comp.cpp

Resolved los ejercicios siguientes de la Relación de Problemas II (página **RP-II.1**):

- *Obligatorios:*

- 5 (Tarifa aérea con descuentos) II_TarifaAerea_Km_Descuentos.cpp
- 9 (Cifrado de caracteres) II_CifradoRotacion.cpp
- 11 (Calcula sanción tráfico) II_Consulta_Multa.cpp
- 18 (Conversión de unidades de medida) II_ConversionUnidadesMedida.cpp

Resuelva, además, los siguientes ejercicios usando algún tipo **enumerado**:

- 20 (Subir salario precario - enumerado) II_Subida_SalarioPrecario_enum.cpp
- 21 (Conversión entre caracteres - enumerado) II_May_a_Min_viceversa_enum.cpp

- *Opcionales:*

- 8 (Haversine, cálculo del mínimo) II_Haversine_con_minimo.cpp
- 14 (Descuento por ventas) II_Descuento_Ventas.cpp
- 15 (Reajustar renta) II_ReajustarRenta.cpp
- 17 (Tarifa aparcamiento (Ampliación)) II_Parking_Ampliacion.cpp

► **Actividades a realizar en las aulas de ordenadores**

Durante esta sesión de prácticas se discutirá sobre las soluciones a las actividades propuestas.

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas, **comentadas** y **modularizadas**.

► **Normas detalladas**

Deberá entregar únicamente el fichero `FP_sesion05.zip` (**importante:** no `.rar` ni otro formato, sólo `.zip`), resultado de empaquetar únicamente los ficheros fuente (`.cpp`), uno por cada ejercicio resuelto.

Los nombres de los ficheros fuente deben ser los indicados (página 25).

La entrega se realizará en PRADO, en el periodo habilitado para ello.

Depuración

" If debugging is the process of removing bugs, then programming must be the process of putting them in. Edsger Dijkstra (1930/2002) "



Un depurador de programas (*debugger* en inglés) permite ir ejecutando un programa sentencia a sentencia (ejecución paso a paso). Además, nos permite ver en cualquier momento el valor de las variables usadas por el programa. El uso de un depurador facilita la localización de errores lógicos en nuestros programas, que de otra forma resultarían bastante difíciles de localizar directamente en el código.

" Debuggers don't remove bugs. They only show them in slow motion "



Para esta sesión de prácticas emplearemos como base el programa `II_Media_int_ampliado.cpp`, disponible en PRADO.

Para poder realizar tareas de depuración en Dev C++ debemos asegurarnos estamos usando un perfil del compilador con las opciones de depuración habilitadas. Si cuando configuramos el compilador seleccionamos **Herramientas | Opciones del Compilador | Compilador a configurar: Debug** nuestro entorno estará preparado para depurar programas.

Si no fuera así, al depurar el programa, Dev C++ nos mostrará la ventana de la figura 4.

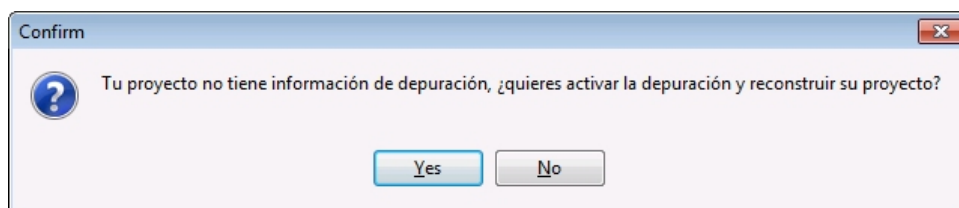


Figura 4: La configuración actual del compilador no permite tareas de depuración

La idea básica en la depuración es ir ejecutando el código *línea a línea* para ver posibles fallos del programa.

Para realizar la depuración del programa debemos dar los siguientes pasos:

1. Establecer una línea del programa en la que queremos que se pare la ejecución. Lo haremos introduciendo un **punto de ruptura** o (*breakpoint*) en dicha línea. Si sospechamos dónde puede estar el error, situaremos el punto de ruptura en dicha línea. En caso contrario, lo situaremos:
 - a) al principio del programa, si no sabemos exactamente dónde falla el programa, o
 - b) al principio del bloque de instrucciones del que desconfiamos, siempre y cuando tengamos confianza en todas las instrucciones que se ejecutan antes.

Para establecer un punto de ruptura podemos mover el ratón en la parte más a la izquierda de una línea de código (o sobre el número de línea) y pulsar el botón izquierdo del ratón en esa posición. La instrucción correspondiente queda marcada en rojo. Si en esa línea ya había un punto de ruptura, entonces será eliminado. También podemos colocar el cursor sobre la instrucción y con el menú contextual (botón derecho del ratón) seleccionar **Añadir/Quitar Punto de Ruptura** o simplemente, pulsar **F4**. Para eliminar un punto de ruptura, se realiza la misma operación que para incluirlo, sobre la instrucción que actualmente lo tiene.

Colocad ahora un punto de ruptura sobre la línea que contiene la primera sentencia condicional `if` (figura 5).

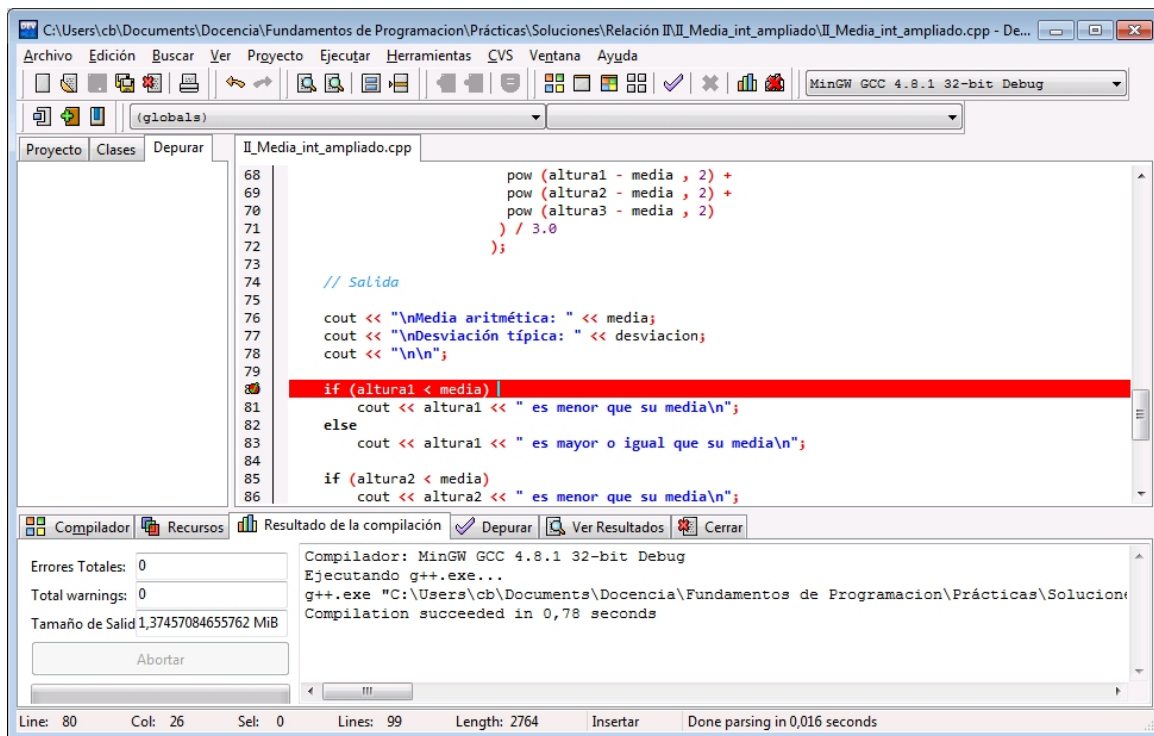


Figura 5: Se ha activado un punto de ruptura

2. Comenzar la depuración:

- pulsar **F5**,
- pulsar sobre el icono ,
- seleccionar en el menú Ejecutar | Depurar, ó
- en la zona inferior, pestaña Depurar, pulsar el botón **Depurar** (figura 6)

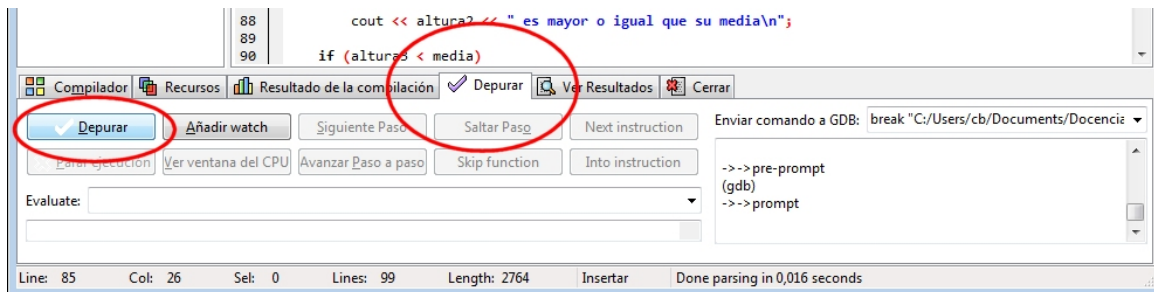


Figura 6: Inicio del proceso de depuración

Muy importante: Si se escoge *ejecutar* en lugar de *depurar*, el programa se ejecuta normalmente, sin detenerse en los puntos de ruptura.

Al iniciarse la depuración se ejecutan todas las sentencias hasta alcanzar el primer punto de ruptura. Llegado a este punto, la ejecución se interrumpe (queda “en espera”) y se muestra en azul (figura 7) la línea que se va a ejecutar a continuación (en este caso, la que contiene el punto de interrupción).

Ahora podemos escoger entre varias alternativas, todas ellas accesibles en la zona inferior (pestaña Depurar) pulsando el botón correspondiente (ver figura 7):

- **Parar ejecución**: Detener la depuración (y ejecución) del programa.
- **Siguiete Paso**: Ejecuta la siguiente instrucción. Si se trata de una llamada a una función, la ejecuta y continúa con la siguiente instrucción, sin entrar a ejecutar las instrucciones internas de la función.
- **Avanzar Paso a paso**: Ejecuta la siguiente instrucción. Si se trata de una llamada a una función, entra en la función y ejecuta la primera instrucción de la función, continuando la depuración dentro de la función.
- **Saltar Paso**: Ejecuta todas las instrucciones hasta encontrar un nuevo punto de ruptura, o llegar al final del programa.

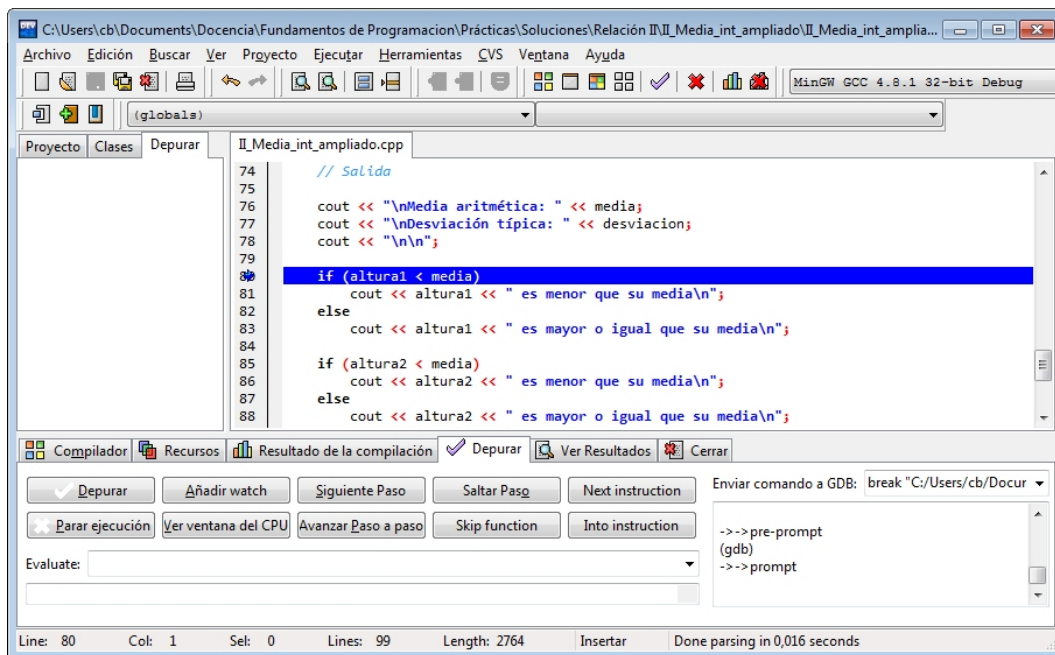


Figura 7: Inicio del proceso de depuración

La posibilidad de ver el valor de los datos que gestiona el programa durante su ejecución hace que sea más sencilla y productiva la tarea de la depuración.

La manera más sencilla de comprobar el valor que tiene una variable es colocar el cursor sobre el nombre de la variable y esperar un instante. Veremos un globo que nos muestra el nombre y valor de la variable (figura 8). El inconveniente es que al mover el ratón desaparece el globo, y cuando queramos inspeccionar nuevamente el valor de la variable debemos repetir la operación.

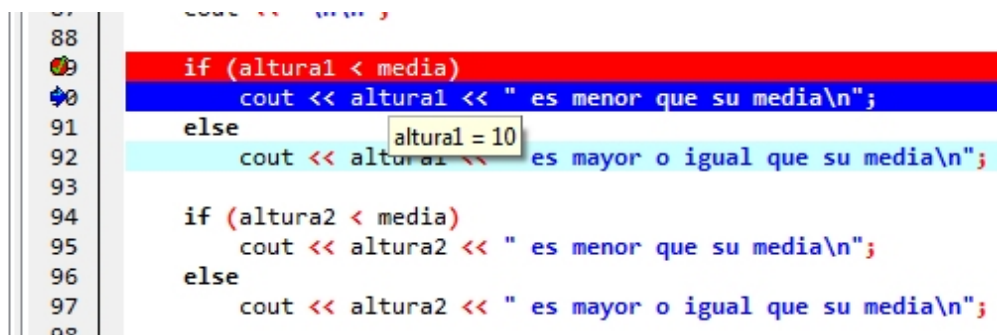


Figura 8: Inspeccionando el valor de una variable

Podemos mantener variables permanentemente monitorizadas. Aparecerán en el Explorador de Proyectos/Clases (seleccionar la pestaña Depurar).

Para añadir una variable podemos:

1. colocar el cursor sobre la variable y con el menú contextual (botón derecho del ratón) seleccionar Añadir watch. Aparecerá una ventana con el nombre de la variable preseleccionado (figura 9.A). Al seleccionar OK aparece la información de esa variable en el Explorador de Proyectos/Clases (figura 9.B).

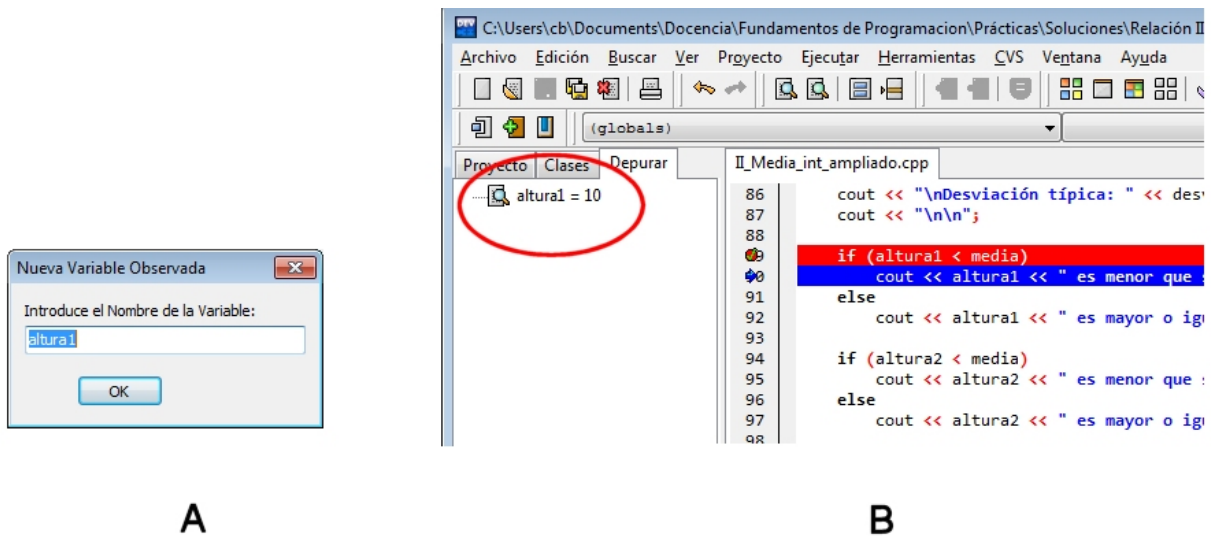


Figura 9: Añadiendo una variable para su inspección permanente

2. abrir el menú contextual (botón derecho del ratón) en cualquier lugar del editor, seleccionar Añadir watch y escribir el nombre de la variable,
3. abrir el menú contextual en el Explorador de Proyectos/Clases (pestaña Depurar), seleccionar Añadir watch y escribir el nombre de la variable,
4. pulsar el botón **Añadir watch** en la zona inferior (pestaña Depurar) y escribir el nombre de la variable.

Conforme se ejecuta el programa podremos ver cómo cambian los valores de las variables monitorizadas.

También podríamos, incluso, modificar su valor directamente pinchando con el botón derecho sobre la variable y seleccionando Modificar Valor.

Otras dos opciones accesibles desde el Explorador de Proyectos/Clases (pestaña Depurar), son Quitar watch para eliminar una variable y Clear All para eliminarlas todas.

Observación final: El depurador ayuda a encontrar errores al permitir ejecutar las sentencias paso a paso y así comprobar por donde va el flujo de control y ver cómo van cambiando las variables. En cualquier caso, nunca nos cansaremos de repetir que el mejor programador es el que piensa la solución en papel, antes de escribir una sola línea de código en el entorno de programación.

"When your code does not behave as expected, do not use the debugger, think".



Una manera informal (¡¡aunque no le falta razón!!) de presentar la efectividad de diversas tácticas de depuración de programas se presenta en la figura 10. La experiencia es un grado que se adquiere practicando (mucho).

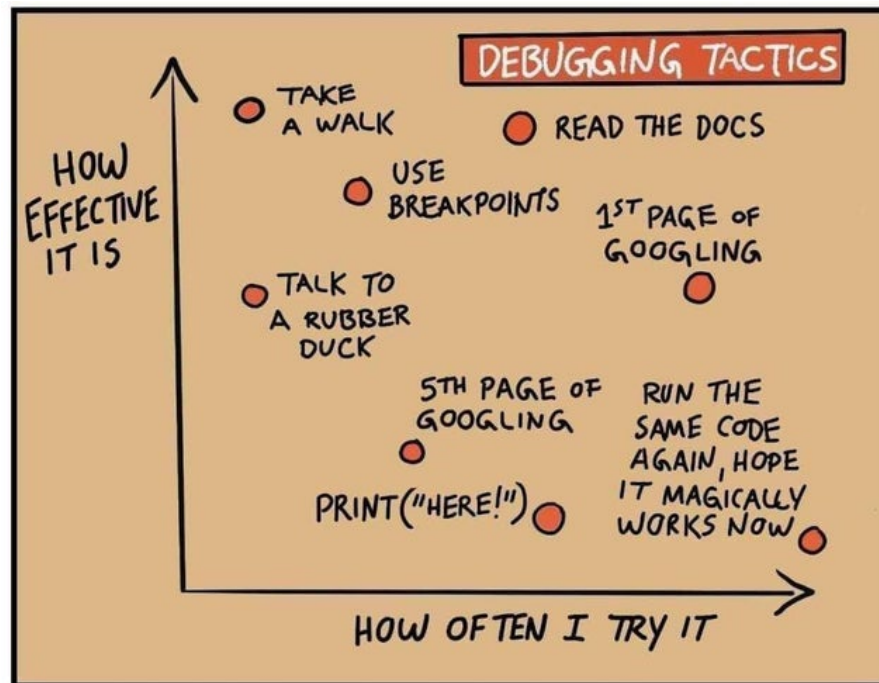


Figura 10: Tácticas de depuración (Fuente: Twitter, autor no citado)

Sesión 6

Estructuras repetitivas (I)

► **Actividades a realizar en casa**

Actividad: Lectura de soluciones de ejercicios resueltos.

Lea la solución de los siguientes ejercicios sobre ciclos, que podrá encontrar en PRADO (sección dedicada a la Sesión 5 de Prácticas):

22 (Conversión a minúsculas - filtros) II_Mayuscula_Minuscula_FiltroEntrada.cpp
23 (Tiempo entre dos instantes -III-)II_DiferenciaTiempo_FiltroEntrada.cpp
25 (Tarifa aérea -III-) II_TarifaAerea_Km_Descuentos_FiltroEntrada.cpp
28 (Pinta dígitos general separando E/C/S) II_PintaDigitos_General_ECS.cpp
29 (Reinvertir inversión) II_Interes_Reinvierte.cpp
35 (Calcular años bisiestos entre dos dados) II_Bisiestos_Intervalo.cpp
39 VIDEO 1 y VIDEO 2 (Sucursal con más ventas) II_Sucursales.cpp
55 (Decimal a binario) II_DecToBin.cpp

Actividad: Resolución de problemas.

Deberá resolver los siguientes ejercicios sobre **ciclos** de la Relación de Problemas II (página RP-II.1):

- **Obligatorios:**

43 (Mayor secuencia ordenada -datos en un rango establecido-) II_SecOrdenadaMasLarga_Rango.cpp
45 (Método RLE) II_RLE.cpp
49 (Acertar un número) II_Adivinar.cpp
51 (Calcular CDF de la gaussiana) II_CDF_Gaussiana.cpp
82 (Aproximación de la raíz cuadrada) II_AproxRaizCuadrada.cpp
83 (Decimal a binario, en rango) II_DecToBin_EnRango.cpp
84 (Binario a decimal) II_BinToDec.cpp

- *Opcionales:*

26 (Divisores de un número) II_Divisores.cpp

30 (Doblar inversión) II_Interes_Doblar.cpp

31 (Parking: calcular tiempo de estancia) II_Parking_LimitadoDinero.cpp

44 (Control carrera) II_Carrera.cpp

52 (Aproximación a π -Madhava-) II_Aprox_PI_Madhava.cpp

53 Aproximación a π -Gregory y Leibniz-) II_Aprox_PI_Gregory-Leibniz.cpp

54 Aproximación a π -Wallis-) II_Aprox_PI_Wallis.cpp

64 (Narcisista) II_Narcisista.cpp

► **Actividades a realizar en las aulas de ordenadores**

En esta sesión se discutirá sobre las soluciones propuestas a los ejercicios de esta sesión.

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas, **comentadas** y **modularizadas**.

► **Normas detalladas**

Deberá entregar únicamente el fichero FP_sesion06.zip (**importante:** no .rar ni otro formato, sólo .zip), resultado de empaquetar únicamente los ficheros fuente (.cpp), uno por cada ejercicio resuelto.

Los nombres de los ficheros fuente deben ser los indicados en la lista de ejercicios propuestos (página 39).

La entrega se realizará en PRADO, en el periodo habilitado para ello.

Sesión 7

Estructuras Repetitivas (II)

► **Actividades a realizar en casa**

Actividad: Lectura de soluciones de ejercicios resueltos.

Lea la solución de los siguientes ejercicios y visualice los videos indicados.

Ejercicios de la Relación de Problemas II (página **RP-II.1**). Las soluciones podrá descargarlas desde PRADO (sección dedicada a la Sesión 7 de Prácticas).

38 (Máxima velocidad imputada) `II_Velocidad_Imputada_Ciclo.cpp`

44 (Control carrera) `II_Carrera.cpp`

62 (Fibonacci) `II_Fibonacci.cpp`

63 **VIDEO** (Número Áureo) `II_Aureo.cpp`

69 (Contar apariciones de un dígito) `II_CuentaDigito.cpp`

Actividad: Resolución de problemas.

- **Obligatorios:**

Resolved los ejercicios siguientes de la Relación de Problemas II (página **RP-II.1**):

47 (Descomposición en factores primos)

`II_Descomposicion_Factores_Primos.cpp`

78 (Gaussiana y CDF en intervalo) `II_Gaussiana_CDF_Intervalo.cpp`

81 (Menú de dos niveles simple) `II_Gestion_Menu_Doble.cpp`

85 (Acertar un número - ampliación) `II_Adivinar_Ampliacion.cpp`

86 (Acertar un numero - autónomo) `II_AdivinarAutonomo.cpp`

87 (Acertar un numero - autónomo e inteligente)

`II_AdivinarAutonomo_Inteligente.cpp`

- *Opcionales:*

Resolved los ejercicios siguientes de la Relación de Problemas II (página **RP-II.1**):

- 48 (Multiplicación rusa) II_Multiplicacion_Rusa.cpp
- 66 (Estudio de inversiones) II_Interes_Reinvierte_Anidado.cpp
- 68 (Divisores de varios números) II_Divisores_Varios.cpp
- 71 (Número perfecto) II_Perfecto.cpp
- 72 (Encontrar enteros - igualdad al cuadrado) II_IgualdadCuadraro.cpp
- 76 (Número secuenciable) II_Secuenciable.cpp
- 80 (Menú de dos niveles - gaussiana) II_Gestion_Menus_Gaussiana.cpp

► **Actividades a realizar en las aulas de ordenadores**

Durante esta sesión de prácticas se discutirá sobre las soluciones a las actividades propuestas.

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas, **comentadas** y **modularizadas**.

► **Normas detalladas**

Deberá entregar únicamente el fichero FP_sesion07.zip (**importante:** no .rar ni otro formato, sólo .zip), resultado de empaquetar únicamente los ficheros fuente (.cpp), uno por cada ejercicio resuelto.

Los nombres de los ficheros fuente deben ser los indicados (página **41**).

La entrega se realizará en PRADO, en el periodo habilitado para ello.

Sesión 8

Funciones

En esta sesión trabajaremos sobre ejercicios del tema 3 (Funciones). Encontrará que algunos de los ejercicios realizados en anteriores sesiones de prácticas deberá reescribirlos usando funciones.

► **Actividades a realizar en casa**

Actividad: Lectura de soluciones de ejercicios resueltos.

Lea la solución de los siguientes ejercicios y visualice los videos indicados.

Ejercicios de la Relación de Problemas III (página [RP-III.1](#)). Las soluciones podrá descargarlas desde PRADO (sección dedicada a la Sesión 8 de Prácticas).

- 4 [VIDEO 1](#) y [VIDEO 2](#) (Número combinatorio, con funciones)
III_Combinatorio_Funcs.cpp
- 6 (Número aureo, con funciones) III_Aureo_Funcs.cpp
- 7 (Pinta dígitos con funciones) III_PintaDigitos_Funcs.cpp
- 12 (Números amigos) III_Amigos_Funcs.cpp
- 13 [VIDEO 1](#) y [VIDEO 2](#) (Contar apariciones de un dígito)
III_CuentaDigito_Funcs.cpp
- 15 [VIDEO 1](#) y [VIDEO 2](#) (Control de una carrera, con funciones)
III_Carrera_Funcs.cpp
- 16 [VIDEO](#) (Recortar string) III_Podar_string_Funcs.cpp
- 17 (Reajustar string) III_Reajustar_string_Funcs.cpp
- 34 [VIDEO](#) (Rectas, con funciones) III_Recta_Funcs.cpp
- 38 [VIDEO 1](#) y [VIDEO 2](#) (Aproximación de la raíz cuadrada, con funciones)
III_AproxRaizCuadrada_Funcs.cpp
- 41 (Decimal a binario, y de binario a decimal, con funciones)
III_DecToBin_BinToDec_Funcs.cpp

Actividad: Resolución de problemas.

- *Obligatorios:*

- 14 (Redondear y truncar un número decimal, con funciones)
III_Redondear_Truncar_Decimal_Funcs.cpp
- 18 (Funciones para leer enteros) III_LecturaEnteros_Funcs.cpp
- 22 (Funciones para dar formato a string) III_FormateaString_Funcs.cpp
- 31 (Gaussiana y CDFs, Regla empírica)
III_GaussianaCDF_Funcs_ReglaEmpirica.cpp
- 37 (Verificar NIF/NIE) III_Comprueba_NIF_NIE.cpp
- 43 (Descomposición en factores primos, con funciones)
III_Descomposicion_Factores_Primos_Funcs.cpp

- *Opcionales:*

- 10 (Número narcisista, con funciones) III_Narcisista_Funcs.cpp
- 19 (Funciones para leer reales) III_LecturaReales_Funcs.cpp
- 20 (Funciones ToString personalizadas) III_ToString_Double_Int_Funcs.cpp
- 23 (Acertar un número - ampliación 2 con la función LeeOpcion2Alternativas)
III_Adivinar_Ampliacion.cpp
- 24 (Progresión geométrica, con funciones) III_ProgresionGeom_Funcs.cpp
- 25 (Suma de una serie, con funciones) III_SumaSerie_Funcs.cpp
- 27 (Multiplicación rusa, con funciones) III_MultiplicacionRusa_Funcs.cpp
- 32 (Parking, con funciones) III_Parking_Funcs.cpp
- 39 (Decimal a binario, con funciones) III_DecToBin_Funcs.cpp
- 40 (Binario a decimal, con funciones) III_BinToDec_Funcs.cpp
- 42 (Máxima velocidad imputada) III_Maxima_Velocidad_imputada_Funcs.cpp

► **Actividades a realizar en las aulas de ordenadores**

Durante esta sesión de prácticas se discutirá sobre las soluciones a las actividades propuestas.

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas, **comentadas** y **modularizadas**.

► **Normas detalladas**

Deberá entregar únicamente el fichero `FP_sesion08.zip` (**importante:** no `.rar` ni otro formato, sólo `.zip`), resultado de empaquetar únicamente los ficheros fuente (`.cpp`), uno por cada ejercicio resuelto.

Los nombres de los ficheros fuente deben ser los indicados (página 44).

La entrega se realizará en PRADO, en el periodo habilitado para ello.

Depuración de funciones

En la sesión 3 trabajamos sobre la depuración de programas usando Dev C++. Entonces no conocíamos cómo escribir funciones y no pudimos sacar partido a todas las opciones de depuración. En esta sesión de prácticas vamos a trabajar sobre la manera en la que se puede monitorizar la ejecución de un programa que incluye funciones.

Usaremos como ejemplo la solución al ejercicio 4 (Número combinatorio con funciones) de la Relación de Problemas III (página **RP-III.1**).

Antes de empezar con las tareas de depuración observaremos el *explorador de clases*. Para acceder a él, basta con seleccionar Ver | Ir al Explorador de Clases (ó simplemente **Ctrl+F3**). En la figura 11 puede observar que el explorador de clases muestra información acerca de las funciones contenidas en el fichero fuente abierto en el editor.

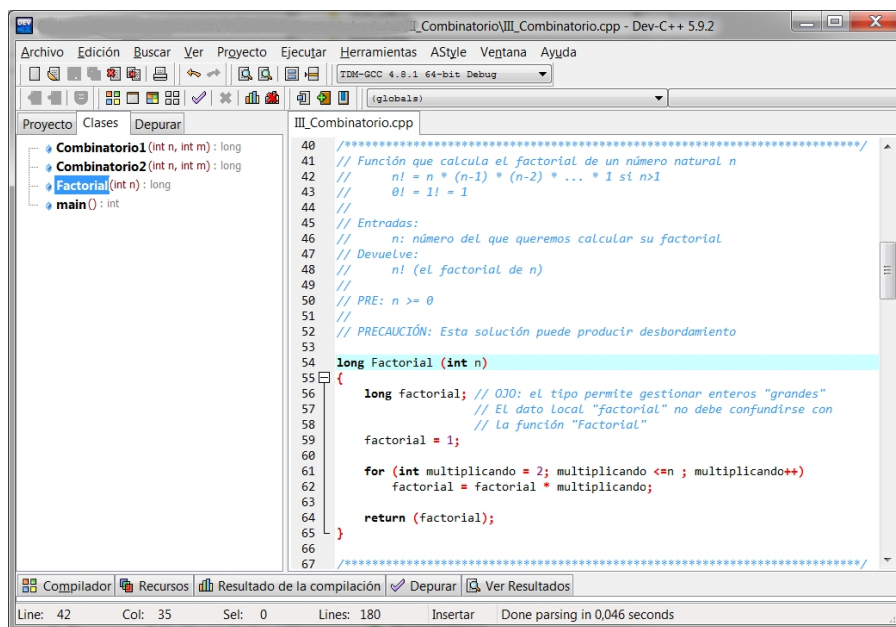


Figura 11: Explorador de clases

Para cada función muestra su *nombre*, los *parámetros formales* y su *tipo*, así como el *tipo de la función* (el tipo del valor devuelto). Se muestran todas las funciones en orden alfabético. Haciendo click sobre el nombre de cualquier función en el explorador, en el editor vemos el código de la función seleccionada. Éste es una manera rápida de acceder al código de cualquier función en nuestros programas.

El proceso de depuración se inicia de la manera habitual:

1. Fijar un punto de ruptura.
2. Comenzar la depuración.

Fijaremos un punto de ruptura, por ejemplo, en la línea 160: buscar la línea

```
combinatorio1 = Combinatorio1 (total_a_elegir, elegidos);
```

y establecer en ella un punto de ruptura. Comenzar la depuración.

El programa se ejecuta hasta llegar a la línea 160, donde se detiene. Ahora podemos monitorizar su ejecución usando los botones disponibles en la zona inferior, bajo la pestaña Depurar (ver figura 7).

- **Parar ejecución**: Detener la depuración (y ejecución) del programa.
- **Siguiente Paso**: Ejecuta la siguiente instrucción. Si se trata de una llamada a una función, la ejecuta y continúa con la siguiente instrucción, sin entrar a ejecutar las instrucciones internas de la función.
- **Avanzar Paso a paso**: Ejecuta la siguiente instrucción. Si se trata de una llamada a una función, entra en la función y ejecuta la primera instrucción de la función, continuando la depuración dentro de la función.
- **Saltar Paso**: Ejecuta todas las instrucciones hasta encontrar un nuevo punto de ruptura, o llegar al final del programa.

En la línea 160, si se pulsara **Siguiente Paso** se ejecuta completamente esa línea: la llamada a la función `Combinatorio1` y la instrucción de asignación, pasando el control a la línea 161 (ver figura 12). Observad cómo la variable `combinatorio1` se ha actualizado correctamente.

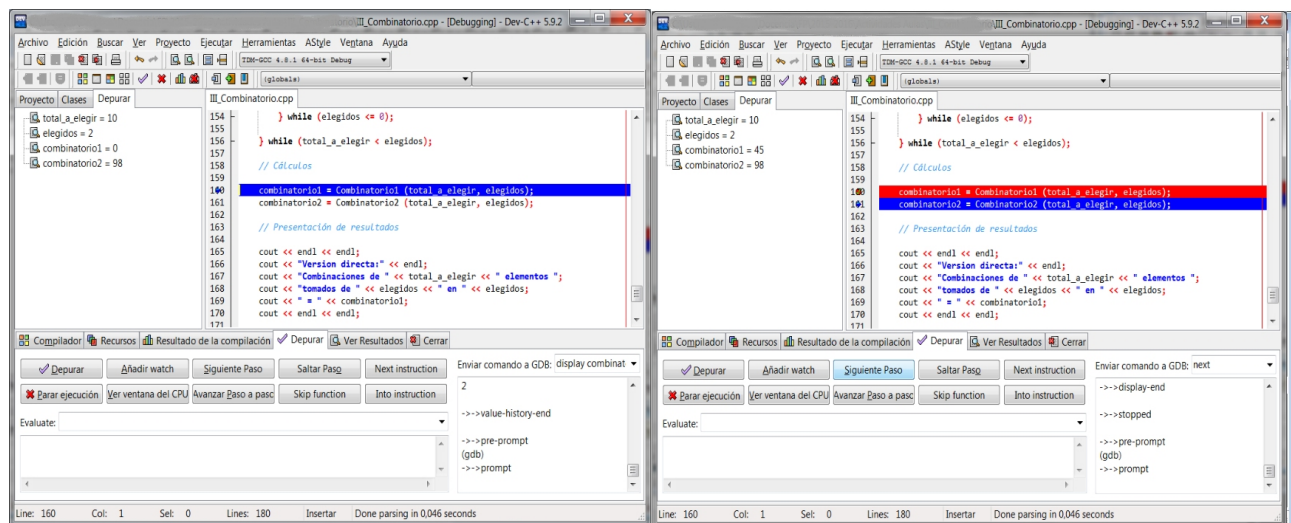


Figura 12: Efecto de *Siguiente Paso*

Si en lugar de ese botón pulsamos **Avanzar Paso a paso** se entra a ejecutar la función, pasando el control a la primera instrucción de la función, la línea 89 (ver figura 13).

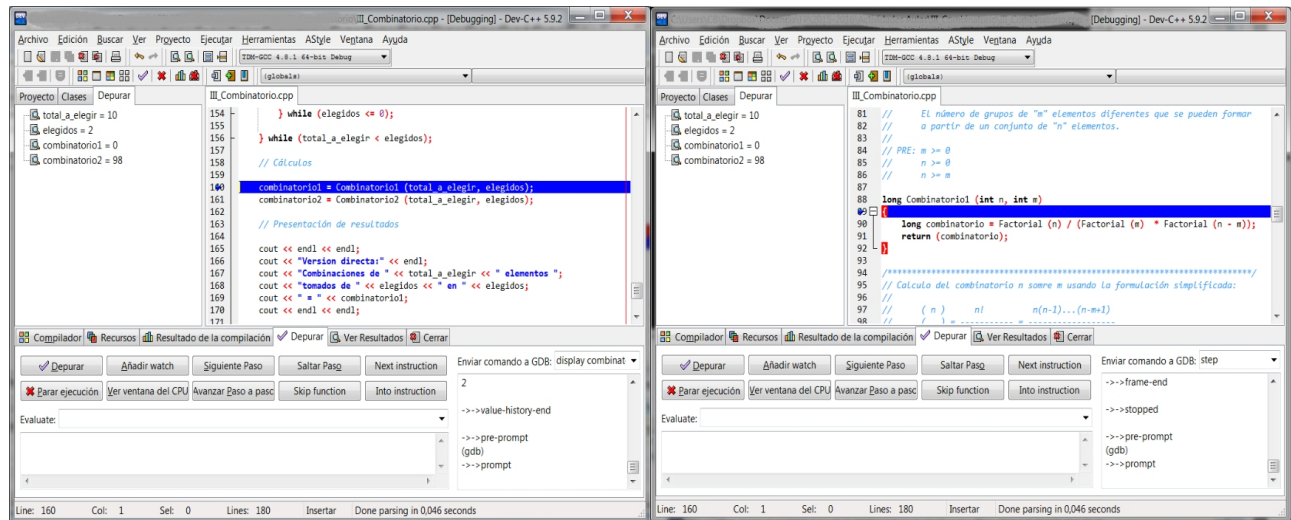


Figura 13: Efecto de *Avanzar Paso a paso*

Durante la ejecución de una función pueden añadirse a la lista de variables monitorizadas cualquiera de las variables locales de la función (incluidas los parámetros formales, por supuesto). Al finalizar la ejecución de la función y dejar de estar activas las variables locales de la función veremos un mensaje de error en estas variables.

La ejecución completa de la línea 90 conlleva la ejecución de tres llamadas a la función `Factorial`:

```
long combinatorio = Factorial(n) / (Factorial(m)*Factorial(n-m));
```

En este punto,

- si se pulsa **Siguiente Paso** se completa la ejecución de esa línea y se cede el control a la línea 91. Observad que la variable local `combinatorio` contiene el valor ya calculado.
- si se pulsa **Avanzar Paso a paso** se entra a ejecutar la función `Factorial`, pasando el control a la primera instrucción de esa función.

Continúe monitorizando la ejecución del programa como desee. No se olvide de probar a establecer un punto de ruptura dentro de una función y observar qué ocurre cuando se pulsan el botón **Siguiente Paso** ¿se detiene la ejecución en el punto de ruptura o lo ignora?

Sesión 9

Registros. Vectores (1)

En esta sesión trabajaremos sobre ejercicios del tema 4 (Registros, vectores y matrices) y concretamente nos centraremos en *registros* y en *vectores* (uso general como estructura lineal de almacenamiento y problemas de búsqueda en vectores).

Los ejercicios sobre vectores se resuelven empleando vectores clásicos (*arrays*). No es necesario, ni conveniente, emplear funciones para su resolución ya que el objetivo es trabajar directamente con *arrays* por lo que **bastará con escribir el código en la función main**. Podría emplearse alguna función siempre que no recibiera como argumento un *array*, evidentemente.

► **Actividades a realizar en casa**

Actividad: Lectura de soluciones de ejercicios resueltos.

Lea la solución de los siguientes ejercicios (podrá encontrarlas en PRADO) y visualice los videos indicados. Encontrará también ficheros de texto para poder ejecutar algunos de los ejercicios usando redirección de entrada.

- **Ejercicio sobre *registros***

- 1 VIDEO (Rectas, con funciones y struct) IV_Recta_Regs.cpp

- **Ejercicios sobre *vectores***

- 16 VIDEO1 y VIDEO2 (Sucursales) IV_Sucursales_Array.cpp

- 17 (Palíndromo) IV_Palindromo_Array.cpp

- 18 (Estadística sobre letras) IV_EstadisticaLetras_Array.cpp

- 22 VIDEO1, VIDEO2 y VIDEO3 (Elimina repetidos -apartados a y b-)
IV_EliminaRepetidos_Array_AyB.cpp

- 23 VIDEO1 y VIDEO2 (Criba de Eratóstenes) IV_Eratostenes_Array.cpp

- 26 (login automático) IV_Login_Automatico_Alumnos.cpp

- La versión que se proporciona emplea únicamente la función sugerida. Considere ampliar esta versión usando las funciones de lectura de un número.

- 32 VIDEO (array de *circunferencias*) IV_Circunferencias_en_Rectangulo.cpp

Actividad: Resolución de problemas.

- **Obligatorios (registros)**

- 4 (Rectángulo y circunferencia, con struct) IV_Rectangulo_Regs.cpp
- 6 (Control carrera, con struct) IV_Carrera_Regs.cpp

- **Obligatorios (vectores)**

- 9 (Leer y comparar dos arrays) IV_Media_y_Comparar_ArraysInt.cpp
- 12 (Inflación) IV_Inflacion.cpp
- 14 (Sueldo de los funcionarios) IV_Valor_SueldoFuncionarios.cpp
- 24 (Descomposición en factores primos y Criba de Eratóstenes)
IV_Eratostenes_Array_para_Descomposicion.cpp
- 39 (Núm.intentos en acertar un numero - autónomo e inteligente)
IV_Intentos_AdivinarAutonomo_Inteligente.cpp

- **Opcionales (registros)**

- 2 (Circunferencia, con funciones y struct) IV_Circunferencia_Regs.cpp
- 3 (CDF, Regla empírica, con struct)
IV_GaussianaCDF_Funcs_Regs_ReglaEmpirica.cpp
- 5 (Máxima velocidad imputada, con funciones y struct)
IV_Maxima_Velocidad_imputada_Regs.cpp

- **Opcionales (vectores)**

- 15 (Cambio de moneda, con caja) IV_CambioMonedas_con_caja.cpp
- 20 (Desnivel) IV_Desnivel.cpp
- 21 (Asignación de escaños con la Ley D'Hont) IV_Ley_DHont.cpp
- 22 (Elimina repetidos -apartado c-) IV_EliminaRepetidos_Array.cpp
- 27 (RLE - una secuencia) IV_RLE_Array.cpp
- 31 (Subsecuencia ordenada más larga (array))
IV_SecuenciaOrdenadaMasLarga_Array.cpp
- 37 (Robot, en línea) IV_Robot_Linea.cpp

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas, **comentadas y modularizadas**.

Sobre la lectura (o no) de espacios y otros separadores

Si interesa leer un carácter separador **no** puede emplear `cin >> c`, sino `c = cin.get()`. Cada vez que se ejecute `cin.get()` se lee un carácter (incluido el espacio en blanco, tabulador ó salto de línea) desde la entrada de datos por defecto.

El ciclo de lectura de datos será del tipo:

```
char c;
.....
c = cin.get();
while (c != TERMINADOR){
    .....
    c = cin.get();
}
```

Asignar más espacio en memoria al programa

Algunos ejercicios pueden usar vectores/matrices muy grandes, como para albergar, por ejemplo, el texto del Quijote. Para guardar en un vector un texto de ese tipo, y que quepa en la pila, debe hacer los siguientes cambios:

- Declare una constante para el tamaño del vector que permita almacenar todos los caracteres.

```
const int TAMANIO = 2500000;
char texto[TAMANIO];
```

Si se prefiere, para que quede más claro el número de caracteres reservados en memoria, puede usar una constante `double` en formato científico y dejar que C++ haga el casting a `int` automáticamente:

```
const int TAMANIO = 25e+5; // int <-- double
char texto[TAMANIO];
```

- Aumente el tamaño de la pila asignada por el sistema operativo al programa generado por el compilador. Para ello, debe seleccionar desde DevC++:

Herramientas -> Opciones del Compilador

Señale la opción

Añadir estos comandos a la línea de comandos del linker

y en la caja de texto añadir lo siguiente (todo seguido, sin espacios en blanco):

`-Wl,--stack,2600000`

con lo que debería quedar (figura 14):

`-static-libgcc -Wl,--stack,2600000`

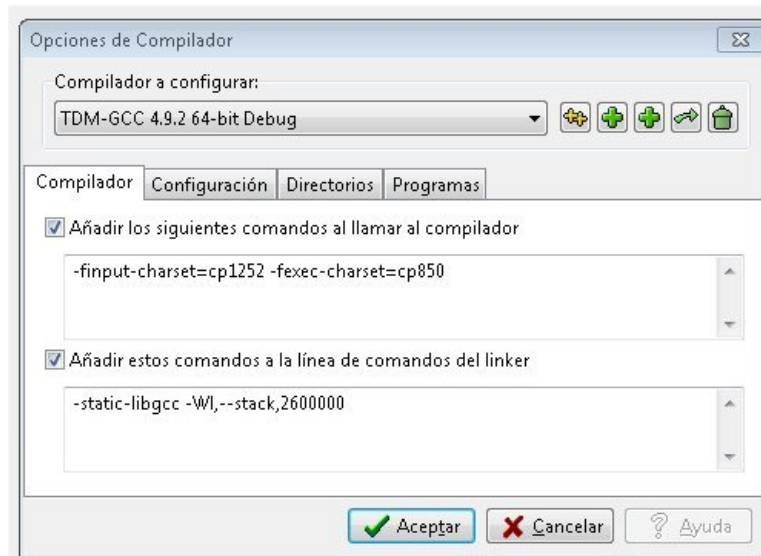


Figura 14: Configurar las *Opciones del compilador*

El último número indica al enlazador (linker) que utilice un tamaño de memoria para la pila de 2600000 bytes. En nuestro ejemplo, teníamos 2.5 millones de caracteres, por lo que necesitaremos 2.5 MB. Reservamos algo más para otras variables del programa: en total 2.6 millones de bytes (2.6 MB).

Tenga en cuenta que aquí no puede usar la notación científica.

► **Actividades a realizar en las aulas de ordenadores**

En esta sesión se discutirá sobre las soluciones propuestas a los ejercicios de esta sesión.

► **Normas detalladas**

Deberá entregar únicamente el fichero `FP_sesion09.zip` (**importante:** no `.rar` ni otro formato, sólo `.zip`), resultado de empaquetar todos los ficheros fuente (`.cpp`), uno por cada ejercicio resuelto.

La entrega se realizará en PRADO, en el periodo habilitado para ello.

Sesión 10

Vectores (2). Matrices

En esta sesión continuamos trabajando con ejercicios del tema 4 (Registros, vectores y matrices). Ahora nos centraremos en problemas sobre *vectores* (ordenación) y *matrices*.

No es necesario, ni conveniente, emplear funciones para su resolución ya que el objetivo es trabajar directamente con arrays/matrices por lo que **bastará con escribir el código en la función main**. En cualquier caso, podría emplearse alguna función siempre que no recibiera como argumento un *array* ó una matriz, evidentemente.

► **Actividades a realizar en casa**

Actividad: Lectura de soluciones de ejercicios resueltos.

Lea la solución de los siguientes ejercicios, que podrá encontrar en PRADO (carpeta Archivos Sesión 10 - previos, sección PRÁCTICA 10) y visualice los videos indicados.

42 VIDEO1 y VIDEO2 (Sucursales, con matrices) IV_Sucursales_Matriz.cpp

43 (Traspuesta y producto de matrices) IV_Matriz_Clasica_Varios.cpp

57 (Liga) IV_Liga.cpp

60 (Matriz aleatoria, ejercicio 1) IV_MatrizAleatoria_1.cpp

61 (Matriz aleatoria, ejercicio 2) IV_MatrizAleatoria_2.cpp

Actividad: Resolución de problemas.

- **Obligatorios**

40 (Listado ordenado de empleados) IV_Orden_ArrayEmpleados.cpp

47 (Inflación en varios países) IV_Inflacion_Matriz.cpp

55 (Robot, en un tablero) IV_Robot_Tablero.cpp

56 (Robot, genera cadena de ordenes) IV_Robot_GeneraOrdenes.cpp

- *Opcionales*

- 38 (Ranking, 1500ML) IV_Ranking_1500ML.cpp
- 46 (RLE - serie de secuencias) IV_RLE_Matriz.cpp
- 48 (Histograma, vocales) IV_Histograma_Vocales.cpp
- 49 (Histograma, todas las letras) IV_Histograma_Letras.cpp
- 51 (ASCII art) IV_ASCII_Art_Matriz.cpp
- 52 (Gráficos de temperaturas) IV_Grafico_Temperaturas.cpp
- 53 (Asignación de asientos) IV_Asignar_Asientos_Matriz.cpp
- 58 (Red de ciudades) IV_Red_Ciudades.cpp
- 59 (Ranking, varias pruebas) IV_Ranking.cpp

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas, **comentadas** y **modularizadas**.

► **Actividades a realizar en las aulas de ordenadores**

En esta sesión se discutirá sobre las soluciones propuestas a los ejercicios de esta sesión.

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas, **comentadas** y **modularizadas**.

► **Normas detalladas**

Deberá entregar únicamente el fichero FP_sesion10.zip (**importante:** no .rar ni otro formato, sólo .zip), resultado de empaquetar únicamente los ficheros fuente (.cpp), uno por cada ejercicio resuelto.

Los nombres de los ficheros fuente deben ser los indicados (página 53).

La entrega se realizará en PRADO, en el periodo habilitado para ello.

Sesión 11

Clases (I): Conceptos básicos - Constructores

► **Actividades a realizar en casa**

Actividad: Visualizar videos didácticos.

En PRADO (sección dedicada al Tema 5) encontrará varios videos explicativos en los que se explica, de manera detallada, la resolución del ejercicio 1 de la *Relación de Problemas V* (Clase Recta). **Use estos vídeos como refuerzo del material didáctico de clase y ayuda para la elaboración de los ejercicios que debe realizar esta semana.** Descargue las soluciones de ese ejercicio (fichero V_Recta.zip)

Este material también está disponible en PRADO, sección PRÁCTICA 11.

Lea el material disponible en la sección CLASES DE INTERÉS, concretamente lo referente a las clases Cronometro y GeneradorAleatorioEnteros.

Actividad: Lectura de soluciones de ejercicios resueltos.

Lea la solución de los siguientes ejercicios, que podrá encontrar en PRADO (carpeta Archivos Sesión 11 - previos, sección PRÁCTICA 11).

2 (Medir el tiempo de ejecución de un bloque de código con la clase Cronometro). Ver:

- V_SumatorioFactoriales_ConFunciones.cpp
- V_SumatorioFactoriales_Crono.cpp

3 (Acertar una serie de números siguiendo las pistas del programa).

Los números aleatorios se generan usando la clase GeneradorAleatorioEnteros. Ver: V_Adivinar.cpp.

5 (Clase Distancia) V_Distancia.cpp

9 (Clase ProgresionGeometrica) V_ProgresionGeometrica_Clase.cpp

Actividad: Resolución de problemas.

En esta sesión de prácticas empezaremos a trabajar en problemas sencillos sobre clases. Los enunciados de los ejercicios propuestos para esta sesión se encuentran en la *Relación de Problemas V* (página **RP-V.1**).

- **Obligatorios:**

- 4 (Clase Dinero) `V_Dinero_Clase.cpp`
- 6 (Clases Rectangulo y Circunferencia)
`V_Rectangulo_y_Circunferencias.cpp`
- 7 (Clases Rectangulo y Circunferencia - Circunferencias inscritas)
`V_Rectangulo_y_CircunferenciasInscritas.cpp`
- 8 (Clase Gaussiana) `V_Gaussiana_Clase.cpp`
- 14 (Lector seguro, con clase) `V_LectorDatos_Clase.cpp`

- **Opcionales:**

- 10 (Clase CoordenadasGPS) `V_CoordenadasGPS_Clase.cpp`
- 11 (Clase Instante) `V_Instante_Clase.cpp`
- 12 (Clase DepositoBancario) `V_DepositoBancario_Clase.cpp`
- 13 (login automático, con clase) `V_LoginAutomatico_Clase.cpp`
- 15 (Manipulador de datos string) `V_ManipuladorString.cpp`

► **Actividades a realizar en las aulas de ordenadores**

En esta sesión se discutirá sobre las soluciones propuestas a los ejercicios de esta sesión.

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas, **comentadas y modularizadas**.

► **Normas detalladas**

Deberá entregar únicamente el fichero `FP_sesion11.zip` (**importante:** no `.rar` ni otro formato, sólo `.zip`), resultado de empaquetar todos los ficheros fuente (`.cpp`), uno por cada ejercicio resuelto.

La entrega se realizará en PRADO, en el periodo habilitado para ello.

► Trabajo personal

Inspección de objetos

En esta sesión de prácticas vamos a trabajar sobre la manera en la que Dev C++ presenta la información sobre la estructura de las clases definidas en el programa sobre el que se está trabajando y las facilidades que ofrece para monitorizar la ejecución de programas que incluyen clases. Usaremos como ejemplo una versión simple de la clase `SegmentoDirigido` que puede encontrar en los apuntes de teoría.

En primer lugar, crearemos la carpeta `V_SegmentoDirigido` y copiaremos en ella el fichero fuente `V_SegmentoDirigido.cpp` (disponible en PRADO (sección dedicada a la Sesión 11 de Prácticas, carpeta Archivos Sesión 11).

Emplearemos de nuevo el *explorador de clases* (ver figura 15): seleccionar **Ver | Ir al Explorador de Clases** (ó simplemente **Ctrl+F3**).

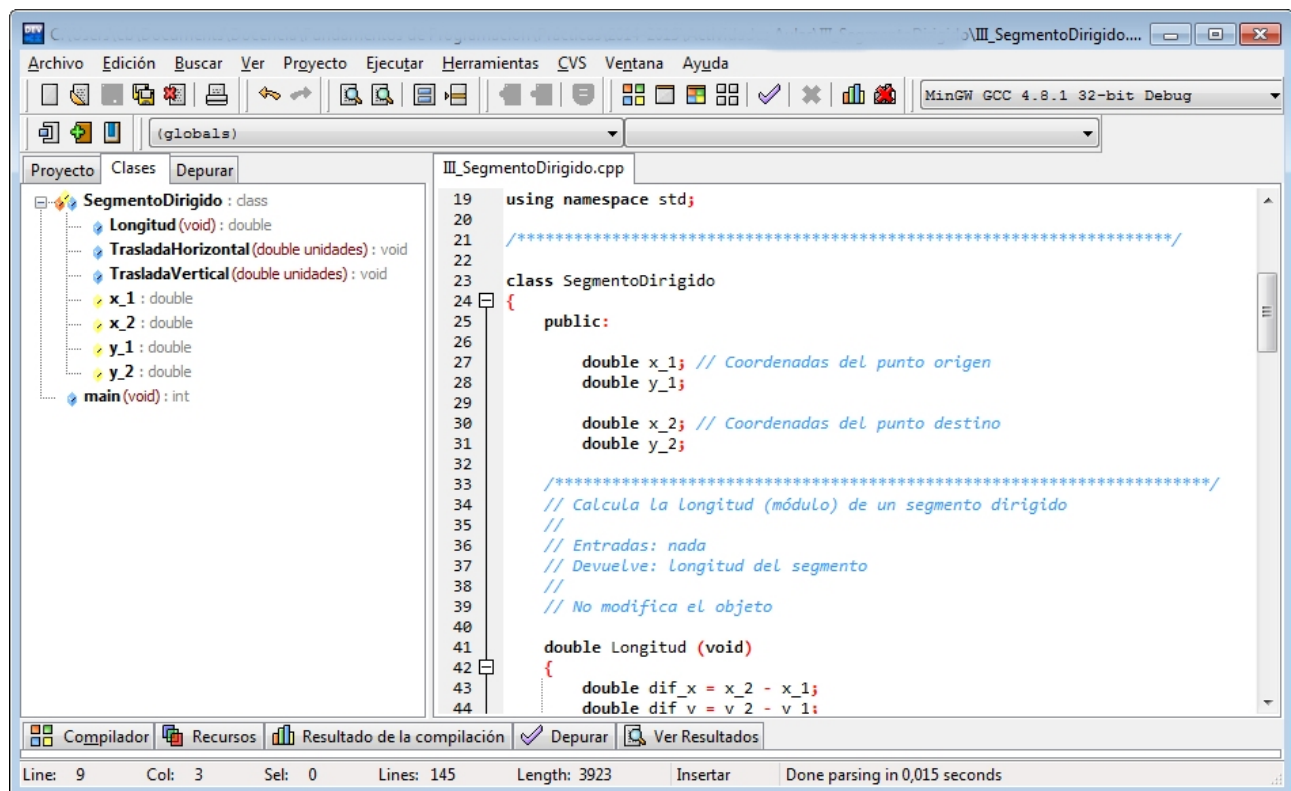


Figura 15: Explorador de clases

En la figura 15 puede observar que el explorador de clases muestra, para este programa,

información acerca de los componentes de la clase `SegmentoDirigido`, definida en el fichero fuente abierto en el editor. También muestra las cabeceras de las funciones definidas en el fichero, como ya conocemos (en este caso, únicamente la función `main`).

En primer lugar observe que el explorador de clases organiza los elementos que muestra de manera jerárquica, por niveles. En este ejemplo encontramos dos elementos en el nivel principal (por orden de aparición en el explorador de clases):

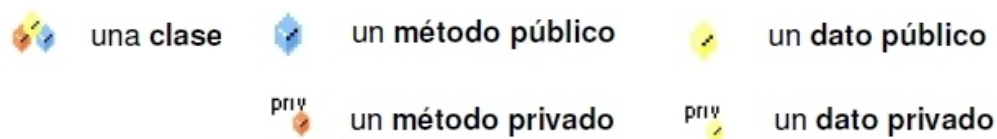
- La clase `SegmentoDirigido`
- La función `main`.

El elemento `SegmentoDirigido` está etiquetado con la palabra `class`. Se enumeran una serie de elementos dentro de esa clase que se muestran asociados con líneas.

Dev C++ organiza la presentación de los elementos de la clase enumerando en primer lugar los *métodos* (por orden alfabético) y a continuación los *datos* (también por orden alfabético).

Haciendo click sobre el nombre de cualquier componente de la clase el editor muestra el código de la función seleccionada (su implementación o definición), o se accede a la declaración del dato seleccionado. Es una manera rápida de acceder al código de cualquier elemento de una clase.

Dev C++ usa un conjunto de iconos para mostrar los diferentes elementos del programa:



Los objetos, como cualquier otro dato, pueden ser monitorizados durante la depuración de un programa. Cuando se añade un objeto para su inspección se muestran sus campos y sus valores. Por ejemplo, en la figura 16 se muestra el contenido del objeto `un_segmento` después de haber asignado los valores a sus cuatro campos.

Para llegar al estado mostrado en la figura 16:

1. se estableció un punto de ruptura en la línea 96,
2. se añadió la variable `un_segmento` a la lista de datos monitorizados (botón derecho sobre `un_segmento` y **Añadir watch**)
3. se inició la depuración del programa (**Ejecutar** | **Depurar**).

Una vez que aparece el objeto `un_segmento` entre los datos monitorizados puede trabajar con él (y con cualquiera de los campos que lo forman) de la misma manera que con cualquier otro dato monitorizado.

No es preciso que muestre un objeto completo, también puede monitorizar cualquier campo individual. Por ejemplo, en la línea 93 marque `un_segmento.x_2`, haga click con el botón derecho y seleccione **Añadir watch**.

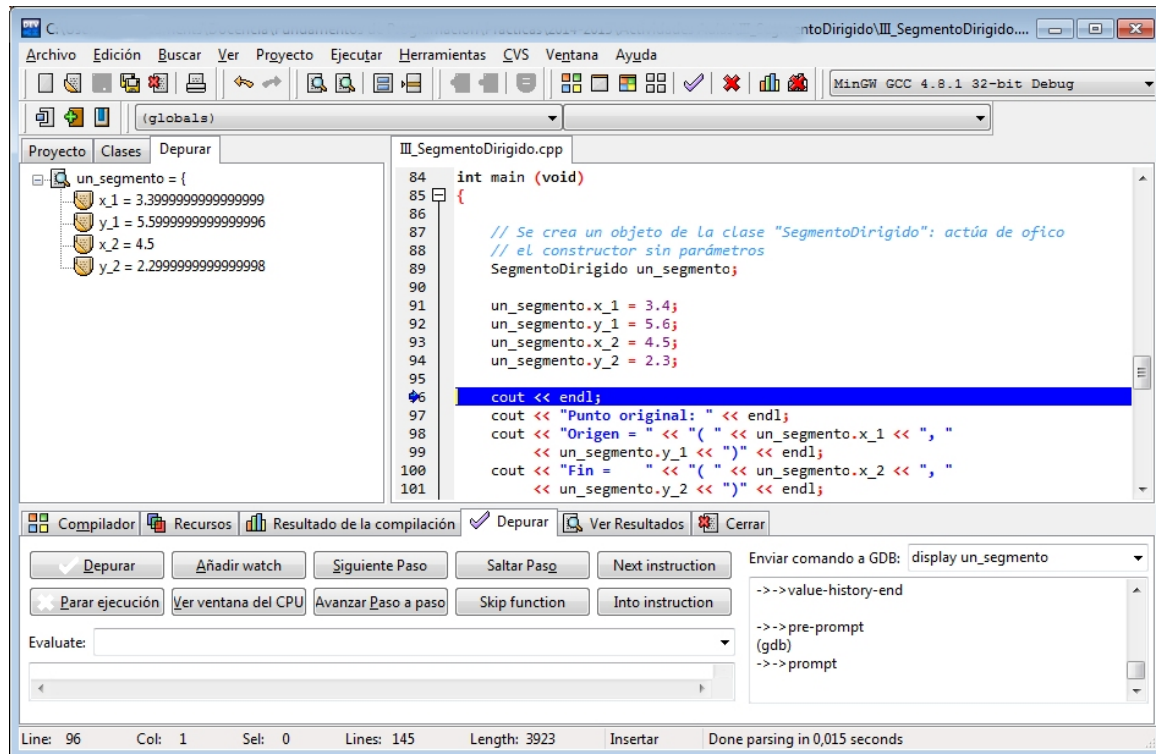


Figura 16: Explorador de clases mostrando el contenido de un objeto SegmentoDirigido

Durante la depuración puede *avanzar paso a paso*, y entrar a ejecutar los métodos de la clase línea a línea, como si se tratara de una función cualquiera. No olvide que al tratarse de un método de una clase, éste se ejecuta sobre un objeto de la clase (el objeto implícito). Por ejemplo, establezca un punto de ruptura en la línea 110

```
un_segmento.TrasladaHorizontal (10);
```

continúe la depuración hasta llegar a esa línea (figura 17.A) y seleccione **Avanzar Paso a paso**. La siguiente línea que se va a ejecutar es la línea 60, la primera del método TrasladaHorizontal (figura 17.B)

El objeto implícito es `un_segmento`. Los identificadores `x_1` y `x_2` que encontramos en las líneas 60 y 61 se refieren a los campos públicos `x_1` y `x_2` del objeto implícito. Si se coloca sobre `x_1` (por ejemplo) observará que el valor que se muestra es 3.4 , el valor del campo `x_1` del objeto `un_segmento`.

Al seleccionar **Siguiete paso** puede observar que **no** se actualiza el valor mostrado del campo `x_1` del objeto `un_segmento`, pero si se coloca sobre `x_1` observará que el valor que se muestra es 13.4, el valor actualizado. La explicación es simple: el objeto `un_segmento` no es accesible (por ese nombre) desde el método `TrasladaHorizontal`. Si añadimos a la lista de inspección de datos `x_1` podremos confirmar que el valor está actualizado.

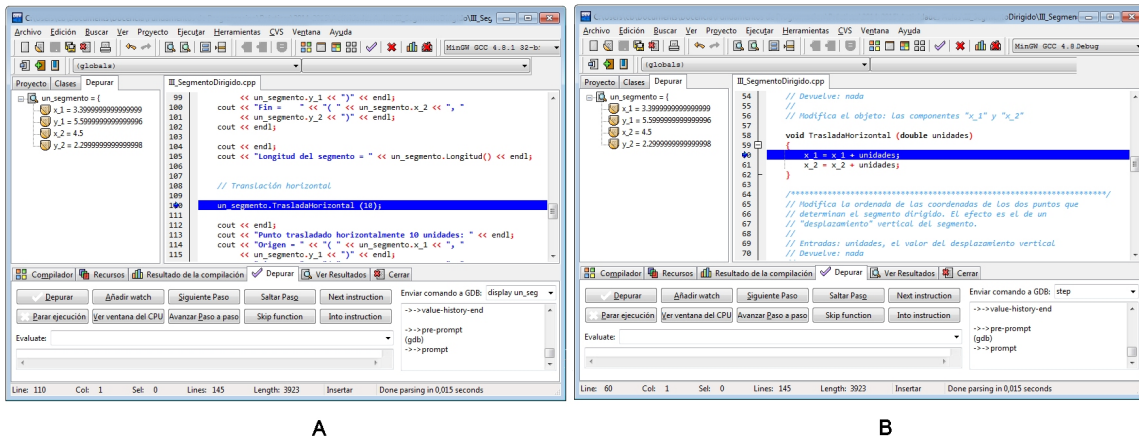


Figura 17: Depuración de métodos de una clase

Si seguimos avanzando paso a paso, al finalizar la ejecución del método y volver a la función `main` comprobaremos que el objeto `un_segmento` se actualiza y se muestran los valores correctos (ver figura 18).

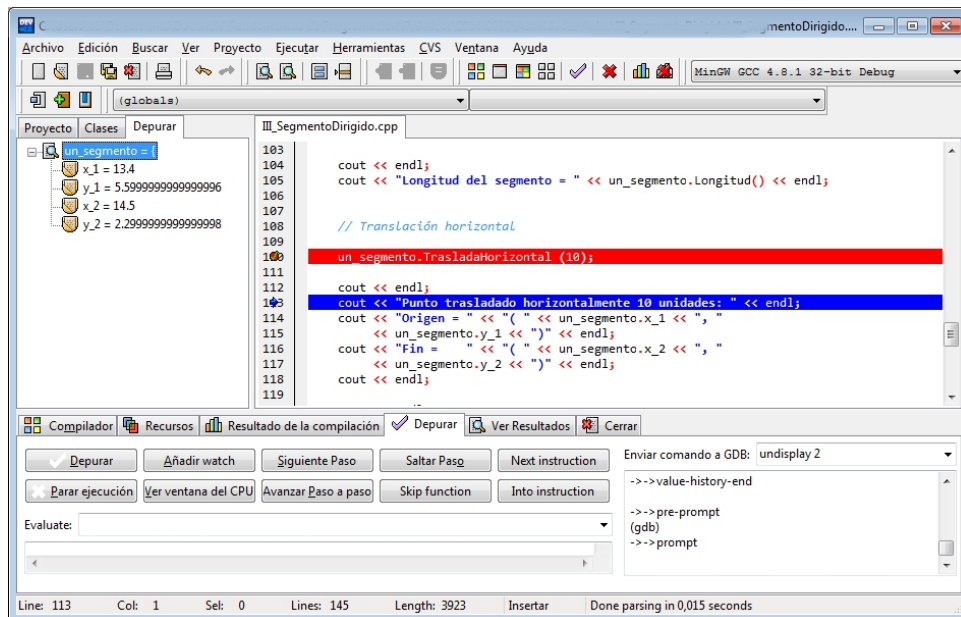


Figura 18: Depuración de métodos de una clase (2)

Sesión 12

Clases (II): Constantes - Objetos con arrays - Secuencias

► **Actividades a realizar en casa**

Actividad: Lectura de soluciones de ejercicios resueltos.

Lea la solución de los siguientes ejercicios, que podrá encontrar en PRADO (carpeta Archivos Sesión 12 - previos, sección PRÁCTICA 12).

- **Ejercicios sobre *constantes***

- 23 (Clase Bicicleta) V_Bicicleta.cpp

- **Ejercicios sobre *objetos con arrays/matrices***

- 28 (Clase Fibonacci) V_Fibonacci_Clase.cpp

- **Ejercicios sobre *secuencias***

- 35 (Constructores adicionales) V_Constructores_Secuencia.cpp

- 36 (Palíndromo) V_Palindromo.cpp

- 37 (Invertir una secuencia de caracteres) V_Invierte.cpp

- 38 VIDEO1 y VIDEO2 (Elimina todas las mayúsculas)

- V_EliminaMayusculas.cpp

- 39 (Eliminar mayúsculas eficientemente) V_EliminaMayusculas_Eficiente.cpp

Actividad: Resolución de problemas.

- **Ejercicios obligatorios sobre *constantes***

- 27 (Inflación en varios países) V_Inflacion_Matriz.cpp

- **Ejercicios obligatorios sobre *objetos con arrays/matrices***

- 25 (Clase GestorNumerosSecuenciabiles con cálculo de las secuencias)

- V_NumerosSecuenciabiles_Clase_ConSecuencia.cpp

- 32 (Listado y cálculos sobre empleados, con clase) V_Empleados.cpp

- *Ejercicios obligatorios sobre **secuencias***

- 40 (Eliminar todas las ocurrencias de un carácter) V_EliminaOcurrencias.cpp
- 43 (Eliminar exceso de blancos) V_EliminaExcesoBlancos.cpp
- 48 (Ordenación de una secuencia, *nuevo orden*)
V_OrdenacionSecuencia_NuevoOrden.cpp

- *Ejercicios opcionales sobre **constantes***

- 22 (Clase Gaussiana, con constante static)
V_Gaussiana_Clase_variante.cpp

- *Ejercicios opcionales sobre **objetos con arrays/matrices***

- 20 (Clase Menu) V_Clase_Menu.cpp
- 24 (Clase ContadorMayusculas)
V_CuentaMayusculas_ClaseContadorMayusculas.cpp
- 26 (Clase Eratostenes) V_Eratostenes_Clase.cpp
- 29 (Representar funciones) V_Gaussiana_Clase_enTablero.cpp
- 30 (Clase RedSocial) V_RedSocial_Clase.cpp

- *Ejercicios opcionales sobre **secuencias***

- 41 (Eliminar todas las repeticiones) V_EliminaRepetidos.cpp
- 42 (Eliminar desde/hasta una posición dada) V_Elimina_Desde_Hasta.cpp
- 47 (Ordenación) V_OrdenacionSecuencia.cpp
- 49 (Búsqueda por interpolación) V_BusquedaInterpolacion.cpp

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas, **comentadas** y **modularizadas**.

► **Actividades a realizar en las aulas de ordenadores**

En esta sesión se discutirá sobre las soluciones propuestas a los ejercicios de esta sesión.

► **Normas detalladas**

Deberá entregar únicamente el fichero FP_sesion12.zip (**importante:** no .rar ni otro formato, sólo .zip), resultado de empaquetar todos los ficheros fuente (.cpp), uno por cada ejercicio resuelto.

Los nombres de los ficheros fuente deben ser los indicados (página 61).

La entrega se realizará en PRADO, en el periodo habilitado para ello.

Sesión 13

Clases (III): Métodos y objetos - Objetos en objetos

En esta sesión se trabajará sobre ejercicios de la *Relación de Problemas VI*. Concretamente, con ejercicios que trabajan sobre sobre: 1) métodos y objetos (página [RP-VI.1](#)) y 2) objetos como datos miembro de otros objetos (página [RP-VI.14](#)).

► **Actividades a realizar en casa**

Actividad: Lectura de soluciones de ejercicios resueltos.

Lea la solución de los siguientes ejercicios, que podrá encontrar en PRADO (carpeta Archivos Sesión 13 - previos, sección PRÁCTICA 13) y visualice los videos indicados.

- **Ejercicios sobre *métodos y objetos***

- 1 (Clase Fraccion) Consultar `VI_Fraccion.cpp`
- 2 (Clase Permutacion) Consultar `VI_Permutacion.cpp`
- 3 (Clase GeneradorPermutaciones) Ver `VI_GeneradorPermutaciones.cpp`
- 10 [VIDEO1](#) y [VIDEO2](#) (Ordenación de una secuencia por *Counting sort*)
`VI_CountingSort.cpp`

- **Ejercicios sobre *Objetos como datos miembro de otros objetos***

- 32 (Clases Punto2D y PoligonoRegular) `VI_PoligonoRegular.cpp`
- 33 (Clases CoordenadasGPS y SecuenciaCoordenadasGPS) `VI_GPS.cpp`
- 34 (Clase Permutacion con una SecuenciaEnteros como dato miembro)
`VI_GeneradorPermutaciones_SecuenciaEnteros.cpp`

Actividad: Resolución de problemas.

- **Ejercicios obligatorios sobre *métodos y objetos***

- 6 (Clase Bin) `VI_Binario_Clase.cpp`
- 20 (Mayor palíndromo dentro de una secuencia) `VI_MayorPalindromo.cpp`

- **Obligatorios sobre *Objetos como datos miembro de otros objetos***

- 24 (Ordenación de un array de datos Empleado, con clases Empleado y Fecha)
`VI_Orden_ArrayEmpleados_ClaseFecha.cpp`

- 25 (Ordenación de un dato SecuenciaEmpleados, con clases Empleado y Fecha)
VI_Orden_SecuenciaEmpleados.cpp
 - 28 (Clases Punto2D, Circunferencia y Rectangulo)
VI_Rectangulo_y_Circunferencias.cpp
 - 31 (Clases Punto2D y ColeccionPuntos2D) VI_ColeccionPuntos.cpp
 - 41 (Array de circunferencias, con clases)
VI_Circunferencias_en_Rectangulo_Clases.cpp
-

- *Ejercicios opcionales sobre **métodos y objetos** (Relación de Problemas VII)*

- 4 (Clase Instante (ampliación)) VI_Instante_Clase.cpp
- 5 (Clase Dinero (ampliación)) VI_Dinero_Clase.cpp
- 7, 8 y 9 (Inserta secuencia) VI_InsertaSecuencia_Todos.cpp
- 11 (Clase ConjuntoOrdenado) VI_ConjuntoOrdenado_Clase.cpp
- 21 (Listado y cálculos sobre empleados, con clase) VI_Empleados.cpp

- *Opcionales sobre **Objetos como datos miembro de otros objetos***

- 23 (Listado y cálculos sobre empleados, con clases Empleado y Fecha)
VI_Empleados_ClaseFecha.cpp
- 26 (Clases Punto2D y Circunferencia) VI_Circunferencia.cpp
- 27 (Clases Punto2D y Rectangulo) VI_Rectangulo.cpp
- 38 (Clase RedSocial con clase Usuario) VI_RedSocial_ClaseUsuario.cpp

► **Actividades a realizar en las aulas de ordenadores**

En esta sesión se discutirá sobre las soluciones propuestas a los ejercicios de esta sesión.

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas, **comentadas** y **modularizadas**.

► **Normas detalladas**

Deberá entregar únicamente el fichero FP_sesion13.zip (**importante:** no .rar ni otro formato, sólo .zip), resultado de empaquetar todos los ficheros fuente (.cpp), uno por cada ejercicio resuelto.

Los nombres de los ficheros fuente deben ser los indicados (página 63).

La entrega se realizará en PRADO, en el periodo habilitado para ello.

Sesión 14

Clases (IV): Tablas

En esta sesión continuamos trabajando sobre ejercicios de la *Relación de Problemas VI*. Concretamente, con ejercicios sobre *tablas* para conocer y ampliar las clases básicas *TablaRectangular* y *TablaDentada* y sus representaciones usando una *matriz clásica* (página [RP-VI.26](#)) y una *secuencia de secuencias* (página [RP-VI.41](#)).

► **Actividades a realizar en casa**

Actividad: Lectura de soluciones de ejercicios resueltos.

Lea la solución de los siguientes ejercicios, que podrá encontrar en PRADO (carpeta Archivos Sesión 14 - previos, sección PRÁCTICA 14).

Ejemplos de **tabla rectangular** que usa como estructura de datos una **matriz clásica**:

47 (Posiciones de suma lateral coincidente en tabla rectangular, matriz clásica)

VI_Posiciones_SumaLateral_TREnteros_Matriz.cpp

48 (*MaxiMin* en tabla rectangular, matriz clásica)

VI_MaxiMin_TREnteros_Matriz.cpp

Ejercicio de **tabla dentada** que usa como estructura de datos una **matriz clásica**:

53 (Insertar fila en tabla dentada, matriz clásica)

VI_Insertar_TDCaracteres_Matriz.cpp

Ejercicio de **tabla rectangular** que usa una **secuencia de secuencias**:

59 (*MaxiMin* en tabla rectangular, secuencia de secuencias)

VI_MaxiMin_TREnteros_SecSec.cpp

Ejemplo de **tabla dentada** que usa una **secuencia de secuencias**:

61 (Insertar fila en tabla dentada, secuencia de secuencias)

VI_Insertar_TDCaracteres_SecSec.cpp

Actividad: Resolución de problemas.

- *Obligatorios*

- 51 (Fila más cercana en tabla rectangular, matriz clásica)
VI_FilaMasCercana_TREnteros_Matriz.cpp
- 57 (Gestión de empleados de varios departamentos en tabla dentada, matriz clásica)
VI_Empleados_VariosDepartamentos_TDEmpleados_Matriz.cpp
- 60 (Fila más cercana en tabla rectangular, secuencia de secuencias)
VI_FilaMasCercana_TREnteros_SecSec.cpp
- 68 (Gestión de empleados de varios departamentos en tabla dentada, secuencia de secuencias)
VI_Empleados_VariosDepartamentos_TDEmpleados_SecSec.cpp

- *Opcionales*

- 50 (Clase MapaDistancias para el problema de la red de ciudades)
VI_Red_Ciudades.cpp
- 54 (Sumar por filas en tabla dentada, matriz clásica)
VI_SumaFilas_TDReales_Matriz.cpp
- 55 (Mayor tabla rectangular en tabla dentada, matriz clásica)
VI_TDReales_a_TRReales_Matriz.cpp
- 65 (Sumar por filas en tabla dentada, secuencia de secuencias)
VI_SumaFilas_TDReales_SecSec.cpp
- 66 (Mayor tabla rectangular en tabla dentada, secuencia de secuencias)
VI_TDReales_a_TRReales_SecSec.cpp

- *Opcionales (clases Texto, Frase y Palabra)*

- 62 (Clases Texto, Frase y Palabra: secuencias y secuencia de secuencias)
VI_Texto_Frase_Palabra.cpp
- 63 (Clase Texto, método Comprime)
VI_Texto_Frase_Palabra_MetodoComprime.cpp
- 64 (Clase Texto, métodos Reemplazar y ReemplazarTodo)
VI_Texto_Frase_Palabra_MetodosReemplazar.cpp

► **Recomendaciones**

Todas las soluciones deben estar correctamente escritas, **comentadas** y **modularizadas**.

► **Actividades a realizar en las aulas de ordenadores**

En esta sesión se discutirá sobre las soluciones propuestas a los ejercicios de esta sesión.

► **Normas detalladas**

Deberá entregar únicamente el fichero FP_sesion14.zip (**importante:** no .rar ni otro formato, sólo .zip), resultado de empaquetar todos los ficheros fuente (.cpp), uno por cada ejercicio resuelto.

Los nombres de los ficheros fuente deben ser los indicados (página 66).

La entrega se realizará en PRADO, en el periodo habilitado para ello.



Doble Grado en Ingeniería Informática y Matemáticas

cb@decsai.ugr.es

Departamento de Ciencias de la Computación e Inteligencia Artificial

Universidad de Granada

RELACIÓN DE PROBLEMAS I. Introducción a C++

1. Indique cuál sería el resultado de las siguientes operaciones:

```
int salario_bas
int salario_final;
int incremento;

salario_base = 1000;
salario_final = salario_base;

incremento = 200;
salario_final = salario_final + incremento;

salario_base = 3500;

cout << "\nSalario base: " << salario_base;

cout << "\nSalario final: " << salario_final;
```

Responda razonadamente a la siguiente pregunta: ¿El hecho de realizar la asignación `salario_final = salario_base`; hace que ambas variables estén ligadas durante todo el programa y que cualquier modificación posterior de `salario_base` afecte a `salario_final`?

Finalidad: Ejemplo básico de asignación a una variable del resultado de una expresión.. Dificultad Baja.

2. Crear un programa que pida un valor de intensidad y resistencia e imprima el voltaje correspondiente, según la *Ley de Ohm*:

$$\text{voltaje} = \text{intensidad} * \text{resistencia}$$

Finalidad: Asignar a una variable el resultado de una expresión. Dificultad Baja.

3. Cread un programa que nos pida la longitud del radio, calcule el área del círculo y la longitud de la circunferencia correspondientes, y nos muestre los resultados en pantalla. Recordad que:

$$\text{longitud circunferencia} = 2\pi r \quad \text{área circulo} = \pi r^2$$

Usad el literal 3.1416 a lo largo del código, cuando se necesite multiplicar por π .

Una vez hecho el programa, cambiad las apariciones de 3.1416 por 3.1415927, recompilad y ejecutad.

Finalidad: Conciernarnos del problema de los literales repetidos a lo largo del código. Dificultad Baja.

4. Escriba un programa que muestre en pantalla el resultado de las siguientes expresiones numéricas que constituyen una aproximación al valor de π . La primera es del año 1800 antes de Cristo, la segunda (también de la misma era) es una aproximación introducida por los matemáticos mesopotámicos y la tercera es del siglo II (introducida por Claudio Ptolomeo)

$$\pi \approx \frac{256}{81} \quad \pi \approx 3 + \frac{1}{8} \quad \pi \approx \frac{377}{120}$$

El resultado debe ser 3.16049, 3.125 y 3.14167

Finalidad: Mezclar tipos enteros y reales. Dificultad Baja.

5. El número π es un número irracional y por tanto tiene infinitas cifras decimales.

Hay diversos métodos para obtener este valor, algunos mejores que otros. Una forma simple de obtenerlo es a través de la función arco-seno, ya que $\pi/6$ es el ángulo (en radianes) cuyo seno vale de 0.5. Matemáticamente se expresa como

$$\frac{\pi}{6} = \text{arco-seno}(0,5)$$

Construir un programa que imprima el valor de π calculado a partir de la anterior fórmula.

El cálculo del arco-seno se realiza en C++ con la función `asin` de la biblioteca `cmath`.

Dificultad Baja.

6. Queremos transformar una cantidad g dada en grados a radianes. Para ello, basta recordar que π radianes equivale a 180° por lo que

$$r = g \frac{\pi}{180}$$

Construya un programa que lea un número real (grados) y muestre los radianes correspondientes.

Para el valor de π puede usar un literal o bien las expresiones de los ejercicios 4 ó 5.

Dificultad Baja.

7. Una compañía aérea establece el precio del billete como sigue: en primer lugar se fija una tarifa base de 150 euros, la misma para todos los destinos. A continuación se suman 10 céntimos por cada kilómetro de distancia al destino.

Cree un programa que lea el número de kilómetros al destino y calcule el precio final del billete.

Dificultad Baja.

8. La compañía aérea del ejercicio 7 quiere aplicar una política de descuentos al precio del billete. Amplíe el programa anterior para después de imprimir el precio del billete pida un porcentaje de descuento (dato `double`) y muestre el precio final después de aplicar el descuento indicado.

Dificultad Baja.

9. Un radar de tramo está organizado en tres sectores: 1) el primero, de 5.3 Km, tiene la velocidad limitada a 80 Km/h, 2) el segundo, de 8.1 Km, tiene la velocidad limitada a 100 Km/h, y 3) el tercero, de 6.4 Km, tiene la velocidad limitada a 80 Km/h. El tramo tiene dos pórticos de control, al inicio y al final.

Escribir un programa que calcule y muestre el *tiempo umbral* de sanción, o sea, el tiempo mínimo que puede emplear un vehículo en recorrer el tramo completo sin ser sancionado. Muestre el resultado en segundos y en minutos y segundos.

Dificultad Baja.

10. Un banco presenta la siguiente oferta. Si se deposita una cantidad de euros `capital` durante un año a plazo fijo, se dará un interés dado por la variable `interes`. Realizad un programa que lea una cantidad `capital` y un interés `interes` desde teclado y calcule en una variable `total` el dinero que se tendrá al cabo de un año, aplicando la fórmula:

$$\text{total} = \text{capital} + \text{capital} * \frac{\text{interes}}{100}$$

Es importante destacar que el compilador primero evaluará la expresión de la parte derecha de la anterior asignación (usando el valor que tuviese la variable `capital`) y a continuación ejecutará la asignación, escribiendo el valor resultante de la expresión dentro de la variable `total`).

A continuación, el programa debe imprimir en pantalla el valor de la variable `total`. Tanto el `capital` como el `interes` serán valores reales. Supondremos que el usuario introduce el interés como un valor real entre 0 y 100, es decir, un interés del 5,4 % se introducirá como 5.4. También supondremos que lo introduce correctamente, es decir, que sólo introducirá valores entre 0 y 100.

Supongamos que queremos modificar la variable original `capital` con el nuevo valor de `total`. ¿Es posible hacerlo directamente en la expresión de arriba?

Finalidad: Resolver un problema real sencillo, usando varias sentencias. Dificultad Baja.

11. Construya un programa para leer el valor de una variable `salario_base` de tipo `double`, la incremente un 2 % e imprima el resultado en pantalla. Para realizar este cálculo, multiplique por 1.02 el valor original. Tiene varias alternativas:

a) Calcular `1.02 * salario_base` dentro de la sentencia `cout`

- b) Introducir una variable `salario_final`, asignarle la expresión anterior y mostrar su contenido en la sentencia `cout`
- c) Modificar la variable original `salario_base` con el resultado de incrementarla un 2%.

Indique qué alternativa elige y justifíquela.

Finalidad: Ejemplo básico de asignación a una variable del resultado de una expresión. Dificultad Baja.

12. Escribir un programa para una caja de un supermercado que lea el total a pagar y la cantidad pagada (se supone que cantidad pagada $\geq$ total a pagar) e indique el número de monedas de 2 euros, 1 euro, 50 céntimos, 20 céntimos, 10 céntimos, 5 céntimos, 2 céntimos y 1 céntimo que se deben dar de cambio. **Se indicará el menor número de monedas de cada clase.**

13. Modificar el programa que resuelve el ejercicio 3 sustituyendo los literales que hacen referencia al valor de π por una *constante* llamada PI.

Modifique su valor, recompile y ejecute. ¿Es esta solución mejor que la del ejercicio 3? ¿por qué?

Finalidad: Entender la importancia de las constantes. Dificultad Baja.

14. Escriba un programa que lea una distancia en millas (como un real) y muestre la cantidad equivalente en kilómetros. A continuación leerá una distancia en kilómetros (como un real) y mostrará la cantidad equivalente en millas.

Debe tener en cuenta que 1 milla equivale a 1.609 kilómetros.

Finalidad: Asignar a una variable el resultado de una expresión. Dificultad Baja.

15. Realizar un programa que nos pida una longitud cualquiera dada en metros. El programa deberá calcular el equivalente de dicha longitud en pulgadas, pies, yardas y millas, y mostrarnos los resultados en pantalla. Para el cálculo, utilice la siguiente tabla de conversión del sistema métrico:

1 pulgada= 25,4 milímetros	1 yarda = 0,9144 metros
1 pie = 30,48 centímetros	1 milla = 1,609344 kilómetros

Finalidad: Plantear la solución con una doble conversión. Dificultad Baja.

16. Escriba un programa que calcule el consumo de gasolina. Pedirá la distancia recorrida (en kms), los litros de gasolina consumidos y los litros que quedan en el depósito. El programa debe informar el consumo en *km/litro*, los *litros/100 km* y cuantos kilómetros de autonomía le restan con ese nivel de consumo.

Finalidad: Resolver un problema real sencillo, usando varias sentencias. Dificultad Baja.

17. Queremos construir un programa que simule un juego inspirado en el de los triles (del que procede el nombre de *trilero*). Suponemos que hay dos participantes y cada uno tiene una caja etiquetada con su nombre. Dentro de cada caja hay una cantidad de dinero y el objetivo es intercambiar las cantidades que hay dentro. Por ahora, sólo se pide construir un programa que haga lo siguiente:

- Debe leer desde teclado dos variables `caja_izda` y `caja_dcha`.
- A continuación debe intercambiar sus valores y finalmente, mostrarlos.

Observe que se desea intercambiar el contenido de las variables, de forma que `caja_izda` pasa a contener lo que tenía `caja_dcha` y viceversa. El siguiente código no es válido ya que simplemente engaña al usuario pero las cajas no se quedan modificadas:

```
cout << "La caja izquierda vale " << caja_dcha << "\n";  
cout << "La caja derecha vale " << caja_izda;
```

Estaríamos tentados a escribir el siguiente código:

```
caja_izda = caja_dcha;  
caja_dcha = caja_izda;
```

pero no funciona correctamente ¿Por qué?

Proponga una solución e impleméntela.

Finalidad: Entender cómo funciona la asignación entre variables. Dificultad Baja.

18. Leed desde teclado tres variables correspondientes a un número de horas, minutos y segundos, respectivamente. Diseñar un algoritmo que calcule las horas, minutos y segundos dentro de su rango correspondiente. Por ejemplo, dadas 10 horas, 119 minutos y 280 segundos, debería dar como resultado 12 horas, 3 minutos y 40 segundos. En el caso de que nos salgan más de 24 horas, daremos también los días correspondientes (pero ya no pasamos a ver los meses, años, etc)

Como consejo, utilizad los operadores `/` (representa la división entera cuando los dos argumentos son enteros) y `%` (representa el resto de la división entera).

Finalidad: Usar expresiones y variables para no repetir cálculos. Dificultad Media.

19. Calcular el número de segundos que hay entre dos instantes del mismo día. Cada instante se caracteriza por la hora (entre 0 y 23), minuto (entre 0 y 59) y segundo (entre 0 y 59).

El programa leerá la hora, minuto y segundo del instante inicial, y la hora, minuto y segundo del instante final (supondremos que los valores introducidos son correctos) y mostrará el número de segundos entre ambos instantes.

Finalidad: Trabajar con expresiones numéricas y algoritmos. Dificultad Media.

20. En atletismo se expresa la rapidez de un atleta en términos de ritmo (*minutos y segundos por kilómetro*) más que en unidades de velocidad (*kilómetros por hora*).

Escribid dos programas para convertir entre estas dos medidas:

- El primero leerá el ritmo (minutos y segundos, por separado) y mostrará la velocidad (kilómetros por hora).
- El segundo leerá la velocidad (kilómetros por hora) y mostrará el ritmo (minutos y segundos).

Finalidad: Trabajar con expresiones numéricas y variables de diferentes tipos. Dificultad Baja.

21. Un intervalo es un espacio métrico comprendido entre dos valores o cotas, a y b , siendo a la cota inferior y b la cota superior. Cada extremo de un intervalo puede ser abierto o cerrado. En este problema no se consideran intervalos con extremos infinitos.

Construya un programa que lea e imprima los datos de un intervalo. Para ello, el programa debe leer los datos en el siguiente orden:

- Un carácter que represente el tipo de intervalo por la izquierda: el usuario deberá introducir el carácter `(` si es abierto y `[` si es cerrado.
- Un número real con la cota izquierda.
- El carácter `,` como separador de las dos cotas
- Un número real con la cota derecha.
- Un carácter que represente el tipo de intervalo por la derecha: el usuario deberá introducir el carácter `)` si es abierto y `]` si es cerrado.

El programa mostrará en pantalla el mismo intervalo introducido.

Finalidad: Manejar entrada de datos de distinto tipo. Dificultad Baja.

22. Queremos construir una expresión numérica que desplace un entero un número dado de posiciones, pero lo mantenga dentro de un intervalo.

Por ejemplo, si el intervalo fijado es $[65, 90]$, el desplazamiento es de 3 unidades y el entero a desplazar es el 70, el resultado sería 73. Si el entero fuese el 89 y el desplazamiento 3, el resultado sería 92. Al no estar el 92 dentro del intervalo, se realiza un *ciclo* de forma que el 91 se transformaría en el 65 y el 92 en el 66.

Se pide construir un programa que lea dos enteros *minimo* y *maximo* que determinarán el intervalo $[\text{minimo}, \text{maximo}]$ (supondremos que el usuario introduce como *maximo* un valor mayor o igual que *minimo*). A continuación el programa leerá un valor entero *desplazamiento* (supondremos que el usuario introduce un valor entre 0 y $\text{maximo} - \text{minimo}$). Finalmente, el programa leerá un entero *a_desplazar*

(supondremos que el usuario introduce un número entre `minimo` y `maximo`). El programa sumará al valor leído el desplazamiento y lo convertirá en un entero dentro del intervalo `[minimo, maximo]` tal y como se ha descrito anteriormente.

Finalidad: Trabajar con los operadores enteros. Dificultad Media.

23. Escribir un programa para calcular la media aritmética muestral y la desviación estándar (o típica) muestral de las alturas de **tres** personas. Éstos valores se leerán con el teclado y serán números reales (de tipo `double`). La fórmula general para un valor arbitrario de n es:

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{X})^2}$$

$\bar{X}$ representa la media aritmética y σ la desviación estándar.

Es necesario usar la función `sqrt` (raíz cuadrada) disponible en `cmath`.

Ejecutad el programa con varios valores y comprobad que el resultado es correcto utilizando una calculadora científica o cualquier calculadora online como por ejemplo la disponible en <http://www.disfrutalasmaticas.com/datos/desviacion-estandar-calculadora.html>

Finalidad: Trabajar con expresiones numéricas y con variables para no repetir cálculos. Dificultad Baja.

24. La función gaussiana es muy importante en Estadística. Es una función real de variable real en la que el parámetro μ se conoce como *esperanza* o *media* y σ como *desviación típica* (*mean* y *standard deviation* en inglés, respectivamente). Su definición viene dada por la expresión:

$$\text{gaussiana}(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{\left\{-\frac{1}{2} \left(\frac{x - \mu}{\sigma}\right)^2\right\}}$$

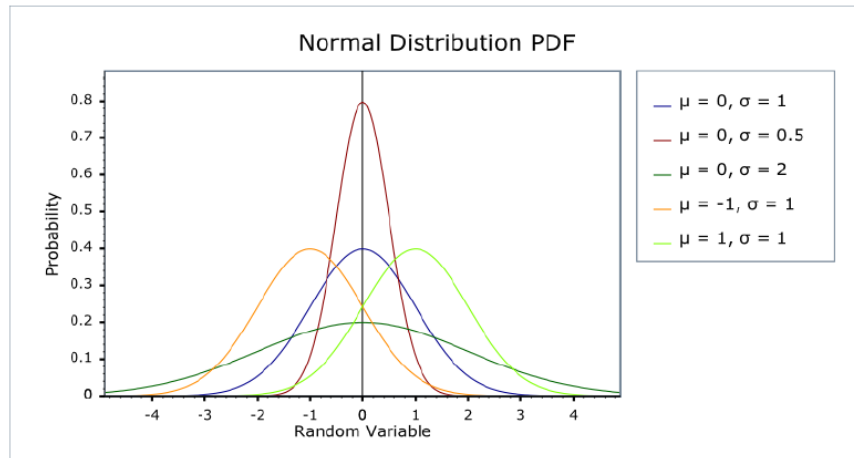
Realizar un programa que lea los coeficientes reales μ y σ de una función gaussiana. A continuación el programa leerá un valor de abscisa x y se imprimirá el valor que toma la función en x .

Para realizar las operaciones indicadas, debe utilizar las siguientes funciones (disponibles en `cmath`):

- Para elevar el número e a un valor cualquiera, use la función `exp`. Por ejemplo, para calcular e^8 debería usar la expresión `exp(8)`
- Para calcular la raíz cuadrada, use `sqrt`. Por ejemplo, para calcular $\sqrt{8}$ debería usar la expresión `sqrt(8)`

- Para elevar un número a otro, utilice la función `pow` en la siguiente forma: `pow(base, exponente)`. Por ejemplo, para calcular 2^{10} debería usar la expresión `pow(2, 10)`

En la gráfica siguiente pueden verse algunos ejemplos de esta función con distintos parámetros.



Comprobad que los resultados son correctos, usando:

<https://www.easycalculation.com/statistics/normal-pdf.php>

Finalidad: Trabajar con expresiones numéricas más complejas. Dificultad Media.

25. Diseñar un programa que lea un carácter (supondremos que el usuario introduce una minúscula), lo pase a mayúscula y lo imprima en pantalla. Hacedlo sin usar las funciones `toupper` ni `tolower` de la biblioteca `cctype`. Para ello, debe considerarse la equivalencia en C++ entre los tipos enteros y caracteres.

Finalidad: Entender la equivalencia entre tipos enteros y de carácter. Dificultad Baja.

26. Escribir un programa que lea un valor entero en un dato de tipo `string`. A continuación lo convierte y asigna a un dato `int` (supondremos que el usuario introduce **siempre** un entero de **tres dígitos**, como por ejemplo 351).

Escribid en pantalla los dígitos separados por dos espacios en blanco. Con el valor anterior la salida sería:

3 5 1

Lo que se muestra en pantalla es el valor de otro dato `string` que se forma concatenando tres parejas de valores compuestas de dos espacios en blanco y un dígito.

Nota: Más adelante aprenderemos a comprobar si el número introducido tiene tres dígitos, y cómo actuar (repitiendo la lectura) en el caso de que no los tenga.

Dificultad Media.

27. Supongamos el siguiente código:

```
int entero;
char character;

cin >> character;
entero = character;
```

Supongamos que ejecutamos el código e introducimos el 7 desde teclado. El programa está leyendo una variable de tipo `char`. Por lo tanto, el 7 se interpreta como un carácter y es como si hiciésemos la siguiente asignación:

```
character = '7';
entero = character;
```

La variable `character` almacena internamente el valor 55 (número ASCII del carácter 7). La variable `entero` almacenará, también, el valor 55

Queremos construir un programa para asignarle a la variable `entero` el *número* 7, asociado al dígito representado en la variable `character`, es decir, el valor 7 (no el 55) ¿Cómo lo haría? El programa debe mostrar el resultado.

Nota. La comilla simple para representar un literal de carácter es la que hay en el teclado del ordenador debajo de la interrogación ?.

Finalidad: Entender la equivalencia entre tipos enteros y de carácter. Dificultad Baja.

28. Para intercambiar mensajes de forma privada, se utilizan distintos algoritmos que codifican/descodifican una cadena de caracteres. Uno de los más sencillos y que fue utilizado por Julio César durante la época del Imperio Romano es el de *rotación*. Consiste en seleccionar una clave (un número entero), y desplazar las letras del alfabeto tantas posiciones como indica la clave. Trabajaremos únicamente con mayúsculas.

Se considera una representación circular del alfabeto, de tal forma que el carácter que sigue a 'Z' es 'A'. Por ejemplo, si `clave=4`, entonces la 'A' se reemplaza por la 'E' y la 'Z' por la 'D'. Utilizando `clave=0` la secuencia cifrada es igual a la original.

Construya un programa que lea un entero representando la clave y un carácter (supondremos que se introduce correctamente una letra mayúscula del alfabeto inglés). El programa codificará el carácter según la clave introducida y lo mostrará por pantalla.

Recomendamos que revise la solución del ejercicio 22 de esta misma Relación de Problemas.

Finalidad: Expresiones con caracteres y enteros. Dificultad Media.

29. Dadas las variables `count = 0`, `limit = 10`, `x = 2` e `y = 7`, calcule el valor de las siguientes expresiones lógicas:

```
count == 0 && limit < 20
limit > 20 || count < 5
!(count == 12)
count == 1 && x < y
!( (count < 10 || x < y) && count >= 0 )
(count > 5 && y == 7) || (count <= 0 && limit == 5*x)
!( limit != 10 && x > y )
```

30. Razonar sobre la falsedad o no de las siguientes afirmaciones:

- a) 'c' es una expresión de caracteres.
- b) $4 < 3$ es una expresión numérica.
- c) $(4+3) < 5$ es una expresión numérica.
- d) `cout << a;` da como salida la escritura en pantalla de una a.

Finalidad: Distinguir entre expresiones de distinto tipo de dato. Dificultad Baja.

31. Declarar las variables necesarias y traducir las siguientes fórmulas a expresiones válidas del lenguaje C++.

- a) $\frac{1 + \frac{x^2}{y}}{\frac{x^3}{1+y}}$
- b) $\frac{1 + \frac{1}{3} \sin h - \frac{1}{7} \cos h}{2h}$
- c) $\sqrt{1 + \left(\frac{e^x}{x^2}\right)^2}$

Algunas funciones de cmath $\text{sen}(x) \rightarrow \sin(x)$ $\text{cos}(x) \rightarrow \cos(x)$ $x^y \rightarrow \text{pow}(x, y)$ $\ln(x) \rightarrow \log(x)$ $e^x \rightarrow \exp(x)$
--

Dificultad Baja.

32. Escribid una expresión lógica que sea verdadera si una variable de tipo carácter llamada `letra` es una letra *mayúscula* y falso en otro caso.

Escribid una expresión lógica que sea verdadera si una variable de tipo entero llamada `edad` es mayor o igual que 18 y menor que 67.

Escribid una expresión lógica que nos informe cuando un año es bisiesto. Los años bisiestos son aquellos que o bien son divisibles por 4 pero no por 100, o bien son divisibles por 400.

Escribid una expresión lógica que nos informe si el valor de una variable `double` llamada `distancia` es menor que la constante `LIMITE`.

Usad una variable lógica que registre si el valor de una variable `int` es menor que otra, otra que informe si son iguales y otra que informe si es mayor. Asegúrese que sólo una, y sólo una de las tres tendrá el valor `true`.

Nota: Cuando se imprime por pantalla (con `cout`) una expresión lógica que es `true`, se imprime 1. Si es `false`, se imprime un 0.

Finalidad: Usar expresiones lógicas, muy usadas en el tema 2. *Dificultad Baja.*

33. El precio final de un automóvil para un comprador es la suma total del costo del vehículo, del porcentaje de ganancia de dicho vendedor y del I.V.A. Diseñar un algoritmo para obtener el precio final de un automóvil sabiendo que el porcentaje de ganancia de este vendedor es del 20 % y el I.V.A. aplicable es del 16 %.

Dificultad Baja.

34. Cread un programa que lea un valor de temperatura expresada en grados Celsius y la transforme en grados Fahrenheit. Para ello, debe considerar la fórmula siguiente:

$$\text{Grados Fahrenheit} = (\text{Grados Celsius} * 180 / 100) + 32$$

Buscad en Internet el por qué de dicha fórmula.

Dificultad Baja.

35. Los compiladores utilizan siempre el mismo número de bits para representar un tipo de dato entero (este número puede variar de un compilador a otro). Por ejemplo, 32 bits para un `int`. Pero, realmente, no se necesitan 32 bits para representar el 6, por ejemplo, ya que bastarían 3 bits:

$$6 = 1 * 2^2 + 1 * 2^1 + 0 * 2^0 \equiv 110$$

Se pide crear un programa que lea un entero positivo n , y calcule el mínimo número de dígitos que se necesitan para su representación. Para simplificar los cálculos, suponed que sólo queremos representar valores enteros positivos (incluido el cero). Consejo: se necesitará usar el logaritmo en base 2 y obtener la parte entera de un real (se obtiene tras el truncamiento que se produce al asignar un real a un entero)

Dificultad Media.

36. Se quiere construir un programa que lea un número real r y un número entero n y trunque r a tantas cifras decimales como indique n . El resultado debe guardarse en una variable diferente. La única función que puede usar de `cmath` es `pow`.

Por ejemplo, si r vale 1.2349 y n vale 2 el resultado será 1.23, si r vale 1.237 y n vale 2 el resultado será 1.23, si r vale 1.237 y n vale 1 el resultado será 1.2 y si r vale 1.237 y n vale 0 el resultado será 1

37. Cread un programa que lea las coordenadas de dos puntos $P_1 = (x_1, y_1)$ y $P_2 = (x_2, y_2)$ y calcule la distancia euclídea entre ellos:

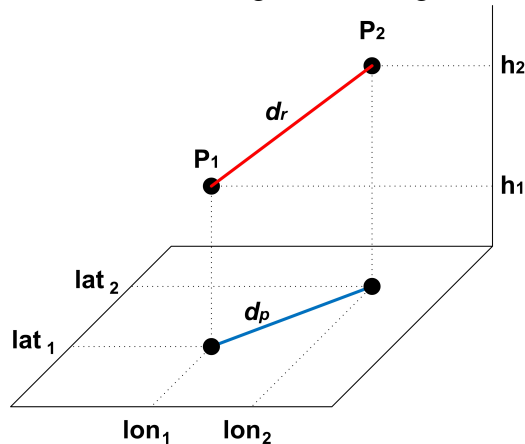
$$d(P_1, P_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Dificultad Baja.

38. El sistema de posicionamiento global, más conocido por sus siglas en inglés, **GPS** (**G**lobal **P**ositioning **S**ystem), es un sistema que permite determinar en toda la Tierra la posición de un objeto.

Un dispositivo GPS es capaz de captar y registrar la posición en el espacio en base a tres coordenadas: **latitud** y **longitud** (*grados*) y **altura** (*metros*). Los valores de latitud y longitud deben verificar $-90 \leq \text{lat} \leq 90$ y $-180 < \text{lon} \leq 180$.

Construir un programa que lea la latitud y longitud (posición en el plano) de dos puntos y calcule la distancia *sobre plano* entre los dos puntos (distancia que no considera la altura de los puntos). Se trata de la longitud del segmento d_p en la figura.



Use la llamada *fórmula del Haversine*:

- a) Calcular a :

$$a = \sin^2\left(\frac{1}{2}(\text{lat}_2 - \text{lat}_1)\right) + \cos(\text{lat}_1) \cos(\text{lat}_2) \sin^2\left(\frac{1}{2}(\text{lon}_2 - \text{lon}_1)\right)$$

- b) Calcular $c = 2 \arcsin(\min(1, \sqrt{a}))$

- c) La distancia será $d_p = R c$

donde $R = 6372797,560856$ m es la longitud media del radio terrestre.

Tenga en cuenta lo siguiente:

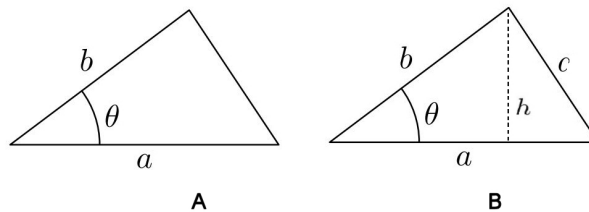
- Los datos de latitud y longitud en las fórmulas vienen expresados en *radianes*. Recuerde que los valores de latitud y longitud que se leen están expresados en grados por lo que habrá de transformarlos a radianes.
- La función arco-seno ($\arcsin$) viene ya implementada en la biblioteca `cmath` con el nombre `asin`
- `min` representa el mínimo de dos valores. Para implementarlo utilice la función `min` que viene definida en la biblioteca `algorithm`.

Una vez calculada la distancia sobre plano, calcular la distancia *real* entre los dos puntos (que considera sus alturas). Se trata de la longitud del segmento d_r de la figura. Observe que puede formar un triángulo rectángulo a partir de los segmentos d_p , d_r y la diferencia de las alturas.

Dificultad Media.

39. El área A de un triángulo se puede calcular a partir del valor de dos de sus lados, a y b , y del ángulo θ que éstos forman entre sí (figura A) con la fórmula $A = \frac{1}{2}ab \sin(\theta)$.

Construid un programa que pida al usuario el valor de los dos lados (en centímetros), el ángulo que éstos forman (en grados), y muestre el valor del área (tenga en cuenta que el argumento de la función `sin` va en radianes).



Dificultad Baja.

Demostración: Nombramos a los lados del triángulo a , b y c , y consideramos que h es la altura sobre el lado a . Consideramos únicamente el ángulo θ delimitado por los lados a y b (figura B). Por la definición de seno, $\sin(\theta) = h/b$, o sea, $h = b \sin(\theta)$.

Aplicando la fórmula del área del triángulo tenemos que $A = \frac{1}{2}a h$ y sustituyendo en la fórmula del área el valor de h tenemos que $A = \frac{1}{2} a b \sin(\theta)$

40. Suponga un caso de elección en el que se decide entre tres opciones: **A**, **B** y **C**.
- El número de votantes totales (censo) está registrado en la constante `VOTANTES`.
 - El número total de de votos registrados es `votos_emitidos`, donde el número total de de votos presenciales es `votos_presenciales` y el número total de de votos por correo es `votos_correo`.
 - Los votos registrados se dividen en válidos (`votos_validos`) y nulos (`votos_nulos`).
 - Entre los votos válidos se distinguen entre los recibidos por correo (`votos_validos_correo`) y presencialmente (`votos_validos_presenciales`).
 - Cada opción recibe un número de votos válidos que se registra en las variables `votos_A`, `votos_B` y `votos_C`, respectivamente.

Escriba expresiones lógicas para reflejar las situaciones que se describen a continuación. Para cada caso, el resultado debe guardarse en una variable lógica (escoja un nombre adecuado).

- a) El número de votos emitidos totales es mayor que el número de votantes.
- b) El número de votos válidos no es igual a la suma de los votos recibidos por las tres opciones.
- c) El número de votos registrados es igual a la suma de los votos válidos y nulos.
- d) Gana la opción **A**.
- e) Gana la opción **B**.
- f) Gana la opción **C**.
- g) Empatán dos opciones.
- h) Hay un *empate técnico* entre las opciones **A** y **B**. Definimos que ocurre un empate técnico cuando la diferencia entre los dos valores es menor que el 5 % de su suma.
- i) La opción **A** obtiene mayoría absoluta.
- j) La coalición **A-C** obtiene mayoría absoluta.
- k) Hay una coalición de dos opciones que obtiene mayoría absoluta.
 - l) La opción **B** obtiene menos de 3 % de los votos válidos registrados.
- m) La participación es mayor que el 75 % del censo de votantes.
- n) Queremos saber si la opción **A** puede gobernar "sólo o en compañía de otros".
- ñ) El número de abstenciones es mayor que el número de votos válidos.
- o) El número de abstenciones es mayor que el número de votos válidos, pero las abstenciones no son mayores que los votos nulos.
- p) El número de votos por correo es mayor que el 20 % de los votos presenciales o el número de abstenciones.
- q) El número de votos nulos por correo es mayor que el número de votos nulos presenciales.
- r) El número de votos nulos por correo es mayor que el número de votos válidos por correo.

41. En este ejercicio se va a procesar el contenido de una cadena de caracteres extrayendo sus componentes. El dato `string` que se va a procesar tiene un formato fijo y predeterminado.

Las cadenas a procesar podrían ser líneas extraídas del índice de un documento. En este caso, en concreto, están formadas por

- un número que indica la **sección** (ocupa las tres primeras casillas y tiene un máximo de tres dígitos),
- un número que indica la **subsección** (ocupa las tres siguientes casillas y tiene un máximo de tres dígitos),
- un número que indica la **subsubsección** (ocupa las tres siguientes casillas y tiene un máximo de tres dígitos),
- una serie de 40 caracteres que contiene el **título** del apartado, y
- un número que indica la **página** (ocupa las cuatro siguientes casillas y tiene un máximo de cuatro dígitos)

Con estas indicaciones las cadenas tendrán una longitud -exacta- de **53** caracteres.

Por ejemplo, en la siguiente cadena (los puntos indican los espacios de la cadena -se escriben en el ejemplo para ayudar a entenderlo- y las barras inicial y final -que no forman parte de la cadena- se escriben para que quede claro dónde empieza y termina la cadena):

```
|..1..1..4El.proceso.de.Compilacion.....13|
```

El resultado que mostrará el programa será:

Componentes:

```
Seccion:      1
Subseccion:   1
Subsubseccion: 4
Pagina:       13
Titulo:       El proceso de Compilacion
```

Línea procesada:

```
1.1.4. El proceso de Compilacion 13
```

Nota: En la cadena leída, las secciones se numeran con un cero en la subsección y subsubsección. Las subsecciones se numeran con un cero en la subsubsección.

Ejemplos:

```
|..1..0..0Introduccion.a.la.Programacion.....1|
|..1..1..0El.ordenador,.algoritmos.y.programas.....2|
```

42. Escriba un programa que lea un número entero (`original`) y construya otro a partir de él (`nuevo`) intercambiando la primera y la última cifra. Después indicará si la primera cifra de `original` es igual, mayor o menor que la última del mismo valor (hágalo comparando `original` y `nuevo`).

Nota: No puede usar ninguna estructura/operador condicional.

RELACIÓN DE PROBLEMAS II. Estructuras de Control

Ejercicios sobre condicionales

1. Cread un programa que lea el valor de la edad (dato de tipo entero) y salario (dato de tipo real) de una persona. Subid el salario un 5 % si éste es menor de 750 euros y la persona es mayor de 55 años.

¿Es mejor incluir otra variable nueva `salario_final` o es mejor modificar la variable que teníamos?

Imprimid el resultado por pantalla. Si no procede la subida imprimid el mensaje "No es aplicable la subida". En ambos casos imprimid el salario final.

Finalidad: Plantear una estructura condicional con una expresión lógica compuesta. Dificultad Baja.

2. Realizar un programa que lea dos valores enteros desde teclado y calcule si cualquiera de ellos divide o no (de forma entera) al otro.
 - En el caso que uno de ellos divida al otro deberá indicarnos que se ha detectado que uno de ellos es múltiplo del otro, y cuál es.
 - Si no fueran múltiplos lo indicará.

Debemos contemplar el caso en el que alguno de los valores introducidos sea cero, en cuyo caso, ninguno divide al otro.

Tened especial cuidado en no mezclar entradas/salidas con cálculos.

Finalidad: Plantear una estructura condicional doble con una expresión lógica compuesta. Dificultad Media.

3. Escribid un programa que lea tres enteros desde teclado y nos diga si están ordenados (da igual si es de forma ascendente o descendente) o no lo están.

Finalidad: Plantear una estructura condicional doble con una expresión lógica compuesta. Dificultad Baja.

4. Retome la solución del ejercicio 7 de la *Relación de Problemas I*. La forma de calcular la tarifa final del billete cambia ahora de la forma siguiente: la tarifa base sigue siendo de 150 euros, la misma para todos los destinos. Ahora bien, si el destino está a menos de 300 kilómetros, el precio final es la tarifa base. Para destinos a más de 300 Km, se suman 10 céntimos por cada kilómetro de distancia al destino (a partir del Km 300).

RELACIÓN DE PROBLEMAS II. Estructuras de Control

Cree un programa para que lea el número de kilómetros al destino y calcule el precio final del billete.

Finalidad: Plantear una estructura condicional simple. Actualizar una variable según una condición. Dificultad Baja.

5. Continuando con el problema del cálculo de las tarifas aéreas que hemos iniciado en el ejercicio 4 de esta relación de problemas, ahora se van a aplicar descuentos a las tarifas ya calculadas. Los criterios a aplicar son los siguientes:

- a) Un primer descuento está determinado por el número de kilómetros del trayecto. Si el trayecto es mayor de 700 km, se aplica un descuento del 2 %
- b) El segundo descuento dependerá del número de puntos de la tarjeta de fidelización del cliente. Si el número de puntos es mayor de 100, se aplica un descuento del 3 % y si es mayor de 200, se aplica un descuento del 4 %.

Los dos descuentos anteriores son independientes y acumulables. En cualquier caso, ambos se aplican sobre el precio del billete: en este programa un cliente podría beneficiarse de un descuento del 2 %, del 3 %, del 4 %, del 5 %, del 6 %, o ninguno.

Finalidad: Plantear una estructura condicional simple. Dificultad Media.

6. Ampliad el ejercicio 19 de la *Relación de Problemas I* de manera que los dos instantes puedan pertenecer a dos días consecutivos. Si los dos instantes pertenecen a dos días diferentes, indicadlo de manera explícita en la salida.

Calcule la solución usando dos algoritmos: 1) en base al número de segundos transcurridos entre los dos instantes, y 2) comparando los valores de horas, minutos y segundos de los dos instantes.

Escribid las dos soluciones en el mismo programa.

Finalidad: Trabajar con condicionales. Dificultad Media.

7. Cread un programa que lea el número de un año e indique si es bisiesto o no. Un año es bisiesto si es múltiplo de cuatro, pero no de cien. Excepción a la regla anterior son los múltiplos de cuatrocientos que siempre son bisiestos. Por ejemplo, son bisiestos: 1600, 1996, 2000, 2004. No son bisiestos: 1700, 1800, 1900, 1998, 2002.

Nota: El programa sólo procesará años del siglo XX en adelante.

Finalidad: Plantear una estructura condicional con una expresión lógica compuesta. Dificultad Baja.

8. Modifique la solución del ejercicio 38 de la *Relación de Problemas I* de manera que el mínimo:

...

- c) Calcular $c = 2 \arcsin(\min(1, \sqrt{a}))$

...

que se calculaba con la función `min` de la biblioteca `algorithm` deberá calcularse ahora con un condicional.

Finalidad: Plantear una estructura condicional simple. Dificultad Baja.

9. Modifique la solución del ejercicio 28 de la *Relación de Problemas I* y cambie la implementación para realizar la codificación del carácter utilizando un condicional simple en vez de una expresión tan compleja como la que se utilizó en dicho ejercicio.

En definitiva, en este ejercicio **no** puede usar el operador módulo %, aunque sí puede usar condicionales simples.

Finalidad: Plantear una estructura condicional simple. Actualizar una variable según una condición. Dificultad Baja.

10. Los radares (o cinemómetros) pueden ser fijos o móviles tienen como misión registrar la velocidad de los vehículos. Estos aparatos presentan un *margen de error* que hay que aplicar a la velocidad captada por el radar y vienen especificados en la orden ITC/3123/2010, de 26 de noviembre https://www.boe.es/diario_boe/txt.php?id=BOE-A-2010-18556

a) En el caso de un radar fijo:

- I) Si la velocidad captada por el radar es menor o igual que 100 km/h, el margen de error es de 5 km/h.
- II) En caso contrario, el margen de error es de un 5 %

b) En el caso de un radar móvil:

- I) Si la velocidad captada por el radar es menor o igual que 100 km/h, el margen de error es de 7 km/h.
- II) En caso contrario, el margen de error es de un 7 %

Los márgenes de error se aplican sobre la velocidad captada y da como resultado la *velocidad imputada*. Por ejemplo, si la velocidad captada es de 95 km/h, la velocidad imputada sería de $95 - 5 = 90$ km/h en el caso de un radar fijo y de $95 - 7 = 88$ km/h en el caso de un radar móvil.

Si la velocidad captada es, por ejemplo, de 104 km/h, la velocidad imputada sería de $104 - 5\% \text{ de } 104 = 104 - 5.2 = 98.8$ km/h en el caso de un radar fijo y de $104 - 7\% \text{ de } 104 = 104 - 7.28 = 96.72$ km/h en el caso de un radar móvil.

Construya un programa que lea desde teclado un carácter que indique el tipo de radar (F para fijo y cualquier otra letra para móvil), la velocidad captada, e imprima la velocidad imputada.

Finalidad: Plantear una estructura condicional anidada y actualizar variables en función de ciertas condiciones. Dificultad Baja.

11. La Dirección General de Tráfico publica en su página web las sanciones a aplicar por infracción de velocidad. Puede consultarse la tabla en https://sede.dgt.gob.es/Galerias/tramites-y-multas/alguna-multa/consulta-de-sanciones-por-exceso-velocidad/cuadro_velocidad.pdf

En este ejercicio queremos determinar la sanción a aplicar en una autovía, cuyo límite de velocidad es 120 Km/h. En la siguiente tabla se muestra la velocidad del vehículo y la sanción correspondiente (número de puntos del carnet de conducir que se restan y la multa en euros)

(120,150]	-> 0, 100
(150,170]	-> 2, 300
(170,180]	-> 4, 400
(180,190]	-> 6, 500
(190, ∞)	-> 6, 600

Escriba un programa que lea la velocidad del vehículo e imprima en pantalla la sanción correspondiente (número de puntos a detraer y multa en euros)

Finalidad: Planteamiento de una estructura condicional anidada. Dificultad Baja.

12. Ampliad el ejercicio 21 de la *Relación de Problemas I*. Después de haber leído los datos que definen el intervalo, el programa debe leer un valor real y determinar si está o no dentro del intervalo.

Finalidad: Plantear una condición compleja. Dificultad Media.

13. Se quiere leer un carácter `letra_original` desde teclado y

- Si es una letra mayúscula, almacenaremos en la variable `letra_convertida` la correspondiente letra minúscula.
- Si es una letra minúscula, almacenaremos en la variable `letra_convertida` la correspondiente letra mayúscula.
- Si es un carácter no alfabético, almacenaremos el mismo carácter en la variable `letra_convertida`

Finalmente, imprimiremos en pantalla alguno de los siguientes mensajes:

- La letra era una mayúscula. Una vez convertida es ...
- La letra era una minúscula. Una vez convertida es ...
- El carácter no era una letra.

Finalidad: Plantear una estructura condicional anidada. Diseñar programas que separen entradas/salidas y cálculos. Dificultad Baja.

14. Construya un programa para calcular el importe total a facturar de un pedido. El programa leerá el número de unidades vendidas y el precio de venta de cada unidad. Si la cantidad vendida es mayor de 100 unidades, se le aplica un descuento del 3 %. Por otra parte, si el precio final de la venta es mayor de 700 euros, se aplica un descuento del 2 %. Ambos descuentos son acumulables. Obtenga el importe final e imprímalo.

Vamos a cambiar el criterio de los descuentos. Supondremos que sólo se aplicará el descuento del 2 % (por una venta mayor de 700 euros) cuando se hayan vendido más de 100 unidades, es decir, para ventas de menos de 100 unidades no se aplica el descuento del 2 % aunque el importe sea mayor de 700 euros.

Cambiar el programa visto en clase para incorporar este nuevo criterio.

Finalidad: Plantear una estructura condicional anidada. Dificultad Baja.

15. Cread un programa que lea los datos fiscales de una persona, reajuste su renta bruta según el criterio que se indica posteriormente e imprima su renta neta final.

- La renta bruta es la cantidad de dinero íntegra que el trabajador gana.
- La retención fiscal es el tanto por ciento que el gobierno *se queda*.
- La renta neta es la cantidad que le queda al trabajador después de quitarle el porcentaje de retención fiscal, es decir:

$$\text{Renta_neta} = \text{Renta_bruta} - \text{Renta_bruta} * \text{Retención} / 100$$

Los datos a leer son:

- Si la persona es un trabajador autónomo o no.
- Si es pensionista o no.
- Si está casado o no.
- Renta bruta (total de ingresos obtenidos).
- Retención inicial.

La modificación se hará de la siguiente forma:

- Bajar un 3 % la retención fiscal a los autónomos
- Para los no autónomos:
 - Se sube un 1 % la retención fiscal a todos los pensionistas
 - Al resto de los trabajadores se les aplica una subida lineal del 2 %. Una vez hecha esta subida, se le aplica (sobre el resultado anterior) las siguientes subidas adicionales a la retención fiscal:
 - Subir otro 2 % si la renta bruta es menor de 20.000 €
 - Subir otro 2.5 % a los casados con renta bruta superior a 20.000 €
 - Subir otro 3 % a los solteros con renta bruta superior a 20.000 €

RELACIÓN DE PROBLEMAS II. Estructuras de Control

Una vez calculada la retención final, habrá que aplicarla sobre la renta bruta para así obtener la renta final del trabajador.

Finalidad: Plantear una estructura condicional anidada. Dificultad Media.

16. La tabla para el cálculo del precio a pagar en un *parking* para 2020 es la siguiente:

Desde el minuto 0 al 30: 0.0412 euros cada minuto
Desde el minuto 31 al 90: 0.0370 euros cada minuto
Desde el minuto 91 al 120: 0.0311 euros cada minuto
Desde el minuto 121 al 660: 0.0305 euros cada minuto
Desde el minuto 661 hasta máximo 24 horas: 24.00 euros

Dadas las horas de entrada y salida del parking (hora y minuto), construya un programa que calcule la tarifa final en euros a cobrar. Ejemplo: si el tiempo de permanencia es de 94 minutos, los primeros 30 minutos se facturan a 0.0412 el minuto, los siguientes 60 se facturan a 0.0370 el minuto, y los 4 restantes a 0.0311 el minuto.

Nota: Se supone que la estancia no sobrepasa las 24 horas.

Finalidad: Utilización del condicional doble. Dificultad Media.

17. Se acuerda la actualización de las tarifas de aparcamiento incorporando un nuevo tramo (desde el minuto 661 hasta el 900 a un coste de 0.0270 euros cada minuto) y subiendo el coste de la tarifa plana a 35 euros (aplicable ahora desde el minuto 901). Actualice el programa con estas nuevas condiciones.

Finalidad: Evaluar informalmente el coste de las actualizaciones. Dificultad Baja.

18. Realizar un programa que realice la conversión entre dos unidades de distancia del SI. Las unidades contempladas serán: mm (milímetros), cm (centímetros), m (metros) y km (kilómetros).

El programa mostrará el *prompt* > y el usuario escribirá cuatro *tokens*:

valor unidad_inicial a unidad_final

y el programa mostrará el resultado de la transformación. Por ejemplo, si quisiéramos transformar 7 centímetros a metros escribiremos:

```
> 7 cm a m
7.000 cm = 0.007 m
```

y para convertir 5.2 kilómetros a milímetros:

```
> 5.2 km a mm
5.200 km = 5200000.000 mm
```

RELACIÓN DE PROBLEMAS II. Estructuras de Control

Notas:

- a) Se permite la transformación entre todas las unidades citadas anteriormente.
- b) Intente minimizar el número de comprobaciones a realizar.
- c) Tenga en cuenta todas las situaciones de error. En caso de error deberá informar del error y terminar la ejecución del programa.

Finalidad: Gestionar múltiples situaciones que resultan de evaluar expresiones lógicas y minimizar el número de comprobaciones. Dificultad Media.

Ejercicios sobre tipos enumerados

19. Modificad el ejercicio 3 para que el programa nos diga si los tres valores leídos están ordenados de forma ascendente, ordenados de forma descendente o no están ordenados.

Para resolver este problema, se recomienda usar una variable de tipo *enumerado*.

Finalidad: Usar el tipo enumerado para detectar cuándo se produce una situación determinada. Dificultad Baja.

20. Cread un programa que lea el valor de la edad (dato de tipo entero) y salario (dato de tipo real) de una persona. Subid el salario un 4 % si es mayor de 65 o menor de 35 años. Si **además** de cumplir la anterior condición, también tiene un salario inferior a 300 euros, se le subirá otro 3 %, mientras que si su salario es mayor o igual que 300 euros pero inferior a 900 euros se le subirá un sólo el 1.5 %.

Imprimid el resultado por pantalla indicando qué subidas se han aplicado.

Finalidad: Plantear una estructura condicional anidada. Usar el tipo enumerado para gestionar categorías. Dificultad Baja.

21. Tome como referencia la solución al ejercicio 13 de esta misma relación de problemas. Ahora deberá emplear un tipo enumerado que permita guardar el *tipo* de carácter de `letra_original`. Considere que ahora podría ser: una letra mayúscula, una letra minúscula, un dígito u otro carácter.

Importante: las vocales con tilde (mayúsculas y minúsculas) y las letras ú, ñ, ò y ñ se englobarán en la categoría de *otros* por simplificar la solución.

Finalidad: Separación de E/S y C. Usar el tipo enumerado para gestionar categorías. Dificultad Media.

Ejercicios sobre bucles

22. Leer un carácter desde teclado, obligando al usuario a que sea una letra mayúscula. Para ello, habrá que usar una estructura repetitiva `do-while`, de forma que si el usuario introduce un carácter que no sea una letra mayúscula, se le volverá a pedir otro carácter (dicho de otra manera, habrá que **filtrar** el valor leído para que sólo admita letras mayúsculas).

Calculad la minúscula correspondiente e imprimidla en pantalla.

Finalidad: Trabajar con bucles con condiciones compuestas. Trabajar con filtros. Dificultad Baja.

23. Ampliad el ejercicio 6 de manera que en esta versión filtre adecuadamente los valores de las horas, minutos y segundos.

Nota: Para calcular si los dos instantes pertenecen al mismo día emplee el algoritmo 2 (comparación entre horas, minutos y segundos de los dos instantes).

Finalidad: Trabajar con filtros. Dificultad Baja.

24. Ampliad el ejercicio 24 de manera que en esta versión se filtre adecuadamente el valor de la desviación típica.

Finalidad: Trabajar con filtros. Dificultad Baja.

25. Reescribid la solución del problema del cálculo de las tarifas aéreas con descuentos (ejercicio 5). Ahora se filtrarán adecuadamente los datos de entrada (distancia a recorrer en el vuelo y puntos del cliente).

Finalidad: Trabajar con filtros. Dificultad Baja.

26. Realizar un programa que lea desde teclado un entero positivo e imprima en pantalla todos sus **divisores propios**. Para obtener los divisores, basta recorrer todos los enteros menores que el valor introducido y comprobar si lo dividen.

Finalidad: Bucle sencillo y filtro de entrada de datos. Dificultad Baja.

27. Ampliad el ejercicio 26 de la *Relación de Problemas I* de manera que en esta nueva versión lea un número entero positivo en una dato `int` o `string` comprobando la validez del dato leído (debe ser positivo).

Por ejemplo, si el número (`int`) es 3519, la salida sería `| 3 | 5 | 1 | 9 |`

En este ejercicio puede mezclar entradas, cálculos y salidas.

Finalidad: Practicar con filtros y ciclos básicos. Dificultad Baja.

RELACIÓN DE PROBLEMAS II. Estructuras de Control

28. Modificad el ejercicio 27 de esta misma relación de problemas para que ahora estén *claramente diferenciadas las etapas de entrada, cálculos y salidas*. Para ello componga un `string` con los dígitos del número leído separados por una barra vertical.

Por ejemplo, si el número (`int`) es 3519, la salida (`string`) sería `| 3 | 5 | 1 | 9 |`

Finalidad: Practicar con filtros y ciclos básicos. Dificultad Media.

29. Modifiquemos el ejercicio 10 de la *Relación de Problemas I*. Supongamos ahora que se quiere reinvertir todo el dinero obtenido (capital más intereses) en otro plazo fijo a un año. Y así, sucesivamente. Construid un programa para que lea el capital C , el interés I y un número de años N , y calcule e imprima todo el dinero obtenido durante cada uno de los N años, suponiendo que se reinvierte todo.

Filtrar adecuadamente los valores leídos de manera que cumplan las condiciones:

- Sobre el capital inicial, $C > 0$,
- Sobre el número de años, $1 \leq N \leq 20$
- Sobre el interés, $0 < I \leq 10$

El programa debe mostrar una salida del tipo:

```
Total en el año número 1 = 240
Total en el año número 2 = 288
.....
```

Finalidad: Usar una variable acumuladora dentro del cuerpo de un bucle (aparecerá a la izquierda y a la derecha de una asignación).

Finalidad: Practicar con filtros y ciclos básicos. Dificultad Baja.

30. Modifiquemos nuevamente el ejercicio 10 de la *Relación de Problemas I*. Ahora se trata de construir un programa para calcular cuantos años han de pasar hasta llegar a doblar, como mínimo, el capital inicial.

Los valores del capital inicial y del interés deben cumplir las restricciones indicadas en el ejercicio 29.

Finalidad: Usar la variable acumuladora en la condición del bucle. Dificultad Baja.

31. Usando como base el programa que soluciona el ejercicio 17 escribid un programa que lea la hora actual (hora y minuto) y una cantidad de dinero. El programa nos indicará cuántos minutos podremos estar estacionados -y la hora límite (hora y minuto)- si vamos a pagar con la cantidad especificada.

Implementar los filtros adecuados al problema.

Nota: Si dispone de 5 euros y 25 céntimos deberá indicar el valor 5.25 cuando se le pida el dinero disponible.

Finalidad: Practicar con filtros y ciclos básicos. Dificultad Baja.

32. De <http://countrysmeters.info> se obtienen los siguientes datos estimados sobre la población de China:

- nace una persona cada 1.87 segundos
- muere una persona cada 3.27 segundos
- emigra una persona cada 71.9 segundos

Escribid un programa que pida un valor de población y calcule cuántos años (enteros) transcurrirán hasta que la población estimada sea mayor o igual al valor dado.

Nota: La población en actual (Octubre de 2019) es de 1.406.862.401 personas,

Filtrar el valor de población introducido para que sea mayor que el de la población actual.

Finalidad: Practicar con filtros y ciclos básicos. Dificultad Baja.

33. Se dice que un número n es **triangular** si se expresa como la suma de los primeros k valores enteros. Por ejemplo, $n = 6$ es triangular ya que $6 = 1 + 2 + 3$ ($k = 3$).

Una forma de obtener los números triangulares es a través de la fórmula $k(k + 1)/2 \quad \forall k \in \mathbb{N}$.

Escriba un programa que, sin aplicar directamente la fórmula anterior, muestre todos los números triangulares que hay menores que un entero n introducido desde teclado.

Finalidad: Practicar con filtros y ciclos básicos. Dificultad Baja.

34. Use como referencia el ejercicio 12 de esta misma relación de problemas.

Después de haber leído los datos que definen el intervalo, el programa debe leer el valor de un entero n y a continuación debe leer n valores reales.

Por cada uno de ellos, el programa nos dirá si pertenece o no al intervalo anterior.

Finalidad: Plantear una estructura repetitiva básica para llevar un contador de acciones. Dificultad Baja.

35. Ampliad el ejercicio 7. El programa pedirá los valores de *dos años del siglo XXI* y mostrará todos los años bisiestos comprendidos entre los dos valores dados.

Finalidad: Practicar con filtros y ciclos básicos. Practicar con algoritmos más elaborados y eficientes. Reutilizar código ya escrito y verificado. Dificultad Media.

36. Realizar un programa que lea números reales **positivos** desde teclado y calcule cuántos se han introducido y cuál es el mínimo y el máximo de dichos valores. Se dejará de leer datos cuando el usuario introduzca el valor 0. Realizad la lectura de los reales dentro de un bucle sobre una única variable llamada `dato`. Es importante controlar los casos extremos, como por ejemplo, que el primer valor leído fuese ya el terminador de entrada (en este caso, el cero).

Finalidad: Destacar la importancia de las inicializaciones antes de entrar al bucle. Ejemplo de lectura anticipada. Dificultad Baja.

RELACIÓN DE PROBLEMAS II. Estructuras de Control

37. Realizar un programa que lea dos secuencias de números reales positivos y nos diga si todos los valores de la primera secuencia son mayores que todos los de la segunda.

Realizad la lectura de los números dentro de sendos bucles sobre una única variable llamada `dato`. El final de cada secuencia viene marcado por el valor 0.

Finalidad: Ejercitar el uso de bucles. Dificultad Baja.

38. Recupere la solución del ejercicio 10 de esta relación de problemas (cálculo de la velocidad imputada). Modifique el programa para que vaya leyendo los siguientes datos dentro de un bucle: en primer lugar, se leerá la matrícula del vehículo (en un dato de tipo `string`), a continuación el tipo de radar (dato de tipo `char`) y finalmente la velocidad del vehículo (en un dato de tipo `double`).

Supondremos que el carácter 'F' está asociado al radar fijo y el carácter 'M' al radar móvil. Supondremos que el usuario introduce un carácter correcto.

El programa calculará las velocidades imputadas e imprimirá en pantalla la matrícula de aquel vehículo que tenga máxima velocidad imputada (también imprimirá dicha velocidad). La lectura terminará cuando la matrícula del vehículo sea la cadena "FIN"

Ejemplo de entrada: 4312BHG F 95 8342DFW F 104 4598IJG M 95 FIN

Salida correcta: 8342DFW 98.8

Finalidad: Practicar con ciclos de lectura adelantada. Dificultad Baja.

39. Una empresa que tiene tres sucursales decide llevar la contabilidad de las ventas de sus productos a lo largo de una semana. Registra cada venta con tres datos: 1) el identificador de la sucursal (1,2 ó 3), 2) el código del producto (a, b ó c) y 3) el número de unidades vendidas.

Diseñar un programa que lea desde el teclado una serie de registros compuestos por `sucursal`, `producto`, `unidades` y diga cuál es la sucursal que más **productos** ha vendido. La lectura termina cuando la sucursal introducida vale -1.

Por ejemplo, con la serie de datos que se detalla a continuación:

```
2 a 20
1 b 10
1 b 4
3 c 40
1 a 1
2 b 15
1 a 1
1 c 2
2 b 6
-1
```

la sucursal que más productos ha vendido es la número 2 con 41 unidades totales.

Para comprobar que el programa funciona correctamente, cread un fichero de texto y redirigid la entrada desde dicho fichero.

Finalidad: Ver un bucle en el que se leen varios datos en cada iteración, pero sólo uno de ellos se usa como terminador de la entrada. Dificultad Media.

40. Se decide informatizar el acta de un partido de baloncesto para saber qué equipo es el ganador del partido. El acta de anotaciones está formada por una serie de *tríos* de números, y cada uno contiene: el número de equipo (1 ó 2), el dorsal del jugador y el número de puntos conseguidos. La última anotación es el valor terminador -1.

Por ejemplo, con la entrada

1 2 4 1 4 1 2 3 6 2 3 2 5 2 5 1 1 3 -1

El programa muestra el equipo ganador (o si hay empate) y la puntuación.

Por ejemplo, con la entrada anterior, gana el equipo 1, y la puntuación es 10-7.

Finalidad: Ver un bucle en el que se leen varios datos en cada iteración, pero sólo uno de ellos se usa como terminador de la entrada. Dificultad Media.

41. La Unión Europea ha decidido premiar al país que más toneladas de hortalizas exporte a lo largo del año. Se dispone de un registro de transacciones comerciales en el que aparecen tres valores en cada apunte. El primer valor es el indicativo del país (E: España, F: Francia y A: Alemania), el segundo valor es un indicativo de la hortaliza que se ha vendido en una transacción (T: Tomate, P: Patata, E: Espinaca) y el tercer valor indica las toneladas que se han vendido en esa transacción. Diseñar un programa que lea desde el teclado este registro, el cual termina siempre al leer un país con indicativo @, y que diga qué país es el que más hortalizas exporta y las toneladas que exporta.

Por ejemplo, con la entrada

E T 10 E T 4 E P 1 E P 1 E E 2 F T 15 F T 6 F P 20 A E 40 @

el país que más vende es *Francia* con un total de 41 toneladas.

Finalidad: Ver un bucle en el que se leen varios datos en cada iteración, pero sólo uno de ellos se usa como terminador de la entrada. Dificultad Media.

42. Construya un programa que calcule cuándo se produjo la mayor secuencia de días consecutivos con temperaturas crecientes. El programa leerá una secuencia de reales representando temperaturas, hasta llegar al -1 y debe calcular la subsecuencia de números ordenada, de menor a mayor, de mayor longitud. El programa nos debe decir la posición donde comienza la subsecuencia y su longitud.

Por ejemplo, ante la entrada siguiente:

17.2 17.3 16.2 16.4 17.1 19.2 18.9 -1

RELACIÓN DE PROBLEMAS II. Estructuras de Control

el programa nos debe indicar que la mayor subsecuencia empieza en la posición 3 (en el 16.2) y tiene longitud 4 (termina en 19.2)

Puede suponer que siempre se introducirá al menos un valor de temperatura.

Finalidad: Trabajar con bucles en los que se compara un valor leído con otro leído en la iteración anterior. Dificultad Media.

43. Modifique el ejercicio 42 para que la lectura de datos finalice cuando se introduce un valor fuera de un rango establecido entre los valores MIN_TEMP_VALIDA y MAX_TEMP_VALIDA.

Notas: En esta versión es posible que el primer valor sea incorrecto (fuera de rango). Además, si la secuencia ordenada más larga tuviera una longitud inferior a MIN_LONG_VALIDA deberá indicar que se trata de una secuencia no válida.

44. Para controlar los tiempos que emplean los corredores de una carrera popular se desea disponer de un programa muy sencillo que se irá ampliando en sucesivas versiones. En esta primera versión el programa empezará pidiendo la hora de inicio de la carrera (hora, minuto y segundo) y la distancia.

Después, para cada corredor que llega a meta (por orden de llegada) se pide el número de dorsal y la hora de llegada (hora, minuto y segundo). Tras leer estos datos el programa mostrará el tiempo que ha empleado en completar la carrera (horas, minutos y segundos) y su ritmo (en km/h y en minutos/km).

El programa finalizará cuando al solicitar el número de dorsal se introduzca un valor negativo.

Nota: Deberá filtrar **todos** los datos leídos.

Finalidad: Trabajar con bucles de lectura anticipada. Trabajar con bucles en los que se compara un valor leído con otro leído en la iteración anterior. Dificultad Media.

45. El método RLE (Run Length Encoding) codifica una secuencia de datos formada por series de valores idénticos consecutivos como una secuencia de parejas de números (valor de la secuencia y número de veces que se repite). Esta codificación es un mecanismo de compresión de datos (zip) sin pérdidas. Se aplica, por ejemplo, para comprimir los ficheros de imágenes en las que hay zonas con los mismos datos (fondo blanco, por ejemplo).

Realizar un programa que lea una secuencia de caracteres terminada con el carácter * y la codifique mediante el método RLE. Leed los datos desde un fichero externo, tal y como se explica en la página 26.

Entrada:	A A A X X X X X D D D D D D Q *
	(tres veces A, cinco veces X, seis veces D, una vez Q)
Salida:	3 A 5 X 6 D 1 Q

Finalidad: Controlar en una iteración lo que ha pasado en la anterior. Dificultad Media.

RELACIÓN DE PROBLEMAS II. Estructuras de Control

46. Implemente un programa que lea bits (ceros y unos) desde teclado, hasta que se introduzca un valor negativo. Si se introduce un positivo distinto de 0 y 1, el programa lo descartará y volverá a leer un valor.

Cada 8 valores de bits leídos, el programa calculará el número entero que representa y lo transformará en el carácter (char) correspondiente. Debe tener en cuenta que el primer bit leído es el más significativo.

Por ejemplo, si el usuario introduce los siguientes enteros:

0 1 0 0 1 3 4 1 1 1

acaba de completar un octeto ya que el 3 y el 4 no son bits (0 o 1) y se descartan. Por tanto, el octeto completado es 0 1 0 0 1 1 1 1 que corresponde al número entero 79 y por tanto al carácter 'O'. El programa debe ir construyendo el entero 79 como suma de potencias de 2 ($0 \cdot 2^7 + 1 \cdot 2^6 + \dots + 1 \cdot 2^0$)

Si no es posible completar el último bloque con 8 bits (porque se haya introducido un negativo antes del octavo bit), se descartarán todos los bits de ese último bloque incompleto.

Si el carácter obtenido corresponde a una letra -mayúscula o minúscula- lo mostrará por pantalla. Una vez terminada la entrada de datos, el programa mostrará el porcentaje de letras y otros símbolos (distintos de letras) leídos.

Ejemplo de entrada:

0 1 0 0 1 3 4 1 1 1 2 0 1 1 0 1 0 1 1 9 0 0 1 7 0 0 0 0 1 1 0 -1

Salida correcta:

Ok

Letras: 66.67%

Otros: 33.33%

Donde las correspondencias son 01001111 (carácter O), 01101011 (carácter k), y 00100001 (carácter !). Los bits finales 10 se descartan ya que no se ha completado un bloque de ocho.

Finalidad: Bucles anidados y lectura de datos. Dificultad Media.

47. Escribir un programa que lea un número entero positivo y muestre su descomposición en factores primos.

Hágalo de manera que se calcule y muestre el resultado de dos maneras:

- a) La primera (más sencilla) consiste en enumerar todos los primos de la descomposición (en el caso de que un primo intervenga más de una vez se mostrará tantas veces como sea preciso). Por ejemplo:

$$360 = 2 * 2 * 2 * 3 * 3 * 5$$

$$121 = 11 * 11$$

$$11 = 11$$

- b) La segunda consiste en expresar el resultado como el producto de potencias (en el caso de que un primo intervenga más de una vez se mostrará una potencia cuya base sea el valor del primo y el exponente será el número de veces que se repite). Evite potencias triviales (de exponente 1) escribiendo, en ese caso, únicamente la base (el valor del primo). Por ejemplo:

$$360 = 2^3 * 3^2 * 5$$

$$121 = 11^2$$

$$11 = 11$$

48. El algoritmo de la multiplicación rusa es una forma distinta de calcular la multiplicación de dos números enteros $n * m$. Para ello este algoritmo va multiplicando por 2 el multiplicador m y dividiendo (sin decimales) por dos el multiplicando n hasta que n tome el valor 1 y suma todos aquellos multiplicadores cuyos multiplicandos sean impares. Por ejemplo, para multiplicar 37 y 12 se harían las siguientes iteraciones

Iteración	Multiplicando	Multiplicador
1	37	12
2	18	24
3	9	48
4	4	96
5	2	192
6	1	384

El resultado de multiplicar 37 y 12 sería la suma de los multiplicadores correspondientes a los multiplicandos impares (en negrita), es decir $12+48+384=444$

Cread un programa que lea dos enteros y calcule su producto con este algoritmo.

Dificultad Media.

49. Diseñar un programa para jugar a adivinar un número. El programa generará un número *aleatorio* (entre MIN y MAX) y el usuario tendrá que adivinarlo. En cada jugada el jugador introducirá un valor y el juego indicará si el número introducido por el jugador está por encima o por debajo del número a adivinar.

Como reglas de parada considerad que el usuario: 1) haya acertado, ó 2) no quiera seguir jugando, y en este caso usad un valor especial (TERMINADOR) para abandonar.

Dificultad Media.

Procedimiento para la generación de números *aleatorios*.

Para poder generar números aleatorios en un rango determinado por MIN y MAX será necesario incluir las siguientes instrucciones:

```
#include<ctime>
#include<cstdlib>
...
const int MIN = 1;
const int MAX = 100;
const NUM_VALORES = MAX-MIN + 1; // Núm. de valores posibles

time_t t;
srand(time(&t)); // Inicializa generador de núms. aleatorios
                // con el reloj del sistema (hora actual)
int incognita;   // Número aleatorio que se genera

// Generación de "incognita" (MIN <= incognita <= MAX)
incognita = (rand() % NUM_VALORES) + MIN;
```

La orden `srand(time(&t))` debe ejecutarse una única vez al principio del programa y sirve para inicializar la secuencia de números aleatorios. Posteriormente, cada vez que se ejecute:

```
incognita = (rand() % NUM_VALORES) + MIN;
```

la instrucción `rand` devolverá valor (pseudo)aleatorio que se modificará para que quede comprendido entre MIN y MAX, y finalmente asignado a la variable `incognita`.

50. Recupere la solución del ejercicio 24 (función gaussiana) de la *Relación de Problemas I*. Construir un programa para imprimir el resultado de aplicar dicha función a varios valores de abscisas.

En primer lugar, se leerán los parámetros que definen la función, es decir, la *esperanza* μ y la *desviación típica* σ . La esperanza puede ser cualquier valor, pero la desviación típica debe ser mayor o igual que cero.

A continuación el programa pedirá un valor *mínimo* (x_i), un valor *máximo* (x_f) y un *incremento* (Δ). Filtrar adecuadamente estos valores.

El programa mostrará el valor de la función gaussiana (g en la ecuación 1) para todos los valores de x (la abscisa) entre x_i y x_f considerando entre dos puntos consecutivos un salto igual al valor dado en el incremento Δ .

En definitiva, se trata de calcular $g(x)$ si $x = x_i + k \Delta$ ($k = 0, 1, 2, \dots$) y $x \leq x_f$

$$g(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{\left\{-\frac{1}{2} \left(\frac{x - \mu}{\sigma}\right)^2\right\}} \quad (1)$$

En la figura 19.A mostramos cuatro funciones gaussianas con diferentes parámetros.

Finalidad: usar código verificado en bucles sencillos. Dificultad Baja.

51. Ahora estamos interesados en obtener el área que cubre la función gaussiana en un intervalo dado. Si consideramos el intervalo $(-\infty, x]$ el valor del área de la región que queda bajo la curva de $g(x)$ se calcula con la *distribución acumulada (cumulative distribution function)* en el punto x , abreviado $CDF(x)$. Matemáticamente:

$$CDF(x) = \int_{-\infty}^x g(x) dx$$

donde $g(x)$ es el valor de la función gaussiana en x (ecuación 1).

Puede obtenerse un valor aproximado de $CDF(x)$ como la *suma* de valores de $g(x)$, empezando por valores alejados de la media (y menores que ella).

En la figura 19 mostramos cuatro funciones gaussianas con diferentes parámetros (A) y sus correspondientes CDFs (B). En la figura 19.A mostramos el área bajo la curva de $g(x)$ con $\mu = -2,0$ y $\sigma^2 = 0,5$ en el intervalo $[-\infty, -2.5]$ y su valor es $CDF(-2.5)$. Puede usar la calculadora de:

https://solveymath.com/online_math_calculator/statistics/cdf_calculator.php para comprobar su resultado.

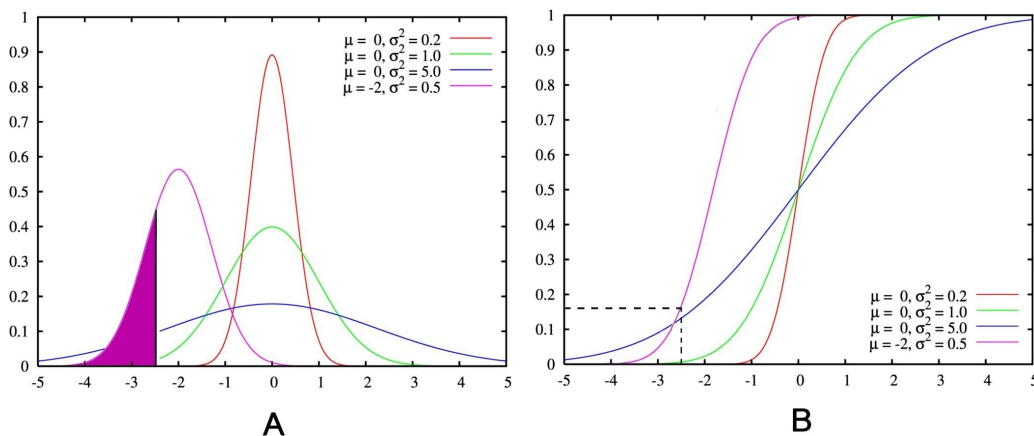


Figura 19: Distribuciones gaussianas y sus correspondientes CDFs

Fuente: Wikipedia: Distribución normal

https://es.wikipedia.org/w/index.php?title=Distribucion_normal

Escribid un programa calcule $CDF(x)$. El programa debe pedir los parámetros que definen una función gaussiana (μ y σ) y el valor de la abscisa, x , para el que se va a calcular $CDF(x)$. Para el cálculo de $CDF(x)$ la integral se convierte en una **suma**, y se requiere concretar cuántas sumas se van a realizar. Bastará con indicar:

- un valor inicial para x (tómese $\mu - 3\sigma$).
- un “salto” entre dos valores consecutivos de x (use una constante).

Dificultad Media.

52. En el siglo XIV el matemático indio Madhava de Sangamagrama calculó el arco tangente a través de un desarrollo de Taylor:

$$\arctan(x) = \sum_{i=0}^{\infty} \frac{(-1)^i x^{2i+1}}{2i+1}$$

Usando como x el valor $\frac{1}{\sqrt{3}}$ obtenemos:

$$\arctan\left(\frac{1}{\sqrt{3}}\right) = \frac{\pi}{6} = \sum_{i=0}^{\infty} \frac{(-1)^i \left(\frac{1}{\sqrt{3}}\right)^{2i+1}}{2i+1}$$

Por lo tanto, podemos usar la siguiente aproximación:

$$\frac{\pi}{6} = \sum_{i=0}^{\text{tope}} \frac{(-1)^i \left(\frac{1}{\sqrt{3}}\right)^{2i+1}}{2i+1}$$

Construya un programa que:

- lea el valor `tope` obligando a que esté entre 1 y 100000, calcule la aproximación de π mediante la anterior serie e imprima el resultado en pantalla.
- lea un entero que indique cuántos decimales de precisión deseamos, calcule la aproximación de π mediante la anterior serie y nos indique cuántas iteraciones -sumas- ha tenido que realizar para conseguir la aproximación deseada.

El valor de π a aproximar es 3.14159265358979323846

Importante: En la implementación de esta solución: 1) **NO** puede usar `pow` (observe que $(-1)^i$ es 1 cuando i es par y -1 si es impar) y 2) tampoco el condicional `if`.

Finalidad: Bucles simples. Acumulación de sumas. Comparación de reales. Dificultad Baja.

53. En el siglo XVII el matemático alemán Gottfried Leibniz y el matemático escocés James Gregory introdujeron una forma de calcular π a través de una serie, es decir, de una suma de términos:

$$\frac{\pi}{4} = \sum_{i=0}^{\infty} \frac{(-1)^i}{2i+1} = 1 - \frac{1}{2*1+1} + \frac{1}{2*2+1} - \frac{1}{2*3+1} + \dots$$

Esta es una serie infinita, pues realiza la suma de infinitos términos. Como no podemos realizar un número infinito de operaciones, habrá que parar en un índice dado,

RELACIÓN DE PROBLEMAS II. Estructuras de Control

llamémosle `tope`, obteniendo por tanto una aproximación al valor de π . Usaremos el símbolo $\approx$ para denotar esta aproximación:

$$\begin{aligned}\frac{\pi}{4} &\approx \sum_{i=0}^{\text{tope}} \frac{(-1)^i}{2i+1} = 1 - \frac{1}{2*1+1} + \frac{1}{2*2+1} - \frac{1}{2*3+1} + \dots + \frac{(-1)^{\text{tope}}}{2*\text{tope}+1} \\ &= 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots + \frac{(-1)^{\text{tope}}}{2*\text{tope}+1}\end{aligned}$$

Construya un programa que lea el valor `tope` obligando a que esté entre 1 y 100000, calcule la aproximación de π mediante la anterior serie e imprima el resultado.

Resuelva este problema de dos formas distintas:

- Usando la función `pow` de `cmath` para implementar $(-1)^i$
- Sin usar la función `pow`. Observe que el valor de $(-1)^i$ es 1 para los valores pares de i y -1 para los impares.

Finalidad: Bucles simples. Acumulación de sumas. Dificultad Baja.

54. Otra aproximación de π introducida en el siglo XVII por el matemático inglés John Wallis viene dada por:

$$\frac{\pi}{2} \approx \frac{2}{1} \cdot \frac{2}{3} \cdot \frac{4}{3} \cdot \frac{4}{5} \cdot \frac{6}{5} \cdot \frac{6}{7} \cdot \dots$$

Construya un programa que lea el valor `tope` obligando a que esté entre 1 y 100000, calcule la aproximación de π mediante la anterior fórmula (multiplicando un total de `tope` fracciones) e imprima el resultado en pantalla.

Debe resolver este problema de dos formas distintas, a saber:

- Observe que el numerador y el denominador varían de forma alternativa (aunque ambos de la misma forma, a saltos de 2). Cuando a uno le toca cambiar, el otro permanece igual. Este comportamiento se puede controlar con una única variable de tipo de dato `bool`.
- Otra forma de implementar los cambios en el numerador y denominador es observando que en cada iteración, el numerador es el denominador de la iteración anterior más 1 y el denominador es el numerador de la iteración anterior más 1.

Ejemplo de entrada: 1000 Salida correcta: 3.1400238186006

Ejemplo de entrada: 100000 Salida correcta: 3.14157694582286

Finalidad: Bucles simples. Acumulación de productos. Usar un `bool` fijado en la iteración anterior. Dificultad Media.

55. Escribir un programa que muestre la representación binaria de un número entero (positivo o cero) expresado en base 10.

El programa pedirá el número decimal que se va a convertir y calculará su representación binaria sobre un dato `string` (los bits que conforman la representación binaria del número serán caracteres '0' ó '1').

El número de bits del resultado (la longitud del `string`) dependerá del valor del número a convertir. Por ejemplo, al convertir el valor 11 se genera el string 1011 (longitud = 4) mientras que al convertir el valor 128 se genera el string 10000000 (longitud = 8).

Comprobar y controlar adecuadamente toda las situaciones de error.

56. Un número entero n se dice que es **desgarrable** (*torn*) si al dividirlo en dos partes izda y dcha, el cuadrado de la suma de ambas partes es igual a n .

Por ejemplo, 88209 es desgarrable ya que $(88 + 209)^2 = 88209$. El número 81 también es desgarrable ya que $(8 + 1)^2 = 81$.

Cread un programa que lea un entero n e indique si es o no desgarrable.

Finalidad: Ejercitar los bucles. Dificultad Baja.

57. Realizar un programa para calcular la suma de los términos de la serie

$$1 - \frac{1}{2} + \frac{1}{4} - \frac{1}{6} + \frac{1}{8} - \frac{1}{10} + \dots - \frac{1}{2n-1} + \frac{1}{2n}$$

para un valor n dado ($n > 0$).

Finalidad: Trabajar con bucles controlados por contador. Dificultad Baja.

58. Realizar un programa que presente una tabla de grados Celsius a grados Fahrenheit desde los 0 grados a los 300, con saltos de 20 grados.

Finalidad: Trabajar con bucles controlados por contador. Dificultad Baja.

59. Calcular mediante un programa la función **potencia** x^n con n un valor entero y x un valor real. No puede usarse la funciones de la biblioteca `cmath`.

Finalidad: Trabajar con bucles controlados por contador. Dificultad Baja.

60. Calcular mediante un programa la función **factorial**.

Se dice que $n!$ es el factorial de un entero n y se define de la forma siguiente:

$$0! = 1$$

$$n! = 1 \times 2 \times 3 \times \dots \times n, \quad \forall n \geq 1$$

Finalidad: Trabajar con bucles controlados por contador. Dificultad Baja.

61. Calcular mediante un programa en C++ el combinatorio $\binom{n}{m}$ con n, m valores enteros. No pueden usarse las funciones de la biblioteca `cmath`.

El combinatorio de n sobre m (con $n \geq m$) es un número entero que se define:

$$\binom{n}{m} = \frac{n!}{m! (n - m)!}$$

Finalidad: Trabajar con bucles controlados por contador. Dificultad Media.

62. En matemáticas, la **sucesión de Fibonacci de orden 2** (a veces mal llamada *serie de Fibonacci*) es la siguiente sucesión infinita de números naturales:

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots$$

La sucesión comienza con los números 1 y 1, y a partir de éstos, cada término puede calcularse como la suma de los dos anteriores (de ahí que se llame *de orden 2*). A los elementos de esta sucesión se les llama *números de Fibonacci*.

Supongamos una sucesión de orden 2. El número de Fibonacci de la posición n , al que llamaremos f_n se puede definir mediante la siguiente relación de recurrencia:

- $f_n = f_{n-1} + f_{n-2}$ para $n > 2$
- $f_1 = f_2 = 1$

Esta sucesión fue descrita en Europa por Leonardo de Pisa, matemático italiano del siglo XIII también conocido como Fibonacci. Tiene numerosas aplicaciones en ciencias de la computación, matemáticas y teoría de juegos. También aparece en diversas configuraciones biológicas.

Escribir un programa que calcule el número de Fibonacci de la posición n (valor introducido por el usuario).

A continuación, el programa solicitará un nuevo valor, k , y mostrará todos los números de Fibonacci $f_1, f_2, f_3, \dots, f_k$.

Finalidad: Trabajar con bucles controlados por contador. Dificultad Media.

63. El **número áureo** se conoce desde la Antigüedad griega y aparece en muchos temas de la geometría clásica. La forma más sencilla de definirlo es como el único número positivo ϕ que cumple que $\phi^2 - \phi = 1$ y por consiguiente su valor es $\phi = \frac{1 + \sqrt{5}}{2}$.

Se pueden construir aproximaciones al número áureo mediante la fórmula $a_n = \frac{f_{n+1}}{f_n}$ siendo f_n el número de Fibonacci de orden n (ver problema 62). La sucesión de valores así calculada proporciona, alternativamente, valores superiores e inferiores a ϕ , siendo cada vez más cercanos a éste, y por lo tanto la diferencia entre a_n y ϕ es cada vez más pequeña conforme n se hace mayor.

RELACIÓN DE PROBLEMAS II. Estructuras de Control

Escribir un programa que calcule el menor valor de n que hace que la aproximación dada por a_n difiera en menos de δ del número ϕ , sabiendo que $n \geq 1$. La entrada del programa será el valor de δ y la salida el valor de n . Por ejemplo, para un valor de $\delta = 0,1$ el valor de salida es $n = 4$

Finalidad: Trabajar con bucles. Reutilizar código ya validado. Dificultad Media.

64. Un número entero de n dígitos se dice que es **narcisista** si se puede obtener como la suma de las potencias n -ésimas de cada uno de sus dígitos. Por ejemplo 153 y 8208 son números narcisistas porque $153 = 1^3 + 5^3 + 3^3$ y $8208 = 8^4 + 2^4 + 0^4 + 8^4$. Construir un programa que nos indique si un entero positivo es narcisista.

Finalidad: Ejercitar los bucles. Dificultad Media.

65. Realizar un programa para calcular los valores de la función:

$$f(x, y) = \frac{\sqrt{x}}{y^2 - 1} \quad \text{para } x = -50, -48, \dots, 50 \quad y = -40, -39, \dots, 40$$

es decir queremos mostrar en pantalla los valores de la función en los puntos

$$(-50, 40), (-50, -39), \dots, (-50, 40), (-48, 40), (-48, -39), \dots, (50, 40)$$

Finalidad: Practicar con bucles anidados. Dificultad Baja.

66. Sobre la solución del ejercicio 29 de esta relación de problemas, se pide lo siguiente. Supondremos que sólo pueden introducirse intereses enteros (1, 2, 3, ...). Calcular el capital obtenido al término de cada año, pero realizando los cálculos para todos los tipos de interés enteros menores o iguales que el introducido (en pasos de 1).

Por ejemplo, si interés es 5, y un número de años es 3, hay que mostrar el capital ganado al término de cada uno de los tres años a un interés del 1 %; a continuación para un interés del 2 %, y así hasta llegar al 5 %. El programa debe mostrar:

Cálculos realizados al 1%:

```
Dinero obtenido en el año número 1 = 2020.00
Dinero obtenido en el año número 2 = 2040.20
Dinero obtenido en el año número 3 = 2060.60
```

Cálculos realizados al 2%:

```
Dinero obtenido en el año número 1 = 2040.00
Dinero obtenido en el año número 2 = 2080.80
Dinero obtenido en el año número 3 = 2122.42
.....
```

Finalidad: Practicar con bucles anidados. Dificultad Baja.

67. Escribid un programa que lea cuatro valores de tipo `char` (`min_izda`, `max_dcha`, `min_dcha`, `max_dcha`) e imprima las parejas que pueden formarse con un elemento del conjunto `{min_izda ... max_izda}` y otro de `{min_dcha ... max_dcha}`. Por ejemplo, si `min_izda = b`, `max_izda = d`, `min_dcha = j`, `max_dcha = m`, el programa debe imprimir las parejas que pueden formarse con un elemento del conjunto `{b, c, d}` y otro de `{j, k, l, m}`, es decir:

```
bj bk bl bm
cj ck cl cm
dj dk dl dm
```

Finalidad: Practicar con bucles anidados. Dificultad Baja.

68. Retome la solución del ejercicio 26 de esta misma relación de problemas y modifíquelo para leer un número entero `tope` e imprima en pantalla los divisores de todos y cada uno de los números positivos menor o iguales que `tope`.

Finalidad: Practicar con bucles anidados. Dificultad Baja.

69. (*Examen Septiembre 2014*) ¿Cuántas veces aparece el dígito 9 en todos números que hay entre el 1 y el 100? Por ejemplo, el 9 aparece una vez en los números 19 y 92 mientras que aparece dos veces en el 99.

Pretendemos diseñar un algoritmo que responda a esta sencilla pregunta, pero de forma suficientemente generalizada. Para ello, se pide construir un programa que lea tres enteros: `cifra` (entre 1 y 9), `min` y `max`, y calcule el número de apariciones del dígito `cifra` en los números contenidos en el intervalo cerrado `[min, max]`.

Finalidad: Ejercitar los bucles anidados. Dificultad Baja.

70. Construya un programa que lea un valor `T` y calcule la siguiente sumatoria:

$$\sum_{i=1}^{i=T} i! = \sum_{i=1}^{i=T} \left(\prod_{j=1}^{j=i} j \right)$$

Por ejemplo, para $T = 4$, la operación a realizar es:

$$1! + 2! + 3! + 4!$$

es decir:

$$1 + (1 * 2) + (1 * 2 * 3) + (1 * 2 * 3 * 4)$$

Finalidad: Practicar con bucles for anidados. Dificultad Media.

71. Un número **perfecto** es aquel que es igual a la suma de todos sus divisores positivos excepto él mismo. El primer número perfecto es el 6 ya que sus divisores son 1, 2 y 3 y $6=1+2+3$. Escribir un programa que muestre el mayor número perfecto que sea menor a un número dado por el usuario.

Finalidad: Practicar con bucles anidados. Dificultad Media.

72. Escribir un programa que encuentre dos enteros n y m mayores que 1 que verifiquen lo siguiente:

$$\sum_{i=1}^m i^2 = n^2$$

Finalidad: Practicar con bucles anidados. Dificultad Media.

73. Supongamos una serie numérica cuyo término general es:

$$a_i = a_1 r^{i-1} = a_1, a_1 r, a_1 r^2, a_1 r^3, \dots$$

Se pide crear un programa que lea desde teclado r , el primer elemento a_1 y el tope k y calcule la suma de los primeros k valores de la serie, es decir:

$$\sum_{i=1}^{i=k} a_i$$

Se proponen dos alternativas:

- Realizad la suma de la serie usando la función `pow` para el cómputo de cada término a_i . Los argumentos de `pow` no pueden ser ambos enteros, por lo que forzaremos a que la base (por ejemplo) sea `double`, multiplicando por `1.0`.
- Si analizamos la expresión algebraica de la serie numérica, nos damos cuenta que es una *progresión geométrica* ya que cada término de la serie queda definido por la siguiente expresión:

$$a_{i+1} = a_i r$$

Es decir, una progresión geométrica es una secuencia de elementos en la que cada uno de ellos se obtiene multiplicando el anterior por una constante denominada razón o factor de la progresión.

Cread el programa usando esta fórmula. NO puede utilizarse la función `pow`.

¿Qué solución es preferible en términos de eficiencia?

Finalidad: Trabajar con bucles que aprovechan cálculos realizados en la iteración anterior. Dificultad Baja.

74. Diseñar un programa para calcular la suma de los 100 primeros términos de la sucesión siguiente:

$$a_i = \frac{(-1)^i(i^2 - 1)}{2i}$$

No puede usarse la función `pow`. Hacedlo calculando explícitamente, en cada iteración, el valor $(-1)^i$ (usad un bucle `for`). Posteriormente, resolvedlo calculando dicho valor a partir del calculado en la iteración anterior, es decir, $(-1)^{i-1}$.

Finalidad: Enfatizar la conveniencia de aprovechar cómputos realizados en la iteración anterior. Dificultad Media.

75. Se dice que un número natural es **feliz** si cumple que si sumamos los cuadrados de sus dígitos y seguimos el proceso con los resultados obtenidos, finalmente obtenemos uno (1) como resultado. Por ejemplo, el número 203 es un número feliz ya que $2^2 + 0^2 + 3^2 = 13 \rightarrow 1^2 + 3^2 = 10 \rightarrow 1^2 + 0^2 = 1$.

Se dice que un número es feliz de grado k si es feliz en un máximo de k iteraciones. Se entiende que una iteración se produce cada vez que se elevan al cuadrado los dígitos del valor actual y se suman. En el ejemplo anterior, 203 es un número feliz de grado 3 (además, es feliz de cualquier grado mayor o igual que 3)

Escribir un programa que diga si un número natural n es feliz para un grado k dado de antemano. Tanto n como k son valores introducidos por el usuario.

Finalidad: Ejercitar los bucles anidados. Dificultad Media.

76. Diremos que un número entero positivo es **secuenciable** si se puede generar como suma de números consecutivos. Por ejemplo, $6 = 1 + 2 + 3$, $15 = 7 + 8$. Esta descomposición no tiene por qué ser única. Por ejemplo, $15 = 7 + 8 = 4 + 5 + 6 = 1 + 2 + 3 + 4 + 5$.

Escribir un programa que lea un entero n y nos diga cuántas descomposiciones posibles tiene. Por ejemplo:

```
15 -> 3 descomposiciones
94 -> 1 descomposición
108 -> 3 descomposiciones
```

Curiosidad: los únicos números con 0 descomposiciones son las potencias de 2.

Dificultad Media.

77. Se pide leer dos enteros sabiendo que el primero no tiene un tamaño fijo y que el segundo siempre es un entero de dos dígitos. Se pide comprobar si el segundo está contenido en el primero.

Entendemos que está contenido si los dos dígitos del segundo entero están en el primer entero de forma consecutiva y en el mismo orden. Por ejemplo, 89 está contenido en 7890, en 7789 y en 8977 pero no en 7980.

Dificultad Media.

78. Ampliad el ejercicio 51 de esta misma Relación de Problemas. Ahora estamos interesados en calcular $g(x)$ y $CDF(x)$ en un intervalo de valores de la abscisa.

Escribid un programa que pida los parámetros que definen una función gaussiana (μ y σ), los extremos del intervalo de valores de la abscisa y la diferencia (salto) entre dos valores consecutivos de x . A continuación mostrará, para cada valor de x los valores de $g(x)$ y de $CDF(x)$.

Como indicamos en el ejercicio 51 de esta misma Relación de Problemas, para el cálculo práctico de $CDF(x)$ bastará con fijar:

- a) un valor inicial de la abscisa para el cálculo de $CDF(x)$ (tómese $\mu - 3\sigma$).
- b) un “salto” entre dos valores consecutivos de x (use una constante).

Dificultad Media.

79. Escribir un programa en el que se presente un menú principal para que el usuario pueda elegir las siguientes opciones:

- a) Calcular factorial.
- b) Calcular potencia.
- c) Salir.

Si el usuario elige la opción de *salir*, el programa terminará.

Si elige *calcular potencia* se le solicitará un valor real x y un valor entero n y calculará x^n (ver ejercicio 59). Si el usuario elige *calcular factorial* se le solicita un valor entero positivo, n y a continuación calculará y mostrará $n!$ (ver ejercicio 60). Después de mostrar el valor calculado el programa mostrará de nuevo el menú.

Finalidad: Ejercitar los bucles anidados. Dificultad Baja.

80. Recupere la solución del ejercicio 79. Escribir un programa en el que se presente un menú principal para que el usuario pueda elegir las siguientes opciones:

- a) Introducir parámetros de la función (esperanza y desviación)
- b) Salir del programa

Si el usuario elige la opción de salir, el programa terminará; si elige la opción de introducir los parámetros, el programa leerá los dos parámetros (esperanza y desviación).

A continuación, el programa presentará un menú con las siguientes opciones:

- Introducir rango de valores de abscisas
- Volver al menú anterior (el menú principal)

Si el usuario elige volver al menú anterior, el programa debe presentar el primer menú; si el usuario elige introducir los valores de abscisas, el programa le pedirá los extremos del intervalo y el incremento entre dos valores consecutivos de la abscisa, y mostrará los valores de la función gaussiana en todos los valores posibles de la abscisa (de manera similar a como se hizo en el ejercicio 78).

Después de mostrar los valores de la función, el programa volverá al menú de introducción del rango de valores de abscisas.

Finalidad: Ejercitar los bucles anidados. Dificultad Media.

81. Escribir un programa en el que se presente un menú principal para que el usuario pueda elegir entre las siguientes opciones:

A→Calcular adición.

P→Calcular producto.

X→Salir.

Si el usuario elige en el menú principal:

a) *Salir*, el programa terminará su ejecución.

b) *Calcular adición* se presenta un menú (secundario) para que el usuario pueda elegir entre las siguientes opciones:

S→Calcular suma.

R→Calcular resta.

X→Salir.

c) *Calcular producto* se presenta un menú (secundario) para que el usuario pueda elegir entre las siguientes opciones:

M→Calcular multiplicación.

D→Calcular división entera.

R→Calcula resto.

X→Salir.

MUY IMPORTANTE: Para implementar las operaciones de *producto* no es posible utilizar los operadores de multiplicación (*) ni división (/). Deberá emplear sumas (+) y restas (-).

En las operaciones seleccionadas desde los menús secundarios el programa pedirá dos números *enteros*, realizará la operación seleccionada, mostrará el resultado y volverá a mostrar el menú secundario seleccionado anteriormente.

En los dos menús secundarios la selección de *Salir* hace que el programa vuelva a mostrar el menú principal.

Finalidad: Ejercitar los bucles anidados. Dificultad Media.

82. El cálculo de la raíz cuadrada puede realizarse con la función `sqrt` disponible en `cmath`. En el manual de referencia (por ejemplo, <https://www.cplusplus.com/reference/cmath/sqrt>) puede leer que en C++ 11 tiene disponibles, básicamente, las siguientes sobrecargas de esa función:

```
double sqrt (double x);  
float sqrt (float x);  
long double sqrt (long double x);
```

Pueden emplearse aproximaciones numéricas para evitar el cálculo con la función `sqrt`. Por ejemplo, en <https://www.youtube.com/watch?v=s0Sdi9z8S6E> puede ver un método muy sencillo. Impleméntelo para el cálculo de la raíz cuadrada de datos `double`.

Implemente a continuación una estimación basada en la proporcionalidad entre triángulos. Se trata de aproximar la función $y = \sqrt{x}$ por *rectas* entre los tramos determinados por los valores con raíces exactas. Por ejemplo, en la figura 20.A mostramos la función $y = \sqrt{x}$ y remarcamos sobre el eje x los valores con raíces exactas (0, 1, 4, 9 y 16) y en el eje y los valores de las raíces exactas (0, 1, 2, 3 y 4). En la figura 20.B unimos los puntos $(x, \sqrt{x})$ que verifican que x tiene raíz exacta.

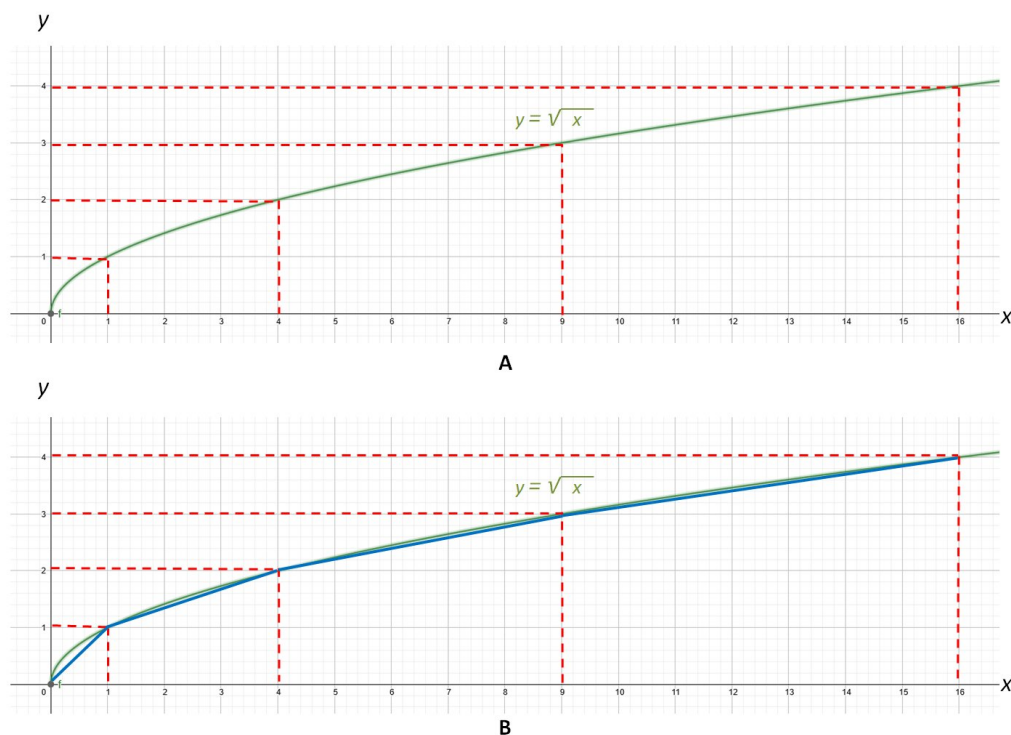


Figura 20: La función $y = \sqrt{x}$ (A) y su aproximación por rectas (B)

Si tomamos un segmento cualquiera en el eje x y lo vemos con detalle observará que hay una región triangular bien delimitada. Por ejemplo en la región entre $x = 1$ y $x = 4$ puede ver el triángulo delimitado por los puntos $(1, 1)$, $(4, 1)$ y $(4, 2)$ (figura 21.A). Por ser estrictos se trata del triángulo delimitado por los puntos $(1, \sqrt{1})$, $(4, \sqrt{1})$ y $(4, \sqrt{4})$.

La raíz de un valor x entre 1 y 4 se puede calcular geoméricamente. En la figura 21.B mostramos cómo se calcula la raíz cuadrada de $x = 3.5$. Observe que el triángulo delimitado por $(1, \sqrt{1})$, $(3.5, \sqrt{1})$ y $(3.5, \sqrt{3.5})$ (triángulo amarillo) es equivalente al delimitado por los puntos $(1, \sqrt{1})$, $(4, \sqrt{1})$ y $(4, \sqrt{4})$ (triángulo azul).

Calcule la raíz cuadrada de un número real positivo usando esta aproximación.

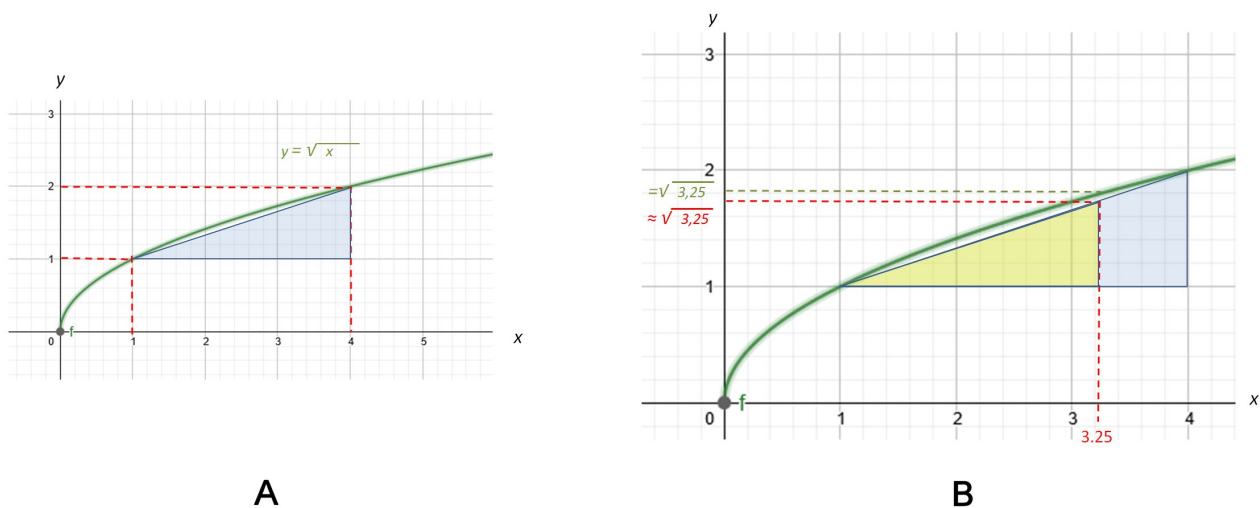


Figura 21: Aproximación a $y = \sqrt{x}$ por triángulos equivalentes

Finalmente muestre una tabla comparativa en la que se muestren los valores estimados por los dos métodos y por la función `sqrt` para valores de x desde 0 a 16, con saltos de 0.5. ¿Puede extraer alguna conclusión de esta tabla? ¿Puede proponer un método de estimación mejor -más ajustado al *verdadero* valor- que los dos implementados?

83. Ampliar la funcionalidad del ejercicio 55. El programa mostrará la representación binaria de un número entero (positivo o cero) en un rango predeterminado.

El programa pedirá en primer lugar el *número de bits* (llamémosle n) que tendrán los valores binarios con los que se va a trabajar. Dado n los números enteros que se admitirán estarán comprendidos entre 0 y $2^n - 1$ (todos ellos pueden representarse estrictamente con n bits o menos).

Igual que se pedía en el ejercicio 55 se calculará su representación binaria sobre un dato `string` aunque ahora el número de bits del resultado siempre será n (si

fuera necesario se rellenarán con '0' a la izquierda del primer '1'). Por ejemplo, si se emplean 8 bits, al convertir el valor 11 ahora se genera el string 00001011 y al convertir el valor 128 se genera el string 10000000 (en todos los casos la longitud será 8).

Comprobar y controlar adecuadamente toda las situaciones de error.

84. Escribir un programa que muestre el valor decimal de un valor dado en el sistema de numeración binario.

El programa pedirá el valor binario, que se leerá sobre un dato `string`, se calculará su valor decimal que se guardará a un entero y se mostrará el resultado.

Comprobar y controlar adecuadamente toda las situaciones de error.

85. Ampliar la funcionalidad del programa escrito para solucionar el ejercicio 49. En esta ocasión, una vez terminado un juego de adivinación podrá volverse a jugar de nuevo: el programa pedirá si queremos volver a jugar o no.

Al terminar (no se quiere jugar más) el programa mostrará cuántos juegos se han realizado (en caso de abandono también se contabiliza ese juego).

Dificultad Media.

86. Vamos a seguir trabajando sobre el problema de la *adivinación* de un número que planteamos en el ejercicio 49 y ampliamos en el ejercicio 85. En ambos ejercicios se requiere la actuación de un usuario.

En este caso el programa se ejecutará sin intervención de un usuario. Cada juego de adivinación consiste en: 1) generar un valor *aleatorio* a adivinar (entre MIN y MAX) y 2) generar números aleatorios en el mismo rango, hasta que el valor generado coincida con el valor buscado. Cada vez que se genera un valor y se compara con el valor buscado se produce una jugada. Para cada juego nos interesa conocer el número de jugadas.

Nota: todos los valores aleatorios generados en un juego son independientes, esto es, el valor aleatorio generado que se compara con el valor buscado no se aporta ninguna información para la siguiente tentativa. Observe que usar la palabra *adivinación* es temerario.

El proceso descrito (cada juego) se repite NUMERO_JUEGOS veces.

El objetivo del programa es conocer el número medio de jugadas por juego, esto es, cuántos intentos esperamos que haya que hacer (en media) para *adivinar* el número.

Dificultad Baja.

87. Mejoraremos la solución autónoma al problema de la *adivinación* planteado en el ejercicio 86 introduciendo una sencilla heurística para dirigir la solución. Evitaremos generar números aleatorios independientes para encontrar el valor buscado. Utilizaremos un valor inicial tentativo y a partir de él, mediante comparaciones con el valor buscado iremos reduciendo el ámbito de la búsqueda. Se tratará de una versión autónoma del problema planteado originalmente en el ejercicio 49 en el que el programa nos daba *pistas* para encontrar el valor buscado.

Heurística: Se parte del valor medio entre MIN y MAX. Si es el valor buscado, terminamos. Si no, el valor buscado estará en la mitad izquierda (si el valor buscado es menor que el valor medio) o en la mitad derecha (si el valor buscado es mayor que el valor medio). Sabiendo en qué mitad hay que buscar, se vuelve a repetir el proceso en la mitad donde corresponde.

Cada vez que se compara un nuevo valor con el valor buscado se produce una jugada. Para cada juego nos interesa conocer el número de jugadas.

El proceso descrito (cada juego) se repite NUMERO_JUEGOS veces.

El objetivo del programa es conocer el número medio de jugadas por juego, esto es, cuántos intentos esperamos que haya que hacer (en media) para *adivinar* el número.

Dificultad Media.

RELACIÓN DE PROBLEMAS III. Funciones

1. Cread una función que calcule el máximo entre tres `double`. La cabecera de la función será la siguiente:

```
double Max(double primero, double segundo, double tercero)
```

Construid un programa principal que llame a dicha función.

Finalidad: Familiarizarnos con la definición de funciones, el paso de parámetros y el ámbito de las variables. Dificultad Baja.

2. Reescribid el ejercicio el ejercicio 59 de la *Relación de Problemas II* de manera que el cálculo de la potencia se realice en una función.

Finalidad: Familiarizarnos con la definición de funciones. Dificultad Baja.

3. Reescribid el ejercicio el ejercicio 60 de la *Relación de Problemas II* de manera que el cálculo del factorial se realice en una función.

Finalidad: Familiarizarnos con la definición de funciones. Dificultad Baja.

4. Implementar la función que calcula el número combinatorio (ejercicio 61 de la *Relación de Problemas II*) y reescribir el programa propuesto en ese ejercicio usando esta función.

Finalidad: Entender el ámbito de los parámetros de las funciones, así como las llamadas entre funciones. Dificultad Baja.

5. Implemente la función que calcula el número de Fibonacci de orden n (ejercicio 62 de la *Relación de Problemas II*) y reescribir el programa propuesto en ese ejercicio usando esta función.

Finalidad: Familiarizarnos con la definición de funciones. Dificultad Baja.

6. Reescribir el programa construido para implementar la solución al ejercicio 63 de la *Relación de Problemas II* (número áureo) modularizando con funciones.

Finalidad: Entender el ámbito de los parámetros de las funciones, así como las llamadas entre funciones. Dificultad Media.

7. Reescribid la solución del ejercicio 28 de la *Relación de Problemas II* (pintar los dígitos de un entero positivo) creando la función `EnteroToString` que reciba un entero cualquiera y un carácter, y devuelva un `string` formado por los dígitos del número separados y delimitados por el carácter dado.

Por ejemplo, la ejecución de `cout << EnteroToString (12345, '|');` muestra `|1|2|3|4|5|`

Finalidad: Familiarizarnos con la definición de funciones y el paso de parámetros. Dificultad Baja.

RELACIÓN DE PROBLEMAS III. Funciones

8. Modificad la solución al ejercicio 50 de la *Relación de Problemas II*. En él se calculaba, repetidamente (para diferentes valores de la abscisa x) el valor de la función gaussiana.

Calcular el valor de la función gaussiana en un punto arbitrario x con una función. Hacedlo de dos maneras: a) con una función que admita un sólo parámetro (x) y b) con una función que admita tres parámetros (x , μ y σ). ¿Cuál es la mejor solución?

Finalidad: Entender el ámbito de los parámetros de las funciones, el uso de constantes globales o locales, el uso de funciones completamente parametrizadas, así como las llamadas entre funciones. Dificultad Media.

9. Reescribid el programa construido para implementar la solución al ejercicio 76 de la *Relación de Problemas II* (número secuenciable) modularizando con funciones. Escriba **dos** funciones: 1) para saber si un número es secuenciable y 2) para saber cuántas secuencias tiene asociadas un número.

El programa leerá dos valores enteros positivos que determinan el intervalo cerrado (los dos extremos incluidos) de trabajo.

Después leerá un valor entero positivo que indica un número de secuencias/descomposiciones de referencia.

A continuación realizará, por orden, las siguientes tareas:

- Calcular y mostrar **cuántos números secuenciables** hay.
- Mostrar los números secuenciables encontrados (sus **valores**) y **cuántas descomposiciones** tienen cada uno de ellos. No mostrar las descomposiciones detalladas, mostrar únicamente cuántas hay.
- Mostrar los números secuenciables (sus **valores**) cuyo número de descomposiciones coincide con el número de referencia leído.
Después, mostrar el total de números secuenciables que cumplen esta última condición.

Finalidad: Entender el ámbito de los parámetros de las funciones, así como las llamadas entre funciones. Dificultad Baja.

10. Reescribid el programa construido para implementar la solución al ejercicio 64 de la *Relación de Problemas II* (número narcisista) modularizando con funciones.

Finalidad: Entender el ámbito de los parámetros de las funciones, así como las llamadas entre funciones. Dificultad Baja.

11. Reescribid el programa construido para implementar la solución al ejercicio 75 de la *Relación de Problemas II* (número feliz) modularizando con funciones.

Finalidad: Entender el ámbito de los parámetros de las funciones, así como las llamadas entre funciones. Dificultad Baja.

12. (*Examen Septiembre 2015*) Dos **números amigos** son dos números naturales a y b , tales que la suma de los divisores propios de a más uno es igual a b , y viceversa. Un ejemplo de números amigos es el par de naturales (220,284), ya que:

- Los divisores propios de 220 son 2, 4, 5, 10, 11, 20, 22, 44, 55 y 110, que suman 283, y $283 + 1 = 284$.
- Los divisores propios de 284 son 2, 4, 71 y 142, que suman 219, y $219 + 1 = 220$.

Realice un programa que implemente estas dos tareas:

- a) En primer lugar debe leer dos números naturales e indicar si son o no amigos
- b) A continuación leerá otro número natural, n , e informará si existe algún número amigo de n en el intervalo centrado en n y de radio $n/3$

Finalidad: Familiarizarnos con la definición de funciones y paso de parámetros. Dificultad Media.

13. Reescribid el programa construido para implementar la solución al ejercicio 69 de la *Relación de Problemas II* (contar las apariciones de un dígito dado en un conjunto de enteros que forman un intervalo) modularizando con funciones.

Finalidad: Entender el ámbito de los parámetros de las funciones, así como las llamadas entre funciones. Dificultad Baja.

14. En `cmath` está definida la función `round` que permite redondear un real al entero más próximo. Por ejemplo: `round(3.6)` devuelve 4, `round(3.5)` devuelve 4, `round(3.1)` devuelve 3 y `round(3.49)` devuelve 3.

Nuestro interés es hacer algo similar pero para permitir el redondeo de la parte decimal, especificando el número de cifras decimales que nos interese. Por ejemplo:

- El resultado de redondear 3.49 a la primera cifra decimal es 3.5
- El resultado de redondear 3.49 a la segunda cifra decimal es 3.49
- El resultado de redondear 3.496 a la segunda cifra decimal es 3.5

Defina la función `Redondea` para calcular un número real aproximado a un número de cifras decimales dado.

En el ejercicio 36 de la *Relación de Problemas I* se pedía truncar la parte decimal de un número real usando un número dado de cifras decimales. Por ejemplo:

- El resultado de truncar 3.49 a una cifra decimal es 3.4
- El resultado de truncar 3.496 a la segunda cifra decimal es 3.49
- El resultado de truncar 3.496 sin decimales es 3 (3.0)

RELACIÓN DE PROBLEMAS III. Funciones

Defina la función `Trunca` para truncar un número real usando un número de cifras decimales dado.

En la implementación de estas funciones, necesitará calcular potencias enteras (potencias de 10 en este caso). Para ello, use la función `Potencia` del ejercicio 2.

15. Reescribid el programa construido para implementar la solución al ejercicio 44 de la *Relación de Problemas II* (control de una carrera) modularizando con funciones.

Bastará con modularizar, al menos, los filtros de lectura de la hora, minuto y segundos. Si encuentra alguna otra tarea que pueda modularizarse sensatamente, hágalo.

Finalidad: Entender el ámbito de los parámetros de las funciones. Variables locales. Dificultad Baja.

16. Escriba la función `EliminaUltimos` que elimine todos los caracteres finales de un `string` que sean igual a un carácter determinado.

Por ejemplo, al borrar la `T` del final de la cadena `TabcTdTTTT` se obtendría la cadena `TabcTd`. Si la cadena fuese `TabcTd`, el resultado de borrar la `T` del final de la cadena sería la misma cadena `TabcTd`, ya que no hay ninguna `T` al final.

NOTA: La manera más sencilla de eliminar el último carácter de un dato `string` es ejecutar el método `pop_back()`,

Escriba la función `EliminaPrimeros` que elimine todos los caracteres iniciales de un `string` que sean igual a un carácter determinado.

Por ejemplo, al borrar la `T` del inicio de la cadena `TabcTdTTTT` se obtendría la cadena `abcTdTTTT`. Si la cadena fuese `abcTdTTTT`, el resultado de borrar la `T` del inicio de la cadena sería la misma cadena `abcTdTTTT`, ya que no hay ninguna `T` al principio.

NOTA: Recomendamos emplear un `string` local a la función para copiar a éste los caracteres de las posiciones p , $p + 1$, ... de la cadena original si todos los caracteres de las posiciones 0 , 1 , ..., $p - 1$ son iguales, e iguales al que se quiere borrar. La función devolverá una copia de ese `string` local.

17. Tomando como base las funciones escritas en el ejercicio 16 escriba las funciones:

```
string EliminaUltimosSeparadores (string cad);  
string EliminaPrimerosSeparadores (string cad);
```

que eliminen todos los separadores finales e iniciales, respectivamente, de la cadena `cad`. Devuelven una copia de `cad` después de haber eliminado los separadores.

NOTA: Use la función `isspace` para determinar si un carácter es un separador.

18. Escribir las siguientes funciones para leer números enteros:

```
int LeeEntero (string titulo)
```

Lee y devuelve un dato `int`. Si el valor leído no fuera un `int` (p.e. 94.5, dos, 77 88, 0xA2, 55.0) la función volvería a pedirlo hasta que se proporcione un valor correcto. La lectura se etiqueta con `titulo`.

Por ejemplo, si `x` es `int`, la ejecución de la instrucción `x = LeeEntero("Introduzca la coordenada x: ");` muestra Introduzca la coordenada x: _ y espera a la introducción de un valor `int`. Si el valor introducido no fuera `int` vuelve a pedirlo.

```
int LeeEnteroEnRango (string titulo, int menor, int mayor)
```

Lee y devuelve un dato `int` en un rango dado. Si el valor leído no fuera `int` ó éste fuera menor que `menor` o mayor que `mayor` la función volvería a pedirlo hasta que se proporcionara un valor correcto. La lectura se etiqueta con `titulo`.

```
int LeeEnteroMayorOIgual (string titulo, int referencia)
```

Lee y devuelve un dato `int` que debe ser mayor o igual que `referencia`. Si el valor leído no fuera `int` ó éste fuera menor que `referencia` la función volvería a pedirlo hasta que se proporcionara un valor correcto. La lectura se etiqueta con `titulo`.

19. Escribir las siguientes funciones para leer números reales:

```
double LeeReal (string titulo)
```

Lee y devuelve un dato `double`. Si el valor leído no fuera un `double` (p.e. dos, 77 88, 0xA2, .) la función volvería a pedirlo hasta que se proporcione un valor correcto. No se admiten valores en formato científico (p.e. 1e5). Se admiten valores como .55 ó 22. ó 78. La lectura se etiqueta con `titulo`. El funcionamiento es similar al de la función `LeeEntero` del ejercicio 18 de esta Relación de Problemas.

```
double LeeRealEnRango (string titulo, double menor, double mayor)
```

El funcionamiento es similar al de la función `LeeEnteroEnRango` del ejercicio 18 de esta Relación de Problemas.

```
double LeeRealMayorOIgual (string titulo, double referencia)
```

El funcionamiento es similar al de la función `LeeEnteroMayorOIgual` del ejercicio 18 de esta Relación de Problemas.

20. Escribir dos funciones para convertir un número a su representación textual en forma de `string`. Estas funciones proporcionan una mejora sobre la función `to_string` de C++11. Recuerde, por ejemplo, que `to_string(23.51)` devuelve la cadena `23.510000` mientras que `to_string(23)` devuelve la cadena `23`. La función `to_string` tiene algunos *inconvenientes* para números reales:
- a) Siempre proporciona **seis** cifras decimales, redondeando la última cifra decimal.
 - b) No suprime los ceros finales en la parte decimal de la cadena de caracteres.
 - c) No permite especificar el número de casillas para la parte entera y decimal.

Las funciones que se piden en este ejercicio resuelven estos problemas.

```
string ToStringInt (int entero, int num_casillas, char relleno);
```

La función convierte el dato `entero` a un `string` con `num_casillas` casillas que contiene el valor textual de `entero`. Las casillas no ocupadas por los dígitos de `entero` (si las hubiera) se rellenan con el carácter `relleno`. El número quedará alineado a la derecha (los huecos, a la izquierda).

Con detalle: si el número de casillas solicitado (`num_casillas`) es mayor que el que se necesita para representar el valor de `entero` sin pérdidas, se rellenan las casillas *iniciales* del `string` con el carácter `relleno` (normalmente será el espacio -ASCII 32-). Si el valor de `entero` tuviera "per se" más de `num_casillas` la cadena devuelta tendrá todas las casillas que se necesite (aunque su número sea mayor que `num_casillas`) para no alterar el valor.

Veamos unos ejemplos. Tenga en cuenta que los caracteres `|` se muestran para ayudar a visualizar el resultado, pero no forman parte de él:

```
ToStringInt (15, 6, ' ') devuelve la cadena |      15|
ToStringInt (15, 6, '#') devuelve la cadena |####15|
ToStringInt (155, 2, ' ') devuelve la cadena |155|
```

Finalidad: Familiarizarnos con la definición de funciones y el paso de parámetros.
Dificultad Baja.

```
string ToStringDouble (double real, int num_casillas,
                      int decimales, char relleno);
```

Convierte el dato `real` a un `string` que contiene su valor textual de acuerdo con el formato indicado según los valores de `num_casillas` (número total de casillas que tendrá el resultado), `decimales` (número de casillas para la parte decimal del resultado) y `relleno` (carácter que se usará para las casillas no ocupadas). El número quedará alineado a la derecha (los huecos, si los hubiera, a la izquierda).

RELACIÓN DE PROBLEMAS III. Funciones

- Si el número de casillas solicitado (`num_casillas`) es mayor que el que se necesita realmente, se completa el *inicio* de la parte entera con el carácter `relleno` (normalmente será el espacio -ASCII 32-).
- La cadena resultante **siempre** tendrá decimales dígitos en la parte decimal. Si tiene que quedarse con menos de los que tuviera `real` redondeará la última cifra decimal.
 - a) Si `decimales` es cero no se considera el punto ni la parte decimal, lo que quiere decir que obtendremos un número entero con (`num_casillas`) casillas.
 - b) Si `decimales` es mayor que el que se necesita la parte decimal se completa con *ceros*.

Veamos unos ejemplos. Tenga en cuenta que los caracteres `|` se muestran para ayudar a visualizar el resultado, pero no forman parte de él:

```
ToStringDouble (3.49, 3, 1, ' ') devuelve |3.5|
ToStringDouble (3.49, 4, 2, ' ') devuelve |3.49|
ToStringDouble (3.496, 4, 2, ' ') devuelve |3.50|
ToStringDouble (3.496, 4, 1, ' ') devuelve | 3.5|
ToStringDouble (3.496, 4, 1, '*') devuelve |*3.5|
ToStringDouble (333.9999999, 1, 0, ' ') devuelve |334|
ToStringDouble (3, 4, 0, ' ') devuelve | 3|
ToStringDouble (3.0, 1, 0, ' ') devuelve |3|
ToStringDouble (3.0, 5, 2, '*') devuelve |*3.00|
```

*Finalidad: Familiarizarnos con la definición de funciones y el paso de parámetros.
Dificultad Media.*

21. Escribir un programa que lea un número indeterminado de números reales positivos y muestre su suma. La lectura finaliza si se introduce un número negativo o el cero.

Use la función `LeeReal` del ejercicio 19 para la lectura y la función `ToString` del ejercicio 20 para mostrar los números reales.

*Finalidad: Uso de funciones en programas. Familiarizarnos con el paso de parámetros.
Dificultad Baja.*

RELACIÓN DE PROBLEMAS III. Funciones

22. Escriba una función para dar formato a un `string`, obteniendo como resultado un nuevo `string` de acuerdo al formato indicado en su llamada. La función tendrá el prototipo:

```
string FormateaString (string cad, int num_casillas,  
                      TipoAjuste ajuste, char relleno=' ');
```

Transforma el contenido del dato `cad` a un nuevo `string` de acuerdo al formato indicado. La función devuelve un *nuevo string*.

El nuevo `string` tendrá un mínimo de `num_casillas` casillas. Si el número de casillas solicitado (`num_casillas`) es mayor que el tamaño de `cad` se completan las casillas necesarias con el carácter `relleno`. Si la longitud de `cad` fuera mayor simplemente se devuelve una copia de `cad`.

Respecto al ajuste se usa un valor del tipo `TipoAjuste` para indicar la alineación de la cadena resultado. Use la siguiente definición de tipo enumerado `TipoAjuste`:

```
enum class TipoAjuste {izquierda, centro, derecha};
```

Veamos unos ejemplos. Tenga en cuenta que los caracteres `|` se muestran para ayudar a visualizar el resultado, pero no forman parte de él:

```
FormateaString ("Maria",10,TipoAjuste::derecha, '*')  
devuelve | *****Maria|
```

```
FormateaString ("Maria",10,TipoAjuste::centro, ' ')  
devuelve |   Maria   |
```

```
FormateaString ("Maria",10,TipoAjuste::izquierda, '-')  
devuelve | Maria-----|
```

```
FormateaString ("Maria",3,TipoAjuste::derecha, '*')  
devuelve | Maria|
```

*Finalidad: Familiarizarnos con la definición de funciones y el paso de parámetros.
Dificultad Baja.*

23. Escriba la función `LeeOpcion2Alternativas` que realiza las siguientes tareas:

- a) imprime en pantalla un mensaje
- b) lee una opción de forma que sólo admita los caracteres 's', 'S', 'n' ó 'N'
- c) devuelve la opción escogida. Piense: ¿qué debería devolver la función, el carácter leído o un valor `bool`?

Aplique esta función en la solución del ejercicio 85 de la *Relación de Problemas II* (adivinar un número repetidamente) para volver (o no) a jugar. Modularice con funciones cuantas tareas cree oportunas en ese programa.

Finalidad: Familiarizarnos con la definición de funciones y el paso de parámetros. Dificultad Baja.

24. Cread las siguientes funciones relacionadas con la progresión geométrica que se vió en el ejercicio 73 de la *Relación de Problemas II*.

Analizad cuáles deben ser los parámetros de estas funciones, y qué valor deben devolver. Cread un programa principal que llame a las funciones escritas.

- a) Una función `SumaHasta` que calcule la suma de los primeros k valores de una progresión geométrica.
- b) Cambiemos ahora la implementación de la anterior función `SumaHasta`. En vez de usar un bucle aplicamos la siguiente fórmula, que calcula la sumatoria aplicando únicamente cinco operaciones:

$$\sum_{i=1}^{i=k} a_i = \sum_{i=1}^{i=k} a_1 r^{i-1} = a_1 \frac{r^k - 1}{r - 1}$$

Es muy importante remarcar que `main` no cambia: hemos cambiado la implementación de la función y lo hemos podido hacer sin necesidad de cambiar la función `main`, ya que ésta no tenía acceso al código que hay dentro de la función. Esto es *ocultación de información* (recordar las clases de teoría).

El cálculo de la potencia r^k puede realizarse con la función `pow` o con nuestra función `Potencia` (ejercicio 2).

En cualquier caso, lo importante es que **mientras no cambiemos la cabecera de la función `SumaHasta`, podemos cambiar su implementación sin tener que cambiar ni una línea de código de la función `main`.**

- c) Añadid una función llamada `SumaHastaInfinito` para que calcular la suma hasta infinito, según la fórmula:

$$\sum_{i=1}^{i=\infty} a_i = \frac{a_1}{1 - r}$$

siempre que el valor absoluto de la razón sea menor que 1.

Es evidente que la implementación de la suma no podrá tener *infinitos* términos, por lo que el resultado será, necesariamente, una aproximación. Plantee una solución razonable.

- d) Una función `MultiplícalaHasta` para que multiplique los k primeros elementos de la progresión, aplicando la fórmula:

$$\prod_{i=1}^{i=k} a_i = \prod_{i=1}^{i=k} a_1 r^{i-1} = \sqrt{(a_1 a_k)^k}$$

Finalidad: Enfatizar la importancia de la ocultación de información.. Dificultad Media.

25. Implementar una solución más general que la pedida en el ejercicio 74 de la *Relación de Problemas II* para el cálculo de la sumatoria de los elementos de una serie.

Queremos que se calculen **independientemente** el elemento i -ésimo de la serie y la sumatoria de los primeros k elementos (valor que introduce el usuario) de la serie.

Escribir la función `Termino` para calcular un elemento cualquiera de la serie. Hacedlo de la manera más eficiente posible. Analizad cuáles deben ser los parámetros de esta función, y qué valor debe devolver.

Finalidad: Entender el ámbito de los parámetros de las funciones, así como las llamadas entre funciones. Dificultad Baja.

26. Reescribir el programa construido para implementar la solución al ejercicio 33 de la *Relación de Problemas II* (número triangular) modularizando con funciones.

Finalidad: Entender el ámbito de los parámetros de las funciones, así como las llamadas entre funciones. Dificultad Baja.

27. Reescribir el programa construido para implementar la solución al ejercicio 48 de la *Relación de Problemas II* (multiplicación rusa) modularizando con funciones.

Finalidad: Entender el ámbito de los parámetros de las funciones, así como las llamadas entre funciones. Dificultad Baja.

28. Reescribir el programa construido para implementar la solución al ejercicio 71 de la *Relación de Problemas II* (número perfecto) modularizando con funciones.

Finalidad: Entender el ámbito de los parámetros de las funciones, así como las llamadas entre funciones. Dificultad Baja.

29. En el ejercicio 51 de la *Relación de Problemas II* se calculó la *distribución acumulada (cumulative distribution function)* ó **CDF** de la función gaussiana para un punto cualquiera x . Reescriba ese ejercicio calculando el valor de la **CDF** en un punto x usando una función.

30. Puede obtenerse un valor aproximado de $CDF(x)$ como el dado por la siguiente fórmula (ver, por ejemplo, en el artículo de Wikipedia https://es.wikipedia.org/w/index.php?title=Distribucion_normal la sección *Aproximaciones numéricas de la distribución normal y su función de distribución*)

$$CDF(x) \approx w = 1 - g(|x|)(b_1t + b_2t^2 + b_3t^3 + b_4t^4 + b_5t^5)$$

$$t = \frac{1}{1 + b_0|x|} \quad b_0 = 0,2316419 \quad b_1 = 0,319381530 \quad b_2 = -0,356563782$$

$$b_3 = 1,781477937 \quad b_4 = -1,821255978 \quad b_5 = 1,330274429$$

Finalmente, si $x \geq 0$ entonces $CDF(x) = w$. Si no, $CDF(x) = 1 - w$.

Modifique el ejercicio 29 de esta misma relación de problemas calculando $CDF(x)$ usando una función que realice los cálculos descritos en este ejercicio.

31. En estadística, la regla **68-95-99.7**, conocida también como la **regla empírica** es una manera rápida de recordar el porcentaje de la población que se encuentra en un entorno centrado en la media (si los datos siguen una distribución normal) y con un ancho de una, dos y tres desviaciones estándar respectivamente. De otra manera: el 68.27 %, 95.45 % y el 99.73 % de la población está dentro del entorno centrado en la media y con un ancho de una, dos y tres desviaciones estándar respectivamente.

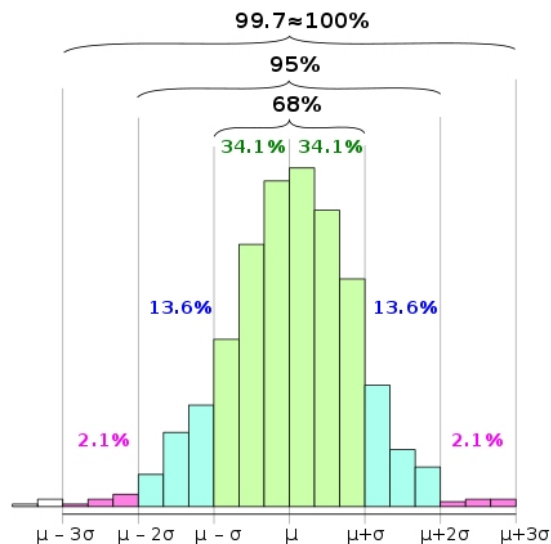


Figura 22: Ilustración de la regla 68 95 99.7
Fuente: Wikipedia: 68 95 99.7 rule

En la figura 22 mostramos los intervalos que contienen el 68.27 %, 95.45 % y el 99.73 % de la población para una distribución gaussiana.

RELACIÓN DE PROBLEMAS III. Funciones

Escribir un programa que lea los parámetros de una distribución gaussiana y a continuación muestre los valores de:

- a) $g(x)$ con la función escrita en el ejercicio 8,
- b) $CDF(x)$ con la función escrita en el ejercicio 29, y
- c) $CDF(x)$ con la función escrita en el ejercicio 30,

para valores de x comprendidos entre $\mu - 3\sigma$ y $\mu + 3\sigma$. Use 0.25 como la diferencia entre dos valores consecutivos de x .

32. Reescribir el programa que calcula la tarifa a pagar en un parking (ver ejercicios 16 y 17 de la *Relación de Problemas II*) modularizando las tareas que estime oportuno.

Finalidad: Familiarizarnos con la definición de funciones y el paso de parámetros, así como las llamadas entre funciones. Dificultad Baja.

33. Evaluemos la bondad de la tres aproximaciones a π sobre las que trabajamos en la *Relación de Problemas II*: 1) Madhava de Sangamagrama (ejercicio 52), 2) Gottfried Leibniz y James Gregory (ejercicio 53), y 3) John Wallis (ejercicio 54)

El programa mostrará un menú con tres opciones:

- 1 **Comparación por número de iteraciones:** El programa pide el valor del número de iteraciones *tope* (entre 1 y 100000, ver enunciados para detalles). A continuación estima el valor de π con los tres métodos de acuerdo al valor de *tope*, calcula sus diferencias con valor *real* de π (tomad como valor a aproximar 3.14159265358979323846) y muestra los valores estimados y las diferencias. Finalmente, indica cuál de los tres métodos da la mejor aproximación.
- 2 **Comparación por calidad de la aproximación:** El programa pide la precisión de la estimación (un número entero entre 1 y 10 que indica el número de decimales coincidentes que se exigen entre el número estimado y el real). A continuación estima el valor de π con los tres métodos de acuerdo al grado de coincidencia indicado y se registra el número de iteraciones que se han necesitado. Se muestran los valores estimados y el número de iteraciones realizadas. Finalmente, indica cuál de los tres métodos da el mejor resultado (menor número de iteraciones).
- 0 **Salir.**

Implementar **tres parejas de funciones** (dos por método) de manera que una de ellas reciba el valor de *tope* y devuelva el valor estimado (para resolver la opción 1) y la otra reciba la aproximación deseada y devuelva el número de iteraciones realizadas (para resolver la opción 2).

34. Se desean realizar diferentes operaciones sobre rectas en un plano.

Una recta r viene determinada por tres coeficientes A , B , C , de forma que todos los puntos (x, y) del plano que pertenecen a la recta verifican:

$$r : Ax + By + C = 0 \text{ (ecuación **general**)}$$

Es más frecuente utilizar la ecuación explícita de la recta, en la que se expresa la abscisa en función de la *pendiente* de la recta (m) y de la *ordenada en origen* (n):

$$r : y = mx + n \text{ (ecuación **explícita**)}$$

de manera que $m = -A/B$ y $n = -C/B$

Escribir un programa que:

- a) Pida los coeficientes de dos rectas,
- b) Muestre los puntos (x, y) por los que pasan las rectas para valores de $x = -10, -9, \dots, 10$ con $\Delta_x = 1$ y para $y = -5, -4, \dots, 5$ con $\Delta_y = 0,5$
Escriba una función para que, dados los coeficientes de una recta y una abscisa, devuelva su ordenada. Escriba otra función para obtener la abscisa a partir de la ordenada.
- c) Escriba la ecuación explícita de las dos rectas
 - i) con una función que escriba directamente la ecuación.
 - ii) con una función que devuelva un `string` con la expresión de la ecuación.En ambos casos deberá calcular la pendiente y la ordenada en el origen. Escriba sendas funciones para estas tareas.
- d) Indique si las rectas son paralelas, secantes o coincidentes (con funciones).

*Finalidad: Familiarizarnos con la definición de funciones y el paso de parámetros.
Dificultad Media.*

35. Escriba la función:

```
string CambiaBase (string val_orig, int base_orig, int base_fin);
```

para cambiar la base de representación de un valor. El dato a transformar es `val_orig` (en base `base_orig`) y se desea transformarlo a base `base_fin`.

Suponga que se van a procesar datos con un máximo de 16 dígitos.

Escriba un programa que lea la base inicial, la base final, el valor a transformar y muestre el resultado de la transformación.

RELACIÓN DE PROBLEMAS III. Funciones

Algunos ejemplos:

base origen	valor origen	base destino	valor destino
10	2054	2	0000100000000110
2	1111001111000001	16	000000000000F3C1
2	1111001111000001	8	0000000000171701
2	0001111	10	0000000000000015
16	CAFE	10	0000000000051966
16	CAFE	2	1100101011111110
2	1100101011111110	10	0000000000051966

Consejo: Para realizar la conversión de base a a base b convierta de base a a base 10 y después de base 10 a base b .

Dificultad Media.

36. El Protocolo de Internet versión 6, en inglés: Internet Protocol version 6 (IPv6), es una versión del Internet Protocol (IP), definida en el RFC 2460 y diseñada para reemplazar a Internet Protocol version 4 (IPv4) RFC 791. Una dirección IPv6 está determinada por 128 bits (posibilita 2^{128} direcciones) frente a IPv4 que trabaja sobre 32 bits (2^{32} direcciones de host diferentes). Las direcciones IPv6, de 128 bits de longitud, se escriben con ocho grupos de cuatro dígitos hexadecimales (cada dígito hexadecimal se representa con 4 bits).

Por ejemplo, 2001:0db8:85a3:08d3:1319:8a2e:0370:7334 es una dirección IPv6 válida.

A grandes rasgos, una dirección IPv6 está formada por dos partes de 64 bits: la primera (prefijo de red) es el identificador de la red y la segunda es el identificador de host. La segunda parte puede ser auto-configurada independientemente para cada host. Un método de autoconfiguración lo proporciona el algoritmo EUI-64. En este ejercicio le pedimos que implemente este algoritmo para generar una dirección de host a partir de su dirección MAC (Media Access Control).

La MAC es un identificador de 48 bits (6 bloques de dos caracteres hexadecimales (4 bits)) que corresponde de forma única a una tarjeta o dispositivo de red. Se la conoce también como dirección física, y es única para cada dispositivo. Por ejemplo,

0a:00:27:00:00:16

El algoritmo EUI-64 consta de dos pasos:

- 1.- Convertir la dirección MAC de 48 bits a un valor de 64 bits. Partimos la dirección MAC en sus dos mitades de 24 bits. Luego se inserta el valor hexadecimal 0xFFFE de 16 bits entre estas dos mitades para formar una dirección de 64 bits.
- 2.- Invertir el bit 7 del primer byte (bit 3 del segundo dígito hexadecimal).

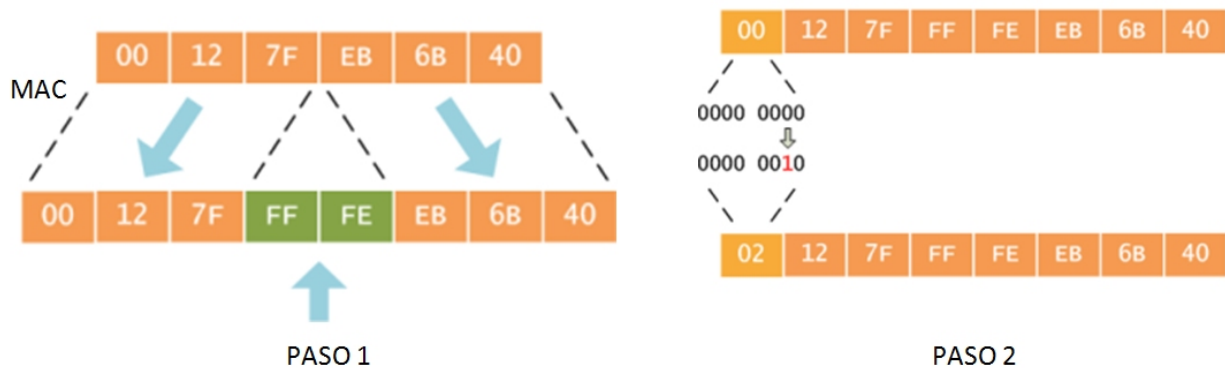


Figura 23: Algoritmo EUI-64

Escribir una función que reciba un string con la dirección MAC del host (12 caracteres hexadecimales) y devuelva un string con la parte de la IPV6 configurada con EUI-64 (16 caracteres hexadecimales).

37. Acerca del DNI, NIF y NIE puede leer un buen resumen en https://es.wikipedia.org/wiki/Número_de_identificación_fiscal

*El Número de Identificación Fiscal (**NIF**) es la manera de identificación tributaria utilizada en España para las personas físicas (con documento nacional de identidad (**DNI**) o número de identificación de extranjero (**NIE**) asignados por el Ministerio del Interior) y las personas jurídicas.*

Escriba una función que verifique la validez de un NIF (de españoles residentes mayores de edad) y del NIE (de extranjeros residentes en España).

En la web referida anteriormente y en <https://www.interior.gob.es/opencms/ca/servicios-al-ciudadano/tramites-y-gestiones/dni/calculo-del-digito-de-control-del-nif-nie/> encontrará las especificaciones técnicas para este cometido.

Escriba un programa que lea un número indefinido de NIFs/NIEs (finaliza cuando de introduzca FIN) y verifique su validez.

Modularice el programa adecuadamente, usando funciones.

Puede comprobar los resultados obtenidos en <http://letradni.appspot.com>

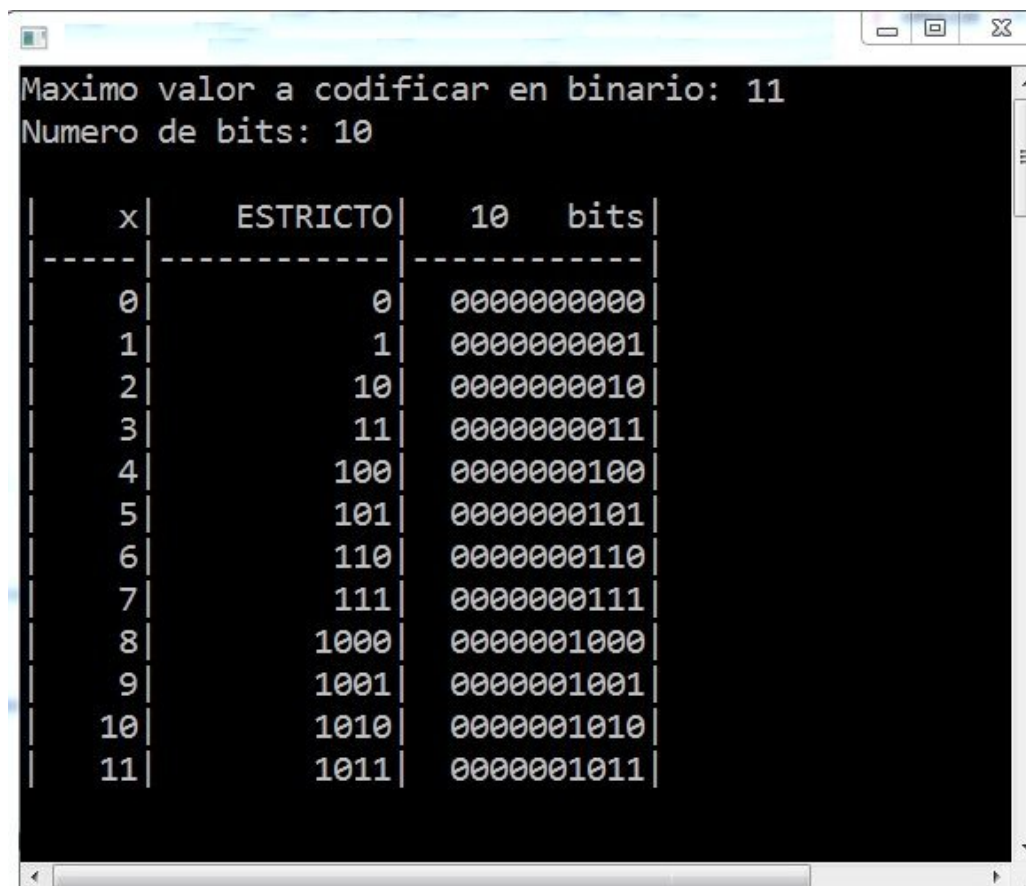
38. En el ejercicio 82 de la *Relación de Problemas II* se describían dos métodos para calcular (de manera aproximada) la raíz cuadrada y se le pedía que implementara un método *propio*.

Encapsule los tres procedimientos de cálculo en **funciones** y reescriba la solución de dicho ejercicio usando las funciones implementadas.

39. En el ejercicio 83 de la *Relación de Problemas II* se calculaba un dato `string` que contenía la representación binaria de un número entero (positivo o cero) usando un número predeterminado de bits.

Escriba un programa que pida un número entero n ($n \geq 0$) y un número de bits t ($t > 0$) y muestre la representación binaria de todos los valores $0 \leq x \leq n$.

El resultado se mostrará en forma de tabla, con tres columnas: x , representación binaria -estricta- de x y representación binaria de x con t bits (ver figura 24).



```
Maximo valor a codificar en binario: 11
Numero de bits: 10
```

x	ESTRICTO	10 bits
0	0	0000000000
1	1	0000000001
2	10	0000000010
3	11	0000000011
4	100	0000000100
5	101	0000000101
6	110	0000000110
7	111	0000000111
8	1000	0000001000
9	1001	0000001001
10	1010	0000001010
11	1011	0000001011

Figura 24: Tabla con la representación binaria de los valores decimales entre 0 y 11 (primera columna), de forma estricta (segunda columna) y con 10 bits (tercera columna)

RELACIÓN DE PROBLEMAS III. Funciones

Esta tarea se va a delegar a una función. Recibirá el número a codificar `num_decimal` y el número de bits `num_bits`. Devolverá un `string` formado por caracteres '0' ó '1' que contiene la representación binaria de `num_decimal`.

```
string DecToBin (int num_decimal, int num_bits);
```

La función proporciona siempre una representación *correcta* por lo que si `num_bits` es menor que el número mínimo de bits que se necesitan, devolverá un `string` con la longitud necesaria. Por ejemplo, $35_{10} = 100011_2$ (6 bits). Las llamadas:

- `DecToBin (35, 8)` devuelve el `string` 00100011. El valor decimal 33 requiere 6 bits, pero al llamar a la función pidiendo 8 bits se completa con dos ceros a la izquierda.
- `DecToBin (35, 4)` devuelve el `string` 100011. El valor decimal 33 requiere 6 bits, pero al llamar a la función pidiendo 4 bits se devuelve la cadena formada por los 6 bits estrictamente necesarios.

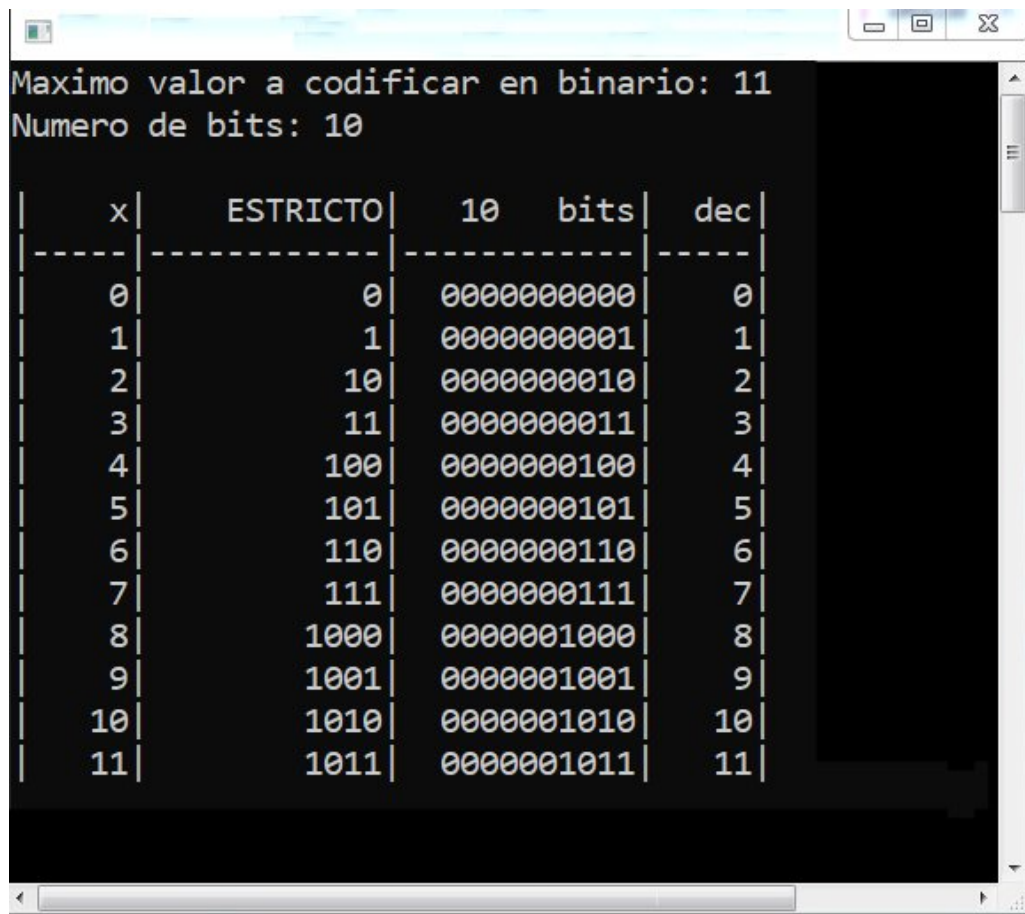
40. Escriba una función que compruebe si un dato `string` contiene la representación binaria de un número entero positivo o cero.

Escriba una función que simplifique la representación binaria de un número entero positivo o cero. La representación binaria se representa en un dato `string`. Se supone que es una representación binaria correcta.

Escriba una función que calcule el valor decimal (`int`) de un valor binario que se suministra en un `string` -se supone que contiene una representación binaria correcta-.

Estas funciones puede usarlas en un programa que tendrá la funcionalidad descrita en el ejercicio 84 de la *Relación de Problemas II*: se pedirá un valor binario (se leerá sobre un dato `string`), se verificará, se simplificará y finalmente se calculará su valor decimal y se mostrará el resultado.

41. Use las funciones implementadas en el ejercicio 40 para verificar los resultados del ejercicio 39 de esta Relación de Problemas.



Maximo valor a codificar en binario: 11
Numero de bits: 10

x	ESTRICTO	10 bits	dec
0	0	0000000000	0
1	1	0000000001	1
2	10	0000000010	2
3	11	0000000011	3
4	100	0000000100	4
5	101	0000000101	5
6	110	0000000110	6
7	111	0000000111	7
8	1000	0000001000	8
9	1001	0000001001	9
10	1010	0000001010	10
11	1011	0000001011	11

Figura 25: Tabla con la representación binaria de los valores decimales entre 0 y 11 (primera columna), de forma estricta (segunda columna), con 10 bits (tercera columna) y después de convertir a decimal el valor binario de la tercera columna.

Use el código de dichos ejercicios y amplíe la tabla añadiendo una nueva columna para comprobar que tomando la representación binaria con t bits, simplificándola y convirtiéndola a decimal, el resultado es igual al valor decimal inicial (ver figura 25).

42. Reescriba la solución del ejercicio 38 de la *Relación de Problemas II* (calcular la máxima velocidad imputada) usando funciones.

Además, para cada vehículo cuya velocidad imputada supere los 120 km/h deberá mostrar la sanción que le corresponde. Escriba una función que devuelva el número de puntos, y otra que devuelva la cuantía económica de la sanción con los datos indicados en el ejercicio 11 de la *Relación de Problemas II*.

Calcule y muestre el número de sanciones que se van a tramitar, el número total de puntos y la media de puntos por sanción. Calcule y muestre la cantidad total de dinero que se va a reclamar y la cuantía media por sanción.

Finalidad: Familiarizarnos con la definición de funciones y el paso de parámetros. Dificultad Baja.

43. Reescriba la solución del ejercicio 47 de la *Relación de Problemas II* usando funciones. En dicho ejercicio se pedía leer un número entero positivo y calcular y mostrar su descomposición en factores primos de dos maneras diferentes.

Escriba ahora dos funciones `void` para *calcular y mostrar* la descomposición en factores primos de `n`, con las cabeceras:

```
void CalculaMuestra_DescFactPrimos_1 (int n);  
void CalculaMuestra_DescFactPrimos_2 (int n);
```

y otras dos para *calcular* la descomposición en factores primos de `n` devolviendo la expresión textual en un `string`. Las cabeceras serán:

```
string Calcula_DescFactPrimos_1 (int n);  
string Calcula_DescFactPrimos_2 (int n);
```

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

Ejercicios sobre *registros*

1. Reescribir el ejercicio 34 de la *Relación de Ejercicios III* usando los tipos de datos

```
struct Punto2D {
    double x, y; // abscisa y ordenada de un punto
};
struct Recta {
    double A, B, C; // Coeficientes de la recta
};
```

para representar un punto un punto y una recta en $\mathbb{R}^2$, respectivamente.

Añadid una función que devuelva el punto de corte entre dos rectas secantes. Escribid en la función `main` el código necesario para probarla.

Añadid una función que indique si un punto pertenece a una recta.

Ampliar la función `main` escribiendo código que lea un número indeterminado de puntos y nos diga si forman parte de la primera recta (finaliza cuando se escriba FIN al leer el valor de la coordenada x).

2. Reutilizad el `struct Punto2D` para representar un punto en $\mathbb{R}^2$.

Definir el tipo de dato `Circunferencia` usando un `struct` de tal manera que para establecer el centro se usará un valor del tipo `Punto2D`.

Escribir funciones para obtener la longitud de la circunferencia y el área del círculo.

Añadir una función que indique si una circunferencia contiene a un punto.

Recordemos que un punto (P_x, P_y) está dentro de una circunferencia con centro (C_x, C_y) y radio r si la distancia Euclídea entre P y C ($d(C, P)$) es menor o igual que r , siendo

$$d(C, P) = \sqrt{(C_x - P_x)^2 + (C_y - P_y)^2}$$

Escribir un programa que lea las coordenadas del centro de una circunferencia y su radio, y a continuación muestre la longitud de la circunferencia y el área del círculo.

Ampliar la función `main` escribiendo código que lea un número indeterminado de puntos y nos diga si están dentro de la circunferencia (finaliza cuando se escriba FIN al leer el valor de la coordenada x).

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

3. Los problemas propuestos en la *Relación de Ejercicios III* relacionados con la función gaussiana tienen el “inconveniente” de que tanto la función que calcula el valor de la gaussiana en x como la que calcula el valor de la CDF en x requieren como argumentos los valores de la *esperanza* y la *desviación típica*.

Cree el tipo de datos *Gaussiana* usando un registro y reescriba las funciones antes mencionadas en base a este nuevo tipo de dato.

Escriba un programa (similar al ejercicio 31 de la Relación de Problemas III) que:

- 1.- lea los parámetros que definen una distribución gaussiana (filtrados como sea conveniente) y los almacene en una dato de tipo *Gaussiana*,
 - 2.- muestre los valores de x , $g(x)$ y $CDF(x)$ para valores de x en el intervalo $[\mu - 3\sigma, \mu + 3\sigma]$, y
 - 3.- pregunte si desea continuar, y de ser así, volver al paso 1.
4. Para modelar un rectángulo se usará el tipo de dato *Rectangulo*. Un rectángulo se caracterizará por la coordenada superior izquierda y la coordenada inferior derecha (ambos son datos de tipo *Punto2D*).

Escribir un programa que lea las coordenadas que definen un rectángulo y calcule la circunferencia centrada en el punto de corte de las diagonales del rectángulo tal que su superficie sea la menor entre todas las circunferencias de área mayor que la del rectángulo.

Use datos de tipo *Rectangulo*, *Circunferencia* y *Punto2D* para la resolución de este problema.

Para el cálculo, considere comenzar con $\text{radio}=0.5$ e ir incrementando su valor 0.25 en cada iteración.

5. Reescribid la solución del ejercicio 42 (calcular la máxima velocidad imputada) de la *Relación de Problemas III* de manera que una sola función devuelva el número de puntos y la cuantía económica de la sanción.

Como en el ejercicio citado, calcule y muestre el número de sanciones que se van a tramitar, el número total de puntos y la media de puntos por sanción. Finalmente, calcule y muestre la cantidad total de dinero que se va a reclamar y la cuantía media por sanción.

6. Reescribid el programa construido para implementar la solución al ejercicio 15 de la *Relación de Problemas III* (control de una carrera, con funciones).

Use datos de tipo *Instante*, para registrar tripletas (*hora*, *minuto*, *segundo*) en la resolución de este problema.

Ejercicios sobre vectores

Todos los ejercicios se resuelven empleando vectores clásicos (*arrays*). No es necesario, ni conveniente, emplear funciones para su resolución ya que el objetivo es trabajar directamente con *arrays*. Por lo tanto, **bastará con escribir el código en la función `main`**.

NOTA: En cualquier caso, podría emplearse alguna función **siempre que no recibiera como argumento un vector**.

7. Declarar un vector de datos `char` para poder guardar todas las letras de la tabla ASCII (minúsculas y mayúsculas, sin caracteres propios del español). Mostrar el resultado.
8. Declarar un vector de datos `char` con TOPE casillas. Pedir que se introduzca por teclado una palabra que tenga, como mucho, TOPE caracteres.

Guardar en el array los caracteres que forman la palabra, invertir el contenido del array (el contenido de la casilla 0 se intercambia por la última útil, el de la segunda por la penúltima, etc.) y mostrar el resultado.

9. Declarar dos *array* de datos `int` con TOPE casillas. Rellenar cada *array* de la siguiente manera: pedir que se introduzca por teclado una serie indeterminada de números enteros (termina cuando se introduce la palabra FIN).

En primer lugar, el programa mostrará el valor medio de los valores introducidos.

Después, indicará si los dos *array* contienen los mismos valores y en el mismo orden.

10. Deberá crear un array de datos Gaussiana (ver ejercicio 3 de la *Relación de Problemas IV*). El array tendrá TOPE casillas. El programa pedirá el número de datos Gaussiana que se van a procesar (n), leerá sus parámetros característicos (media y desviación típica) y los guardará en el array indicado. Así, tendremos una colección $\{g_i : i = 1, 2, \dots, n \text{ donde } g_i \rightsquigarrow N(\mu_i, \sigma_i)\}$

A continuación pedirá que se introduzca un valor de abscisa, x . Después mostrará los valores $g_i(x)$ para $i = 1, 2, \dots, n$

11. Declare dos datos de tipo Gaussiana (ver ejercicio 3 de la *Relación de Problemas IV*) y lea sus parámetros característicos (media y desviación típica)

Deberá calcular $g_1(x)$ para $\mu_1 - 3\sigma_1 \leq x \leq \mu_1 + 3\sigma_1$ y $g_2(x)$ para $\mu_2 - 3\sigma_2 \leq x \leq \mu_2 + 3\sigma_2$ y guardar los valores calculados en sendos *array* de datos `double`.

Previamente debe haber fijado un valor de SALTO entre dos valores consecutivos de x . Como el valor de SALTO está preestablecido, una vez leídos los parámetros de g_1 y g_2 puede saber si dispondrá de suficiente espacio para guardar $g_1(x)$ y $g_2(x)$. Si no es así, terminar. Si dispone de espacio, calcule y guarde los valores de $g_1(x)$ y $g_2(x)$.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

Finalmente, usando los datos guardados en los array, calcule los puntos de corte (si los hubiera) entre las dos funciones, muestre cuántos hay y cuáles son.

NOTA: Optimice el número de cálculos.

12. En la tabla 1 mostramos el **IPC** de España entre 1990 y 2021. IPC es es acrónimo de **Índice de Precios al Consumo**, que el **INE**³ define como *una medida estadística de la evolución de los precios de los bienes y servicios que consume la población residente en viviendas familiares en España. El conjunto de bienes y servicios, que conforman la cesta de la compra, se obtiene básicamente del consumo de las familias y la importancia de cada uno de ellos en el cálculo del IPC está determinada por dicho consumo*⁴.

Año	IPC	Año	IPC	Año	IPC	Año	IPC
2021	6,50 %	2013	0,30 %	2005	3,70 %	1997	2,00 %
2020	-0,50 %	2012	2,90 %	2004	3,20 %	1996	3,20 %
2019	0,80 %	2011	2,40 %	2003	2,60 %	1995	4,30 %
2018	1,20 %	2010	3,00 %	2002	4,00 %	1994	4,30 %
2017	1,10 %	2009	0,80 %	2001	2,70 %	1993	4,90 %
2016	1,60 %	2008	1,40 %	2000	4,00 %	1992	5,30 %
2015	0,00 %	2007	4,20 %	1999	2,90 %	1991	5,50 %
2014	-1,00 %	2006	2,70 %	1998	1,40 %	1990	6,50 %

Cuadro 1: Datos históricos del IPC en España (Fuente: INE)

Queremos conocer el valor actualizado según las variaciones del IPC que debería tener un bien cualquiera conociendo su valor monetario en un año concreto. Por ejemplo, en 2001 una barra de pan costaba 0.30 € y una entrada de cine 3,90 €. En 2021 la barra de pan costaba alrededor de 0.70 € y la entrada de cine 7,78 €. ¿El incremento de precio es razonable?

Escriba un programa que solicite un valor monetario y el año de referencia de ese valor. A continuación solicite un otro año (puede ser anterior al primero). El programa debe mostrar el valor monetario dado actualizado según el IPC en el año indicado.

Repetir hasta que se introduzca un valor monetario negativo.

13. En la tabla 2 mostramos la subida del sueldo de los funcionarios en España desde 2010⁵. Considerando la subida del IPC detallada en la tabla 1 escriba un programa que solicite dos años y calcule la pérdida/ganancia de poder adquisitivo (en porcentaje) de un funcionario entre los dos años dados.

³Instituto Nacional de Estadística

⁴<https://www.ine.es/>

⁵En <https://www.elindependiente.com> publicado el 04/25/2022

Repetir hasta que se introduzca un año negativo.

Año	Subida	Año	Subida
2021	0,90 %	2015	0,00 %
2020	2,00 %	2014	0,00 %
2019	2,50 %	2013	0,00 %
2018	1,75 %	2012	0,00 %
2017	1,00 %	2011	0,00 %
2016	1,00 %	2010	-5,00 %

Cuadro 2: Subida de los sueldos de los funcionarios

14. Considerando las tablas de la subida del IPC (tabla 1) y de la variación de los sueldos de los funcionarios en España desde 2010 (tabla 2) escriba un programa que calcule la ganancia/pérdida absoluta de poder adquisitivo de un funcionario.

El programa leerá el valor de un sueldo y el año de referencia de ese sueldo. A continuación leerá el un segundo año (debe ser posterior). Calcule el salario que le correspondería en ese segundo año si se aplicaran las variaciones del IPC y el salario que le corresponde de acuerdo a las subidas reales aplicadas. Muestre ambos valores, su diferencia y el porcentaje de ganancia/pérdida respecto al salario calculado con las variaciones del IPC.

Repetir hasta que se introduzca un valor monetario negativo.

15. Este ejercicio se basa en el ejercicio 12 de la *Relación de Problemas I*.

Se trataba de calcular el mínimo número de monedas de cada tipo (2 euros, 1 euro, 50 céntimos, 20 céntimos, 10 céntimos, 5 céntimos, 2 céntimos y 1 céntimo) que se entregaban como vuelta después de haber recibido pagado euros (dato `double`) por una compra de un importe de `total` euros (también es un dato `double`).

Ahora hay que considerar las monedas disponibles en la caja, de manera que si no hay monedas de un determinado valor, debe suplirlo usando unidades de valor inferior.

NOTA: Deberá tener en cuenta la contingencia de no disponer de suficiente cambio, por supuesto.

Proponemos usar dos *array*:

- a) *caja*, para guardar las unidades de cada tipo de moneda,
- b) *valor*, para guardar el valor (p.e. en céntimos) de cada tipo de moneda.

de manera que si *valor[i]* es 20 y *caja[i]* es 5 indica que disponemos de 5 monedas de 20 céntimos.

El programa leerá los valores de *total* y *pagado* y mostrará la cantidad de monedas que deben devolverse de cada tipo.

NOTA: Por comodidad, inicialice explícitamente el contenido de *caja*. Por ejemplo:

```
const int TIPOS_MONEDA = 8;
.....
int caja[TIPOS_MONEDA] = {10, 0, 10, 10, 3, 0, 10, 25};
```

ó

```
const int TIPOS_MONEDA = 8;
.....
int caja[TIPOS_MONEDA];
.....
caja[0] = 10; // Monedas de 1 ct.
caja[1] = 0;  // Monedas de 2 ct.
.....
```

16. Vamos a ampliar la funcionalidad de la aplicación desarrollada como solución al ejercicio 39 de la *Relación de Problemas II* (sucursal con mayor número de ventas).

El programa debe leer un número indeterminado de datos de ventas (código de sucursal, código de producto y número de unidades vendidas).

Nota: Recuerde que la lectura de datos finaliza cuando se introduce -1 como código de sucursal.

La empresa dispone de **100 sucursales**, aunque algunas no han vendido ningún producto. Los códigos de los productos son caracteres entre 'a' y 'j'. El programa sólo leerá datos de venta de las sucursales que hayan realizado operaciones de ventas, por lo que no todas las sucursales aparecerán.

No se conoce a priori el número de operaciones de venta, ni el número de sucursales que se van a procesar, ni el código de las sucursales que se van a procesar, aunque sí se sabe que los códigos de éstas son números entre 1 y 100.

Después de leer los datos de ventas el programa mostrará la sucursal que más unidades ha vendido y cuántas unidades.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

A continuación, presentará un listado en el que aparecerán código de sucursal y número total de productos vendidos en la sucursal. En el listado aparecerán únicamente las sucursales que hayan vendido algún producto.

Finalmente mostrará el número total de operaciones de venta, el número de sucursales que hayan vendido algún producto y el número total de unidades vendidas.

17. Escribir un programa que lea un número indeterminado de caracteres (la lectura termina cuando se introduce un `*`) y los guarde en un vector.

El programa nos dirá si los caracteres guardados en el vector forman un **palíndromo**, es decir, que se leen igual de izquierda a derecha que de derecha a izquierda.

Tened en cuenta que los espacios y separadores **no** se tienen en cuenta y que **no** se diferencian mayúsculas y minúsculas.

Por ejemplo, `{ 'a', 'b', 'b', 'a' }` y `{ 'a', ' ', ' ', 'b', ' ', ' ', 'b', 'a' }` serían un palíndromo (realmente el mismo palíndromo).

Nota: Como los separadores también se guardan en el vector **no** podrá emplear el operador `>>` para la lectura de los caracteres. Use el método `get` para la lectura. Su uso es muy sencillo: si `c` es un dato `char`, la instrucción `c = cin.get();` lee el valor de un carácter (puede ser cualquier carácter) de `cin` y lo guarda en `c`.

18. Escribir un programa que lea un número indeterminado de caracteres (la lectura termina cuando se introduce un `*`) y muestre **el número total de apariciones cada letra**.

Después indicará cuántos caracteres (total) se han leído, y de ellos, cuántos son letras y cuántos son separadores

A continuación mostrará cuántas letras no han aparecido ninguna vez, y cuáles son.

Finalmente mostrará cuáles son las letras más y menos frecuentes y el número de veces que aparecen (no tenga en cuenta para el cálculo de la letra menos frecuente aquellas que no aparecen ninguna vez).

Nota: Para los cálculos sobre una letra se consideran iguales la mayúscula y la minúscula correspondiente.

Puede utilizar el texto del Quijote, disponible en PRADO.

19. Escribir un programa que recibe un número indeterminado de caracteres terminado por el carácter `#`. Los caracteres leídos deben ser 0 ó 1 (dígitos binarios) ó cualquier separador. El programa deberá construir caracteres a partir de los dígitos binarios leídos y seleccionar algunos de ellos, copiándolos a un array de `char`.

Cada bloque de 8 dígitos binarios leídos sirve para reconstruir un carácter (un dato `char`). Los dígitos binarios se leen desde el más significativo al menos significativo.

Con el objetivo de minimizar el tiempo de cálculo, para formar un carácter se empleará un array de datos `int` que contiene el peso de cada dígito binario en un byte: 256, 128, 64, ..., 1.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

No todos los caracteres reconstruidos serán de nuestro interés. El programa seleccionará únicamente los caracteres alfabéticos, numéricos, la coma, el punto y el espacio en blanco. Los caracteres seleccionados se copiarán en una array (mensaje) que tendrá un tamaño máximo predeterminado de 200.

Finalmente mostrará el mensaje reconstruido.

Por ejemplo, dada la secuencia:

```
01001000 001 10010 01 111 11000001111 0100111 10011#
```

se genera el mensaje: H20 porque los ocho primeros bits (01001000) forman el número decimal 72 ('H'), los ocho siguientes (00110010) forman el 72 ('2'), los siguientes (01111110) forman el 126 ('~', que no es aceptable), los siguientes (00001111) forman el 15 (no es aceptable), los siguientes (01001111) forman el 79 ('O') y los cuatro últimos (0011) no forman un byte completo.

```
01001000 001 10010 01 111 11000001111 0100111 10011#
----- *** ***** .. ... ^^^^^^^ ----- -****
72 ('H') 52 ('2') 126 ('~') 15 79 ('O') INCOMPLETO
```

20. Supongamos un vector de enteros relativo a un conjunto de alturas de un track GPS. Cada entero representa la altura de la posición GPS en un instante dado. Se quiere calcular lo siguiente:

- **Máximo desnivel:** Se define como la máxima diferencia (en valor absoluto) entre dos alturas consecutivas.
Por ejemplo, si el vector es 1 3 2 4 7 5, las diferencias son +2 -1 +2 +3 -2, siendo 3 la máxima en valor absoluto. Si el vector fuese 1 3 2 4 1 2, las diferencias serían +2 -1 +2 -3 +1, siendo 3 la máxima en valor absoluto.
- **Desnivel acumulado positivo.** La idea es ir buscando puntos consecutivos en los que la altura vaya aumentando. El desnivel acumulado se calcula como la suma de todas esas diferencias positivas. Por ejemplo, si el vector es 1 3 2 4 7 5, las diferencias son +2 -1 +2 +3 -2, por lo que el desnivel acumulado positivo será 7 (la suma de +2, +2 y +3)

Construir un programa que, en primer lugar, lea un número entero que indique el número de valores de altura que se van a introducir. A continuación el programa leerá dichos valores y los almacenará en un vector.

El programa calculará el máximo desnivel y el desnivel acumulado positivo y los imprimirá en pantalla.

21. (*Basado en el Examen de Febrero de 2017*) El sistema (o ley) **D'Hondt** es el método que se utiliza en España para asignar los escaños asignados a una circunscripción electoral.

Se quiere construir un programa que lea el número total de escaños a distribuir, el número de partidos que han participado en las elecciones y los votos obtenidos por cada uno de ellos. El programa mostrará cuántos escaños obtuvo cada partido.

La asignación de los escaños se hace a través de un proceso iterativo en el que en cada iteración se asigna un escaño a un partido y así hasta llegar al número total de escaños a repartir. Se calculan cocientes sucesivos para cada partido político. La fórmula de los cocientes es: $\text{cociente} = IV_i / (S_i + 1)$ donde IV_i es el número total de votos obtenidos por el partido i y S_i es el número de escaños que ha obtenido hasta el momento el partido i .

El número de votos recibidos por cada partido se divide sucesivamente por cada uno de los divisores, desde 1 hasta el número total de escaños a repartir. La asignación de cada escaño se determina hallando el **máximo** de los cocientes y asignando a cada uno un escaño, hasta que estos se agoten.

<https://www.europapress.es/nacional/> $\Rightarrow$
 $\Rightarrow$ noticia-consiste-ley-dhondt-20150519142619.html

Ejemplo. Supongamos unas elecciones a las que se presentan $n = 4$ partidos, entre los que deben repartirse 5 escaños. La asignación de escaños se hace como sigue: Cada escaño se asigna al partido cuyo cociente sea máximo (en el ejemplo, se indica con * y en negrita).

Votos	Partido A $IV_A = 340000$	Partido B $IV_B = 280000$	Partido C $IV_C = 160000$	Partido D $IV_D = 60000$
Escaño 1	(340000/1 =) 340000*	(280000/1 =) 280000	(160000/1 =) 160000	(60000/1 =) 60000
Escaño 2	(340000/2 =) 170000	(280000/1 =) 280000*	(160000/1 =) 160000	(60000/1 =) 60000
Escaño 3	(340000/2 =) 170000*	(280000/2 =) 140000	(160000/1 =) 160000	(60000/1 =) 60000
Escaño 4	(340000/3 =) 113333	(280000/2 =) 140000	(160000/1 =) 160000*	(60000/1 =) 60000
Escaño 5	(340000/3 =) 113333	(280000/2 =) 140000*	(160000/2 =) 80000	(60000/1 =) 60000
E. asignados	2	2	1	0

Sugerencia: represente la información de cada partido con un struct. Por ejemplo:

```
struct Partido {
    char sigla; // "Nombre" del partido
    long votos; // Votos recibidos
    int escagnos; // Escaños conseguidos
};
```

22. Escribir un programa que lea un número indeterminado de números positivos (termina la lectura cuando se introduce un negativo) aunque nunca leerá más de 50. Conforme los va leyendo, los va almacenando en un vector, datos.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

A continuación elimina del vector los valores repetidos, dejando una sola copia. No se dejan huecos en el vector y todos los números quedan agrupados en las posiciones más bajas del vector.

Implementar **tres** versiones del borrado:

- a) Usar un **vector auxiliar** `sin_repetidos` en el que almacenamos una única aparición de cada componente:

```
Copiar la primera componente de "datos" en "sin_repetidos"
Desde la segunda casilla de "datos", hasta la última:
    Si el valor de la casilla NO está en "sin_repetidos",
        añadirla al vector "sin_repetidos"
Sustituir todas las componentes de "datos"
por las de "sin_repetidos"
```

- b) El problema del algoritmo anterior es que usa otro vector, lo que podría suponer una carga importante de memoria si trabajásemos con vectores grandes. Por lo tanto, vamos a resolver el problema sin usar vectores auxiliares.

Si una componente está repetida, **se borrará del vector**. Para borrar la componente de la casilla `p`:

```
Desde la casilla "p" hasta la penúltima:
    Copiar en la casilla "p" el contenido de la "p"+1
```

- c) El anterior algoritmo nos obliga a desplazar muchas componentes. Implementar el algoritmo que usa dos *apuntadores*, `posicion_lectura` y `posicion_escritura`, que nos van indicando, en cada momento, la componente que se está leyendo y el sitio dónde tiene que escribirse.

Por ejemplo, supongamos que en un determinado momento la variable `posicion_lectura` vale 6 y `posicion_escritura` 3. Si la componente en la posición 6 está repetida, simplemente avanzaremos `posicion_lectura`. Por el contrario, no estuviese repetida, la colocaremos en la posición 3 y avanzaremos una posición **ambas** variables.

23. (*Examen Septiembre 2012*) La **criba de Eratóstenes** (Cirene, 276 a. C. Alejandría, 194 a. C.) es un algoritmo que permite hallar todos los números primos menores que un número natural dado n .

El procedimiento “manual” consiste en escribir todos los números naturales comprendidos entre 2 y n y *tachar* los números que *no* son primos de la siguiente manera: el primero (el 2) se declara primo y se tachan todos sus múltiplos; se busca el siguiente número entero que no ha sido tachado, p , se declara primo y se procede a tachar todos sus múltiplos, y así sucesivamente. El proceso de criba termina cuando el cuadrado de p es mayor que n .

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

Si MAX_PRIMO es el mayor número que se va a considerar (establezca su valor a 5000, p.e.) escribir un programa que lea un valor n ($1 \leq n \leq \text{MAX_PRIMO}$) y calcule y muestre todos los primos menores o iguales que n .

Recomendaciones:

- a) Para realizar la criba use el vector `es_primo` de datos `bool` de manera que si `es_primo[i]` es `true` entonces, i es primo.
- b) El resultado (los números primos calculados) se guardarán en el vector `primos`, de datos `int`, almacenados de manera consecutiva, sin huecos. Se establece un tamaño máximo para el vector de MAX_DATOS casillas (piense en un valor sensato para MAX_DATOS basándose en MAX_PRIMO).
- c) Tenga en cuenta las posibles situaciones de error, détéctelas y actúe correctamente.

24. En el ejercicio 47 de la *Relación de Problemas II* se pedía escribir un programa que lea un número entero positivo y muestre su descomposición en factores primos.

Ahora deberá realizar un programa que lea un número indeterminado de enteros positivos (finalizar cuando se introduce un cero) y exprese cada uno de ellos en base al producto de sus factores primos. Exprese el resultado como el producto de potencias (apartado b del ejercicio antes mencionado). Por ejemplo:

```
360 = 2^3 * 3^2 * 5
121 = 11^2
11 = 11
```

Para la solución **deberá considerar únicamente los factores primos**. Para ello deberá calcular (y guardar en un array) los números primos antes de leer ningún número. Use la Criba de Eratóstenes descrita en el ejercicio 23 de esta misma relación de problemas.

Recomendamos usar un struct del tipo:

```
struct Pareja {
    int primo;
    int potencia;
};
```

para guardar cada pareja primo-potencia. Cada una de estas parejas se guardará en una casilla del array `descomposicion`:

```
const int CASILLAS_DESCOMPOSICION = 50;
Pareja descomposicion[CASILLAS_DESCOMPOSICION] = {{0,0}};
int utilizados_descomposicion = 0;
```

25. (*Versión simplificada de Examen Ordinario 2018*) Se dispone de los array de caracteres `grande` y `peque`, y del carácter `a_buscar`. Se quiere saber si `a_buscar` **siempre** viene seguido de la secuencia `peque` en la secuencia `grande`. Tenga en cuenta lo siguiente:

- Cada aparición de `a_buscar` debe ir asociado a una secuencia `peque` **distinta** y ésta ha de estar a la derecha de `a_buscar` (no necesariamente contigua).
- Entre cada carácter y su secuencia asociada, no debe haber más de n caracteres entre ambos. Puede haber n pero no más.

A continuación se muestran algunos ejemplos. Supondremos que el carácter `a_buscar` es `b` y la secuencia `peque` es `TU`. Por simplificar, supondremos que `*` representa la aparición de 0 o más caracteres distintos de `b`, `T` y `U`

- En la secuencia `* b * b * T U`, el carácter `b` no va seguido siempre de `TU`. Se puede apreciar que la primera aparición de `b` sí viene seguida de `TU`, pero no ocurre así con la segunda aparición de `b`.
- En cualquiera de la siguientes secuencias:
`* b * b * T U * T U *`
`* b * T U * b * T U *`
el carácter `b` va seguido siempre de `TU`.

En cualquier caso, también hay que tener en cuenta el número de caracteres que hay entre el carácter y la secuencia. Anunciamos que *entre cada carácter y su secuencia asociada, no debe haber más de n caracteres*.

Por ejemplo, si la secuencia es `a b b T U a a a T U` podemos observar que entre la primera `b` y la primera aparición de `TU` hay 1 carácter y entre la segunda aparición de `b` y la segunda aparición de `TU` hay 5 caracteres. Por lo tanto, si fuese $n = 6$, el resultado sería positivo ya que tanto 1 como 5 son menores o iguales que $n = 6$. Si fuese $n = 5$, el resultado también sería positivo. Pero si fuese $n = 4$, entonces el resultado sería negativo ya que 5 no es menor o igual que $n = 4$.

26. (*Versión simplificada de Examen Febrero 2013*) Se está diseñando un sistema web que recolecta datos personales de un usuario y, en un momento dado, debe sugerirle un nombre de usuario (*login*). Dicho *login* estará basado en el nombre y los apellidos; en concreto estará formado por los n primeros caracteres de cada nombre y apellido (en minúsculas, unidos y sin espacios en blanco). Por ejemplo, si el nombre es Antonio Francisco Molina Ortega y $n = 2$, el nombre de usuario sugerido será `anfrmoor`.

Debe tener en cuenta que el número de palabras que forman el nombre y los apellidos puede ser cualquiera. Además, si n es mayor que alguna de las palabras que aparecen en el nombre, se incluirá la palabra completa. Por ejemplo, si el nombre es Ana CAMPOS de la Blanca y $n = 4$, entonces la sugerencia será anacampdelablan.

Construya un programa que lea el valor de n (int) y el nombre y apellidos a codificar (string). El programa deberá calcular el *login* que le corresponde a la cadena leída de acuerdo al valor de n y componer el resultado en otra variable string.

Sugerencia: para calcular el *login* utilice la función:

```
string Codifica (string cad, int n)
```

donde cad recibirá el nombre y apellidos del usuario y n el número de caracteres deseados. La función devuelve el *login* calculado.

27. En el ejercicio 45 de la *Relación de Problemas II* trabajamos sobre la codificación **RLE** (Run Length Encoding), que codifica una secuencia de datos formada por series de valores idénticos consecutivos como una secuencia de parejas (valor, número de veces que se repite).

En ese ejercicio estábamos obligados a procesar los datos conforme se leían ya que no podíamos almacenarlos. Ahora que conocemos cómo guardar en un array una serie de datos es posible separar claramente las fases de lectura, procesamiento y presentación de resultados.

Escriba un programa que haga, en este orden:

- a) Leer y almacenar en un vector *s* un número indeterminado (no más de 50) de **caracteres**.

La secuencia se lee en una cadena de caracteres (use un dato string). Las cadenas vacías o de longitud mayor que 50 no se consideran.

Un ejemplo de secuencia sería:

```
aaabddeeeeeddssssa
```

Mostrar la secuencia original.

- b) **Codificar** mediante el método RLE el vector *s*. Guardar el resultado de la codificación en otro vector (*s_codificada*).

Mostrar la secuencia codificada.

- c) Usando el vector *s_codificada* **descodificar** la secuencia y almacenar el resultado en un tercer vector (*s_reconstruida*).

Mostrar la secuencia reconstruida.

- d) Comparar las secuencias *s* y *s_reconstruida* e informar del resultado de la comparación.

28. (*Examen Febrero 2016*) Dado un vector de caracteres que contiene un mensaje cifrado, se pide construir otro vector nuevo con el mensaje descifrado y mostrarlo.

La forma de descifrado consiste en coger la primera y última letra de cada palabra. Las palabras están separadas por uno o más espacios en blanco o el final del vector. Si la última palabra no tiene espacios en blanco a su derecha, se coge sólo el primer carácter.

Por ejemplo, si denotamos el inicio y final de la secuencia con un corchete, entonces:

[Hidrógeno limpia] se descodificaría como [Hola]

Reserve memoria para trabajar con un máximo de 1000 caracteres.

Ejemplo de entrada: [Hidrógeno limpia] – Salida correcta: [Hola]

Ejemplo de entrada: [Hidrógeno limpia] – Salida correcta: [Hol]

Ejemplo de entrada: [Hidrógeno] – Salida correcta: [H]

Ejemplo de entrada: [Hidrógeno] – Salida correcta: [Ho]

Ejemplo de entrada: [H] – Salida correcta: [H]

Ejemplo de entrada: [H] – Salida correcta: [H]

29. (*Examen Febrero 2015*) Se dispone de una serie de datos pluviométricos de varios años, en concreto del número de litros que han caído en un día en cada una de las ciudades del mundo. Se quiere calcular los k mayores índices pluviométricos, ordenados de mayor a menor.

Aplique el siguiente algoritmo para calcular el Top- k :

Vector original: "v" (contiene datos de tipo long int)

Vector que contendrá los "k" mayores valores: "topk"

Leer "v" (terminar si el dato leído es cero o negativo)

Copiar "v" en topk

Leer "k"

Ordenar COMPLETAMENTE "topk" de MAYOR a MENOR

(recomendamos el algoritmo de ordenación por inserción)

El programa mostrará los k primeros valores del vector topk.

Por ejemplo, para la secuencia de datos 2.1 0.7 12.4 2.6 3.2 y $k = 2$ mostrará como resultado: 12.4 3.2 mientras que para la misma secuencia, si $k = 22$ mostrará como resultado: 12.4 3.2 2.6 2.1 0.7.

30. Un futbolista chuta una pelota que está en el suelo con una velocidad inicial de v m/s y un ángulo de α grados. Supongamos que la pelota se desplaza en un plano. El punto del que parte la pelota será el punto $(0, 0)$. Indique a qué distancia (eje x) del punto inicial llegará la pelota al suelo, y cual es la altura (eje y) máxima que alcanza.

Obtenga y muestre la secuencia de puntos (x, y) que definen la trayectoria de la pelota. Considere saltos de 0.5 m entre dos puntos consecutivos en el eje x .

Utilice el tipo `struct Punto2D` para representar cada punto del plano por el que se mueve la pelota. Los puntos se van guardando en un *array* conforme se van calculando. Establezca su capacidad con una constante con un valor alto.

31. En este ejercicio se trata de resolver el mismo problema que el planteado en el ejercicio 42 de la *Relación de Problemas II*.

Recordemos que se trataba de calcular la subsecuencia ordenada de mayor longitud en una secuencia de datos numéricos que indicaban la temperatura media diaria de una serie de días consecutivos. El programa indica la posición donde comienza la mayor subsecuencia y su longitud. Por ejemplo, ante la entrada siguiente:

17.2 17.3 16.2 16.4 17.1 19.2 18.9 -1

el programa nos debe indicar que la mayor subsecuencia empieza en la posición 3 (en el 16.2) y tiene longitud 4 (termina en 19.2)

En la versión anterior no se podían guardar los valores leídos por lo que los cálculos se realizaban conforme se iban leyendo los datos. Ahora la secuencia de temperaturas se puede guardar en un array de datos `double` y los cálculos pueden realizarse posteriormente a la lectura. Establecer el tamaño máximo del vector en 100 casillas.

32. Recuperar los tipos de datos `Punto2D`, `Circunferencia` y `Rectangulo` usados en el ejercicio 4 de esta misma Relación de Problemas.

Escribir un programa que lea las coordenadas que definen un rectángulo y guarde en un array una serie de datos de tipo `Circunferencia`, todas centrada en el punto de corte de las diagonales del rectángulo.

Las circunferencias en las que estamos interesadas serán todas las circunferencias **inscritas** en el rectángulo. Para ello comience con una circunferencia de radio $r=0.5$ y vaya incrementando su valor 0.25 en cada iteración.

Muestre cuántas circunferencias se han generado y a continuación, sus propiedades (centro y radio).

Por ejemplo, dado el rectángulo de la figura 26, el array de datos de tipo `Circunferencia` contendrá **tres** casillas con datos válidos (la circunferencia con trazo discontinuo no se guardará porque no está inscrita en el rectángulo).

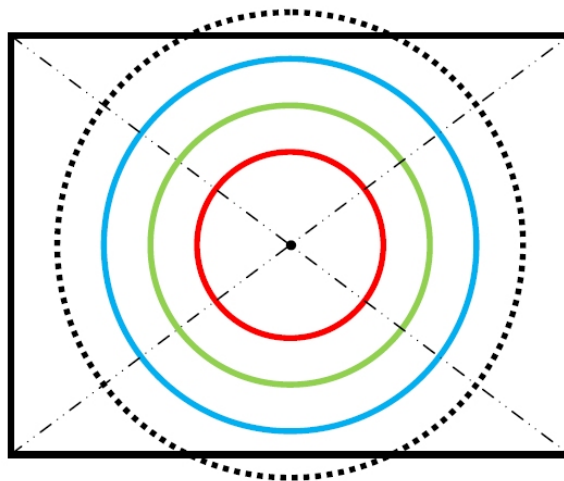


Figura 26: Tres circunferencias inscritas en un rectángulo y una no inscrita

33. (*Examen Ordinario 2019*) Se pretende establecer un nuevo orden para comparar valores enteros positivos. Un valor entero a será mayor que otro b si el número de dígitos nueve es mayor en a que en b . Si el número de nueves es igual en los dos, el mayor será el que tiene más ochos. Si hay empate también con este dígito, se considera el siete. Así hasta llegar al cero, si fuese necesario. Si la frecuencia de todos los dígitos es igual en ambos valores, se les considera iguales, bajo este orden. Se pide lo siguiente:
- Diseñar una función que reciba dos enteros positivos, a y b , y devuelva `true` si a es mayor estricto a b , atendiendo al orden especificado y `false` en caso contrario.
 - Realizar una función `main` que ordene de forma ascendente `v`, un array de `util` enteros siguiendo este nuevo criterio. La ordenación se realizará en el mismo vector `v`.

Suponga declarados

```
const int TAM = 100;
int v[TAM];
int util; // util <= TAM
```

El programa pedirá el valor de `util` (cuántas casillas se van a ocupar) y a continuación rellenará esas casillas con números aleatorios entre 0 y 999.

Por ejemplo, si `v` contiene los valores {5000, 2244, 9000, 99, 550} tras la ordenación quedará {2244, 5000, 550, 9000, 99}.

34. (*Examen Ordinario 2018*)

Dado un vector de enteros v y dado un número t , se quiere ver si hay una serie consecutiva de elementos del vector que sume t . Por ejemplo, si $v = [4, 1, 3, 9, 2]$:

- Si $t = 6$, no hay ninguna secuencia.
- Si $t = 12$, sí hay. Sería la secuencia $[3, 9]$
- Si $t = 14$, sí hay. Sería la secuencia $[3, 9, 2]$

Suponga que ya tiene declarado un vector v en `main` y ha leído sus valores, así como el valor de t . Escriba el código que compruebe lo especificado anteriormente e imprima en pantalla el índice en el que comienza la secuencia (-1 si no existe dicha secuencia).

35. En el ejercicio 37 de la *Relación de Problemas III* escribió una función que verificaba la validez de un NIF ó un NIE.

Escriba un programa que reciba un número indeterminado de palabras (finaliza cuando se lea `FIN`) y seleccione y guarde aquellas que correspondan con un NIF/NIE válido. Deberá guardarlos en un array, indicar cuántos ha encontrado y mostrarlos.

Nota: Tenga en cuenta que es posible que el NIF/NIE aparezca en un texto y en ese contexto puede aparecer seguido del carácter punto (.), coma (,) ó punto y coma (;). Debe considerar estas situaciones y seleccionarlos como válidos.

36. La función del ejercicio 37 de la *Relación de Problemas III* solo verificaba la validez de un NIF ó un NIE por lo que un NIF incorrecto como 12345678L (debería tener asociada la letra Z) lo considera igual que MARÍA.

Escriba una función que indique si una palabra pudiera ser un NIF/NIE válido, esto es, no es un NIF/NIE válido porque la última letra no es la correcta. El resto de la palabra es propia de un NIF/NIE válido.

Escriba un programa que reciba un número indeterminado de palabras (finaliza cuando se lea `FIN`) y seleccione y guarde aquellas que correspondan con un NIF/NIE que pudiera ser válido. Deberá guardarlos en un array, indicar cuántos ha encontrado y mostrarlos. Deberá indicar cual es la letra que le correspondería.

Nota: Tenga en cuenta que es posible que el NIF/NIE aparezca en un texto y en ese contexto puede aparecer seguido del carácter punto (.), coma (,) ó punto y coma (;). Debe considerar estas situaciones y seleccionarlos como válidos.

37. (Examen Ordinario 2020)

Se dispone de un robot que se mueve **en una línea**, utilizando pasos de longitud fija. Inicialmente, el robot se ubica en la posición pos , siendo pos un valor entero positivo $1 \leq pos \leq 100$. Luego, el robot ejecuta una serie de órdenes, indicadas mediante un array ord de tipo `char`, con longitud lon . Cada orden es una letra `I` ó `D`, indicando si el robot se mueve a la izquierda (decrementando la posición actual pos en 1 unidad) o a la derecha (incrementando la posición actual pos en 1 unidad). Las posiciones **válidas** del robot cumplen $1 \leq pos \leq 100$. Se dice que **una serie de órdenes es correcta si el robot nunca se sale de las posiciones válidas**.

Se pide implementar un programa (directamente en la función `main`) para que, dada una posición inicial pos y una lista de órdenes ord de longitud lon , haga lo siguiente:

- Si la serie de órdenes es correcta, muestre cuántas veces se visitó cada posición.
- Si la serie de órdenes NO es correcta, el programa terminará indicando cuántas órdenes se pudieron ejecutar.

Ejemplos

Posición inicial pos: 10 Longitud lon: 6 Órdenes ord: DDIIII Posiciones (válidas) visitadas: 10→11→12→11→10→9→8 Salida del programa Serie de órdenes: correcta Frecuencia de visitas por posición: (8,1), (9,1), (10,2), (11,2), (12,1).	Posición inicial pos: 1 Longitud lon: 4 Órdenes ord: DIID Posiciones (válidas) visitadas: 1→2→1 Salida del programa Serie de órdenes: incorrecta. Se ejecutaron 2 órdenes.
---	---

38. Se quiere elaborar el ránking anual nacional de una prueba de atletismo, Se trata de los 1500 ML en pista (AL). Cada Federación territorial ha recopilado los tiempos oficiales de todas las competiciones celebradas y se han remitido a la Federación nacional que ha elaborado un fichero de texto. En el fichero no hay ningún orden en los datos aunque la información de cada registro (un resultado concreto) ocupa una sola línea. Cada línea tiene la siguiente información: *fecha de evento*, *código de licencia* y *tiempo*.

- La fecha se encuentra en una palabra de longitud 10 con formato `dd/mm/aaaa`
- El código de licencia se encuentra en una palabra de longitud variable (su formato no tiene importancia para este problema).
- El tiempo se encuentra en una palabra de longitud 12 con formato `hh:mm:ss:mmm`

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

El número de registros que se van a procesar se desconoce. Se sabe que no hay más cuando se encuentra la palabra FIN en lugar de una fecha. Por ejemplo:

```
07/05/2021    GR-4567    00:03:45:105
12/04/2021    MA-67    00:03:43:129
    17/10/2021    J-1237    00:03:47:587
01/02/2021    GR-1234    00:04:01:045
05/12/2021    M-11234    00:03:52:345
22/05/2021    B-1092    00:03:52:345
FIN
```

Escribir un programa que elabore el ránking, ordenando los datos de manera creciente de acuerdo al *tiempo*. El resultado debe aparecer bien formateado. Por ejemplo, para los datos dados anteriormente:

Ranking de la prueba: 1500ML

1	MA-67	12 Abril	2021	03:43:129
2	GR-4567	7 Mayo	2021	03:45:105
3	J-1237	17 Octubre	2021	03:47:587
4	M-11234	5 Diciembre	2021	03:52:345
5	B-1092	22 Mayo	2021	03:52:345
6	GR-1234	1 Febrero	2021	04:01:045

Algunas notas prácticas sobre la resolución:

- Para mejorar la capacidad de abstracción y facilitar la resolución recomendamos usar tipos de datos `struct` para caracterizar a las entidades básicas *fecha* (tipo `Fecha`) y *tiempo* (tipo `Tiempo`).
- La tripleta (*fecha*, *licencia*, *tiempo*) caracteriza a la entidad *marca* por lo que también deberían agruparse en un dato `struct` (llámelo `RegistroDeMarca`).
- Las línea del resultado empiezan con un número entero (tres casillas) que indica la posición en el ránking. Puede usar la función `ToString` escrita como solución al ejercicio 20 de la *Relación de Problemas III*.
- El código de licencia ocupa 10 casillas.
- La fecha se presenta en un formato diferente al que se ha leído:
 - Para mostrar el día del mes (dos casillas) use la función `ToString` recientemente mencionada (ejercicio 20 de la *Relación de Problemas III*).
 - se escribe el *nombre* del mes (el número de casillas está determinado por el nombre de mes más largo).
Recomendamos escribir la función `string ToString (Fecha la_fecha);`

- f) El tiempo de carrera se puede presentar de manera diferente a como se lee. En el ejemplo observe que el campo *horas* no se muestra -para una prueba como los 1500ML no procede- y en otros casos como la prueba de 100ML no sería lógico mostrar los minutos (menos aún las horas). Recomendamos escribir una función `ToString` para componer una cadena adecuada al tipo de tiempo que se quiere mostrar. Decida su formato.
- g) La ordenación deberá realizarse en base al *tiempo*. Tenga en cuenta que no podrá comparar dos datos `Tiempo` directamente (con los operadores de relación) por lo que deberá escribir alguna función que compare dos datos de tipo `Tiempo`.

El siguiente código muestra el contenido del array `marcas` después de haber sido ordenado. Puede servirle de referencia.

```
// Array que se usa para guardar los datos leídos y procesados

const int MAX_ATLETAS = 100;
RegistroDeMarca marcas[MAX_ATLETAS];
int utilizados_marcas = 0;

.....

for (int i=0; i<utilizados_marcas; i++) {

    cout << ToString(i+1, 3, ' ')
         << setw(10) << marcas[i].licencia   << "   "
         << ToString(marcas[i].fecha_marca) << "  "
         << ToString(marcas[i].tiempo_marca,
                     CategoriaTiempo::minutos)
         << endl;
}
```

39. En el ejercicio 87 de la *Relación de Problemas II* planteábamos una heurística sencilla para *adivinar* un valor aleatorio comprendido entre MIN y MAX basado en la *búsqueda dicotómica*.

Queremos saber cuántos intentos se realizan para *localizar* cada uno de los valores comprendidos entre MIN y MAX. Para cada valor i ($\text{MIN} \leq i \leq \text{MAX}$) calcule y guarde el número de intentos. A continuación muestre una tabla que indique los valores de i y el número de intentos realizados para localizarlo.

Deberá separar los procesos de cálculo y presentación de resultados.

Nota: no necesita generar números aleatorios.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

40. Una empresa tiene los datos de sus empleados guardados en un fichero de texto. Por simplicidad, cada línea guarda los datos de un único empleado y no hay líneas vacías (en consecuencia, habrá tantas líneas como empleados). Los datos registrados son los siguientes:

- apellidos (string),
- nombre (string),
- fecha de nacimiento (string, en formato dd/mm/aaaa), y
- sueldo bruto (double).

y se encuentran en este orden y separados por el carácter *. Por ejemplo:

```
Martinez Lopez*Juan Carlos*10/12/2000*2345.55*
Abad y Paz de Molina*Maria del Carmen*01/01/1999*3002.88*
Pi Garcia*Jose*05/05/1987*1987.44*
Martin Martinez*Patricia*04/06/1998*1234.56*
```

Debe escribir un programa para mostrar un listado *ordenado* (ascendente) de los empleados. Use **redirección de entrada** para la lectura de los datos. Guarde los datos en un (único) array.

Escriba, en primer lugar, el código para realizar la ordenación por el *sueldo bruto*. El listado deberá presentar los datos de manera similar a ésta:

Martin Martinez	Patricia	04 Jun 1998	1234.56
Pi Garcia	Jose	05 May 1987	1987.44
Martinez Lopez	Juan Carlos	10 Dic 2000	2345.55
Abad y Paz de Molina	Maria del Carmen	01 Ene 1999	3002.88

Si el listado fuera, en cambio, por *apellidos*, el resultado sería:

Abad y Paz de Molina	Maria del Carmen	01 Ene 1999	3002.88
Martin Martinez	Patricia	04 Jun 1998	1234.56
Martinez Lopez	Juan Carlos	10 Dic 2000	2345.55
Pi Garcia	Jose	05 May 1987	1987.44

mientras que si fuera por *fecha de nacimiento*:

Pi Garcia	Jose	05 May 1987	1987.44
Martin Martinez	Patricia	04 Jun 1998	1234.56
Abad y Paz de Molina	Maria del Carmen	01 Ene 1999	3002.88
Martinez Lopez	Juan Carlos	10 Dic 2000	2345.55

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

La ordenación deberá poder realizarse por cualquiera de los campos indicados (salario, apellidos o fecha de nacimiento) aunque la versión que se compilará, tal cual se entregue, será la que ordena por el salario.

El programa, no obstante, debe poder modificarse para obtener la ordenación por los otros criterios con el mínimo número de cambios posibles y evitando la repetición de código.

Realice las pruebas necesarias para asegurar que se realizan correctamente todas las ordenaciones. Cada vez que cambie el criterio de ordenación deberá recompilar el programa, necesariamente⁶. Asegúrese de que los cambios realizados son mínimos y razonables.

41. Reescribir la solución del ejercicio 11 de la *Relación de Problemas II*. En dicho ejercicio se determinaba la sanción a aplicar en una autovía a partir de una tabla donde se indicaba la velocidad del vehículo y la sanción correspondiente (número de puntos del carnet de conducir que se restan y la multa en euros). La información sobre sanciones se guarda ahora en un array.

```
// Tipo enumerado para categorias de riesgo
enum class TipoRiesgo {Ninguno, Bajo, Medio_Bajo,
                      Medio, Medio_Alto, Alto};

// Un struct TramoSancion guarda información de un tramo
// de velocidad que tienen asociado una sanción.
struct TramoSancion {
    int      min_veloc_tramo; // Mínima veloc. del tramo.
    int      puntos;          // Puntos a detraer
    double   sancion;         // Sanción económica.
    TipoRiesgo riesgo;        // Clase de riesgo.
};
.....
const int NUM_TRAMOS = 5;
TramoSancion tramos [NUM_TRAMOS] = {
    {121, 0, 100.0, TipoRiesgo::Bajo},
    {151, 2, 300.0, TipoRiesgo::Medio_Bajo},
    {171, 4, 400.0, TipoRiesgo::Medio},
    {181, 6, 500.0, TipoRiesgo::Medio_Alto},
    {191, 6, 600.0, TipoRiesgo::Alto}
};
```

Reescriba la solución en base a estas declaraciones.

⁶Al menos en esta asignatura y con los conocimientos actuales

Ejercicios sobre *matrices*

Todos los ejercicios se resuelven empleando matrices clásicas. No es necesario, ni conveniente, emplear funciones para su resolución ya que el objetivo es trabajar directamente con los vectores y las matrices. Por lo tanto, bastará con escribir el código en la función `main`. En cualquier caso, podría emplearse alguna función siempre que no recibiera como argumento una matriz.

42. En este problema se propone reescribir otra solución del problema de las sucursales (ejercicio 16 de esta misma relación de problemas). Queremos almacenar en memoria no sólo las ventas totales de cada sucursal (resuelto en el ejercicio indicado anteriormente), sino la información referente a los productos vendidos. En definitiva:

- Total de unidades vendidas de cada producto.
- Producto más vendido.
- Cuántos tipos de producto han sido vendidos (número de tipos de producto con unidades vendidas mayores que cero).
- Número total de unidades vendidas entre todas las sucursales.

Para poder guardar en memoria la información requerida para los cálculos proponemos una matriz bidimensional `ventas` con tantas filas como número (máximo) de sucursales y columnas como número (máximo) de productos. Emplead las constantes:

```
// Num. máximo de sucursales
const int MAX_SUCURSALES_EMPRESA = 100;

const char MAX_PRODUCTO = 'j'; // Cod. máximo de producto
const int NUM_PRODUCTOS = MAX_PRODUCTO - 'a' + 1; // Num. productos

// Todas las ventas se registran en la matriz "ventas":
// * las de la sucursal i [int] en la fila i-1
// * las del producto 'j' [char] en la columna 'j'-'a'

int ventas[MAX_SUCURSALES_EMPRESA][NUM_PRODUCTOS];
```

No se conoce a priori el número de operaciones de venta, ni el número de sucursales que se van a procesar, ni el código de éstas (algunas sucursales puede que no hayan realizado ventas). Se sabe que los códigos de sucursales son números entre 1 y 100.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

Tampoco se conoce a priori los productos que se han vendido, ni el código de éstos (algunos productos puede que no hayan vendido). Se sabe que los códigos de producto son caracteres entre 'a' y 'j'.

Nota: Aconsejamos que una vez leídos los datos, y actualizada la matriz `ventas`:

- a) Usar un vector, `ventas_sucursal`, con tantas casillas como filas tenga la matriz `ventas`. Guardará el número total de unidades vendidas por cada sucursal.
- b) Usar un vector, `ventas_producto`, con tantas casillas como columnas tenga `ventas`. Guardará el número total de unidades vendidas de cada producto.

43. Definir las constantes que especifican el número de filas y columnas de la matriz (su *capacidad*) y leer (filtrando adecuadamente) los valores de dos variables `filas` y `columnas` que contendrán el número de filas y columnas que se utilizarán.

Leer los `filas` $\times$ `columnas` datos de la matriz, y realizar las siguientes tareas:

- a) Calcular la traspuesta de la matriz, guardando el resultado en otra matriz (siempre que sea posible).
- b) Leer los datos de otra matriz (incluidas sus dimensiones) que tenga la misma capacidad y multiplicar ambas matrices. Guardar el resultado en otra matriz (siempre que sea posible efectuar la multiplicación).

44. (*Examen Septiembre 2011*) Definir las constantes que especifican el número de filas y columnas de la matriz (su *capacidad*) y leer (filtrando adecuadamente) los valores de dos variables `filas` y `columnas`.

Leer los `filas` $\times$ `columnas` datos de la matriz, y calcular la posición del elemento que sea el mayor de entre los mínimos de cada fila.

Por ejemplo, dada la matriz M (3×4), el máximo entre 4, 2 y 1 (los mínimos de cada fila) es 4 y se encuentra en la posición (0,2).

9	7	4	5
2	18	2	12
7	9	1	5

45. (*Examen Ordinario 2018*) Una empresa de servicios dispone de n técnicos y n pedidos por atender. La empresa dispone de una matriz $T^{n \times n}$ de tarifas que indica el precio que cobra el técnico i por atender el pedido j . Suponed que $0 \leq T_{ij} \leq 100$

Resolver el problema implica asignar los técnicos a los pedidos. Esta asignación se puede modelizar mediante una matriz de datos `bool` $A^{n \times n}$ donde $A_{ij} = \text{true}$ significa que el técnico i atiende el pedido j , y $A_{ij} = \text{false}$ en caso contrario. Una asignación válida es aquella en la que a cada técnico solo le corresponde un pedido y cada pedido está asignado a un único técnico.

Dada una asignación válida, definimos el coste de dicha asignación como:

$$\sum_{i=1}^n \sum_{j=1}^n A_{ij} \times T_{ij}$$

Ejemplo: si dada la matriz T se realiza la asignación a los técnicos 1, 2 y 3 (en este orden) se obtienen las asignaciones indicadas en A

$$T = \begin{bmatrix} 21 & 12 & 31 \\ 16 & 14 & 25 \\ 12 & 18 & 20 \end{bmatrix} \quad A = \begin{bmatrix} \textit{false} & \textit{true} & \textit{false} \\ \textit{true} & \textit{false} & \textit{false} \\ \textit{false} & \textit{false} & \textit{true} \end{bmatrix}$$

46. Seguimos trabajando sobre la codificación RLE. En esta ocasión se va a codificar una serie indeterminada (no más de 20) de secuencias de caracteres de las características descritas en el problema 27 de esta misma relación de problemas.

Cada secuencia se lee en una cadena de caracteres (use un dato `string`). Las cadenas vacías o de longitud mayor que 50 no se consideran. Para terminar de introducir secuencias bastará con escribir `FIN`.

Por ejemplo, la siguiente entrada proporciona al programa dos secuencias:

aaabddeeeeeddssssa

bbdttttrrrrrr

aaaaaaaaaaaaaaaaaaaaassssssssssssssssssssssssfrrrrrrrrrrrrrrrrrrrr
FIN

(Solo se consideran dos, porque una está *vacía* y otra tiene más de 50 caracteres).

Escriba un programa que haga, en este orden:

- Leer y almacenar en la matriz `m` un número indeterminado (no más de 20) secuencias de (no más de 50) **caracteres** (cada secuencia se guarda en una fila).
Mostrar las secuencias que se van a procesar.
- Codificar** mediante el método RLE.
Guardar el resultado de la codificación en otra matriz (`m_codificada`). Cada secuencia codificada se guarda en una fila.
Mostrar las secuencias codificadas.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

c) Usando la matriz `m_codificada`, **descodificar** las secuencias y almacenar el resultado en una tercera matriz (`m_reconstruida`). Cada secuencia descodificada se guarda en una fila.

Mostrar las secuencias reconstruidas.

d) Comparar las secuencias guardadas en `m` y `m_reconstruida` e informar del resultado de la comparación.

47. En la figura 27 mostramos el **IPC** de varios países entre 2001 y 2021.

Año	España	Inglaterra	USA	Zona Euro
2021	6,50%	5,40%	7,00%	5,00%
2020	-0,50%	0,60%	1,40%	-0,30%
2019	0,80%	1,30%	2,30%	1,30%
2018	1,20%	2,10%	1,90%	1,50%
2017	1,10%	2,90%	2,10%	1,30%
2016	1,60%	1,60%	2,10%	1,10%
2015	0,00%	0,23%	0,70%	0,20%
2014	-1,00%	0,55%	0,80%	-0,20%
2013	0,30%	2,00%	1,50%	0,80%
2012	2,90%	2,71%	1,70%	2,20%
2011	2,40%	4,20%	3,00%	2,80%
2010	3,00%	3,73%	1,50%	2,20%
2009	0,80%	2,83%	2,70%	0,90%
2008	1,40%	3,11%	0,10%	1,60%
2007	4,20%	2,12%	4,10%	3,10%
2006	2,70%	2,97%	2,50%	1,90%
2005	3,70%	1,92%	3,40%	2,20%
2004	3,20%	1,64%	3,30%	2,40%
2003	2,60%	1,25%	1,90%	2,00%
2002	4,00%	1,69%	2,40%	2,30%
2001	2,70%	1,07%	1,60%	2,10%

Figura 27: Variación de la inflación (Fuente: <https://www.invertirenbolso.info/Historico-IPC-Espana/datos-historicos-del-ipc-en-espana.html>)

Escriba un programa que solicite un número n de años ($2 \leq n \leq 5$) y muestre qué país (España, Inglaterra, USA ó Zona Euro) es el que 1) tiene el mayor IPC medio en n años y 2) tiene el menor IPC medio en n años. Muestre además, sus valores y los años entre los que se han calculado los resultados.

Repita hasta introducir un valor de n negativo.

48. (*Examen Extraordinario 2016*) Queremos calcular la frecuencia absoluta de las vocales presentes en un texto y mostrar el resultado en forma de histograma usando caracteres. El histograma resultante deberá tener un aspecto como el de la figura 28.

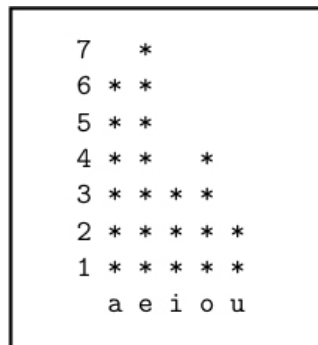


Figura 28: Histograma mostrado con caracteres (número de apariciones de las vocales)

El histograma que nos interesa, tal como se muestra en la figura, se puede mostrar a partir de una matriz de caracteres. El del ejemplo tiene tantas líneas como la frecuencia de la vocal que más se repite (7), y tantas columnas como el número de vocales (5). Se emplea el carácter *asterisco* (ASCII 42) para indicar que la casilla está ocupada y el carácter *espacio* (ASCII 32) para indicar que la casilla está libre.

Escribir un **programa completo** que lea una serie indefinida de caracteres de caracteres de la entrada estándar (terminados por *) y que muestre el histograma que representa la frecuencia *absoluta* de las vocales introducidas.

Suponed que la frecuencia mayor de la vocal que más se repite fuera 500.

49. Vamos a mejorar el ejercicio 48 calculando el histograma para **todas** las letras del alfabeto español*.

Escribir un **programa completo** que lea una serie indefinida de caracteres de caracteres de la entrada estándar (terminados por *) y que muestre el histograma que representa la frecuencia de las letras introducidas.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

Por ejemplo, en la figura 29 mostramos el histograma que muestra las frecuencias de las apariciones de todas las letras del alfabeto español en un texto dado.

Letras y sus frecuencias

a	b	c	d	e	f	g	h	i	j	k	l	m
9	0	6	6	8	1	1	2	8	2	0	5	3
n	o	p	q	r	s	t	u	v	w	x	y	z
4	10	2	2	4	1	4	3	0	0	0	1	0

Histograma

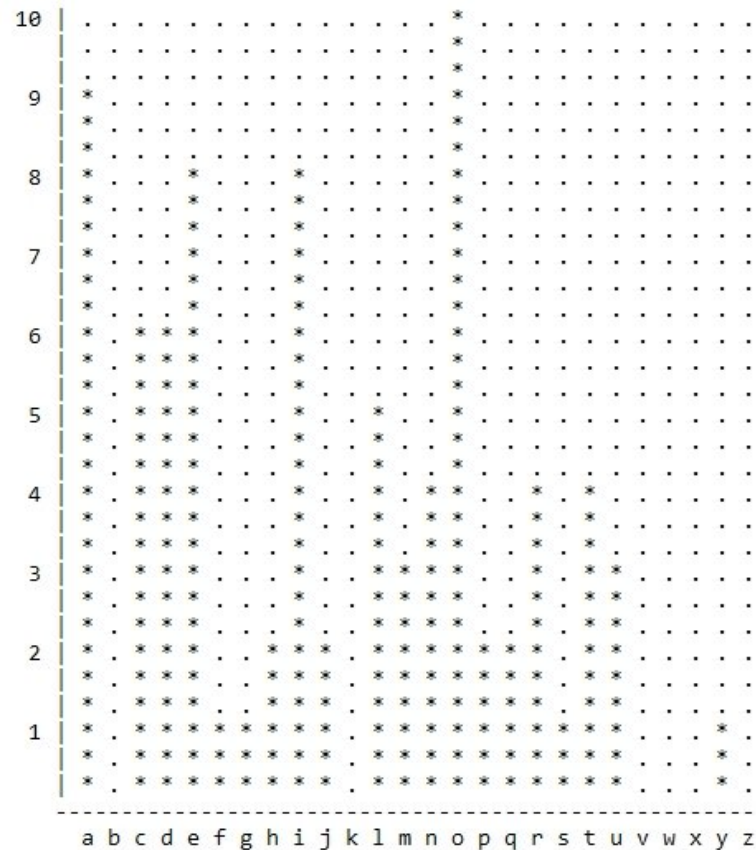


Figura 29: Histograma mostrado con caracteres

Como ayuda, considere las definiciones:

```
// Numero de caracteres a considerar
const int MUM_CARACTERES = 'z'-'a'+1;

char las_letras [MUM_CARACTERES]; // Letras a procesar
int veces_letras[MUM_CARACTERES] = {0}; // Contadores
```

```
// Dimensiones del tablero de dibujo
const int NUM_FILAS = 30; // Núm. de filas
const int NUM_COLS  = MUM_CARACTERES; // Núm. de columnas
char histograma [NUM_FILAS] [NUM_COLS];
```

A la letra más frecuente (en el ejemplo, la 'o' siempre se le asociará una barra con NUM_FILAS (en el ejemplo, 30) caracteres y a las demás letras se les asociará una barra con una altura proporcional a su frecuencia tomando como referencia la de la letra más frecuente.

* Algunas consideraciones prácticas:

- a) No se van a diferenciar mayúsculas de minúsculas. En el ejemplo que les mostramos hemos considerado las letras minúsculas por lo que las mayúsculas se han transformado a minúsculas antes de procesarlas.
- b) Considere las vocales acentuadas junto a sus correspondientes no acentuadas. Por ejemplo, si encuentra el carácter á ó el carácter Á deberá considerarlos como si fueran el carácter a.
- c) Considere al carácter ü (resp.Û) junto a la vocal u.
- d) Considere al carácter ñ (resp. Ñ) junto al carácter n.

La lectura y procesamiento de los caracteres especiales (los específicos del alfabeto español lo son) no se realiza exactamente igual que los demás. Para entender cómo debe realizarse disponen en PRADO de un programa completo (Prueba_de_letras.cpp) para entender cómo deben procesarlos.

50. (*Examen Extraordinario 2018*) Un diligente agente de comercio tiene que visitar periódicamente a sus n clientes. Cada uno de ellos vive en una ciudad distinta. Su jefe le ordena que prepare el plan de viaje según las siguientes reglas:

- a) Tiene que visitar a todos los clientes y una sola vez a cada uno.
- b) El recorrido se hará de forma que el coste total sea el menor posible.
- c) No importa por qué ciudad comience, pero debe finalizar en la misma ciudad en la que comenzó. Por tanto, el recorrido forma un ciclo. Y, obviamente, no puede volver a la ciudad inicial hasta haber visitado todas las demás.

Como datos de entrada se facilitan el número de clientes, n , y el coste de viaje que hay entre cada par de ciudades, expresada como una matriz, C , de tamaño $n \times n$, donde C_{ij} es el coste de viajar desde la ciudad i a la j (dicha matriz no tiene por qué ser simétrica). Supondremos que todos estos valores son positivos, $C_{ij} > 0$ y que se introducen correctamente.

Este es un problema muy difícil de resolver para el que existen distintas soluciones aproximadas. Uno de esos algoritmos, llamado *heurística del vecino más cercano*, funciona así: se selecciona una ciudad de partida, a , y se busca la ciudad más económica para llegar, b . A continuación, se busca la ciudad aún no visitada más económica a b . Y así hasta completar la visita a todas las ciudades. El coste del recorrido es la suma de todos los costes intermedios.

Implemente el algoritmo descrito anteriormente. Construya un programa que lea el valor de n y los valores de la matriz C , así como la ciudad inicial. Luego calculará el recorrido y posteriormente su coste. Finalmente mostrará ambos resultados por la salida estándar.

Por ejemplo, supongamos un problema con $n = 4$ cuya matriz de costes es:

$$C = \begin{bmatrix} - & 10 & 15 & 9 \\ 11 & - & 13 & 8 \\ 17 & 21 & - & 15 \\ 26 & 7 & 14 & - \end{bmatrix}$$

Por simplicidad, asumimos que los nombres de las ciudades son $1, \dots, n$. Si partimos de la ciudad 1 (índice 0), entonces la solución encontrada es: 1, 4, 2 y 3 con un coste de $9 + 7 + 13 + 17 = 46$. Si el viaje comienza por la ciudad 3 (índice 2), el recorrido final sería 3, 4, 2 y 1 con un coste de $15 + 7 + 11 + 15 = 48$.

51. “Dibujar” un círculo y un rectángulo.

En realidad se trata de hacer algo más *simple*, para lo que tengamos herramientas, pero el resultado se asemejará a un dibujo hecho con trazo grueso: si mostramos el contenido de una **matriz de caracteres** (habiendo elegido estratégicamente *los caracteres* y *dónde colocarlos*) podemos obtener resultados muy interesantes. Es lo que se conoce como **ASCII art**. En la figura 30 mostramos unos ejemplos.

El proceso de dibujo consiste en mostrar el contenido de la matriz recorriéndola por filas. Esto significa que la matriz es el tablero de dibujo, en la que la casilla de la fila 0 y columna 0 se corresponde con la esquina superior izquierda del dibujo, que cada “punto” del dibujo es una casilla de la matriz y que cada tipo de “trazo” es un carácter diferente (recuerde el ejercicio 48 de esta misma relación de problemas).

Nuestro propósito es poder hacer un dibujo sencillo:

- tiene un número muy reducido de caracteres: uno para el *fondo* y otro para las *figuras*
- es generado automáticamente, es decir, cada “punto” es dibujado o no según el resultado de una función/test lógico.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

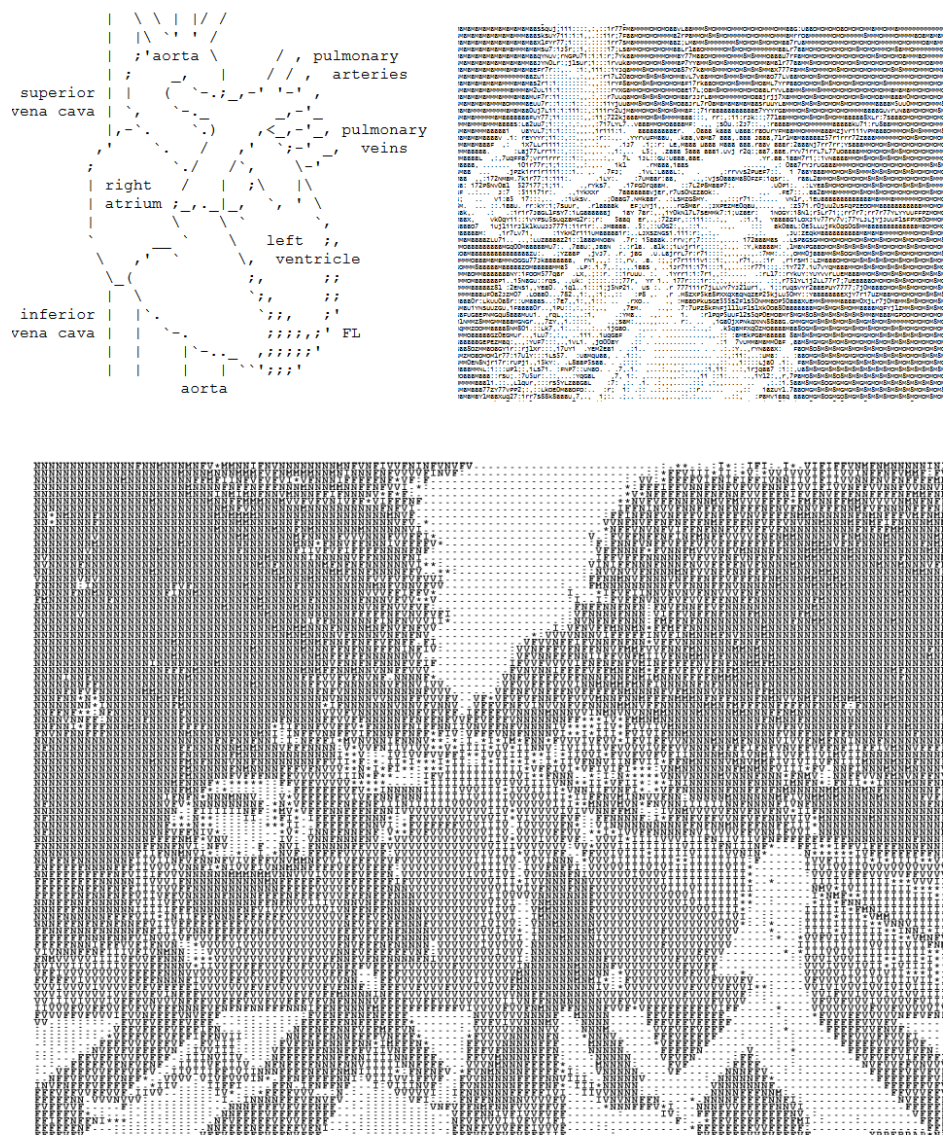


Figura 30: Ejemplos de ASCII art

Para simplificar suponemos que:

- el tablero tiene `NUM_FILAS=51` filas y `NUM_COLS=51` columnas,
- asociamos el tablero al primer cuadrante, esto es, el origen de coordenadas (0, 0) estará asociado a la esquina inferior izquierda del tablero, la casilla `[50][0]`
- El *fondo* se dibuja con el carácter punto ('.'), las figuras con el carácter '*' y los *puntos de intersección entre figuras* con el carácter 'x'.

El programa pedirá el valor del centro de la circunferencia y el radio, y dibujará los puntos que configuran el círculo. A continuación pedirá las coordenadas que definen un rectángulo y dibujará el círculo (de nuevo) y el rectángulo en el mismo tablero.

Recupere las definiciones de los tipos `Circunferencia` y `Rectangulo` del ejercicio 2 y úselos en este ejercicio.

Importante: Es posible que parte del círculo o del rectángulo quede fuera del tablero. Debe dibujar lo que sea posible (los puntos que quedan dentro del tablero: figura 31.B).

En la figura 31 mostramos dos ejemplos. En la figura 31.A aparece un círculo centrado en el punto (25, 15) y con radio 10 y en la figura 31.B un círculo centrado en el punto (5, 5) y con radio 10 y un rectángulo determinado por los puntos (5, 20) -esquina superior izquierda- y (30, 5) -esquina inferior derecha-.

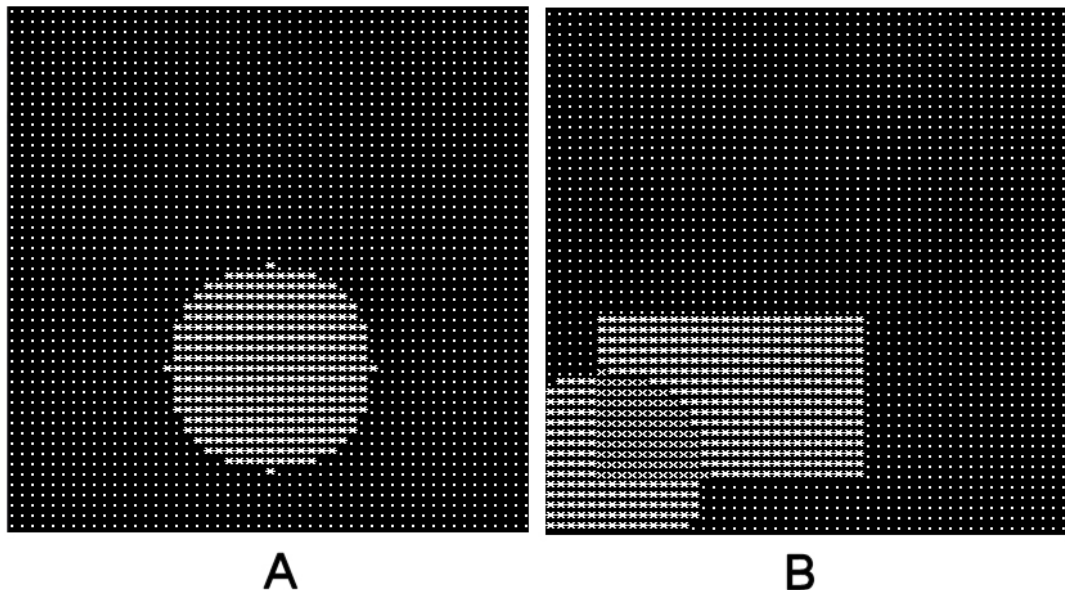


Figura 31: A) Círculo con centro (25, 15) y con radio 10. B) Círculo con centro (5, 5) y radio 10, y rectángulo determinado por los puntos (5, 20) -esquina superior izquierda- y (30, 5) -esquina inferior derecha-

52. Queremos “visualizar” dos gráficos que resuman las temperaturas registradas diariamente: 1) las medias semanales de las mínimas y 2) las medias semanales de las máximas. En la figura 32 mostramos el gráfico de las temperaturas medias mínimas semanales de Granada en 2019. Las medias de las temperaturas máximas se representarán de manera similar.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

Medias de las minimas semanales:

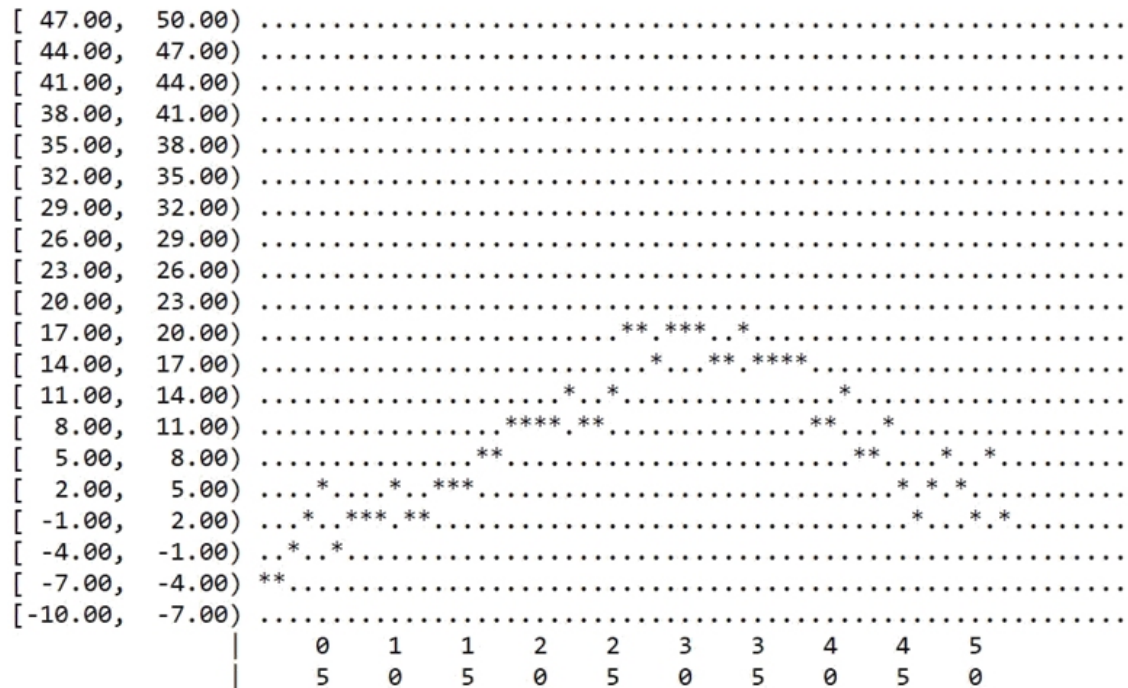


Figura 32: Medias semanales de las temperaturas mínimas de Granada en 2019

Esta representación se ha obtenido a partir de la matriz de caracteres tablero:

```
const int NUM_FILAS = 20; // Número de filas del tablero
const int NUM_COLS = 60; // Número de columnas del tablero

char tablero [NUM_FILAS] [NUM_COLS];

const char TRAZO = '*'; // Caracter del "trazo" (dibujo)
const char HUECO = '.'; // Caracter del "hueco" (no dibujo)
```

Para este ejemplo concreto se ha asociado el valor de temperatura -10.0 a la *primera* fila y el valor 50.0 a la *última* fila. Cada fila de la matriz tiene asociada un intervalo de temperaturas.

Consejo: en primer lugar resuelva el ejercicio sin etiquetar los ejes.

53. Escribir un programa para asignar asientos de pasajeros en un avión. Asumimos un avión pequeño con la numeración de asientos como se indica en la figura 33.A. El programa mostrará con una X el asiento que está ya asignado. Por ejemplo, después de asignar los asientos 1A, 2B, y 4C, lo que se mostrará en pantalla tendrá un aspecto como el que se muestra en la figura 33.B.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

1	A	B	C	D	1	⊗	B	C	D
2	A	B	C	D	2	A	⊗	C	D
3	A	B	C	D	3	A	B	C	D
4	A	B	C	D	4	A	B	⊗	D
5	A	B	C	D	5	A	B	C	D
6	A	B	C	D	6	A	B	C	D
7	A	B	C	D	7	A	B	C	D

Figura 33: (A) Antes de reservar dos asientos. (B) Después

Después de mostrar los asientos disponibles, el programa pregunta por el asiento deseado: el usuario teclea un asiento (número de fila, después la letra) y el programa actualiza la asignación mostrando el esquema anterior. Esto continua hasta que todos los asientos se asignen o hasta que el usuario indique que no quiere asignar más asientos (introduciendo el valor -1 en el número de la fila).

Si el usuario introduce un asiento ya asignado, el programa mostrará un mensaje indicando que el asiento está ocupado y volverá a solicitarlo.

54. Unifique en un único programa las tareas realizadas en los ejercicios 35 y 36 de esta misma relación de problemas, pero guardando los NIFs/NIEs localizados en matrices de datos `char` en lugar de usar arrays de `string`.

Deberá usar dos matrices, una para los NIFs/NIEs correctos y otra para los NIFs/NIEs incorrectos. Cada fila tendrá, en consecuencia, 9 columnas.

55. Se va a dotar de mayor funcionalidad al robot desarrollado en el ejercicio 37 de esta misma Relación de Problemas para que se mueva en un tablero de 10 filas y 10 columnas.

Inicialmente, el robot se ubica en la posición `pos`, siendo `pos` una pareja de enteros positivos (f, c) donde $1 \leq f \leq 10$ y $1 \leq c \leq 10$. Luego, el robot ejecuta una serie de órdenes, indicadas mediante un array `ord` de tipo `char`, con longitud `lon`. Cada orden es una letra 'I', 'D', 'A' o 'B' indicando si el robot se mueve a la izquierda, derecha, arriba o abajo. Las posiciones **válidas** del robot cumplen que para cada posición `pos` sus coordenadas (f, c) son correctas. Se dice que **una serie de órdenes es correcta si el robot nunca se sale de las posiciones válidas**.

Se pide implementar un programa que lea la posición inicial `pos` (fila y columna, por separado) y la secuencia de órdenes a ejecutar. Le recomendamos que use un dato `string`, p.e. `cad_ordenes` para la lectura. Si el string es correcto, el array `ord` se rellenará a partir de `cad_ordenes`, siendo `lon` la longitud de `cad_ordenes`.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

Después de la lectura de los datos haga lo siguiente:

- Si la serie de órdenes es correcta, muestre cuántas veces se visitó cada posición. Use una representación bidimensional (figura 34).
- Si la serie de órdenes NO es correcta, el programa terminará indicando cuántas órdenes se pudieron ejecutar.

En la figura 34 mostramos el resultado de ejecutar el programa empezando por la posición (5, 5) y con la secuencia de órdenes II AAAA DDBDDDBB

```
Posicion inicial = [5, 5]
Ordenes = II AAAA DDBDDDBB
Num. ordenes = 14

-----

La serie de ordenes es : correcta
Se ejecutaron 14 ordenes

      :  1  2  3  4  5  6  7  8  9 10
-----
10 :  .  .  .  .  .  .  .  .  .  : 10
 9 :  .  .  1  1  1  .  .  .  .  :  9
 8 :  .  .  1  .  1  1  1  1  .  :  8
 7 :  .  .  1  .  .  .  .  1  .  :  7
 6 :  .  .  1  .  .  .  .  1  .  :  6
 5 :  .  .  1  1  1  .  .  .  .  :  5
 4 :  .  .  .  .  .  .  .  .  .  :  4
 3 :  .  .  .  .  .  .  .  .  .  :  3
 2 :  .  .  .  .  .  .  .  .  .  :  2
 1 :  .  .  .  .  .  .  .  .  .  :  1
-----
      :  1  2  3  4  5  6  7  8  9 10
```

Figura 34: Casillas recorridas (número de veces) por el robot empezando por la posición (5, 5) y con la secuencia de órdenes II AAAA DDBDDDBB

56. Vamos a seguir trabajando con un robot similar al del ejercicio 55 de esta misma Relación de Problemas. Ahora va a poder moverse por una región rectangular el tablero de NUM_FILAS filas y NUM_COLS columnas.

El objetivo es hacer que recorra todas las casillas del tablero pasando una vez por

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

cada casilla. Empezará por la casilla (1, 1) y desde allí se desplazará, casilla a casilla, hasta cubrir todo el tablero.

El programa solicitará la casilla inicial (fila y columna, por separado) y generará una cadena de órdenes para: 1) mover el robot a la casilla (1, 1) y 2) recorrer todo el tablero.

El programa realizará las tareas descritas en el ejercicio 55 solo que ahora, después de leer la posición inicial, la cadena de órdenes que debe ejecutar el robot no se lee, sino que la genera el propio programa. Deberá ser, evidentemente, correcta.

En la figura 35 mostramos el resultado de ejecutar el programa empezando por la posición (5, 5) en un tablero de 6 filas y 7 columnas. Las primeras 8 órdenes IIIIBBBB tienen como objetivo mover el robot a la casilla (1, 1). Las restantes órdenes son las generadas para recorrer todo el tablero (en este caso **en zig-zag**).

```
Posicion inicial = [5, 5]
Ordenes = IIIIBBBBDDDDDDAIIIIIIADDDDDDAIIIIIIADDDDDDAIIIIII
Num. ordenes = 49
```

La serie de ordenes es : correcta
Se ejecutaron 49 ordenes

```
      :  1  2  3  4  5  6  7
-----
6 :  1  1  1  1  1  1  1 :  6
5 :  2  2  2  2  2  1  1 :  5
4 :  2  1  1  1  1  1  1 :  4
3 :  2  1  1  1  1  1  1 :  3
2 :  2  1  1  1  1  1  1 :  2
1 :  1  1  1  1  1  1  1 :  1
-----
      :  1  2  3  4  5  6  7
```

Figura 35: Casillas recorridas (número de veces) por el robot empezando por la posición (5, 5) y con la secuencia de órdenes generada por el programa.

Reescriba el algoritmo que genera las órdenes para que el recorrido sea **en espiral**, recorriendo los bordes en primer lugar.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

57. *Basado en el Examen Ordinario de Enero de 2020* Para gestionar un campeonato de n equipos se utiliza una matriz cuadrada. En cada casilla se puede almacenar uno de tres posibles valores: '1', 'X' ó '2'. La casilla (f, c) contendrá un valor que representa el resultado del partido entre f y c : si vale '1' indica que ha ganado f , si vale 'X' han empatado y si vale '2' ha ganado c .

Suponga las declaraciones para una liga de 5 equipos:

```
const int NUM_EQUIPOS = 5;  
char liga[NUM_EQUIPOS][NUM_EQUIPOS];
```

Un ejemplo de la matriz `liga` puede verlo en la figura 36.

	0	1	2	3	4	
0		1	1	1	2	
1	2		X	X	1	
2	1	X		1	2	Partido 2 contra 4: Gana 4
3	2	X	1		X	Partido 3 contra 4: Empate
4	1	2	X	2		Partido 4 contra 2: Empate

Liga de 5 equipos:
1: Gana fila
2: Gana columna
X: Empate

Diagonal principal no se usa

Figura 36: Explicación del contenido de la matriz `liga`

Observe que dados dos equipos, f y c , habrá dos partidos: uno en el que se enfrentan f y c (el equipo local es f) y el recíproco, de c contra f (el equipo local es c). Además, el valor de la diagonal no se usa, ya que un equipo no se enfrenta a sí mismo.

Escriba un programa que lea el contenido de la matriz `liga` y después calcule la clasificación final del campeonato.

Los puntos se contabilizan, para cada equipo, como sigue: una victoria implica 3 puntos, un empate 1 punto, y una derrota 0 puntos.

Para este ejemplo, el resultado sería similar a lo que mostramos a la derecha.

PUNTOS			CLASIFICACION		
-----			-----		
EQUIPO	1:	15	EQUIPO	1:	15
EQUIPO	2:	10	EQUIPO	5:	11
EQUIPO	3:	9	EQUIPO	2:	10
EQUIPO	4:	9	EQUIPO	3:	9
EQUIPO	5:	11	EQUIPO	4:	9

58. (*Examen Ordinario 2017*) Se desea construir una matriz de datos `double` para almacenar las distancias de los caminos directos que conectan un conjunto de ciudades. Si entre dos ciudades no existe un camino directo, se almacenará un cero. Se supone que la distancia de una ciudad consigo misma será cero y que las distancias son simétricas. Un ejemplo con 5 ciudades sería el mostrado en la figura 37.

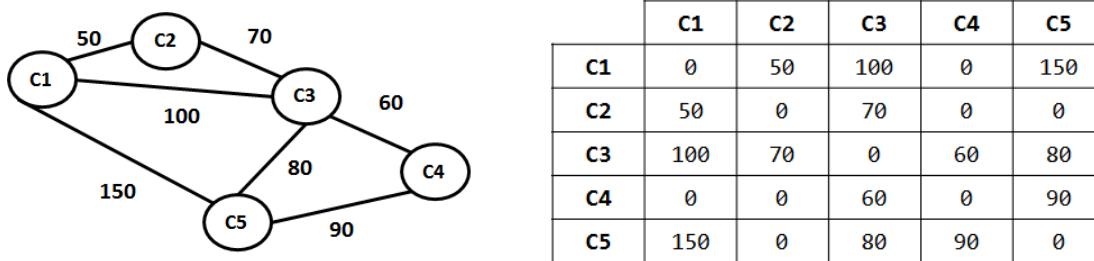


Figura 37: Red de ciudades. Grafo y su representación con una matriz.

Suponga que nunca se trabajará con más de 50 ciudades.

Escriba un programa que lea el número de ciudades de la red, y las distancias entre las ciudades (recuerde que 0 indica que entre dos ciudades no existe un camino directo) y rellene adecuadamente la matriz de distancias.

A continuación, resuelva las siguientes tareas, mostrando el resultando en pantalla:

- Obtener la ciudad (su índice) con mayor número de conexiones directas.
En el ejemplo, sería la ciudad 3 con 4 conexiones.
- Dada una ciudad j , obtener las ciudades conectadas directamente con j .
En el ejemplo, si consideramos la ciudad 4, vemos que está conectada directamente con las ciudades 3 y 5.
- Dadas dos ciudades i y j para las cuales no existe un camino directo, obtener aquella ciudad intermedia z que permita hacer el trayecto entre i y j de la forma más económica posible. Es decir, se trata de encontrar una ciudad z tal que $d(i, z) + d(z, j)$ sea mínima ($d(a, b)$ es la distancia entre las ciudades a y b). Si dicha ciudad no existiera, se indicará expresamente.
Por ejemplo, si se desea viajar desde la ciudad 2 a la 5, hacerlo a través de la ciudad 1 tiene un costo de $50 + 150 = 200$ mientras que si se hace a través de la ciudad 3, el costo sería $70 + 80 = 150$.
- Lea una serie de valores `int` -números de ciudades- e indique si todas las ciudades tienen conexiones directas entre sí. El terminador será -1.

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

59. Se desea ampliar la funcionalidad del programa escrito en el ejercicio 38 de esta misma Relación de Problemas (calcular el ranking en la prueba de 1500ML). En esta ocasión se va a calcular el ranking en varias pruebas seleccionadas anteriormente.

La solución, no obstante, no debería ser muy distinta. Se proponía la ordenación del *array* marcas en base al *tiempo*. Piense en el problema actual como una extensión: *como* si dispusiéramos de *varios array* (uno por prueba) y cada uno contuviera sus marcas, en cantidades posiblemente distintas a los de las otras pruebas.

Concretando, se propone organizar todas las marcas en la *matriz* marcas:

```
const int MAX_PRUEBAS = 20;
const int MAX_MARCAS_EN_PRUEBA = 100;
RegistroDeMarca marcas[MAX_PRUEBAS][MAX_MARCAS_EN_PRUEBA];

// Número de filas ocupadas de "marcas"
int num_pruebas; // PRE: 0 <= num_pruebas <= MAX_PRUEBAS
```

En cada fila se guardarán todas las marcas de la misma prueba. Su número podría ser distinto para diferentes filas (no todas las pruebas tienen el mismo número de marcas). Para guardar el número de casillas ocupadas por fila habrá que usar un array:

```
int num_marcas_en_prueba[MAX_PRUEBAS];
// PRE: 0<=num_marcas_en_prueba[i]<=MAX_MARCAS_EN_PRUEBA
```

de manera que `num_marcas_en_prueba[f]` guardará cuántas marcas de la prueba `f` ($0 \leq f < \text{num_pruebas}$) están guardadas en la matriz `marcas`.

Los datos sobre los que se va a trabajar se leen de la entrada estándar. El formato es muy simple. Por ejemplo:

```
5 100ML 200ML 400ML 800ML 1500ML

1500ML 07/05/2021 GR-4567 00:03:45:105
800ML 22/01/2021 GR-237 00:01:21:147
200ML 05/08/2021 GR-465 00:00:22:122
1500ML 01/02/2021 GR-1234 00:04:01:045
100ML 22/05/2021 B-1092 00:00:12:345
.....
100ML 07/11/2021 SE-123 00:00:12:345
400ML 22/07/2021 GR-237 00:00:43:258
FIN
```

La primera fila: 5 100ML 200ML 400ML 800ML 1500ML indica que se van a considerar 5 pruebas (`num_pruebas` valdrá 5) y las pruebas serán las etiquetadas con las palabras 100ML, 200ML, 400ML, 800ML y 1500ML (el orden no debería ser importante).

RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

Después aparecerán las marcas. El final de la serie de marcas está indicada por la palabra FIN. Observe que la líneas tienen un formato similar al del ejercicio 38 solo que ahora aparece un nuevo elemento al principio: una palabra que identifica la prueba (debe suponer que será una de las etiquetas que aparece en la primera línea).

El programa leerá los datos, los guardará en la matriz *marcas*, ordenará *por filas* los registros de marcas de acuerdo al *tiempo* y presentará el resultado de manera legible. Por ejemplo:

Ranking general.

100ML

1	GR-1234	1 Febrero	2021	00:11:045
2	GR-6789	5 Agosto	2021	00:12:000
3	B-1092	22 Mayo	2021	00:12:345
4	SE-123	7 Noviembre	2021	00:12:345

200ML

1	MA-67	12 Abril	2021	00:21:129
2	GR-465	5 Agosto	2021	00:22:122
3	GR-6789	5 Agosto	2021	00:24:135

400ML

1	MA-1247	15 Junio	2021	00:41:002
2	AL-543	18 Marzo	2021	00:41:098
3	MA-4547	2 Enero	2021	00:41:132
4	AL-47	6 Octubre	2021	00:42:232
5	GR-237	22 Julio	2021	00:43:258

800ML

1	GR-237	22 Enero	2021	01:21:147
2	M-11234	15 Octubre	2021	01:22:001
3	V-35	12 Septiembre	2021	01:24:345

1500ML

1	MA-67	12 Abril	2021	03:43:129
2	GR-4567	7 Mayo	2021	03:45:105
3	J-1237	17 Octubre	2021	03:47:587
4	B-1092	22 Mayo	2021	03:52:345
5	M-11234	5 Diciembre	2021	03:52:345
6	GR-1234	1 Febrero	2021	04:01:045

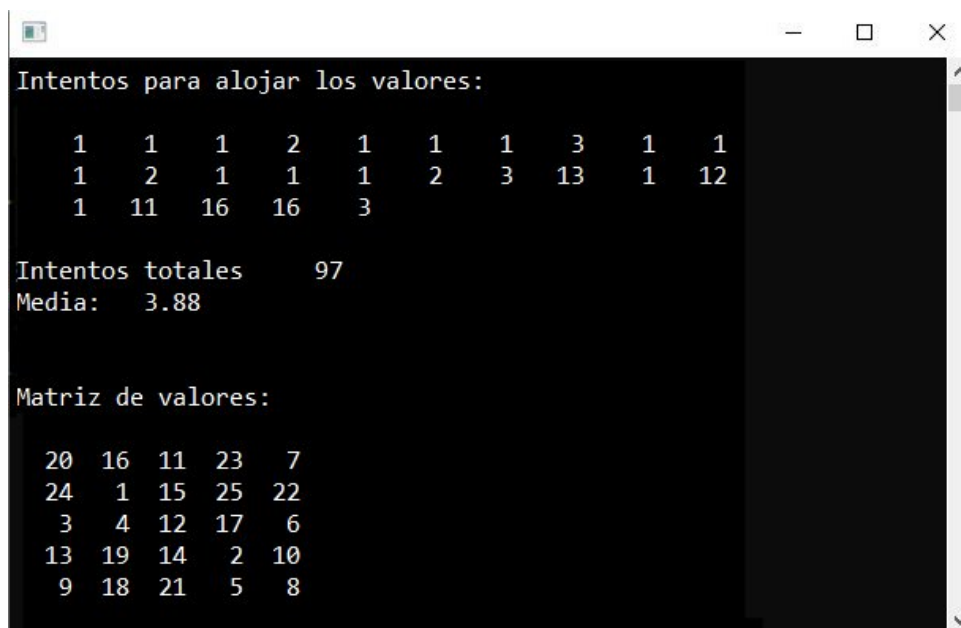
60. Suponga una matriz que puede guardar datos enteros cuyas dimensiones son NUM_FILAS y NUM_COLS columnas. Queremos rellenar completamente la matriz con todos los valores comprendidos entre 1 y NUM_FILAS*NUM_COLS pero colocándolos en posiciones aleatorias. También queremos saber cuántos intentos hemos necesitado para cada valor, el total de intentos para llenar la matriz y cuál es la media de intentos por valor.

El algoritmo para llenar la matriz será el siguiente:

- Para cada valor a colocar en la matriz, genere una posición aleatoria de la matriz (genere un número de fila y un número de columna).
- Si la casilla está libre, coloque ahí el valor, y si está ocupada vuelva a generar otra posición aleatoria.

El programa mostrará en primer lugar el número de intentos para alojar cada valor (NUM_DATOS_LINEA datos por línea), a continuación el número total de intentos y el valor medio. Finalmente mostrará la matriz de valores.

En la figura 38 mostramos un ejemplo usando una matriz de 5 filas y 5 columnas.



```
Intentos para alojar los valores:
  1  1  1  2  1  1  1  3  1  1
  1  2  1  1  1  2  3 13  1 12
  1 11 16 16  3

Intentos totales      97
Media:      3.88

Matriz de valores:
20 16 11 23  7
24  1 15 25 22
 3  4 12 17  6
13 19 14  2 10
 9 18 21  5  8
```

Figura 38: Ejemplo de ejecución usando una matriz de 5 filas y 5 columnas

61. Suponga una matriz que puede guardar datos enteros cuyas dimensiones son NUM_FILAS y NUM_COLS columnas. Queremos rellenar completamente la matriz con valores aleatorios comprendidos entre MIN_VALOR y MAX_VALOR. Tenga en cuenta que un mismo valor podía aparecer varias veces en la matriz y que algunos valores podrían no aparecer.

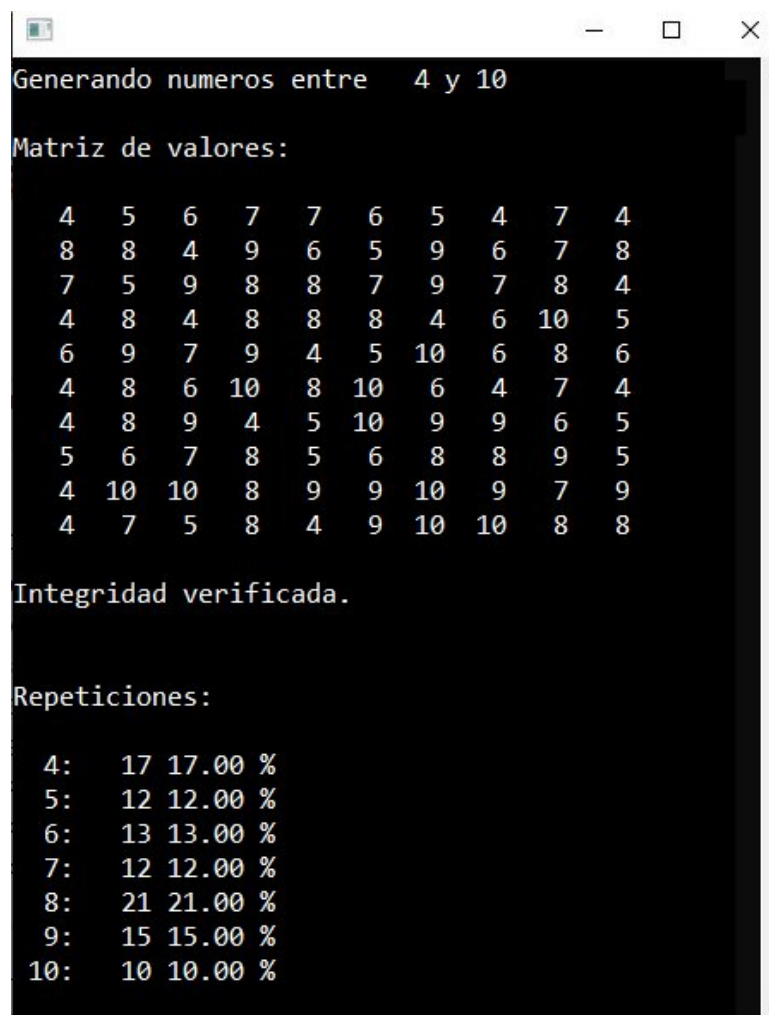
RELACIÓN DE PROBLEMAS IV. Registros, vectores y matrices

Queremos calcular cuántas veces aparece en la matriz cada uno de los valores entre MIN_VALOR y MAX_VALOR y su frecuencia relativa (expresada como porcentaje).

Escriba un programa que rellene la matriz y la muestre. Después calculará cuántas veces aparece cada valor y realizará una prueba de integridad (mostrando el resultado). La prueba consiste en comprobar si la suma de las apariciones de los posibles valores que se pueden generar coincide con la cantidad de valores de la matriz.

Finalmente mostrará para cada uno de los potenciales valores que pueden generarse cuántas veces aparece y en qué porcentaje sobre el total.

En la figura 39 puede ver un ejemplo de ejecución usando una matriz de 10 filas y 10 columnas rellena con números aleatorios entre 4 y 10.



```
Generando numeros entre 4 y 10

Matriz de valores:

 4  5  6  7  7  6  5  4  7  4
 8  8  4  9  6  5  9  6  7  8
 7  5  9  8  8  7  9  7  8  4
 4  8  4  8  8  8  4  6 10  5
 6  9  7  9  4  5 10  6  8  6
 4  8  6 10  8 10  6  4  7  4
 4  8  9  4  5 10  9  9  6  5
 5  6  7  8  5  6  8  8  9  5
 4 10 10  8  9  9 10  9  7  9
 4  7  5  8  4  9 10 10  8  8

Integridad verificada.

Repeticiones:

4:  17 17.00 %
5:  12 12.00 %
6:  13 13.00 %
7:  12 12.00 %
8:  21 21.00 %
9:  15 15.00 %
10: 10 10.00 %
```

Figura 39: Ejecución con una matriz 10×10 rellena con números aleatorios entre 4 y 10

Se valorará el uso correcto y adecuado de las matrices y vectores así como la eficiencia en tiempo y espacio.

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

Ejercicios básicos sobre clases

1. En este ejercicio se plantean varias modificaciones. Debe entregar un fichero cpp por cada uno de los apartados.

Se desea implementar una clase `Recta` para representar una recta en el plano. Una recta viene determinada por tres coeficientes `A`, `B`, `C`, de forma que todos los puntos (x,y) que pertenecen a la recta verifican lo siguiente (ecuación general de la recta):

$$Ax + By + C = 0$$

Deberá tener en cuenta que `A` y `B` no pueden ser simultáneamente 0.

a) Definición de la clase y creación de objetos

Defina la clase `Recta`. En este apartado utilice únicamente datos miembro públicos. Cree un programa principal que haga lo siguiente:

- Defina dos objetos de la clase `Recta`.
- Lea seis reales desde teclado.
- Le asigne los tres primeros a los coeficientes de una recta y los otros tres a la segunda recta.
- Calcule e imprima la pendiente de cada recta aplicando la fórmula:

$$\text{pendiente} = -A / B$$

b) Métodos públicos

En vez de calcular la pendiente en el programa principal, vamos a ponerlo como un método de la clase y así lo reutilizaremos todas las veces que necesitemos. Añada un método para el cálculo de la pendiente y modifique el `main` para tener en cuenta este cambio. ¿Añadimos `pendiente` como dato miembro de la recta? La respuesta es que no ¿Por qué? Añadir también los siguientes métodos:

- Método `ToString` para obtener una representación textual (en un `string`) de la ecuación general de la recta.
- Obtener la ordenada (`y`) dado un valor de abscisa `x`, aplicando la fórmula:
$$(-C - xA) / B$$
- Obtener la abscisa (`x`) dado un valor de ordenada `y`, aplicando la fórmula:
$$(-C - yB) / A$$

En la función `main`, después de reescribir el código de la primera versión usando los métodos propuestos ahora, leed un valor de abscisa e imprimir la ordenada según la recta y leed un valor de ordenada e imprimir la abscisa que le corresponde. Hacedlo sólo con la primera recta.

c) *Datos miembro privados*

Cambie ahora los datos miembro públicos y póngalos privados. Tendrá que añadir métodos para asignar y ver los valores de los datos miembro. Añada métodos para asignar un valor a cada uno de los tres datos miembro. Modifique el `main` para tener en cuenta estos cambios.

A partir de ahora, todos los ejercicios deben resolverse utilizando únicamente datos miembro privados.



d) *Política de acceso a los datos miembros*

En vez de usar un método para asignar un valor a cada dato miembro, defina un único método `SetCoeficientes` para asignar los tres a la misma vez. Observe que los métodos permiten definir la política de acceso a los datos miembro. Si tengo previsto cambiar por separado los coeficientes de la recta, usaré métodos de asignación individuales. En caso contrario, usaré un único método que modifique a la misma vez todos los datos miembro. Incluso pueden dejarse en la clase ambos tipos de métodos para que así el cliente de la clase pueda usar los que estime oportunos en cada momento. Por ahora, mantenga únicamente el método de asignación *en bloque* `SetCoeficientes`.

e) *Constructor*

Modifique el programa principal del último apartado e imprima los valores de los datos miembros de una recta, **antes** de asignarles los coeficientes. Mostrará, obviamente, un valor indeterminado. Para evitar este problema, añada un constructor a la recta para que el objeto esté en un estado válido en el mismo momento de su definición. El constructor deberá tener como parámetros, obligatoriamente, los tres coeficientes de la recta. Tendrá que modificar convenientemente el `main` para tener en cuenta este cambio.

f) *Política de acceso a los datos miembro*

Suprima ahora el método `SetCoeficientes`. De esta forma, una vez creado el objeto (pasándole los datos apropiados en el constructor) ya no podremos modificar los datos miembro. Esto es útil en aquellas situaciones en las que no queremos permitir que el estado del objeto cambie, una vez se ha creado.

g) *Métodos privados*

Añada el método *privado* `Simplifica` para simplificar una recta. El método modificará los coeficientes de la recta dividiéndolos por el máximo común divisor de los tres coeficientes. La recta obtenida será una recta equivalente a la original.

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

Calcule el máximo común divisor con la función `int MCD (int a, int b)` que calcula el máximo común divisor de `a` y `b`. Esta función ¿será pública o privada? ¿Por qué? ¿Qué método(s) llamará(n) a `Simplifica`? ¿Tiene sentido que sea público?

Finalidad: Trabajar con los conceptos básicos de clases. Dificultad Baja.

2. En este ejercicio se va a emplear la la clase `Cronometro`, que sirve para medir el tiempo de ejecución de un conjunto de instrucciones. Sus métodos públicos son:

```
+ void Reset()
+ double NanoSegundosTranscurridos()
+ double MiliSegundosTranscurridos()
```

Puede descargar el código completo de PRADO (sección: **Clases de interés**). El código incluye la explicación de cómo usar la clase.

Para realizar una prueba de esta clase, use el programa que propocionamos (`VI_SumatorioFactoriales_ConFunciones.cpp`) donde reescribimos la solución al ejercicio 70 de la *Relación de Problemas II* usando dos funciones:

- `long SumaFactorial_Directo (int tope)`
- `long SumaFactorial_Optimizado (int tope)`

Queremos medir el tiempo (con la clase `Cronometro`) que necesita cada función para realizar su trabajo y compararlos. Para ello usaremos el mismo valor de `tope`.

Como el código original:

```
suma1 = SumaFactorial_Directo(tope);
.....
suma2 = SumaFactorial_Optimizado(tope);
```

se va a ejecutar muy rápidamente, si queremos medir y comparar el tiempo, deberíamos repetir la ejecución de cada función un número alto de veces. Así, proponemos cambiar las instrucciones originales por:

```
for (int i=0; i < VECES; i++)
    suma1 = SumaFactorial_Directo(tope);
.....
for (int i=0; i < VECES; i++)
    suma2 = SumaFactorial_Optimizado(tope);
```

donde `VECES` es un número *muy grande*, p.e., `const double VECES = 2e+7;`

Finalidad: Enfatizar la importancia de la ocultación de información y de la interfaz pública de una clase. Dificultad Baja.

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

3. Recuperad la solución del ejercicio 49 de la *Relación de Problemas II* (acertar un número siguiendo las pistas del programa). En este caso, el número aleatorio que genera el programa para ser adivinado se calcula con la clase `GeneradorAleatorioEnteros`. No hace falta que entienda el código de la clase sino únicamente cómo utilizar sus métodos públicos.

La clase se usa para generar números aleatorios enteros en un rango de valores.

Los métodos públicos de la clase `GeneradorAleatorioEnteros` son:

```
+ GeneradorAleatorioEnteros()  
+ GeneradorAleatorioEnteros(int min, int max)  
+ int Siguiente()
```

Puede descargar el código completo de PRADO. El código incluye la explicación de cómo usar la clase. Esta clase tiene dos constructores:

- El primero no tiene parámetros y establece que únicamente se van a generar ceros y unos (este constructor llama al segundo).
- El segundo tiene dos parámetros, `min` y `max` que delimitan el rango.

Cada vez que se llame al método `Siguiente`, éste devolverá un valor aleatorio en el rango especificado.

Nota: Realmente, los números generados son *pseudoaleatorios* (puede consultar Internet para tener una idea de este concepto). En este ejemplo se han generado según una distribución uniforme pero se pueden generar números con las probabilidades dadas por muchas otras distribuciones -disponibles en la biblioteca `random` de C++11-.

Finalidad: Enfatizar la importancia de la ocultación de información y de la interfaz pública de una clase. Dificultad Baja.

4. Definid una clase `Dinero` para trabajar de forma precisa con datos monetarios. La clase tendrá dos datos miembro: `euros` y `centimos`.

La clase deberá permitir que se introduzca un número de céntimos arbitrario (siempre que sea positivo o cero) y se encargará de ajustar las cantidades debidamente. Por ejemplo, si asignamos 20 euros y 115 céntimos, el objeto deberá almacenar el valor 21 en euros y 15 en centimos. No olvide añadir un método `ToString()` para representar de forma textual el contenido de un objeto `Dinero`.

Hacer un programa que calcule el precio final de un producto a partir de su precio inicial, de un incremento fijo mensual y de un número de meses. El programa irá mostrando, mes a mes, el precio del producto.

Nota: Implemente métodos que modifiquen una cantidad de dinero.

Finalidad: Trabajar con una clase como una abstracción. Dificultad Baja.

5. La sonda Mars Climate Orbiter fue lanzada por la NASA en 1998 y llegó a Marte el 23 de septiembre de 1999. Lamentablemente se estrelló contra el planeta ya que se acercó demasiado. El error principal fue que los equipos que desarrollaron los distintos módulos de la sonda usaron sistemas de medida distintos (el anglosajón y el métrico). Cuando un componente software mandaba unos datos en millas (o libras), otro componente software los interpretaba como si fuesen kilómetros (o Newtons). El problema se habría arreglado si todos hubiesen acordado usar el mismo sistema. En cualquier caso, cada equipo se encuentra más a gusto trabajando en su propio sistema de medida. Por tanto, la solución podría haber pasado por que todos utilizasen una misma clase para representar distancias (idem para fuerzas, presión, etc), utilizando los métodos que les resultasen más cómodos.

Para ello, se pide construir la clase `Distancia` que contendrá métodos como `SetKilometros`, `SetMillas`, etc. Internamente se usará un único dato miembro privado llamado `kilometros` al que se le asignará un valor a través de los métodos anteriores, realizando la conversión oportuna (una milla es 1,609344 kilómetros). La clase también proporcionará métodos como `GetKilometros` y `GetMillas` para lo que tendrá que realizar la conversión oportuna (un kilómetro es 0,621371192 millas).

Observad que la implementación de la clase podría haber utilizado como dato miembro privado, una variable `millas`, en vez de `kilómetros`. Esto se oculta a los usuarios de la clase, que sólo ven los métodos `SetKilometros`, `SetMillas`, `GetKilometros` y `GetMillas`.

Nota. Otro de los fallos del proyecto fue que no se hicieron suficientes pruebas del software antes de su puesta en marcha, lo que podría haber detectado el error. Esto pone de manifiesto la importancia de realizar una batería de pruebas para comprobar el correcto funcionamiento del software en todas las situaciones posibles. Esta parte en el desarrollo de un proyecto se le conoce como *pruebas de unidad (unit testing)*

Finalidad: La clase como una abstracción de un concepto. *Dificultad Baja.*

6. Este ejercicio consiste en reescribir la solución al problema planteado en el ejercicio 4 de la *Relación de Problemas IV*. Ahora se utilizarán **clases** para entidades tipo *rectangulo* y *circunferencia*. En cualquier caso, la solución no será la definitiva ya que necesitamos profundizar en la materia para poder diseñar una correcta estructura de clases: en este momento no sabemos cómo trabajar con una clase que tiene datos miembro de otra clase (p.e. la clase *Circunferencia* se caracteriza a partir de un dato *Punto2D* -el *centro*- además del *radio*). En un ejercicio de la *Relación de Problemas VI* se solucionará correctamente este problema. El nuevo enunciado es el siguiente:

Para modelar un rectángulo se usará la clase `Rectangulo`. Un rectángulo se caracterizará por la coordenada superior izquierda y la coordenada inferior derecha (ambos son datos `struct` de tipo `Punto2D`).

Para modelar una circunferencia se usará la clase Circunferencia. Una circunferencia se caracterizará por la coordenada del centro (un dato struct de tipo Punto2D) y el radio.

Escribir un programa que lea las coordenadas que definen un rectángulo y calcule la circunferencia centrada en el punto de corte de las diagonales del rectángulo tal que su superficie sea la menor entre todas las circunferencias de área mayor que la del rectángulo.

Para el cálculo, considere comenzar con $\text{radio}=0.5$ e ir incrementando su valor 0.25 en cada iteración.

Finalidad: Diseño preliminar de una clase compleja. Dificultad Baja.

7. Recuperar las clases escritas en el ejercicio 6 de esta misma Relación de Problemas.

Escribir un programa que lea las coordenadas que definen un rectángulo y calcule y muestre una serie de datos de tipo Circunferencia, todas centrada en el punto de corte de las diagonales del rectángulo.

Las circunferencias en las que estamos interesadas serán todas las circunferencias **inscritas** en el rectángulo. Para ello comience con una circunferencia de radio $\text{radio}=0.5$ y vaya incrementando su valor 0.25 en cada iteración.

Muestre cuántas circunferencias inscritas se han generado. Finalmente, muestre las propiedades de la última circunferencia inscrita y de la primera no inscrita.

Por ejemplo, dado el rectángulo de la figura 40, los objetos Circunferencia que nos interesan son **tres** (la circunferencia con trazo discontinuo no se guardará porque no está inscrita en el rectángulo).

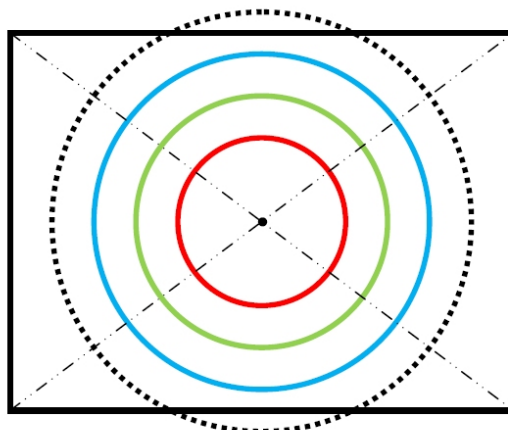


Figura 40: Tres circunferencias inscritas en un rectángulo y una no inscrita

Finalidad: Diseño preliminar de una clase compleja. Dificultad Baja.

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

8. Recuperad el ejercicio 31 de la *Relación de Problemas III* sobre la función gaussiana. En vez de trabajar con funciones, plantead la solución con una clase.

Debe diseñar la clase teniendo en cuenta que la función matemática gaussiana viene determinada unívocamente por el valor de la esperanza y la desviación, es decir, son estos dos parámetros lo que distinguen a una función gaussiana de otra.

Recuerde añadir un método `ToString` para la clase. Sugerimos el siguiente formato: si g es $N(0, 1)$ y el objeto g representa a esa función, la ejecución de $g.ToString()$ devolverá la cadena `N (0.000000, 1.000000)`.

Finalidad: Familiarizarnos con la definición de clases. Dificultad Baja.

9. En el ejercicio 24 de esta relación de problemas se definieron varias funciones para operar sobre una progresión geométrica. Definid ahora una clase para representar una progresión geométrica.

- Diseñad la clase pensando cuáles serían los datos miembro esenciales que definen una progresión geométrica, así como el constructor de la clase.
- Definid el método `Termino` que devuelva el término k -ésimo,
- Definid los métodos `SumaHastaInfinito`, `SumaHasta` y `MultiplicaHasta`.
- Cread un programa principal que llame a los anteriores métodos.

Finalidad: Comparar la ventaja de un diseño con clases frente a otro realizado con funciones. Dificultad Baja.

10. Se quiere construir la clase `CoordenadasGPS` para representar las coordenadas geográficas de una posición terrestre, dada por tres datos reales, a saber, su longitud, latitud y altura, tal y como se describe en el ejercicio 38 de la *Relación de Problemas I*. La clase debe proporcionar métodos para asignar y recuperar los datos de latitud y longitud. Se pide hacerlo de la siguiente forma:

- Sólo se podrán asignar los datos de longitud y latitud en grados. El/Los método/s correspondiente/s transformarán esos grados en radianes y los asignarán a los datos miembro. La altura vendrá en metros. Debe comprobar que los grados sean correctos, es decir, los grados de latitud deben estar en el intervalo $[-90, 90]$ y los de longitud en $[-180, 180]$. La altura debe estar entre -423 (Valle del Jordán) y 8848 (monte Everest).
- Para consultar los datos de longitud y latitud se definirán métodos específicos para cada caso: `GetLatitudGrados()`, `GetLatitudRadianes()`, `GetLongitudGrados()` y `GetLongitudRadianes()`.

Por ahora, la clase sólo debe proporcionar los métodos para asignar y recuperar los valores de latitud, longitud y altura. Escriba también un método `ToString`. Posteriormente se ampliará la funcionalidad de esta clase.

Finalidad: Diseño de una clase básica. Dificultad Baja.

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

11. Defina la clase `Instante` para representar un instante de tiempo dentro de un día. Por lo tanto, la clase debe representar un número de horas, minutos y segundos.

Añada métodos para:

- Obtener el número de segundos y minutos transcurridos desde las 0h 0min 0seg. Por ejemplo, si el instante es 1h 2min 1seg, el número total de segundos transcurridos es de 3721 y el de minutos 62.
- Establecer el instante a partir del número de segundos transcurridos. Por ejemplo, si se establece a 3721, el instante debe contener 1h 2min 1seg.

No olvide el método `ToString` para representar en forma de texto el valor de un objeto `Instante`. Por ejemplo, si `hora` es un dato instante que contiene el valor de la hora 2:30:00 la ejecución de `hora.ToString()` devuelve la cadena (2 h: 30 m : 0 s).

Puede definir todos los métodos adicionales que estime oportuno.

Cree un programa que lea un número de horas, minutos y segundos iniciales/finales.

El programa creará los objetos `instante_inial` e `instante_final` y calculará el número de segundos que hay de diferencia entre ambos instantes. A continuación creará un objeto `instante_diferencia` correspondiente a la cantidad de horas, minutos y segundos de diferencia entre el instante final y el inicial e imprimirá en pantalla el valor de `instante_diferencia`. El programa también imprimirá, por separado, el total de segundos y minutos enteros que hay en `instante_diferencia`.

Puede reescribir cualquiera de los (muchos) ejercicios que trabajan con instantes empleando la clase `Instante`.

Finalidad: Diseño de una clase básica. Dificultad Baja.

12. Se quiere construir una clase `DepositoBancario` para simular préstamos, ofreciendo la funcionalidad descrita en los ejercicios 29 (reinvierte capital e interés un número de años) y 30 (reinvierte capital e interés hasta obtener el doble de la cantidad inicial) de la *Relación de Problemas II*. Por tanto, la clase debe proporcionar, para un capital y unos intereses dados, métodos para:

- calcular el capital que se obtendrá al cabo de un número de años,
- calcular el número de años que deben pasar hasta doblar la cantidad inicial.

A la hora de diseñar la clase, tendremos que analizar cuestiones como:

- a) ¿Cuáles son sus datos miembro? Es evidente que el *capital* y el *interés* sí lo serán ya que cualquier operación con un objeto de la clase `DepositoBancario` involucra a ambas cantidades. ¿Pero y el *número de años*?
- b) ¿Qué constructor definimos?
- c) ¿Queremos modificar el capital y el interés una vez creado el objeto?

- d) ¿Queremos poder modificarlos de forma independiente?
- e) ¿Hay alguna restricción a la hora de asignar un valor al capital e interés?
- f) ¿Es mejor un método para calcular el número de años hasta obtener el doble de la cantidad inicial, o por el contrario es mejor un método para calcular el número de años hasta obtener una cantidad específica?

Finalidad: Diseñar la interfaz de una clase. Dificultad Baja.

13. **Nota:** En el ejercicio 26 de la *Relación de Problemas IV* se trabajó sobre una versión simplificada de este problema.

(Examen Febrero 2013) Se está diseñando un sistema web que recolecta datos personales de un usuario y debe sugerirle un nombre de usuario (login). Dicho login estará basado en el nombre y los apellidos; en concreto estará formado por los n primeros caracteres de cada nombre y apellido (en minúsculas, unidos y sin espacios en blanco). Por ejemplo, si el nombre es Antonio Francisco Molina Ortega y $n = 2$, el nombre de usuario sugerido será anfrmoor.

Debe tener en cuenta que el número de palabras que forman el nombre y los apellidos puede ser cualquiera. Además, si n es mayor que alguna de las palabras que aparecen en el nombre, se incluirá la palabra completa. Por ejemplo, si el nombre es Ana CAMPOS de la Blanca y $n = 4$, entonces la sugerencia será anacampdelablan.

Implemente la clase `Login` que tendrá como único dato miembro el número máximo de caracteres que nos interesan de cada palabra del nombre. Hay que definir el método `Codifica` que recibirá una cadena de caracteres (tipo `string`) formada por el nombre y apellidos (separados por uno o más espacios en blanco) y devuelva otra cadena con la sugerencia de login.

```
class Login {  
  
    private:  
        int num_caracteres_a_coger;  
  
    public:  
        Login (int n) : num_caracteres_a_coger (n) { }  
        string Codifica(string nombre_completo) { ..... }  
        .....  
};
```

Los únicos métodos que necesita usar de la clase `string` son `size` (o su sinónimo `length`) y `push_back`. Para probar el programa o bien lea una cadena de caracteres con `getline(cin, cadena)`; o bien use directamente literales de tipo `string`.

Dificultad Media.

14. Encapsule en la clase `Lector` las funciones escritas en los ejercicios 18 y 19 de la *Relación de Problemas III*. El objetivo es disponer de una clase (`Lector`) que ofrezca un conjunto de métodos **seguros** de lectura **etiquetada** de datos `int` y `double`.

Deberá deducir la estructura de la clase a partir del siguiente ejemplo. Los datos `double` se leerán de manera similar.

```
// Lectura de un entero

Lector lector_temp("Temperatura: ");
int temp = lector_temp.LeeEntero();

cout << "Temperatura leida = " << temp << endl;

// Lectura de enteros acotados superior e inferiormente

Lector lector_hora;

lector_hora.SetTitulo ("Introduzca hora: ");
int h = lector_hora.LeeEnteroEnRango (0, 23);

lector_hora.SetTitulo ("Introduzca minuto: ");
int m = lector_hora.LeeEnteroEnRango (0, 59);

lector_hora.SetTitulo ("Introduzca segundo: ");
int s = lector_hora.LeeEnteroEnRango (0, 59);

cout << "Hora: " << h << ":" << m << ":" << s << endl;

// Lectura de un entero acotado inferiormente

Lector lector_ingreso("Cantidad a ingresar: ");

int ingreso = lector_ingreso.LeeEnteroMayorOIgual (0);

cout << "Valor del ingreso = " << ingreso << endl;
```

Finalidad: Presentar una clase lectora. Dificultad Media.

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

15. Escriba la clase `ManipuladorString`. Esta clase se encargará de modificar un dato `string` y realizar cálculos complejos sobre él con funciones no ofertadas por la clase `string`. Proponemos que trabaje sobre el siguiente esquema:

ManipuladorString
- <code>string cadena</code>
+ <code>ManipuladorString (void);</code> + <code>ManipuladorString (string la_cadena);</code> + <code>void SetCadena (string la_cadena);</code> + <code>string GetCadena (void);</code> + <code>void EliminaPrimerosSeparadores (void);</code> + <code>void EliminaUltimosSeparadores (void);</code> + <code>void EliminaPrimerosYUltimosSeparadores (void);</code> + <code>void ConvierteAMayusculas (void);</code> + <code>void ConvierteAMinusculas (void);</code>

Mostraremos la manera de trabajar con esta clase con un ejemplo. Suponga que quiere eliminar los espacios iniciales y finales de un dato `string` llamado `nombre`.

Necesitamos un objeto `ManipuladorString` que contenga una copia del dato `string` que queremos manipular. Hay dos opciones:

- a) con el constructor con un argumento:

```
ManipuladorString m (nombre);
```

- b) con el constructor sin argumentos y el método `SetCadena`:

```
ManipuladorString m;  
m.SetCadena(nombre);
```

Después se ejecuta el método deseado sobre `m`:

```
m.EliminaPrimerosYUltimosSeparadores();
```

que modifica el valor del campo privado `cadena`. Después la instrucción

```
nombre = m.GetCadena();
```

deja en `nombre` el resultado de la manipulación (en este caso, `cadena nombre` sin espacios iniciales ni finales).

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

En cambio, si quisiéramos mantener el valor original de `nombre` tendremos que usar otra variable `string` para guardar el resultado que devuelve el método `GetCadena`.

```
string nombre_limpio = m.GetCadena();
```

Finalidad: Presentar una clase manipuladora que incrementa la funcionalidad de clases ya existentes. Dificultad Baja.

16. Añada a la clase `ManipuladorString` los métodos públicos:

```
void EliminaExcesoSeparadores (void);  
int  CuentaPalabras (void);
```

Finalidad: Mejorar la funcionalidad de una clase existente. Dificultad Media.

17. Reescriba la solución del ejercicio 14 de esta misma Relación de Problemas utilizando las clases `Lector` y `ManipuladorString` para su resolución.

Finalidad: Trabajar con varias clases en un programa. Dificultad Media.

18. Reescriba la solución del ejercicio 9 de la *Relación de Problemas III* (números secuenciables) utilizando una clase. Se trata de una clase *gestora* o *manipuladora* del tipo desarrollado en el ejercicio 15 de esta misma Relación de Problemas.

GestorNumerosSecuenciables
- int valor; // PRE: 1 <= valor
 + GestorNumerosSecuenciables (void); + GestorNumerosSecuenciables (int n); + void SetValor (int n); + int GetValor (void); + bool EsSecuenciable (void); + int NumSecuencias (void);

Finalidad: Trabajar con clases manipuladoras. Dificultad Baja.

19. En el ejercicio 15 de la *Relación de Problemas II* (actualización de la retención fiscal y renta) se leían caracteres del teclado para saber si una persona era autónomo, pensionista, etc. Se propuso una solución basada en funciones para este problema en el ejercicio 23 de la *Relación de Problemas II* con la función `LeeOpcion2Alternativas` que se encargaba de realizar la lectura de estas propiedades binarias.

Escribid ahora la clase `LectorOpcion2Alternativas` que encapsule esta funcionalidad y cambiar el programa principal para que use esta clase.

Finalidad: Diseñar y emplear una clase lectora simple. Dificultad Baja.

20. En este ejercicio proponemos que construya la clase `Menu` con el objetivo de poder gestionar rápidamente menús en los que las opciones se presentan con texto, asociando un número a cada opción y se selecciona la opción introduciendo el número asociado a la opción deseada. Se intentará generalizar la solución desarrollada de manera que sea adaptable al mayor número de problemas.

Desde la función `main` la clase se verá de la siguiente manera. Suponga que va a ofrecer cuatro opciones (más la opción de salida). Las siguientes instrucciones bastan para crear el un objeto `Menu` llamado `menu_principal`:

```
string opciones = "Terminar|Alta|Baja|Consulta|Modificación|";
Menu menu_principal (opciones);
```

La gestión del menú podría hacerse en una estructura de programa como la siguiente:

```
bool terminar = false;

while (!terminar) {
    menu_principal.ImprimeOpciones();
    int tarea = menu_principal.EligeOpcion();

    switch (tarea) {

        case (0) : { // Terminar
            terminar = true;
            break;
        } // case (0)

        case (1) : { // Alta
            .....
        } // case (1)

        .....
    } // switch

} // while
```

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

La ejecución de `ImprimeOpciones` muestra la lista de tareas disponibles y la de `EligeOpcion` permite que el usuario introduzca un número que indica la tarea que desea. El número suministrado debe ser correcto (en este ejemplo entre 0 y 4) y si no es así, se vuelve a pedir repetidamente. En este ejemplo sería como sigue:

```
1.- Alta
2.- Baja
3.- Consulta
4.- Modificacion
0.- Terminar
```

Introduzca la opcion elegida -->

Finalidad: Diseñar una clase que facilite la creación y gestión de menús. Hacerlo de la manera más general posible. Dificultad Media.

21. Escribir un programa que haga:

- a) Se presentará al usuario un *menú principal*, que ofrece dos alternativas: 1) introducir los valores de los coeficientes (A, B y C) de una recta, y ii) salir del programa.
- b) Si se eligió la primera opción, una vez introducidos los coeficientes se presentará al usuario un *menú secundario*, para que elija alguna de las siguientes opciones:
 - Mostrar el valor de la pendiente de la recta.
 - Mostrar la ordenada, dada una abscisa (el programa pedirá la abscisa)
 - Mostrar la abscisa, dada una ordenada (el programa pedirá la ordenada)
 - Volver al menú principal.

Proponemos usar la clase `Menu` del ejercicio 20 de esta Relación de Problemas para crear dos objetos `Menu`: 1) `menu_principal` y 2) `menu_secundario`.

Finalidad: Trabajar con varias clases en un programa. Dificultad Media.

22. Modifique la clase `Gaussiana` del ejercicio 8 de esta misma Relación de Problemas y haga que ahora π sea una constante a nivel de clase (`static`).

Finalidad: trabajar con constantes estáticas no enteras. Dificultad Baja.

23. Se quiere construir una clase para representar la tracción de una bicicleta, es decir, el conjunto de platos (engranaje delantero), cadena y piñones (engranaje trasero). Supondremos que se dispone de tres platos (numerados de 1 a 3, siendo 1 el más pequeño) y 7 piñones (numeradas de 1 a 7, siendo 1 el piñón más grande).

La clase debe proporcionar métodos para cambiar de plato y piñón, sabiendo que los platos avanzan o retroceden de 1 en 1 y los piñones cambian a saltos de uno o de

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

dos. Si ha llegado al límite superior (inferior) y se llama al método para subir (bajar) de plato, la posición no variará. Lo mismo se aplica al piñón.

Cread un programa principal que lea desde un fichero externo los movimientos realizados e imprima la situación final de plato y piñón. Los datos se leerán en el siguiente formato: tipo de engranaje (piñón o plato) seguido del tipo de movimiento.

Para codificar esta información se usarán las siguientes letras: E indica un plato, P un piñón, S para subir una posición, B para bajar una posición, T para subir dos posiciones y C para bajar dos posiciones. T y C sólo se aplicarán sobre los piñones.

E S P S P S P S P C E S E B E B X

En este ejemplo los movimientos serían: el plato sube, el piñón sube en tres ocasiones sucesivas, el piñón baja dos posiciones de golpe, el plato sube y vuelve a bajar. Supondremos siempre que la posición inicial del plato es 1 y la del piñón 1. Así pues, la posición final será Plato=1 y Piñón=2.

Mejorad la clase para que no permita cambiar la marcha (con el plato o el piñón) cuando haya riesgo de que se rompa la cadena. Este riesgo se produce cuando la marcha a la que queremos cambiar es de la siguiente forma:

- Plato igual a 1 y piñón mayor o igual que 5
- Plato igual a 2 y piñón o bien igual a 1 o bien igual a 7
- Plato igual a 3 y piñón menor o igual que 3

*Finalidad: Diseñar la interfaz de una clase. Trabajar con datos miembro constantes..
Dificultad Media.*

24. Realizar un programa que vaya leyendo caracteres hasta que se encuentre un punto ' . '. Queremos contar el número de veces que aparece cada mayúscula.

Cread una clase específica `ContadorMayusculas` que implemente los métodos necesarios para realizar la cuenta de las mayúsculas. La clase debe proporcionar los métodos:

```
void IncrementaCuenta (char mayuscula)
int  CuantasHay (char mayuscula)
```

Los contadores deberán almacenarse en un vector, que deberá ser un dato privado de la clase. Use como referencia el ejercicio 18 de la *Relación de Problemas IV*.

Ampliar el programa para que muestre el número de veces que aparece cada mayúscula en un dato `string`.

Dificultad Media.

25. Amplíe la clase `GestorNumerosSecuenciabiles` del ejercicio 18 de esta Relación de Problemas. En esta nueva versión calculamos y guardamos todas las secuencias (hasta un máximo de 100) asociadas a un número secuenciable.

Una secuencia está determinada únicamente por una pareja de valores enteros (p, q) siendo $p < q$ de manera que si n es un número secuenciable, entonces $n = p + (p + 1) + \dots + q$

Para probar esta adición, amplíe la funcionalidad del apartado c de manera que ahora deberá realizar estas tareas:

Mostrar los números secuenciabiles (sus **valores**) cuyo número de descomposiciones coincide con el número de referencia leído.

Para cada uno de ellos deberá mostrar todas sus descomposiciones.

Finalmente, mostrar el total de números secuenciabiles que cumplen esta última condición.

Por ejemplo, para números entre 1 y 20 con dos secuencias, el resultado sería:

3. Numeros secuenciabiles con 2 secuencias:

9 (2 secuencias)

$$9 = 2 + 3 + 4$$

$$9 = 4 + 5$$

18 (2 secuencias)

$$18 = 3 + 4 + 5 + 6$$

$$18 = 5 + 6 + 7$$

Hay 2 numeros con 2 secuencias

26. (*Examen Septiembre 2012*) La **criba de Eratóstenes** permite hallar todos los números primos menores que un número natural dado n . Hemos trabajado sobre este problema en el ejercicio 23 de la *Relación de Problemas IV*.

En esta ocasión se trata de definir una **clase** llamada `Eratostenes`. Un objeto de la clase podrá almacenar los números primos calculados en un vector de enteros:

```
// Todos los primos calculados serán menores que TAMANIO.  
static const int TAMANIO = 500;  
int vector_privado[TAMANIO];
```

```
//PRE: 0 <= total_utilizados <= TAMANIO  
int total_utilizados; // Número de casillas ocupadas
```

lo que significa que en un objeto `Eratostenes` se podrá guardar un **máximo** de `TAMANIO` números primos.

El usuario de la clase quiere generar primos menores o iguales que un valor dado, por ejemplo:

```
Eratostenes primos;  
primos.CalculaHasta(100);
```

El método `void CalculaHasta(int n)` calcula y guarda en el array interno los primos menores o iguales que `n`. Este es el método en el que se implementa la criba. Cuando se ejecute el método, se almacenarán en un vector interno del objeto todos los primos menores que `n`.

Se dará cuenta que el valor proporcionado puede ser demasiado ambicioso. Por ejemplo, si `n` vale 5000 no habrá espacio en el vector interno (500 casillas) para todos los primos menores que 5000. Debería guardar en el objeto `Eratostenes` dos valores:

- `n` el valor que se suministró para calcular los primos guardados.
- `n_real` guarda el menor valor entero mayor o igual todos los primos del vector. Si se pudieron guardar todos los primos solicitados, `n` y `n_real` coincidirán.

La clase ofrecerá, además, como mínimo, estos métodos:

- `int GetReferencia()` devuelve el valor que se suministró para calcular los primos (el valor del campo `n`).
- `int GetReferenciaReal()` devuelve el menor valor entero mayor o igual todos los primos del vector (el valor del campo `n_real`).
- `int TotalCalculados()` devuelve cuántos primos hay almacenados.
- `int Elemento(int k)` devuelve el `k`-ésimo primo almacenado.

Un programa que use esta clase podría ser, esquemáticamente:

```
Eratostenes primos;  
primos.CalculaHasta(100);  
  
cout << "Primos <= " << primos.GetReferencia() << endl;  
cout << "Obtenidos <= "  
    << primos.GetReferenciaReal() << endl;  
  
cout << "El mayor es:  
    << primos.Elemento(primos.TotalCalculados()-1) << endl;  
  
cout << "Primos: " << endl;  
  
for (int i=0; i<primos.TotalCalculados(); i++)  
    cout << setw(5) << primos.Elemento(i);
```

Dificultad Media.

27. Reescriba la solución del ejercicio 47 de la Relación de Problemas IV usando una clase, `GestorIPC`, que proporcione los valores del IPC y métodos para su gestión. La estructura de la clase que proponemos es la indicada en la figura 41. Los valores de las constantes estáticas, del array `PAISES` y de la matriz `IPC` puede consultarlos en la figura 27 en la que se mostraba el **IPC** de varios países entre 2001 y 2021.

GestorIPC
<pre>- PRIMER_ANIO = 2001 - ULTIMO_ANIO = 2021 - NUM_ANIOS = ULTIMO_ANIO - PRIMER_ANIO + 1 - NUM_PAISES = 4 - string PAISES [NUM_PAISES]; - double IPC [NUM_ANIOS] [NUM_PAISES];</pre>
<pre>+ int PrimerAnio (void); + int UltimoAnio (void); + int NumAnios (void); + int NumPaises (void); + string NombrePais (int pais); + double ValorIPC (int anio, int pais); + double IPC_Medio_PaisPeriodo (int el_pais, int el_primer_anio, int los_anios); + InfoIPC Info_IPC_MinMedio_Periodo (int num_anios); + InfoIPC Info_IPC_MaxMedio_Periodo (int num_anios);</pre>

Figura 41: La clase `GestorIPC`

Para reducir el número (innecesario, en la práctica) de métodos, se ha decidido aprovechar los cálculos que involucra el máximo (resp. mínimo) valor medio de un periodo haciendo que el método devuelva un struct de tipo `InfoIPC` cuya estructura es:

```
struct InfoIPC {
    int    indice_pais;           // 0 <= indice_pais <= NUM_PAISES
    double valor_medio_IPC;      // Máximo ó mínimo IPC medio (valor)
    int    anio_inicio;          // Primer año del periodo
    int    num_anios;            // Número de años del periodo
};
```

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

28. La sucesión de Fibonacci de orden n es una secuencia de números en la que los dos primeros son el 1 y el 1. A partir del tercero, los elementos se calculan como la suma de los n anteriores, si ya hay n elementos disponibles, o la suma de todos los anteriores si hay menos de n elementos disponibles.

Por ejemplo, la sucesión de Fibonacci de orden 4 sería la siguiente:

1, 1, 2, 4, 8, 15, 29, ...

Escribir un programa que lea los valores de dos enteros, n y k y calcule, almacene y muestre por pantalla los k primeros términos de la sucesión de Fibonacci de orden n .

Definid una clase llamada `Fibonacci`. Para almacenar los elementos de la serie se usará un *vector* de TAMANIO enteros. Al constructor se le pasará como parámetro el valor de n . Escribid, al menos, los siguientes métodos:

- `GetOrden()` para obtener el valor de n .
- `void CalculaPrimeros(int k)` para que calcule los k primeros elementos.
- `int TotalCalculados()` que devuelva cuántos elementos hay almacenados.
- `int Elemento(int i)` para que devuelva el elemento i -ésimo.

Este problema es parecido al del ejercicio 26 de esta misma Relación de Problemas, en el sentido de que puede ocurrir que se desea generar los primeros k elementos de la sucesión pero no es posible guardarlos todos en el vector interno. Complete la clase teniendo en cuenta esta restricción.

El programa principal podría quedar de la forma:

```
Fibonacci fib(4);
fib.CalculaPrimeros(100);

cout << "Creada sucesion de Fibonacci de orden "
      << fib.GetOrden() << endl;
cout << "Pedidos los primeros
      << fib.GetNumValoresPedidos() << " valores. " << endl;
cout << "Obtenidos los primeros
      << fib.TotalCalculados() << " valores. " << endl;
cout << endl;
cout << "Valores calculados: " << endl;
for (int i=0; i<fib.TotalCalculados(); i++)
    cout << setw(5) << fib.Elemento(i);
```

Dificultad Media.

29. En los ejercicios 8 y 22 de esta relación en el que construíamos la clase Gaussiana. En ese ejercicio se pedía calcular y mostrar, para valores $x \in [\mu - 3\sigma, \mu + 3\sigma]$ con $\Delta_x = 0,5$ los valores de $g(x)$ y $CFD(x)$. Muestre las gráficas de $g(x)$ y $CFD(x)$.

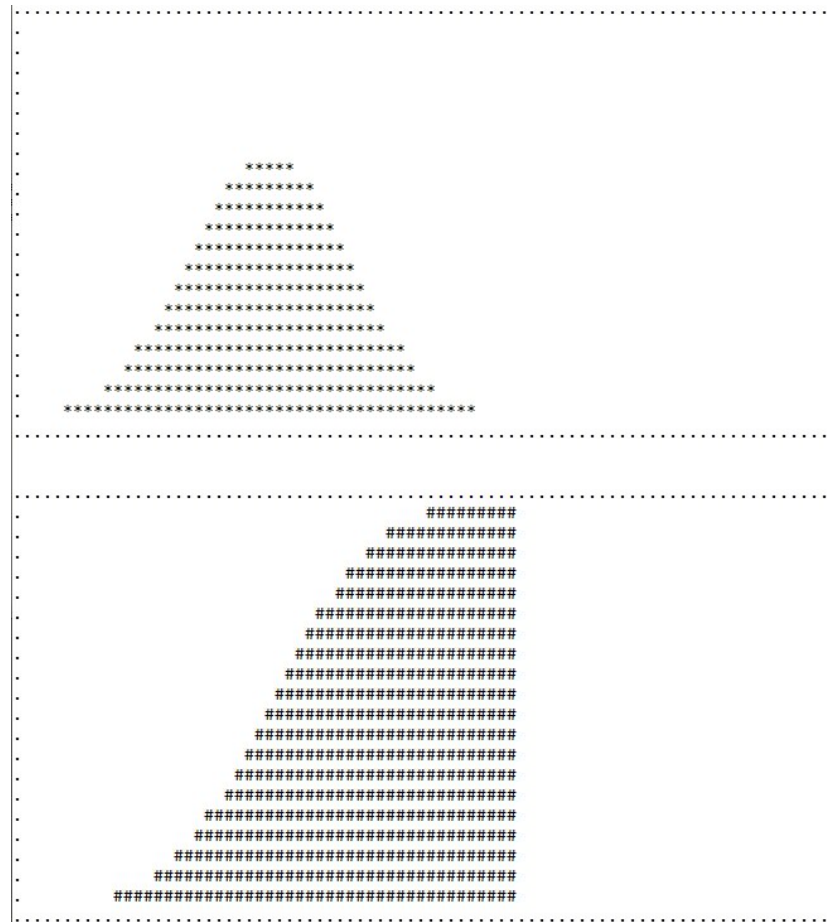


Figura 42: $g(x)$ y $CFD(x)$ de una función de Gauss con $\mu = 0$ y $\sigma = 2$. Los valores de x representados pertenecen al intervalo $[\mu - 3\sigma, \mu + 3\sigma]$ con $\Delta_x = 0,25$

Para esta tarea recordad el ejercicio 51 de la *Relación de Problemas IV* en el empleábamos una matriz de char como “tablero”. Sugiero usar la **clase** Lienzo para dibujar sobre él las gráficas. Básicamente será una matriz de char, por ejemplo:

```
const char HUECO = ' ';
const char RELLENO1 = '*';
const char RELLENO2 = '#';
const int FILAS = 20;
const int COLS = 80;
char tablero[FILAS][COLS];
```

- a) Las *columnas* representan la variable independiente x . Los valores que toma están predeterminados: entre $\mu - 3\sigma$ y $\mu + 3\sigma$ y el incremento entre dos valores consecutivos de x es $\Delta_x = 0,5$
La columna 0 corresponde al primer valor de x ($\mu - 3\sigma$), la columna 1 al siguiente ($\mu - 3\sigma + 0.5$), y así sucesivamente.
- b) Las *filas* representan la variable $y = f(x)$ por lo que habrá que tener en cuenta los valores posibles de y para poder asignarles la fila que les corresponde.
Considere que los valores de y estarán en el intervalo $[0.0, 0.3]$ (use constantes) para $g(x)$ y $[0.0, 1.0]$ (use constantes) para $CDF(x)$.
Para la realización de este ejercicio, y teniendo en cuenta que estamos representando funciones $y = f(x)$ en un tablero cuyas filas están numeradas desde 0 hasta FILAS-1 y cuyas columnas desde 0 hasta COLS-1 tenga en cuenta:
- c) Si representa únicamente las parejas $(x, f(x))$ obtendría la gráfica de la función (ver figura 43). Estamos interesados en que se resalte también la parte interior a la gráfica, como en la figura 42.
- Una vez calculada la fila que le corresponde a $y = f(x)$ tendrá que rellenar también en el tablero las casillas que quedan *por debajo* de esa fila.

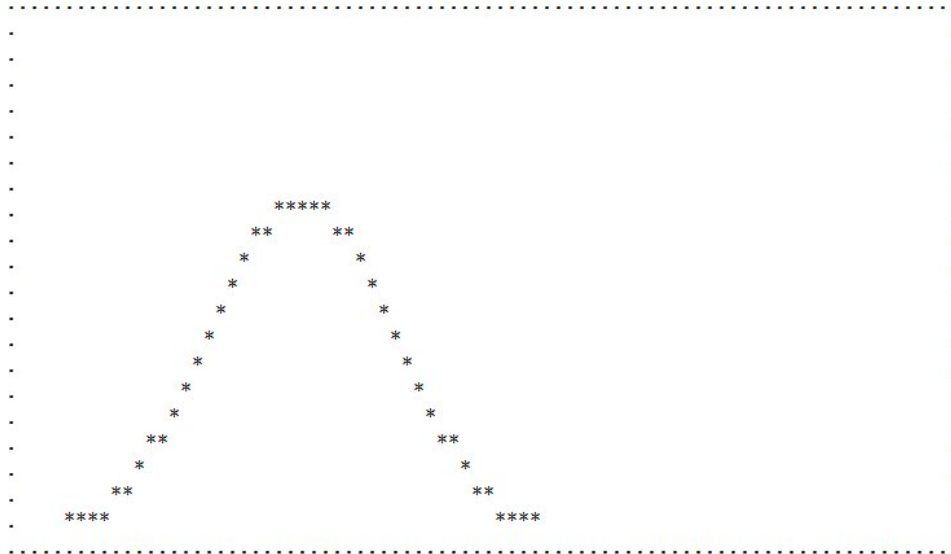


Figura 43: $g(x)$ de una función de Gauss con $\mu = 0$ y $\sigma = 2$. Los valores de x representados pertenecen al intervalo $[\mu - 3\sigma, \mu + 3\sigma]$ con $\Delta_x = 0,25$

Dificultad Media.

30. (*Basado en el Examen Ordinario 2019*) Implementar una clase `RedSocial` para gestionar una red social. La red debe almacenar los nombres de los usuarios y las relaciones de amistad. Se propone la siguiente representación:

```
class RedSocial {
    private:
        static const int MAXIMO_USUARIOS = 100;
        string usuarios[MAXIMO_USUARIOS];
        bool relaciones_amistad[MAXIMO_USUARIOS][MAXIMO_USUARIOS];
        int usuarios_uitiles;
    public:
        ...
};
```

donde `relaciones_amistad` es una matriz binaria tal que la componente (i, j) es `true`, si el usuario i es amigo del usuario j , y `false`, en otro caso. Asumiremos que las relaciones de amistad son *simétricas*, por lo que la matriz binaria también lo es.

Se pide implementar, al menos, los siguientes métodos básicos (los usuarios vienen especificados por sus nombres):

- `RedSocial (void);`
Constructor sin argumentos. Crea una red vacía.
- `int NumUsuarios (void)`
Devuelve el número de usuarios de la red.
- `void AniadeUsuario (string usr)`
Añade el usuario `usr` a la red (no tiene amigos).
- `void EliminaUsuario (string usr)`
Elimina el usuario `usr` de la red.
- `bool Existe (string usr)`
Informa si `usr` es un usuario de la red.
- `void HacerAmigos(string usr1, string usr2);`
Registra una relación de amistad entre `usr1` y `usr2`
- `void DeshacerAmigos(string usr1, string usr2);`
Elimina una relación de amistad entre `usr1` y `usr2`
- `bool SonAmigos(string usr1, string usr2);`
Informa si hay relación de amistad entre `usr1` y `usr2`

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

Escriba un programa que cree una red y lea los nombres de los usuarios de la red (termina cuando se introduce FIN en el primer nombre). Después lee una serie de líneas que contienen parejas de nombres que indican las relaciones de amistad y va actualizando las relaciones de amistad en la red.

Queremos ampliar la funcionalidad de la red con:

- a) Un método que, dado un usuario *A* de la red, sugiera un amigo potencial para *A*. Un amigo potencial es aquel usuario de la red (no amigo de *A*) que tiene más amigos en común con *A*.

Escriba en la función `main` código para que solicite el nombre de un usuario y sugiera una relación de amistad por él.

- b) Un método que decida si dos usuarios son *amigos circunstanciales*, es decir, si son amigos pero no tienen amigos en común.

Escriba en la función `main` código para que muestre todos los amigos circunstanciales presentes en la red.

Implemente cualquier otro método adicional, público o privado, que considere útil. Use cualquier clase adicional que le resulte útil.

- 31. En este ejercicio van a trabajar con las clases `Menu` y `RedSocial` (ejercicios 20 y 30, respectivamente, de esta misma Relación de Problemas).

Deberá usar un **menú** para gestionar una red social. El menú ofrecerá las siguientes opciones: 1) *Añadir usuario* 2) *Eliminar usuario*, 3) *¿Existe usuario?*, 4) *Crear amistad*, 5) *Deshacer amistad*, 6) *¿Hay amistad?*, 7) *Sugerir amigos potenciales*, 8) *¿Amigos circunstanciales?* y 9) *Mostar red social*. La opción 0 se reserva para *terminar*

Finalidad: Construir un programa con varias clases. Reutilizar código ya escrito y verificado. Dificultad Media.

- 32. Este ejercicio está basado en el ejercicio 40 de la *Relación de Problemas IV*. En ese ejercicio se leía un número indeterminado de datos referentes a los empleados de una empresa que se almacenaban en registros, que a su vez se organizaban en un array y después se ordenaban.

Ahora deberá usar la clase `Empleado` para representar a los empleados. Deberá diseñar la clase correctamente usando datos privados, constructores, métodos `Set`, métodos `Get` y otros métodos de gestión si fueran necesarios.

Nota: la fecha de nacimiento seguirá siendo un `struct`.

Use un array de objetos `Empleado` para almacenar los empleados leídos. Muestre su contenido después de leer los datos de los empleados. Calcule los salarios mínimos y máximos, muestre sus valores y los nombres y apellidos de los empleados que ganan menos y más. Calcule el salario medio, muestre su valor y después muestre un listado de los empleados cuyo salario es superior al salario medio.

Finalidad: Definición de clases. Usar un array de objetos. Dificultad Media.

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

33. Reescribir el ejercicio 33 de la *Relación de Problemas V* usando clases. En ese ejercicio (consulte el enunciado) se pedía escribir un programa que recibe un número indeterminado de *palabras* (finaliza cuando se lee FIN) y selecciona y guarda aquellas que correspondan con un NIF/NIE válido. Las palabras seleccionadas las guarda en un array, indica cuántos NIF/NIE válidos ha encontrado y finalmente los muestra.

Le proponemos crear la clase `Verificador_NIF_NIE` para esta tarea. Use un dato string como campo privado en el que se almacena la palabra que se va a verificar.

Verificador_NIF_NIE
- string cadena
+ Verificador_NIF_NIE (void);
+ Verificador_NIF_NIE (string la_cadena);
+ void SetNIFNIE (string cad);
+ string GetNIFNIE (void);
+ bool Verifica_NIF_NIE (void);
- char LetraNIF (int n)

Recomendamos procesar la cadena usando la clase `ManipuladorString`.

34. Reescribir el ejercicio 36 de la *Relación de Problemas IV* usando clases.

Para esta tarea (consulte el enunciado) puede añadir el método

```
bool PuedeSer_NIF_NIE (void);
```

a la clase `Verificador_NIF_NIE` desarrollada en el ejercicio 33 de esta Relación de Problemas.

Recomendamos procesar la cadena usando la clase `ManipuladorString`.

Ejercicios sobre *secuencias*

En los ejercicios que se refieran a la clase `SecuenciaCaracteres` use la siguiente definición (de los apuntes de clase):

```
////////////////////////////////////
```

```
class SecuenciaCaracteres {
```

```
private:
```

```
    static const int TAMANIO = 50; // Núm.casillas disponibles  
    char vector_privado[TAMANIO];
```

```
    // PRE: 0<=total_utilizados<=TAMANIO  
    int total_utilizados; // Núm.casillas ocupadas
```

```
public:
```

```
    /*****  
    // Constructor sin argumentos
```

```
    SecuenciaCaracteres (void) : total_utilizados (0) {}
```

```
    /*****  
    // Devuelve el número de casillas ocupadas
```

```
    int TotalUtilizados (void)  
    {  
        return (total_utilizados);  
    }
```

```
    /*****  
    // Devuelve el número de casillas disponibles
```

```
    int Capacidad (void)  
    {  
        return (TAMANIO);  
    }
```

```
    /*****/pre>
```

```

/*****/
// "Vacía" completamente la secuencia

void EliminaTodos (void)
{
    total_utilizados = 0;
}

/*****/
// Añade un elemento ("nuevo") al vector.
// PRE: total_utilizados < TAMANIO
// La adición se realiza si hay alguna casilla disponible.
// El nuevo elemento se coloca al final del vector.
// Si no hay espacio, no se hace nada.

void Aniade (char nuevo)
{
    if (total_utilizados < TAMANIO){
        vector_privado[total_utilizados] = nuevo;
        total_utilizados++;
    }
}

/*****/
// Devuelve el elemento de la casilla "indice"
// PRE: 0 <= indice < total_utilizados

char Elemento (int indice)
{
    return (vector_privado[indice]);
}

/*****/
// Cambia el contenido de la casilla "indice" por el valor
// dado en "nuevo". El tamaño de la secuencia no cambia.
// PRE: 0 <= indice < total_utilizados

void Modifica (int indice, char nuevo)
{
    if ((indice >= 0) && (indice < total_utilizados)) {
        vector_privado[indice] = nuevo;
    }
}
/*****/
```

```

/*****/
// Eliminar el carácter de la posición dada por "indice".
// Realiza un borrado físico (desplazamiento y sustitución).
// PRE: 0 <= indice < total_utilizados

void Elimina (int indice)
{
    if ((indice >= 0) && (indice < total_utilizados)) {

        int tope = total_utilizados-1; // posic. del último

        for (int i = indice ; i < tope ; i++)
            vector_privado[i] = vector_privado[i+1];

        total_utilizados--;
    }
}

/*****/
// Inserta el carácter "nuevo" en la posición dada por "indice".
// Desplaza todos los caracteres una posición a la derecha
// antes de copiar en "indice" en valor "nuevo".
// PRE: 0 <= indice < total_utilizados

void Inserta (int indice, char valor_nuevo)
{
    if ((total_utilizados < TAMANIO) && (indice >= 0)
        && (indice < total_utilizados)) {

        for (int i = total_utilizados ; i > indice ; i--)
            vector_privado[i] = vector_privado[i-1];

        vector_privado[indice] = valor_nuevo;
        total_utilizados++;
    }
}

/*****/
```

```

/*****/
// Compone un string con todos los caracteres que están
// almacenados en la secuencia y lo devuelve.

string ToString()
{
    string cadena;

    for (int i=0; i<total_utilizados; i++)
        cadena = cadena + vector_privado[i];

    return (cadena);
}
/*****/

};

////////////////////////////////////
```

Importante:

- Para todos los ejercicios, se ha de diseñar una batería de pruebas.
- Para poder leer un espacio en blanco **no** puede emplear `cin >> caracter`, sino `caracter = cin.get()`. Cada vez que se ejecute `cin.get()` el compilador lee un carácter (incluido el espacio en blanco) desde la entrada de datos por defecto.

35. Añadir dos constructores a clase `SecuenciaCaracteres`:

- a) Constructor que recibe un dato `string` e inicializa la secuencia con el contenido del dato `string`. Si la secuencia no pudiera contener todos los caracteres del `string` se llenará *completamente*, empezando por la primera posición del `string`.

Por ejemplo, las siguientes instrucciones:

```
SecuenciaCaracteres sec_hola ("¡Hola, mundo!");
cout << sec_hola.ToString() << endl;
```

mostrarán:

```
¡Hola, mundo!
```

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

- b) Constructor que recibe un carácter *c* y un entero *n* para crear una secuencia formada por *n* caracteres, todos ellos iguales al carácter *c*.

Por ejemplo, las siguientes instrucciones:

```
SecuenciaCaracteres sec_asteriscos ('*', 10);  
cout << sec_asteriscos.ToString() << endl;
```

mostrarán:

```
*****
```

Finalidad: Métodos que acceden a las componentes del vector. Dificultad Baja.

36. Añadir un método con la cabecera:

```
bool EsPalindromo (void)
```

que nos diga si la secuencia es un **palíndromo**, es decir, que se lee igual de izquierda a derecha que de derecha a izquierda. Por ejemplo, {'a', 'b', 'b', 'a'} sería un palíndromo.

Escribid un programa que forme una secuencia y compruebe si es un palíndromo.

Finalidad: Método que accede a las componentes del vector. Dificultad Baja.

37. Añadir el método `IntercambiaComponentes` para intercambiar dos componentes del vector. Por ejemplo, si el vector contiene {'h', 'o', 'l', 'a'}, después de intercambiar las componentes 1 y 3, se quedaría con {'h', 'a', 'l', 'o'}.

¿Qué debería hacer este método si los índices no son correctos?

Añadir otro método llamado `Invierte` para invertir el vector, de forma que si el vector contenía, por ejemplo, los caracteres {'m', 'u', 'n', 'd', 'o'}, después de llamar al método se quedará con {'o', 'd', 'n', 'u', 'm'}.

Nota: Para implementar este método, llamad a `IntercambiaComponentes()`.

Finalidad: Ejemplo de método que modifica el estado del objeto. Gestión de precondiciones. Dificultad Baja.

38. Añadir el método `EliminaMayusculas` para eliminar todas las mayúsculas. Por ejemplo, después de aplicar dicho método al vector {'S', 'o', 'Y', ' ', 'y', 'O'}, éste debe quedarse con {'o', ' ', 'y'}.

En este ejercicio proponemos implementar el siguiente algoritmo:

```
Recorrer todas las componentes del vector  
Si la componente es una mayúscula, borrarla
```

Para borrar la mayúscula llamad al método `Elimina()`.

Finalidad: Recorrido sencillo de un vector. Dificultad Baja.

39. El algoritmo del ejercicio 38 es muy ineficiente ya que requiere trasladar muchas veces muchas posiciones. En el ejercicio 22 (apartado c) de la *Relación de Problemas IV* propusimos una solución más eficiente, que empleaba dos variables (*apuntadores*): una que nos permitía recorrer el vector completamente (*posicion_lectura*) y otra (*posicion_escritura*) que nos indica el sitio dónde tiene que escribirse.

Implementar este algoritmo para eliminar todas las mayúsculas de una secuencia de caracteres.

Finalidad: Modificar un vector a través de dos apuntadores. Practicar el ocultamiento de información. Dificultad Media.

40. Añadir el método `EliminaOcurrencias` para eliminar todas las apariciones que hubiese de un determinado carácter `a_borrar` en la secuencia.

Por ejemplo, después de eliminar el carácter 'o' de la secuencia {'S','o','Y',' ','y','o'} ésta debe quedar: {'S','Y',' ','y'}.

Para conseguir una buena solución aplicad el algoritmo que emplea dos apuntadores.

Finalidad: Modificar un vector a través de dos apuntadores. Practicar el ocultamiento de información. Dificultad Media.

41. Añadir un método `EliminaRepetidos` que quite los elementos repetidos, de forma que cada componente sólo aparezca una única vez. Se mantendrá la primera aparición, de izquierda a derecha. Por ejemplo, si el vector contiene {'b','a','a','h','a','a','a','a','c','a','a','a','g'} después de quitar los repetidos, se quedaría como sigue: {'b','a','h','c','g'}

Para conseguir una buena solución aplicad el algoritmo que emplea dos apuntadores.

Finalidad: Modificar un vector a través de dos apuntadores. Practicar el ocultamiento de información. Dificultad Media.

42. Añadir los métodos `EliminaDesde` y `EliminaHasta` para eliminar todos los caracteres desde una posición dada y hasta una posición dada (incluida).

Por ejemplo, en la secuencia {'*',' ','H','o','l','a',' ',' ','*'} la ejecución de `EliminaDesde(6)` da como resultado que la secuencia se queda en {'*',' ','H','o','l','a'}. Si sobre esta última secuencia se ejecuta `EliminaHasta(1)` da como resultado la secuencia {'H','o','l','a'}.

Finalidad: Modificar un vector. Dificultad Baja.

43. Añadir un método `EliminaExcesoBlancos()` para eliminar el exceso de caracteres en blanco, es decir, que sustituya todas las secuencias de espacios en blanco por un sólo espacio. Por ejemplo, si el vector original es {' ','a','h',' ',' ',' ','c'}, que contiene una secuencia de tres espacios consecutivos, el vector resultante debe ser {' ','a','h',' ','c'}.

RELACIÓN DE PROBLEMAS V. Clases (Primera parte)

Nota: Debe hacerse lo más eficiente posible.

Finalidad: Recorrido de las componentes de un vector, en el que hay que recordar lo que ha pasado en la iteración anterior. *Dificultad Media.*

44. *(Basado en el Examen Ordinario 2019)* Añadir a la clase `SecuenciaCaracteres` un método que compruebe si después de cada aparición de un carácter determinado `c` siempre viene una secuencia de caracteres determinada `sec`. Cada `c` debe estar asociada a una secuencia `sec` **diferente**.

Ejemplos en los que el método devuelve `true`:

```
secuencia principal = a b T T U d a T T U    c = a    sec = T T U
secuencia principal = a b a c T T U T T U    c = a    sec = T T U
```

Ejemplos en los que el método devuelve `false`:

```
secuencia principal = a b T T U d d T T U a    c = a    sec = T T U
secuencia principal = a b a c T T U T T c      c = a    sec = T T U
```

Tenga en cuenta lo siguiente:

- Si la secuencia `sec` contiene al carácter `c`, el método devolverá `false`
- No puede utilizar el tipo de dato `string` en ningún sitio.
- Puede definir los métodos auxiliares que estime oportuno.

Dificultad Media.

45. *(Basado en el Examen Ordinario 2020)* A partir de la implementación básica de la clase `SecuenciaCaracteres` implemente un método que compruebe si una secuencia `s1` rima de forma asonante con otra secuencia `s2` con grado `k`. Esto significa que las últimas `k` vocales de `s1` y `s2` deberán coincidir. El método devolverá `true` si las dos secuencias `s1` y `s2` riman de forma asonante, y `false` en caso contrario.

Se valorará la eficiencia en tiempo y espacio.

Dificultad Media.

46. *(Examen Septiembre Doble Grado 2013)* Sobre la clase `SecuenciaEnteros`, defina lo que sea necesario para calcular el número de secuencias ascendientes del vector.

Por ejemplo, en el vector `{2,4,1,1,7,2,1}` encontramos 4 secuencias ascendentes, que son: `{2,4}`, `{1,1,7}`, `{2}`, `{1}`.

Dificultad Media.

47. Añada a la clase `SecuenciaEnteros` los métodos que implementan los algoritmos de ordenación que se describen en los apuntes de Teoría:

```
void OrdenaSeleccion()  
void OrdenaInsercion()  
void OrdenaIntercambio()  
void OrdenaIntercambioMejorado()
```

Para probar su funcionamiento, escriba un programa que use las clases `Cronometro` y `GeneradorAleatorioEnteros`. El programa rellenará una secuencia de 10000 datos enteros aleatorios, los ordenará con los cuatro métodos implementados y medirá el tiempo empleado por cada uno de ellos.

Dificultad Media.

48. (*Basado en el Examen Ordinario 2019*) En el ejercicio 33 de la *Relación de Problemas IV* definimos un nuevo criterio de orden entre dos números enteros positivos: Un valor entero a será mayor que otro b si el número de dígitos nueve es mayor en a que en b . Si el número de nueves es igual en los dos, el mayor será el que tiene más ochos. Si hay empate también con este dígito, se considera el siete. Así hasta llegar al cero, si fuese necesario. Si la frecuencia de todos los dígitos es igual en ambos valores, se les considera iguales.

Añada cuatro nuevos métodos a la clase `SecuenciaEnteros`:

```
void OrdenaSeleccion_NuevoOrden()  
void OrdenaInsercion_NuevoOrden()  
void OrdenaIntercambio_NuevoOrden()  
void OrdenaIntercambioMejorado_NuevoOrden()
```

Bastará una ligera modificación en los métodos implementados en el ejercicio 47).

escriba un programa que pida el número de elementos de la secuencia (cuántas casillas se van a ocupar), rellene esa secuencia con números aleatorios entre 0 y 999, la ordene con los métodos implementados y muestre el resultado.

Dificultad Baja.

49. Implementad la **Búsqueda por Interpolación** en la clase `SecuenciaEnteros`. El método busca un valor buscado entre las posiciones `izda` y `dcha`.

Este algoritmo recuerda a la búsqueda binaria porque 1) requiere que el vector en el que se va a realizar la búsqueda esté ordenado y 2) en cada consulta sin éxito se descarta una parte del vector para la siguiente búsqueda.

La diferencia fundamental con la búsqueda binaria es la manera en que se calcula el elemento del vector que sirve de referencia en cada consulta (que ocupa la posición `pos`). Ahora el valor de referencia no es el que ocupa la posición central del subvector en el que se efectúa la búsqueda (el delimitado únicamente por `izda` y `dcha`), sino que depende también del contenido de esas casillas, de manera que 1) `pos` será más

cercana a dcha si buscado es más cercano a $v[dcha]$ y 2) pos será más cercana a izda si buscado es más cercano a $v[izda]$. En definitiva, se cumple la relación:

$$\frac{\text{pos} - \text{izda}}{\text{dcha} - \text{izda}} = \frac{\text{buscado} - v[\text{izda}]}{v[\text{dcha}] - v[\text{izda}]}$$

Dificultad Media.

50. *(Basado en el Examen Extraordinario 2020)* En la figura 44 se muestra un *Triángulo de Pascal* (o *Tartaglia*). Observe que, exceptuando los 1 que siempre están en los extremos, cada número es igual a la suma de los dos números que tiene justo encima.

Fila 1																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																													</
--------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----

Figura 44: Triángulo de Pascal con 7 filas.

Se desea implementar la clase `TrianguloPascal`. Para usar eficientemente el espacio se decide emplear un objeto de la clase `SecuenciaEnteros` para guardar los valores del triángulo (use la implementación básica de la clase `SecuenciaEnteros`). La parte privada de la clase `TrianguloPascal` será:

```
SecuenciaEnteros valores;  
int num_filas;
```

Por ejemplo, para el triángulo mostrado en la figura 44, el campo `num_filas` valdrá 7 y la secuencia `valores` contendrá los valores: 1, 1, 1, 1, 2, 1, 1, 3, 3, 1, 1, 4, 6, 4, 1, 1, 5, 10, 10, 5, 1, 1, 6, 15, 20, 15, 6 y 1. Observe que los datos de la fila f ($f \geq 1$) del triángulo empiezan en la posición $p = (f(f - 1))/2$ de la secuencia ($p \geq 0$).

La clase debe tener la siguiente funcionalidad:

- a) Métodos simples. a) Constructor sin argumentos: crea y rellena un triángulo de Pascal con *las dos primeras* filas. b) Métodos de consulta simples: `NumFilas`, para saber cuántas filas tiene el triángulo, y `NumCasillas`, para saber cuántas casillas tiene una fila dada del triángulo.
- b) Método `AniadeFilas`: para añadir al triángulo un número de filas dado.

- c) Método `ObtenerFila`: para obtener todos los valores de una fila dada. Los valores se obtienen en un objeto de la clase `SecuenciaEnteros`. Si la fila solicitada no estuviera calculada, se calcularían (y añadirían al triángulo) tantas filas como fueran necesarias.
- d) Método `Valor`: para obtener el valor que ocupa la casilla `c` en la fila `f`. Si la fila solicitada no estuviera calculada, se calcularían (y añadirían al triángulo) tantas filas como fueran necesarias.

Escriba un programa completo que lea un valor positivo `n` y cree un triángulo de Pascal con `n` filas. A continuación mostrará los valores del triángulo, fila a fila. Finalmente mostrará los valores centrales de las filas impares del triángulo. En el caso de un triángulo de 5 filas, por ejemplo, mostrará los valores 1 (fila 1), 2 (fila 3) y 6 (fila 5).

Escriba todos los métodos (públicos o privados) adicionales que pudiera necesitar. Escriba también las precondiciones aplicables a los datos y métodos.

Dificultad Media.

51. Cread una clase `Frase` para almacenar un conjunto de caracteres. Internamente, como dato miembro privado, se usará un vector de caracteres. Aunque esta clase es similar a la clase `SecuenciaCaracteres` en su representación, veremos que en la práctica es muy diferente.

Una frase está formada por palabras. Una palabra es una serie de caracteres consecutivos delimitada por espacios en blanco. La primera palabra de una frase no tiene por qué tener espacios a su izquierda y la última no tiene por qué tener espacios a su derecha. Puede haber varios espacios en blanco consecutivos entre dos palabras.

Añadid los siguientes métodos:

- Constructor que recibe como argumento un `string`.
- `void EliminaBlancosIniciales()` para borrar todos los blancos iniciales.
- `void EliminaBlancosFinales()` para borrar todos los blancos finales.
- `int NumeroPalabras()` que indique cuántas palabras hay en la frase.
- `int LongitudPalabra(int k_esima)` que indique la longitud (número de caracteres) de la `k`-ésima palabra.
- `int PosPalabra(int k_esima)` que indique la posición donde empieza la `k`-ésima palabra.
- `void BorraPalabra(int k_esima)` para que borre la palabra `k`-ésima.
- `string Palabra(int k_esima)` que devuelva la `k`-ésima palabra.

Dificultad Media.

52. (*Basado en el Examen Ordinario de 2020*) Una palabra es **seudosimétrica** si, cuando se parte por la mitad, da como resultado dos palabras que tienen las mismas letras con las mismas frecuencias. Si la longitud de la palabra es un número impar, la decisión se toma sin considerar la letra central. Por ejemplo *gaga* es una palabra *seudosimétrica*. Las dos mitades *ga* y *ga* tienen los mismos caracteres con la misma frecuencia. Otros ejemplos son *rotor*, *aabccaba*, *xyzxy*. En cambio, la palabra *abbaab* NO es *seudosimétrica*.

Use un objeto de la clase `SecuenciaCaracteres` en este ejercicio para representar una palabra. Supondremos que los caracteres almacenados pertenecen al conjunto de letras minúsculas consecutivas del alfabeto inglés (`abcdefghijklmnopqrstuvwxyz`).

Implemente los siguientes métodos en la clase:

```
SecuenciaCaracteres SubSecuencia (int num, int inicio);
```

Devuelve una secuencia de caracteres compuesta por un máximo de `num` caracteres, siendo el primero el que se encuentra en la posición `inicio`. Deberá considerar todas las situaciones posibles que puedan ocurrir con los valores de `inicio` y `num` y el tamaño de la secuencia, intentando siempre devolver una secuencia de caracteres (posiblemente vacía).

```
bool EsSeudoSimetrica (void);
```

Comprueba si la secuencia representa una palabra *seudosimétrica*.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

Copiando objetos - Métodos y objetos

1. Diseñad la clase `Fraccion` con la siguiente funcionalidad:

- Definid tres constructores para la clase:
 - Constructor sin argumentos. Pensad y establecer adecuadamente los valores por defecto que se consideren adecuados.
 - Constructor con un argumento.
 - Constructor con dos argumentos.
- Añadir métodos para sumar, restar, multiplicar y dividir dos fracciones. En todos ellos el objeto implícito (sobre el que se aplica el método) y el objeto explícito no se modifican. Además, la fracción resultante será una fracción irreducible.

Finalidad: Ver métodos a los que se les pasa como parámetro y devuelven un objeto de su misma clase. Dificultad Baja.

2. Cread una clase `Permutacion` para representar una permutación de enteros.

- Para almacenar los valores usaremos un *vector* de enteros.
- Pasaremos como parámetro al constructor un objeto de la clase `SecuenciaEnteros`. Supondremos como precondition que el vector es una **permutación correcta**, es decir, que contiene todos los enteros sin repetir entre el mínimo y el máximo de dichos valores. Por ejemplo, $(2, 3, 6, 5, 4)$ es una permutación correcta pero no lo es $(7, 7, 6, 5)$ (tiene el 7 como valor repetido) ni tampoco $(7, 6, 4)$ (le falta el 5).
- Añadid un método que calcule el número de lecturas de dicha permutación. Una permutación de un conjunto de enteros, $C = \{1, \dots, n\}$, tiene k lecturas, si para leer sus elementos en orden creciente (de izquierda a derecha) tenemos que recorrer la permutación k veces. Por ejemplo, la siguiente permutación del conjunto $\{0, \dots, 8\}$:

4 0 8 1 2 5 3 6 7

necesita 3 lecturas. En la primera obtendríamos 0, 1, 2 y 3. En la segunda 4, 5, 6 y 7 y finalmente, en la tercera, 8.

Finalidad: Métodos que reciben objetos de otra clase. Dificultad Media.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

3. Crear la clase `GeneradorPermutaciones` para generar permutaciones aleatorias de un conjunto de enteros entre un valor mínimo y un valor máximo.

La clase tendrá un único método con la siguiente cabecera:

```
Permutacion Genera(int primero, int ultimo)
```

dónde la clase `Permutacion` es la vista en el problema 2 de esta misma relación de problemas. Por ejemplo, si `primero = 1` y `ultimo = 6`, una permutación válida sería `{3, 1, 6, 4, 5, 2}`. Como puede observarse, no pueden aparecer elementos repetidos y deben estar todos los valores entre 1 y 6.

Finalidad: Métodos que devuelven objetos de otra clase y usan localmente otros objetos. Dificultad Media.

4. Amplíe la clase `Instante` sobre la que se empezó a trabajar en el ejercicio 11 de la *Relación de Problemas V*. Esa clase servía para representar un instante de tiempo dentro de un día. Añada los métodos:

- `bool EsAnterior(Instante otro)`
para saber si el instante del objeto implícito es anterior a otro.
- `Instante Avanza (Instante otro)`
para calcular el instante que resulta de sumar otro al instante registrado en el objeto implícito. En este caso, otro representa una cantidad de tiempo más que un instante concreto.
- `Instante Retrocede (Instante otro)`
para calcular el instante que resulta de restar otro al instante registrado en el objeto implícito.
- `Instante Salto (int num_segundos)`
para calcular el instante que resulta de sumar/restar num_segundos al instante registrado en el objeto implícito.
- `Instante Diferencia (Instante otro)`
para calcular la diferencia de tiempo (en horas, minutos y segundos) entre el instante implícito y otro.

Cree un programa para probar los nuevos métodos.

Finalidad: Ver métodos a los que se les pasa como parámetro (y/o devuelven) un objeto de la misma clase. Dificultad Baja.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

5. Ampliad la clase Dinero. Partimos del siguiente esquema (resultado del ejercicio 4 de la *Relación de Problemas V*):

Dinero
- int euros; // PRE: 0 <= euros - int centimos; // PRE: 0 <= centimos < 100
+ Dinero (void); + Dinero (double cantidad); // PRE: 0 <= cantidad + Dinero (int los_euros, int los_centimos); // PRE: 0 <= los_euros 0 <= los_centimos + int GetEuros (void); + int GetCentimos (void); + double GetValor (void); + void Incrementa (double cantidad); // PRE: 0 <= cantidad + void Incrementa (int euros_suman, int centimos_suman); // PRE: 0 <= euros_suman 0 <= centimos_suman + string ToString (int num_casillas_euros, char caracter_hueco); // PRE: 0 <= num_casillas_euros

Descarte el método

```
void Incrementa (int euros_suman, int centimos_suman);
```

Añada los métodos:

- Dinero Incrementa(Dinero cantidad)
calcula y devuelve el resultado de sumar cantidad a la cantidad de dinero del objeto implícito. El valor de cantidad deber ser positivo.
- Dinero Decrementa(Dinero cantidad)
- Dinero Decrementa(double cantidad)
calcula y devuelve el resultado de restar cantidad a la cantidad de dinero del objeto implícito. El valor de cantidad deber ser positivo. Si no es posible realizar la resta, no se hace nada.
- Dinero Modifica(double cantidad)
que calcula y devuelve el resultado de sumar/restar cantidad a la cantidad de dinero del objeto implícito. Si no es posible realizar la resta, no se hace nada.

- `bool EsMayor(Dinero otro)`
- `bool EsMayor(double cantidad)`
- `bool EsMenor(Dinero otro)`
- `bool EsMenor(double cantidad)`
- `bool EsIgual(Dinero otro)`
- `bool EsIgual(double cantidad)`

devuelve *true* si la cantidad de dinero del objeto implícito es mayor/menor/igual que la de otro (o la que indica cantidad).

- `Dinero Diferencia(Dinero otro)`
- `Dinero Diferencia(double otro)`

que calcula y devuelve la diferencia entre la cantidad de dinero del objeto implícito y otro. Para poder realizar esta operación, el objeto implícito tiene un valor mayor que otro, evidentemente.

Cree un programa para probar los nuevos métodos.

Finalidad: Ver métodos a los que se les pasa como parámetro (y/o devuelven) un objeto de la misma clase. Sobrecarga de métodos. Dificultad Baja.

6. Queremos disponer de una clase que nos permita trabajar con datos *binarios* (la clase `Bin`) y realizar las operaciones básicas típicas sobre éstos.

Vamos a restringir el tamaño a un máximo de 32 bits y sólo consideraremos valores positivos. Así, podremos operar con valores (decimales) entre 0 y $2^{32} - 1 = 4294967295$. Investigue y vea que el tipo `unsigned int` requiere 4 bytes (32 bits) para su almacenamiento y que el máximo valor que puede representarse es 4294967295 (consulte `UINT_MAX` en `climits`). Esto nos ayudará a adoptar un dato `unsigned int` como campo privado de la clase. Es económico (4 bytes para cada valor a representar) y disponemos de un amplio surtido de operaciones aritméticas que pueden realizarse con datos `unsigned int`.

La clase `Bin` responderá al esquema dado en la figura 45. Los métodos se comportan como se describe a continuación.

Nota: Escriba las precondiciones lógicas para cada caso.

- Constructores.
`Bin (void);`
`Bin (unsigned int el_valor_inicial);`
`Bin (string el_valor_inicial);`

Crean un objeto `Bin` y lo inicializan al valor dado. El constructor sin argumentos inicializa el objeto con el valor 0.

Bin
<pre>- unsigned int valor_bin; // PRE: 0 <= valor_bin < 2^{32}-1</pre>
<pre>+ Bin (void); + Bin (unsigned int el_valor_inicial); + Bin (string el_valor_inicial); + void SetValor (unsigned int el_valor); + void SetValor (string int el_valor); + int MinNumBits (void); + unsigned int GetValorDecimal (void); + string ToString (int num_casillas); + Bin Suma (Bin otro); + Bin Resta (Bin otro); + Bin AND (Bin otro); + Bin OR (Bin otro); + Bin DespDcha (int veces); + Bin DespIzda (int veces);</pre>
<pre>- string DecToBin (unsigned int num_decimal, int num_casillas); - unsigned int BinToDec (string cad);</pre>

Figura 45: La clase Bin

- Métodos que modifican el objeto (*set*).

```
void SetValor (unsigned int el_valor);
void SetValor (string int el_valor);
```

Modifica el valor del dato binario sustituyéndolo por `el_valor`.

- Métodos de consulta (*get*).

```
int MinNumBits (void);
```

Devuelve cuántos bits se requieren para representar el valor.

```
unsigned int GetValorDecimal (void);
```

Devuelve el valor decimal del dato binario.

```
string ToString (int num_casillas);
```

Devuelve un string con el valor binario. El valor binario ocupa num_casillas. En el caso de no precisar tantas casillas se rellenan con ceros a la izquierda. Si se necesitan más casillas se usan las que se requieran para no perder información. El valor binario obtenido siempre está delimitado por los caracteres [y]

- Métodos de cálculo.

```
Bin Suma (Bin otro);  
Bin Resta (Bin otro);  
Bin AND (Bin otro);  
Bin OR (Bin otro);
```

Realiza la operación indicada y devuelve un **nuevo** objeto con el resultado. Ni el objeto implícito ni el explícito (otro) se modifican.

- Métodos de desplazamiento de bits.

```
Bin DespDcha (int veces);  
Bin DespIzda (int veces);
```

Realiza la operación indicada (desplazamiento de los bits tantas casillas como indique veces) y devuelve un **nuevo** objeto con el resultado. Los “huecos” se rellenan con ceros. El objeto implícito no se modifica.

- Los métodos privados DecToBin y BinToDec son, básicamente, las funciones desarrolladas en los ejercicios 39 y 40 de la *Relación de Problemas III*.

Finalidad: Ver métodos a los que se les pasa como parámetro (y/o devuelven) un objeto de la misma clase. Sobrecarga de métodos. Dificultad Baja.

7. Sobre la clase SecuenciaCaracteres, defina un método que inserte otra secuencia (otra) a partir de una posición (pos_insercion).

```
void InsertaSecuenciaCaracteres(int pos_insercion,  
                                SecuenciaCaracteres otra)
```

Implemente este método utilizando un método que inserta un único carácter:

```
void Inserta(int pos_insercion, char valor_nuevo)
```

Inserta el carácter valor_nuevo en la posición pos_insercion. Tras la inserción, valor_nuevo ocupará la posición pos_insercion.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

Tendrá que llamar a este método para insertar, uno a uno, todos los caracteres de la secuencia otra.

Finalidad: Métodos que reciben como parámetro un objeto de la misma clase. Dificultad Media.

8. Implemente el método del ejercicio 7 de una forma eficiente, sin tener que llamar al método de inserción de un carácter por cada carácter que se vaya a insertar.

Finalidad: Métodos que reciben como parámetro un objeto de la misma clase. Dificultad Media.

9. Modifique el método del ejercicio 8 de esta Relación de Problemas de manera que ahora devuelva una **nueva** secuencia. Las secuencia original (la implícita) y la secuencia otra no se modifican.

Finalidad: Métodos que reciben como parámetro un objeto de la misma clase y devuelve un objeto de la misma clase. Dificultad Media.

10. ([Examen Septiembre 2014](#)) Sobre la clase SecuenciaCaracteres implemente el algoritmo **Counting Sort** para ordenar **todos** sus valores.

```
SecuenciaCaracteres CountingSort (void);
```

El método no modificará las componentes del vector privado sino que debe construir una secuencia nueva y devolverla.

El algoritmo funciona de la siguiente forma:

- 1) Calcular los valores mínimo y máximo del vector. Use métodos específicos para estas tareas.

Por ejemplo, si el vector contiene

c b b a b c c a g c b g c

el mínimo es 'a' y el máximo 'g'.

- 2) Construir un *vector local* para contabilizar las frecuencias de todos los caracteres que hay entre el mínimo y el máximo.

Con el ejemplo anterior, el vector de frecuencias será

2 4 5 0 0 0 2

que corresponden a las frecuencias de las letras que hay entre 'a' y 'g'.

Nota: También podría usar una *secuencia de enteros* para guardar los contadores en lugar del array citado.

- 3) Recorrer el vector de frecuencias almacenando cada carácter tantas veces como indique su frecuencia.

En el ejemplo: 2 veces el 'a', cuatro veces el 'b', etc.

a a b b b b c c c c c g g

Ampliación: Ordenar **una parte** de la secuencia, dados los extremos:

`SecuenciaCaracteres CountingSort (int izda, int dcha);`

Finalidad: Trabajar con métodos con vectores locales / secuencias locales. Devolver un objeto de la misma clase. Dificultad Media.

11. (*Examen Septiembre 2012*) Definid la clase `ConjuntoOrdenado` para almacenar una secuencia **ordenada** de números enteros **sin repetidos**. Definid métodos para:

- Añadir un entero (de forma ordenada y sin almacenar repetidos).
- Calcular la unión con otro conjunto. En la unión se deben incluir los elementos que estén en cualquiera de ellos.
- Calcular la intersección con otro conjunto. En la intersección se deben incluir los elementos que sean comunes a ambos conjuntos.

12. (*Examen Convocatoria Ordinaria 2019*) En matemáticas, un *multiconjunto* es una modificación del concepto de conjunto que, a diferencia de éste, permite múltiples instancias para cada uno de sus elementos. El número entero positivo de instancias de cada elemento se llama *multiplicidad* de este elemento en el multiconjunto. Por ejemplo, en el multiconjunto $\{5, 5, 5, 3, 3, 2\}$, 5 tiene multiplicidad 3, 3 tiene multiplicidad 2 y 2 tiene multiplicidad 1.

Los elementos de un multiconjunto se toman en un conjunto fijo U (*universo*). Dado un multiconjunto A en un universo U , para cada elemento $x \in U$, definimos $m_A(x)$ como la multiplicidad de x en A . Entonces, se pueden definir las siguientes operaciones sobre multiconjuntos:

- El **soporte** de un multiconjunto A en un universo U , se define como $Sop(A) = \{x \in U \mid m_A(x) > 0\}$.
- A está **incluido** en B , denotado como $A \subseteq B$, si $\forall x \in U, m_A(x) \leq m_B(x)$.
- La **intersección** de A y B es un multiconjunto C tal que $m_C(x) = \min\{m_A(x), m_B(x)\} \forall x \in U$.

Se requiere implementar la clase `Multiconjunto` con $U = \{1, 2, \dots, 1000\}$.

Observe que no es necesario representar explícitamente todas las repeticiones de cada valor. Así pues, en vez de representar el multiconjunto $\{5, 5, 5, 3, 3, 2\}$ como una secuencia de enteros repetidos, resulta más conveniente trabajar con la multiplicidad de cada valor del universo U . Por lo tanto, para representar el anterior multiconjunto basta con considerar cada valor del dominio (5, 3, 2) y sus multiplicidades asociadas (3,2,1).

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

Nota: Use la clase `SecuenciaEnteros`

Deberá realizar:

- a) Defina los datos miembro de la clase y los constructores que estime oportunos.
 - b) Implemente un método para obtener la multiplicidad de un número perteneciente a U en un multiconjunto y un método para añadir un elemento x al multiconjunto con cierta multiplicidad k . Si el elemento ya existe en el multiconjunto, se incrementará su multiplicidad en k .
 - c) Diseñe métodos que implementen las tres operaciones definidas con anterioridad (soporte, inclusión e intersección). El soporte devolverá un objeto de la clase `SecuenciaEnteros`, la inclusión un `bool` y la intersección un objeto de la clase `Multiconjunto`.
 - d) Implemente un método que devuelva un objeto de la clase `SecuenciaEnteros` con **todos** los elementos de un multiconjunto. No se requiere que estén ordenados. Para el ejemplo anterior, se devolvería $\{5, 5, 5, 3, 3, 2\}$.
13. En el ejercicio 10 de la *Relación de Problemas V* se planteaba el diseño básico de la clase `CoordenadasGPS` y donde se escribieron los métodos `set` y `get` para fijar y recuperar los valores de latitud, longitud y altura de un punto.

Ampliad la clase con métodos que permitan:

- Calcular la diferencia de altura entre dos puntos (entre dos objetos `CoordenadasGPS`).
- Calcular la distancia *sobre plano* entre dos puntos (ver ejercicio 38 de la *Relación de Problemas I*).
- Calcular la distancia *real* entre dos puntos (ver ejercicio 38 de la *Relación de Problemas I*).

Construir un programa que pida las coordenadas GPS de dos puntos y calcule la diferencia de altura, la distancia sobre plano, la distancia real entre ellos.

14. Ampliar la clase `Recta` (ver apuntes) para añadir métodos que realicen las tareas:

- a) Calcular si dos rectas son paralelas, secantes, perpendiculares o coincidentes (métodos `bool`).
- b) Calcular y devolver la recta paralela que pasa por un punto dado.
- c) Calcular y devolver la recta perpendicular que pasa por un punto dado.
- d) Si dos rectas son secantes, devolver el punto de corte.

Finalidad: Métodos que reciben y devuelven objetos. Dificultad Media.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

15. Cread la clase RLE para codificar/descodificar una secuencia de enteros. Los métodos a implementar tendrán las siguientes cabeceras:

```
SecuenciaEnteros Codifica      (SecuenciaEnteros
                                secuencia_descodificada)
SecuenciaEnteros Descodifica   (SecuenciaEnteros
                                secuencia_codificada)
```

Por ejemplo, la llamada al método `Codifica()` pasándole como parámetro el vector (1 1 1 2 2 2 2 3 3 3 3 3 5) debe devolver el vector (3 1 5 2 6 3 1 5) mientras que el método `Descodifica()` hace lo contrario.

Nota: La clase RLE no necesita ningún campo.

Escribid un programa principal que lea un número indeterminado de enteros (terminando con un -1) y los guarde en un objeto de la clase `SecuenciaEnteros`. A continuación la codificará y descodificará en una nueva secuencia.

Añadir un método público a la clase `SecuenciaEnteros`:

```
bool EsIgual (SecuenciaEnteros otro);
```

que sirva para comparar dos datos de tipo `SecuenciaEnteros`: serán iguales si contienen el mismo número de elementos, son iguales y están en el mismo orden. Usad este método para demostrar que la secuencia original es igual a la secuencia obtenida después de codificarla y descodificarla.

Finalidad: Ver métodos a los que se les pasa como parámetro (y/o devuelven) un objeto de otra clase. Dificultad Baja.

16. *(Examen Febrero 2009)* Según un estudio de una universidad ignifera, no importa el orden en el que las letras están escritas, la única cosa importante es que la primera y la última letra estén escritas en la posición correcta. El resto pueden estar totalmente mal y aún podrías leerlo sin problemas. Esto es porque no leemos cada letra por sí misma sino la palabra como un todo.

Se pide crear la clase `Palabra` que permita almacenar un conjunto de caracteres que representarán una palabra. Definid un método `EsIgual_Debil` al que se le pase como parámetro otra palabra y determine si son iguales atendiendo al siguiente criterio: *La primera letra de ambas palabras es igual, la última letra de ambas palabras también es igual y el resto de las letras son las mismas, pero no están necesariamente en las mismas posiciones.*

Dificultad Baja.

17. (*Examen Septiembre 2012*) Defina la clase `SecuenciaParejas_ValorFrecuencia` que permite almacenar un conjunto de parejas en la que cada pareja representa un valor y su frecuencia. Por ejemplo, si los datos que se manejan son edades, cada pareja indica una edad (el valor) y las veces que aparece (su frecuencia).

En este problema se trabajará con datos `char`. Por simplicidad, cada pareja puede representarse con un dato `struct`:

```
struct Pareja_ValorFrecuencia{
    char valor;
    int  frecuencia;
}
```

La clase `SecuenciaParejas_ValorFrecuencia` se describe en la figura 46. Implemente la clase de acuerdo a esta especificación.

SecuenciaParejas_ValorFrecuencia	
- const int	<u>TAMANIO</u>
- Pareja_ValorFrecuencia	vector_privado[TAMANIO]
- int	total_utilizados
+	Pareja_ValorFrecuencia()
+ int	TotalUtilizados()
+ int	Capacidad()
+ void	Aniade(Pareja_ValorFrecuencia nuevo)
+ Pareja_ValorFrecuencia	Elemento(int indice)
+ void	Elimina(int indice)
+ string	ToString()

Figura 46: La clase `SecuenciaParejas_ValorFrecuencia`

Escribir un método de la clase `SecuenciaCaracteres` que reciba un objeto de la clase `SecuenciaParejas_ValorFrecuencia` de manera que elimine en la secuencia de caracteres el carácter indicado en cada pareja, y tantas veces como indique su frecuencia.

Por ejemplo, si una secuencia de parejas, `sp`, contiene las parejas `{(a, 1), (b, 2)}` y una secuencia de caracteres, `s`, los valores `{a, b, a, b, c, a, b, d, a}` la llamada

```
s.Borrar (sp);
```

dejará el objeto `s` en este estado: `{a, c, a, b, d, a}`

Finalidad: Trabajar con un vector de objetos. Dificultad Media.

18. (*Examen Septiembre 2016*) Añadir a la clase `SecuenciaEnteros` un método llamado `BusquedaHaciaArriba` que devuelva otra secuencia de enteros que contenga los n elementos ordenados iguales o mayores a un elemento valor de referencia dado.

La implementación deberá ser lo más eficiente posible.

El método recibirá el valor de referencia r y el número de elementos a devolver n . Hay que realizar todas las comprobaciones necesarias para que el método funcione correctamente. En caso de que no se puedan devolver n elementos, se devolverán el máximo posible de ellos. La `SecuenciaEnteros` original permanecerá inalterada tras la terminación de la ejecución. Cualquier otro método que emplee para la resolución del ejercicio tendrá que ser declarado y definido dentro de la clase.

Por ejemplo, dada la secuencia:

(12 0 4 2 6 6 19 21 14 28 9 14 2 11 14)

la búsqueda hacia arriba con $p = 6$ y $n = 3$ sería $\{6, 6, 9\}$, y con $p = 15$ y $n = 5$ sería $\{19, 21, 28\}$.

19. En el ejercicio 15 construimos una clase gestora (RLE) que era capaz de codificar y decodificar secuencias de enteros mediante el algoritmo RLE. El resultado era siempre una secuencia de enteros.

Si la secuencia que se quiere codificar no es de datos enteros (por ejemplo, `SecuenciaCaracteres`) no puede emplearse la clase RLE para guardar el resultado de la codificación porque el resultado debería ser una secuencia de parejas formadas por un *entero* (la frecuencia, o número de veces) y un *carácter* (el valor). Podría modificarse la clase RLE para que en lugar de generar/recibir una secuencia de enteros pudiera recibir/devolver una secuencia de parejas (`char`, `int`), o (`int`, `int`), de manera que la clase RLE fuera lo más independiente posible del tipo de la secuencia.

Modificar la clase RLE de acuerdo a estas indicaciones, introduciendo las clases `SecuenciaParejas...` en la nueva solución.

Finalidad: Trabajar con varias clases. Trabajar con un vector de objetos. Trabajar con métodos que reciben/devuelven objetos. Dificultad Media.

20. (*Examen Conv. Extraordinaria Febrero 2022*) Un **palíndromo** es una secuencia que se lee igual de izquierda a derecha que de derecha a izquierda. Se pide que defina un método que calcule y devuelva el *palíndromo de mayor longitud* contenido en una secuencia. Dicho palíndromo siempre existe y será un objeto de la clase `SecuenciaCaracteres`.

`SecuenciaCaracteres MayorPalindromoContenido (void);`

Si hay varios con la misma longitud, tiene libertad para elegir cuál de ellos devolver.

Por ejemplo, el mayor palíndromo contenido en CABBADÉ es la subsecuencia ABBA, mientras que el mayor palíndromo contenido en ABCDEF es cualquiera de las subsecuencias de tamaño 1, por ejemplo A. El mayor palíndromo contenido en ABBA es la subsecuencia ABBA, y el mayor palíndromo en gAAtySHHSvvABCCBAfh es la subsecuencia ABCCBA.

Nota: Se valorará especialmente la **eficiencia** del método y la **modularidad**.

21. Reescriba la solución al ejercicio 32 de la *Relación de Problemas V*. En ese ejercicio se leía un número indeterminado de datos referentes a los empleados de una empresa que se almacenaban en un array.

En el ejercicio citado se pedía usar la clase `Empleado` para representar a cada uno de los empleados, aunque el campo que guarda la fecha de nacimiento seguía siendo un `struct`. También se pedía usar un array de objetos `Empleado` para almacenar los empleados leídos. Proceda de igual manera en este ejercicio.

El programa que debe escribir realiza las mismas tareas que el indicado: 1) muestre el contenido del array después de leer los datos de los empleados, 2) calcule los salarios mínimos y máximos, muestre sus valores y los nombres y apellidos de los empleados que ganan menos y más, y finalmente, 3) calcule el salario medio, muestre su valor y después muestre un listado de los empleados cuyo salario es superior al salario medio.

Muy importante: En esta versión deberá realizar las comparaciones entre datos `Empleado`.

Finalidad: Métodos que reciben objetos. Dificultad Baja.

Objetos como datos miembro de otros objetos

22. En el ejercicio 12 de la *Relación de Problemas V* construimos la clase `DepositoBancario`. En ese momento el capital se representó con un dato `double` pero ahora parece más razonable representarlo con un dato de la clase `Dinero` (ver ejercicio 5 de esta misma relación de problemas).

```
class DepositoBancario {
    private:
        Dinero capital; // PRE: 0 < capital.GetValor()
        double interes; // PRE: 0 < interes <= 100
    public:
        .....
};
```

Reescriba la clase `DepositoBancario` y la solución del ejercicio 12 de la *Relación de Problemas V*.

23. Se trata de una pequeña modificación del ejercicio 32 de la *Relación de Problemas V*. La modificación consiste en escribir la clase `Fecha` para representar la fecha de nacimiento de cada empleado. Realice los cambios necesarios en la solución del ejercicio mencionado. La funcionalidad debe ser la misma.

Finalidad: Trabajar con clases que contienen datos de otras clases. Trabajar con arrays de objetos. Dificultad Media.

24. Reutilice las clases `Empleado` y `Fecha` con las que ha trabajado en el ejercicio 23 de esta misma *Relación de Problemas* para adaptar el ejercicio 40 de la *Relación de Problemas IV* de manera que las ordenaciones propuestas se realicen sobre un **array** de objetos de la clase `Empleado` (que contienen un campo de clase `Fecha`).

Finalidad: Trabajar con clases que contienen datos de otras clases. Trabajar con arrays de objetos. Métodos que reciben/devuelven objetos. Dificultad Media.

25. Reescriba el ejercicio 24 de esta *Relación de Problemas* para ordenar una colección de datos de clase `Empleado` que contienen un dato de clase `Fecha`. Organice ahora los datos `Empleado` en una **secuencia**. Deberá crear la clase `SecuenciaEmpleados` y realizar las ordenaciones propuestas sobre la secuencia.

Finalidad: Trabajar con clases que contienen datos de otras clases. Trabajar con secuencias de objetos. Métodos que reciben/devuelven objetos. Dificultad Media.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

26. Usad la clase `Punto2D` para representar un punto en $\mathbb{R}^2$ y cread la clase `Circunferencia` que represente el centro con un dato del tipo `Punto2D`.

Además de los constructores necesarios añade el siguiente conjunto mínimo de métodos:

- a) obtener la longitud de la circunferencia,
- b) obtener el área del círculo interior,
- c) obtener el valor del centro de la circunferencia,
- d) obtener el valor del diámetro de la circunferencia,
- e) saber si la circunferencia contiene a un punto. Un punto (x, y) está dentro de una circunferencia con centro (x_0, y_0) y radio r si se verifica que:

$$(x_0 - x)^2 + (y_0 - y)^2 \leq r^2$$

(El valor de π es constante, y el mismo para todos los objetos `Circunferencia`).

Asociada a la clase `Punto2D` deberá implementar una *clase lectora* (lo que llamamos una "fábrica de objetos". Le recomendamos estudiar los apuntes para más detalles. Como guía, considere que la clase `LectorPunto2D` tendría una estructura como indicamos a continuación:

LectorPunto2D
- string mensaje
+ LectorPunto2D (string el_mensaje);
+ void SetMensaje (string nuevo_mensaje));
+ void ImprimeMensaje (void);
+ Punto2D Lee (void);

Finalidad: Declarar un objeto como dato miembro de otro objeto. Clases lectoras. Dificultad Media.

27. Usando como referencia la clase `Punto2D` (y su clase lectora asociada, `LectorPunto2D`) para representar un punto en $\mathbb{R}^2$, diseñar e implementar la clase `Rectangulo`.

Proponemos usar las propiedades (figura 47.A):

- e_{si} , esquina superior izquierda (de la clase `Punto2D`),
- b , longitud de la base (`double`), y
- a , longitud de la altura (`double`).

Notas:

- a) Puede escoger cualquier otra representación si lo desea, siempre que no tenga datos miembro redundantes.
- b) Considere únicamente rectángulos cuyos lados sean paralelos a los ejes de coordenadas.

Además de los constructores necesarios añada el siguiente conjunto de métodos:

- a) obtener los valores de cada esquina (un método por esquina),
- b) obtener la longitud de la base,
- c) obtener la longitud de la altura,
- d) obtener el perímetro del rectángulo,
- e) obtener el área del rectángulo,
- f) obtener el valor del punto central del rectángulo, y
- g) calcular si un punto está dentro de un rectángulo.

Finalidad: Declarar un objeto como dato miembro de otro objeto. Clases lectoras. Dificultad Media.

28. Reescribir la solución del ejercicio 6 de la *Relación de Problemas V* con las nuevas definiciones de las clases `Punto2D`, `Rectangulo` y `Circunferencia` desarrolladas en los ejercicios 26 y 27 de esta misma Relación de Problemas.

Nota: No olvide el uso de la clase lectora asociada, `LectorPunto2D`.

Finalidad: Declarar un objeto como dato miembro de otro objeto. Clases lectoras. Dificultad Media.

29. Usando como referencia la clase `Punto2D` para representar un punto en $\mathbb{R}^2$ diseñar e implementar la clase `Triangulo`. Proponemos usar as coordenadas de los tres vértices (V_A , V_B , V_C) para la representación de un dato `Triángulo` (figura 47.B):

Además de los constructores necesarios añada el siguiente conjunto de métodos:

- a) obtener los valores de cada esquina (un método por esquina),
- b) obtener los valores de los ángulos interiores (un método por ángulo),
- c) calcular el perímetro del triángulo, y
- d) calcular si un triángulo es rectángulo.

Finalidad: Declarar un objeto como dato miembro de otro objeto. Dificultad Baja.

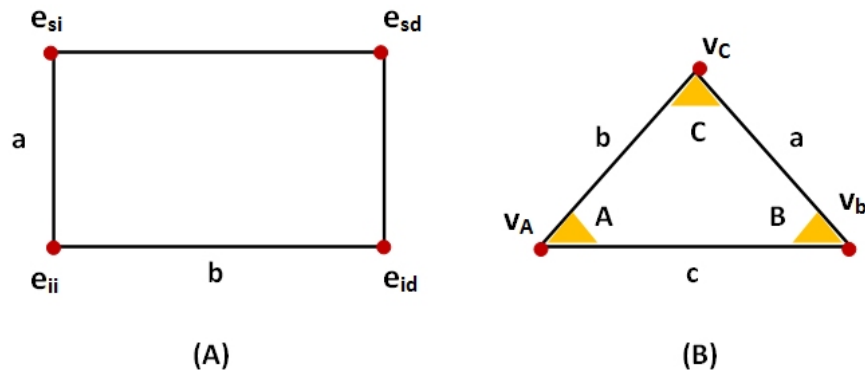


Figura 47: Características de los objetos de clase Rectángulo (A) y Triángulo (B)

30. Usando como referencia la clases Punto2D, Rectángulo y Triángulo incorpore los métodos necesarios para realizar las siguientes tareas:
- a) obtener el triángulo inferior de un rectángulo, considerando su diagonal principal.
 - b) obtener el triángulo superior de un rectángulo, considerando su otra diagonal.
 - c) calcular el área del triángulo.
 - d) calcular si un punto está dentro de un triángulo.
 - e) calcular si un rectángulo está dentro de otro.

Finalidad: Declarar un objeto como dato miembro de otro objeto. Métodos que usan objetos de otras clases. Dificultad Alta.

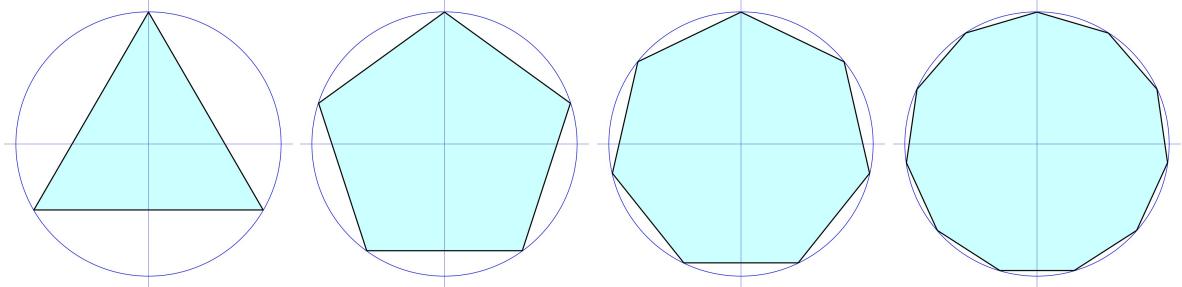
31. Considerar la clase ColeccionPuntos2D que se usará para almacenar y gestionar una colección de datos de tipo Punto2D.

Realizar un programa que lea los datos necesarios para la creación de un objeto Circunferencia. A continuación lee un número indeterminado de datos de tipo Punto2D y los almacena en un objeto ColeccionPuntos2D (la lectura finaliza cuando el usuario escribe FIN en la lectura de la *abscisa* de un nuevo punto).

Finalmente muestra cuáles de los puntos almacenados en la colección ColeccionPuntos2D está en el círculo delimitado por la circunferencia.

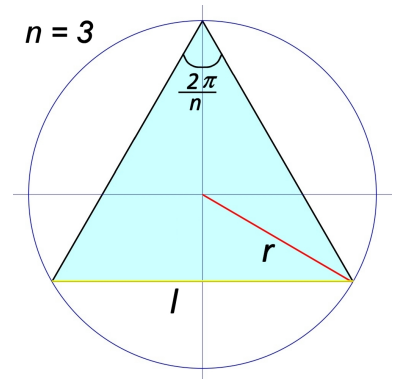
Finalidad: Clase con una secuencia de objetos de otra clase. Dificultad Media.

32. (*Examen Febrero 2016*) Un polígono regular de n caras tiene n lados de la misma longitud y todos los ángulos interiores son iguales. Su centro geométrico es el centro de la circunferencia circunscrita (la que lo envuelve). Supondremos que dos polígonos son distintos si se diferencian o bien en sus centros geométricos, o bien en el número de lados o bien en la longitud de cualquiera de ellos. Así pues, por ejemplo, no tendremos en cuenta las distintas posiciones en el plano que se podrían obtener girando el polígono sobre su centro.



Si llamamos n al número de lados y l a su longitud, tenemos que:

- la longitud r del radio de la circunferencia circunscrita viene definida por $r = l / (2 \sin(\pi/n))$
- El área del polígono es $A = (n/2) \cdot r^2 \sin(2\pi/n)$
- Si queremos construir un polígono inscrito en la misma circunferencia, pero multiplicando por un entero k el número de lados, la longitud de cada uno de los kn lados viene dada por



$$l' = r \sqrt{2(1 - \cos(2\pi/(kn)))}.$$

Se quiere diseñar la clase `PoligonoRegular` para poder representar este tipo de polígonos y realizar las siguientes tareas:

- Definir métodos que permitan la consulta y modificación de los campos del objeto.
- Calcular el perímetro del polígono.
- Calcular el área del polígono.
- Calcular la diferencia entre el área del polígono y la del círculo correspondiente a la circunferencia circunscrita.
- Comprobar si un polígono es mayor que otro (considerando al área de cada uno)
- Construir un nuevo polígono que tenga la misma circunferencia circunscrita y con un número de lados que sea múltiplo del número de lados del polígono.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

Debe tener los siguientes constructores:

- a) Sin parámetros. Los valores a asignar por defecto sean: 3 para el número de lados (triángulo), 1 para la longitud y (0,0) para las coordenadas del centro.
- b) Un constructor que cree un polígono regular con una longitud y número de lados concretos (parámetros) y deje como centro el valor por defecto (0,0).
- c) Un constructor que cree un polígono regular con una longitud, número de lados y centro concretos (parámetros).

Defina un programa que realice lo siguiente:

- Cree dos polígonos, `poligono1` con los valores por defecto y `poligono2` con 6 lados de longitud 4 y centrado en (0,0). El programa comprobará si `poligono1` es mayor que `poligono2`.
- Construya `poligono3` a partir de `poligono1`, con la misma circunferencia circunscrita y con el doble número de lados. Imprima el área del nuevo polígono.
- Repita el proceso anterior generando polígonos con el doble número de lados en cada iteración, hasta que el polígono generado tenga un área *similar* a la del círculo delimitado por la circunferencia circunscrita. Consideraremos que las áreas son similares si no se diferencian en más de 10^{-5} .

Debe mostrar el número de lados del polígono que aproxima a la circunferencia.

Dificultad Media.

33. Un *track GPS* es una sucesión de coordenadas que forman un camino. Se supone que se han ido adquiriendo por un dispositivo dotado con un sensor GPS y las coordenadas se han ido almacenando de manera consecutiva.

Se desea modelar un *track GPS* mediante una *secuencia de datos de tipo CoordenadasGPS* (ver ejercicio 13 de esta misma Relación de Problemas). Diseñe e implemente la clase `SecuenciaCoordenadasGPS`. Implemente los métodos:

- Constructor sin argumentos.
- Métodos `get` para obtener la capacidad y el número de puntos del *track*.
- Método para añadir una nueva coordenada al *track*.
- Método para calcular el **máximo desnivel**. Se define como la máxima diferencia (en valor absoluto) entre dos alturas consecutivas (ejercicio 20 de la *Relación de Problemas IV*).
- Método para calcular el **desnivel acumulado positivo**. Se calcula como la suma de las diferencias de altura positivas entre cada dos puntos consecutivos (ejercicio 20 de la *Relación de Problemas IV*).
- Método para calcular la **distancia real** del *track*.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

Escribir un programa que lea un número indeterminado de coordenadas GPS (finaliza cuando se introduce -100), calcule el máximo desnivel, el desnivel acumulado positivo, y la distancia real del *track*.

34. Reescribir la solución al ejercicio 3 de esta relación de problemas (generador de permutaciones aleatorias) usando como dato miembro de la clase `Permutacion` un objeto de la clase `SecuenciaEnteros`, en vez de un *vector* clásico.

Al constructor le seguimos pasando un objeto de la clase `SecuenciaEnteros`. Observad que al cambiar la representación, habrá que modificar los métodos de la clase `Permutacion` porque el acceso que realizaban, libremente, al vector donde se guardaban los valores de la permutación ya no es posible. El acceso a los valores de la permutación se hará con los métodos públicos de la clase `SecuenciaEnteros`.

Finalidad: Trabajar con un objeto como dato miembro de otro objeto. Dificultad Media.

35. Reescribid la solución al ejercicio 51 de la *Relación de Problemas V* (clase `Frase`) para que use como dato miembro privado un *vector de palabras*.

Cada palabra será un objeto de la clase `Palabra`. Un objeto de esta clase representa a una sucesión de caracteres que no contiene espacios en blanco.

Observen que las tareas que se realizan en algunos de los métodos de la antigua clase `Frase` corresponden ahora a la clase `Palabra` (por ejemplo, calcular la longitud de una palabra concreta) mientras que algunos de ellos ya no tienen sentido (por ejemplo, `EliminaBlancosIniciales` y `EliminaBlancosFinales`) y otros deben reescribirse completamente (por ejemplo, `BorraPalabra`).

Finalidad: Métodos que devuelven objetos y usan localmente otros objetos, de la misma clase u otra. Dificultad Media.

36. Ampliar la colección de clases del ejercicio 35 (clases `Frase` y `Palabra`) con la clase `Parrafo` que modela una serie de frases.

Escriba los métodos que considere oportunos.

Escriba un programa que lea una serie indeterminada de cadenas, finalizando cuando se detecta el *fin de fichero*. Cada cadena formará una frase y todas ellas un párrafo. El programa mostrará cuantas frases, palabras y caracteres componen el párrafo.

Finalidad: Métodos que devuelven objetos y usan localmente otros objetos, de la misma clase u otra. Dificultad Media.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

37. (*Examen Extraordinario 2020*) Un **píxel de color** es una terna (r, g, b) con $0 \leq r, g, b \leq 255$ donde r es el nivel de rojo, g es el nivel de verde y b es el nivel de azul. Una **imagen en color** es una matriz de píxeles de color. Podemos representar un píxel y una imagen mediante las siguientes clases:

PixelRGB
- unsigned char rojo;
- unsigned char verde;
- unsigned char azul;
+

ImagenRGB
- static const int MAX_DIM = 500;
- int ancho;
- int alto;
- PixelRGB imagen[MAX_DIM][MAX_DIM];
+

donde *imagen* es una matriz de objetos *PixelRGB* en la que la posición (0,0) se corresponde con el píxel en la esquina superior izquierda de la imagen.

Se pide:

- Un constructor de la clase *PixelRGB* que inicialice un píxel a un color RGB dado.
- Un constructor de la clase *ImagenRGB* que, dado un entero $1 \leq d \leq \text{MAX_DIM}$ y un *PixelRGB* p , cree una imagen *cuadrada* y *plana*: crea un objeto *ImagenRGB* de dimensiones $d \times d$ donde todos los píxeles tienen el mismo color (plana) p .
- Un método de la clase *ImagenRGB* que calcule y devuelva un **histograma** de una de las tres capas de color (R, G ó B) de la imagen. Su prototipo será:

SecuenciaEnteros *Histograma* (char color);

donde *color* será R, G ó B y la secuencia devuelta contendrá 256 casillas.

Así, la ejecución de *paisaje.Histograma('R')* devuelve siempre una secuencia de 256 datos *int* (es un objeto de la clase *SecuenciaEnteros*) con la frecuencia absoluta de cada valor en la capa R de *paisaje*: la casilla 0 de la secuencia contiene el número de veces que aparece el valor 0 en la capa R, la casilla 1 de la secuencia contiene el número de veces que aparece el valor 1 en la capa R, y así sucesivamente.

- Escribir un método que concatene horizontalmente dos imágenes. El resultado es otra imagen mayor que las que intervienen: el ancho es la suma de los anchos de las dos imágenes, y el alto es el mayor entre las dos imágenes. Los "huecos" se rellenan con píxeles en negro (en RGB sería (0,0,0)).

Se deben implementar todos los métodos (públicos y privados) que estime necesarios. Escriba también las precondiciones aplicables a los datos y métodos.

38. (*Basado en el Examen Ordinario 2019*) Trabajaremos sobre el problema planteado en el ejercicio 30 de la *Relación de Problemas V* en el que se pedía implementaba la clase `RedSocial` para gestionar una red social.

Es ese ejercicio, los usuarios de la red se caracterizaban únicamente por su nombre (un dato `string`). Los nombres de los usuarios se guardaban en el campo `usuarios`:

```
class RedSocial {  
  
    private:  
        static const int MAXIMO_USUARIOS = 100;  
        string usuarios[MAXIMO_USUARIOS];  
        bool relaciones_amistad[MAXIMO_USUARIOS][MAXIMO_USUARIOS];  
        int usuarios_utiles;  
  
    public:  
        ...  
};
```

donde `usuarios_utiles` indicaba el número de casillas del array `usuarios_utiles` y el número de filas y columnas de `relaciones_amistad`.

Ahora deberá escribir la clase `Usuario` que tendrá, entre otros campos, el campo `nombre_usuario` (`string`) que será único en la red y caracterizará a un usuario concreto. Decida los demás campos. Ahora, la clase `RedSocial` será:

```
class RedSocial {  
  
    private:  
        static const int MAXIMO_USUARIOS = 100;  
        Usuario usuarios[MAXIMO_USUARIOS];  
        bool relaciones_amistad[MAXIMO_USUARIOS][MAXIMO_USUARIOS];  
        int usuarios_utiles;  
  
    public:  
        ...  
};
```

Reescriba los programas que resolvían los ejercicios 30 y 31 de la *Relación de Problemas V* con las modificaciones introducidas. Deberá tener en cuenta que algunos métodos escritos para esos programas ya no son válidos. Por ejemplo, el método

```
void AniadeUsuario (string usr);
```

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

para añadir el usuario cuyo nombre es `usr` a la red ya no es válido. Ahora debería disponer de un método como éste:

```
void AniadeUsuario (Usuario usr);
```

y previamente debió crear un objeto `Usuario` y debió inicializarlo convenientemente. Recuerde que no es posible añadir un usuario a la red con el mismo nombre que otro que ya existe.

Finalidad: Trabajar con matrices de objetos. Dificultad Media.

39. En el ejercicio **38** de la *Relación de Problemas IV* se elaboraba el ránking anual de una prueba de atletismo (1500 ML en pista). En ese ejercicio se usaba un `struct` de tipo `RegistroDeMarca` que contenía los datos necesarios de una *marca* (una tripleta *fecha*, *licencia* y *tiempo*).

El procedimiento consistía en leer datos de diferentes marcas de esa prueba que se almacenaban en un array, *marcas*, que finalmente se ordenaban en base al *tiempo*. El array *marcas* se definía:

```
// Array que se usa para guardar los datos leídos y procesados

const int MAX_ATLETAS = 100;
RegistroDeMarca marcas[MAX_ATLETAS];
int utilizados_marcas = 0;
```

Reescriba la solución de ese ejercicio modelizando las entidades que intervenían en la solución (*Fecha*, *Tiempo* y `RegistroDeMarca`) usando **clases**.

Notas:

- a) Siga usando una *array clásico* en la función `main` para almacenar las marcas.
- b) No es preciso que proporcione una funcionalidad plena a las clases *Tiempo* y *Fecha*. Hacerlo así sería un trabajo demasiado largo y complejo.

Finalidad: Trabajar con clases que contienen datos de otras clases. Trabajar con arrays de objetos. Dificultad Media.

40. Reescriba la solución del ejercicio **59** de la *Relación de Problemas IV* usando **clases**, siguiendo las pautas dadas en el ejercicio **39** de esta Relación de Problemas.

Finalidad: Trabajar con clases que contienen datos de otras clases. Trabajar con matrices de objetos. Dificultad Alta.

41. Reescriba la solución del ejercicio 32 de la *Relación de Problemas IV* usando las **clases** Punto2D, Circunferencia y Rectangulo.

Las circunferencias en las que estamos interesados se guardarán en un *array* conforme se vayan calculando. Después, se mostrará cuántas circunferencias se han guardado y sus propiedades.

Finalidad: Trabajar con clases que contienen datos de otras clases. Trabajar con arrays de objetos. Dificultad Media.

42. *Basado en el Examen Extraordinario de 2020* Use la clase SecuenciaCaracteres, para representar una palabra en una secuencia de caracteres. Considere la clase ConjuntoSecuencias (figura 48) para almacenar un conjunto de secuencias de caracteres, accesibles cada una de ellas por un índice (con base 0).

ConjuntoSecuencias	
-	const int TAMANIO = 100
-	SecuenciaCaracteres ConjuntoSecuencias[TAMANIO]
-	int num_componentes
+	ConjuntoSecuencias()
+	int NumComponentes()
+	int Capacidad()
+	void Aniade(SecuenciaCaracteres nuevo)
+	SecuenciaCaracteres Componente(int indice)

Figura 48: La clase ConjuntoSecuencias

Defina los métodos necesarios para resolver los siguientes problemas (puede definir los métodos auxiliares que estime oportuno). Debe especificar y justificar a **qué clase** pertenece cada método y si es **público o privado**.

- a) Supongamos dos secuencias p y q, cuyas longitudes (número de elementos utilizados) son l_p y l_q respectivamente. Llamemos d_p al número de caracteres **distintos** que aparecen en p y d_q al número de caracteres **distintos** que aparecen en q. Teniendo en cuenta estos valores defina los siguientes métodos:

- EsMayor

Comprueba si una secuencia es mayor que otra según el criterio:

- Si $d_p > d_q$ entonces p es mayor que q
- Si $d_p = d_q$ y $l_p > l_q$ entonces p es mayor que q
- En cualquier otro caso p no es mayor que q

- **EsIgual.**

Comprueba si dos secuencias p y q son iguales.

Dos secuencias p y q son iguales si se cumple que $d_p = d_q$ y $l_p = l_q$

Por ejemplo, la secuencia $\{aaaaa\}$ es menor que $\{osos\}$ ya que la primera tiene sólo *una* letra distinta y la segunda tiene *dos* letras distintas. La secuencia $\{seso\}$ es menor que $\{sosas\}$ ya que las dos tienen *tres* caracteres distintos, pero la primera tiene una longitud (número de elementos utilizados) de *cuatro* y la segunda, *cinco*. Las secuencias $\{abb\}$ y $\{ede\}$ son iguales: ambas tienen el mismo número de caracteres distintos (*dos*) y la misma longitud (*tres*).

b) Escribir un método para insertar ordenadamente una secuencia en un conjunto de secuencias. Tenga en cuenta:

- El método modifica el conjunto de secuencias sobre el que se aplica.
- El orden es **decreciente**, considerando el criterio definido anteriormente.
- Si el conjunto de secuencias no está ordenado, el método no hará nada. Se entiende que un conjunto de secuencias está ordenado si cada secuencia es mayor o igual que la siguiente.

Por ejemplo, dado el conjunto

$$S = \{ \{osas\}, \{eso\}, \{sosos\}, \{aaaaa\} \},$$

al insertar ordenadamente la secuencia $\{osos\}$, obtendríamos como resultado

$$S = \{ \{osas\}, \{eso\}, \{sosos\}, \{osos\}, \{aaaaa\} \}.$$

Si $S = \{ \{eso\}, \{aaaaa\}, \{osas\}, \{sosos\} \}$, entonces la inserción ordenada no modifica el conjunto puesto que éste no se encuentra ordenado.

c) Escribir un método para construir y devolver un nuevo conjunto con todas aquellas secuencias **menores** (estrictamente) que una secuencia de referencia dada. Tenga en cuenta lo siguiente:

- El orden a considerar es el definido anteriormente.
- Ni el conjunto del que se toman las secuencias, ni el resultante, tienen que estar necesariamente ordenados.

Por ejemplo, si el conjunto fuera

$$S = \{ \{oso\}, \{osas\}, \{aaaaa\}, \{esos\}, \{si\}, \{asisto\} \}$$

y la secuencia de referencia $\{isa\}$, el conjunto devuelto sería

$$S = \{ \{oso\}, \{aaaaa\}, \{si\} \}.$$

Tablas (representación: matriz clásica)

La representación basada en una matriz clásica quiere decir que la estructura de datos que se empleará para almacenar los datos será una *matriz clásica*, declarada convenientemente en la zona privada de la clase.

Por ejemplo, si se trata de una **tabla rectangular** de caracteres (todas las filas tienen el mismo número de columnas *ocupadas*):

```
static const int NUM_FILS = 50; // Filas disponibles
static const int NUM_COLS = 40; // Columnas disponibles

char matriz_privada[NUM_FILS][NUM_COLS];

// PRE: 0 <= filas_utilizadas <= NUM_FILS
// PRE: 0 <= col_utilizadas <= NUM_COLS

int filas_utilizadas;
int cols_utilizadas;
```

Si las filas pueden tener diferente número de casillas *ocupadas* hablamos entonces de una **tabla dentada**:

```
static const int NUM_FILS = 50; // Filas disponibles
static const int NUM_COLS = 40; // Columnas disponibles

char matriz_privada[NUM_FILS][NUM_COLS];

// PRE: 0 <= filas_utilizadas <= NUM_FILS

int filas_utilizadas;

// PRE: 0 <= num_cols_utilizadas[i] <= NUM_COLS
//       para i=0,1,...,filas_utilizadas-1
// Número de columnas ocupadas en cada fila

int num_cols_utilizadas[NUM_FILS];
```

TABLA RECTANGULAR USANDO UNA MATRIZ

43. Cread la clase `TablaRectangularCaracteres`. Un objeto de esta clase representa tablas de datos en las que todas las filas tienen el mismo número de columnas, es decir, representa una *matriz rectangular*.

Escribir los métodos descritos en la figura 49.

TablaRectangularCaracteres	
- const int	<u>NUM_FILS</u>
- const int	<u>NUM_COLS</u>
- char	matriz_privada[NUM_FILS] [NUM_COLS]
- int	filas_utilizadas
- int	cols_utilizadas
+	TablaRectangularCaracteres(void)
+	TablaRectangularCaracteres(int num_columnas)
+	TablaRectangularCaracteres(SecuenciaCaracteres primera_fila)
+ int	CapacidadFilas(void)
+ int	CapacidadColumnas(void)
+ int	FilasUtilizadas(void)
+ int	ColumnasUtilizadas(void)
+ char	Elemento(int fila, int columna)
+ SecuenciaCaracteres	Fila(int indice_fila)
+ SecuenciaCaracteres	Columna(int indice_fila)
+ void	Aniade(SecuenciaCaracteres fila_nueva)
+ void	EliminaTodos (void)
+ bool	EstaVacía (void)
+ bool	EsIgual (TablaRectangularCaracteres otra)

Figura 49: Métodos básicos de la clase `ClaseTablaRectangularCaracteres`

Con más detalle se trata de los métodos:

- Constructores: 1) sin argumentos, para crear una tabla vacía, 2) que recibe el número de columnas, y 3) que recibe la primera fila (un objeto de la clase `SecuenciaCaracteres`).
- Obtener la capacidad en filas (resp. columnas).

- c) Obtener el número de filas (resp.columns) utilizadas.
- d) Obtener el carácter que haya en una fila y columna.
- e) Devolver una fila (resp. columna) completa. Devolverá una copia de la fila (resp. columna) en un objeto de la clase `SecuenciaCaracteres`.
- f) Añadir una fila entera (será un objeto de la clase `SecuenciaCaracteres`). La fila proporcionada pasará a ser la última de la tabla.
- g) Eliminar todos los elementos de la tabla.
- h) Consultar si la tabla está vacía.
- i) Comprobar si es igual a otra tabla rectangular.

Escribir una función `main` que pruebe todos los métodos de la clase.

Finalidad: Trabajar con matrices. Pasar a un método de una clase un parámetro de otra clase. Dificultad Baja.

44. (*Examen Febrero 2011*) Añada a la clase `TablaRectangularCaracteres`. un método que proporcione la matriz **traspuesta**. Si no es posible, deberá devolver una matriz vacía.

Añada un método para comprobar si la matriz es **simétrica**. Hacedlo primero calculando la traspuesta de la matriz y viendo si la matriz original es igual a su traspuesta. Hacedlo también comprobando directamente si cada componente es igual a su *simétrico*.

Finalidad: Trabajar con matrices. Devolver matrices. Pasar a un método de una clase un parámetro de su misma clase. Dificultad Media.

45. Añadid dos métodos a la clase `TablaRectangularCaracteres` para que inserte una fila completa en una posición (de fila) determinada, o borre la fila indicada. Los métodos serán:

```
void Inserta (int num_fila, SecuenciaCaracteres fila_nueva);  
void Elimina (int num_fila);
```

El valor de `num_fila` en el método `Inserta()` indica la posición que ocupará la nueva fila en la matriz tras la inserción. Así, si queremos que la nueva fila sea la primera `num_fila` será 0 mientras que si queremos que sea la última, `num_fila` será `TotalUtilizados()`.

El valor de `num_fila` en el método `Elimina()` indica la fila que queremos eliminar.

Finalidad: Trabajar con matrices. Pasar a un método de una clase un parámetro de otra clase. Dificultad Baja.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

46. Definid la clase `TablaRectangularEnteros` de forma análoga a la clase `TablaRectangularCaracteres` (ejercicios 43, 44 y 45 de esta relación de problemas). Escribir para esta clase los métodos implementados en esos ejercicios y añadir los siguientes métodos para encontrar un valor:

```
bool Contiene (int buscado)
```

Devuelve true si la matriz contiene el valor buscado.

```
PosicionEnMatriz PrimeraOcurrencia (int buscado)
```

Encontrar un dato en la matriz y devolver su posición. El valor devuelto contiene información acerca de la posición (fila y columna) de buscado. En el caso de que no esté contendrá (-1,-1). El tipo `PosicionEnMatriz` será:

```
struct PosicionEnMatriz {  
    int fila;  
    int columna;  
};
```

(Examen Septiembre 2011) Calcular la posición de aquel elemento que sea el mayor de entre los mínimos de cada fila. Por ejemplo, dada la matriz M (3×4),

9	7	4	5		9	7	4	5		4		4
2	18	2	12	$\Rightarrow$	2	18	2	12	$\Rightarrow$	2	$\Rightarrow$	2
7	9	1	5		7	9	1	5		1		1

el máximo entre 4, 2 y 1 (los mínimos de cada fila) es 4 y está en (0, 2).

Implementar cuantas clases (y métodos) considere necesario.

47. Amplíe la clase `TablaRectangularEnteros` con un nuevo método llamado `PosicionesSumaLateral` que devuelva las posiciones (f, c) en las que la suma del valor de la izquierda $(f, c - 1)$ y de la derecha $(f, c + 1)$ coincide con la del valor de la casilla (f, c) .

Considere que es posible que haya más de una casilla que cumpla la condición indicada. Además, las posiciones en una tabla pueden representarse con el tipo `PosicionEnMatriz` (ejercicio 46) por lo que además de la clase `TablaRectangularEnteros` debería disponer de las clases:

- a) `SecuenciaEnteros` para poder construir y dar valores a la tabla rectangular. Recuerde que un objeto `TablaRectangularEnteros` “crece” fila a fila, siendo cada fila un objeto `SecuenciaEnteros`.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

- b) `SecuenciaPosicionEnMatriz` para poder guardar todas las posiciones de la matriz (datos de tipo `PosicionEnMatriz`) que cumplen la condición estipulada.

Por ejemplo, dada la matriz M (3×4),

	Col 0	Col 1	Col 2	Col 3
Fila 0	1	2	3	4
Fila 1	4	5	7	2
Fila 2	3	6	3	9

La secuencia de posiciones calculadas sería: $\{ (1, 2), (2, 1) \}$

48. Escribir un programa que calcule si existe un valor *MaxiMin* en una tabla rectangular de enteros. Un *MaxiMin* es aquel que es a la vez máximo de su fila y mínimo de su columna. Por ejemplo, en la matriz M (3×4),

9	7	4	5	9	7	4	5	9	7	4	5
2	18	2	12	2	18	2	12	2	18	2	12
1	4	3	2	1	4	3	2	1	4	3	2

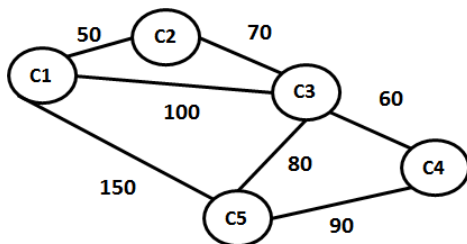
existe *MaxiMin*. Su valor es 4 y está en (2, 1).

Usar la clase `TablaRectangularEnteros`, e implementar cuantas clases (y métodos) considere necesario.

49. (*variante del examen de Febrero de 2011*) Escribir **una función (no un método)** que reciba M , un objeto de la clase `TablaRectangularEnteros`, y un valor entero k ($k \geq 1$) y calcule la suma de aquellos valores M_{ij} de la tabla M que verifiquen que $i + j$ sea igual a:

$$\sum_{h=1}^k h^2 \quad \text{para algún valor de } k \geq 1$$

50. (*Examen Ordinario 2017*) Las distancias entre un conjunto de ciudades se almacenan en una **tabla rectangular** de datos `double`. En realidad se almacenan las distancias de los **caminos directos** que conectan un conjunto de ciudades. Si entre dos ciudades no existe un camino directo, se almacenará un cero. Se supone que la distancia de una ciudad consigo misma será cero y que las distancias son simétricas. Un ejemplo con 5 ciudades sería:



	C1	C2	C3	C4	C5
C1	0	50	100	0	150
C2	50	0	70	0	0
C3	100	70	0	60	80
C4	0	0	60	0	90
C5	150	0	80	90	0

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

Diseñe e implemente la clase `MapaDistancias` para representar este tipo de datos. Use como base la clase `TablaRectangular`. Suponga que nunca se trabajará con más de 50 ciudades.

Proporcione métodos que doten a la clase de funcionalidad para resolver el problema que se detalla a continuación.

Escriba un programa que lea el número de ciudades de la red, y las distancias entre las ciudades (recuerde que 0 indica que entre dos ciudades no existe un camino directo) y rellene adecuadamente la matriz de distancias.

A continuación, resuelva las siguientes tareas, mostrando el resultando en pantalla:

- a) Obtener la ciudad (su índice) con mayor número de conexiones directas.
En el ejemplo, sería la ciudad 3 con 4 conexiones.
- b) Dada una ciudad j , obtener las ciudades conectadas directamente con j .
En el ejemplo, si consideramos la ciudad 4, vemos que está conectada directamente con las ciudades 3 y 5.
Devuelva el resultado en un objeto `SecuenciaEnteros`.
- c) Dadas dos ciudades i y j para las cuales no existe un camino directo, obtener aquella ciudad intermedia z que permita hacer el trayecto entre i y j de la forma más económica posible. Es decir, se trata de encontrar una ciudad z tal que $d(i, z) + d(z, j)$ sea mínima ($d(a, b)$ es la distancia entre las ciudades a y b). Si dicha ciudad no existiera, se indicará expresamente.
Por ejemplo, si se desea viajar desde la ciudad 2 a la 5, hacerlo a través de la ciudad 1 tiene un costo de $50 + 150 = 200$ mientras que si se hace a través de la ciudad 3, el costo sería $70 + 80 = 150$.
- d) Lea una serie de valores `int` que representan números de ciudades (el terminador será el valor -1) e indique si todas las ciudades tienen conexiones directas entre sí.
Reciba los números de ciudades en un objeto `SecuenciaEnteros`.

51. Ampliar la clase `TablaRectangularEnteros` con un **método** que busque la fila de la matriz que más se parezca a un vector de enteros (clase `SecuenciaEnteros`) al que llamaremos *referencia*. El método devolverá el *número* de la fila.

La similitud entre dos vectores $x = (x_1 \cdots x_p)$ e $y = (y_1 \cdots y_p)$ vendrá dada por la distancia euclídea entre ambos:

$$\text{dist}(x, y) = \sqrt{(x_1 - y_1)^2 + \cdots + (x_p - y_p)^2}$$

Además, la búsqueda solo se hará sobre un conjunto seleccionado de **filas** de la tabla, enumeradas en la secuencia de enteros `filas_a_comparar` (un objeto de la clase `SecuenciaEnteros`).

Por ejemplo, dada la matriz M (7×4),

→	3	1	0	8
	4	5	1	5
→	5	7	1	7
	7	9	6	1
→	4	9	5	5
→	2	8	2	2
	7	3	2	5

y los vectores `referencia = 2, 8, 1, 1` y `filas_a_comparar = 0, 2, 4, 5`, el programa deberá encontrar 5 como la fila más cercana a la secuencia `referencia` (en el dibujo anterior se han marcado con una flecha las filas indicadas por el vector `filas_a_comparar`).

52. (*Examen Extraordinario 2018*) Un diligente agente de comercio tiene que visitar periódicamente a sus n clientes. Cada uno de ellos vive en una ciudad distinta. Su jefe le ordena que prepare el plan de viaje según las siguientes reglas:
- Tiene que visitar a todos los clientes y una sola vez a cada uno.
 - El recorrido se hará de forma que el coste total sea el menor posible.
 - No importa por qué ciudad comience, pero debe finalizar en la misma ciudad en la que comenzó. Por tanto, el recorrido forma un ciclo. Y, obviamente, no puede volver a la ciudad inicial hasta haber visitado todas las demás.

Como datos de entrada se facilitan el número de clientes, n , y el coste de viaje que hay entre cada par de ciudades, expresada como una matriz, C , de tamaño $n \times n$, donde C_{ij} es el coste de viajar desde la ciudad i a la j (dicha matriz no tiene por qué ser simétrica). Supondremos que todos estos valores son positivos, $C_{ij} > 0$ y que se introducen correctamente.

Este es un problema muy difícil de resolver para el que existen distintas soluciones aproximadas. Uno de esos algoritmos, llamado *heurística del vecino más cercano*, funciona así: se selecciona una ciudad de partida, a , y se busca la ciudad más económica para llegar, b . A continuación, se busca la ciudad aún no visitada más económica a b . Y así hasta completar la visita a todas las ciudades. El coste del recorrido es la suma de todos los costes intermedios.

Diseñe e implemente la clase `PlanificadorVisitas` para representar este tipo de datos. Use como base la clase `TablaRectangular`. Suponga que nunca se trabajará con más de 50 ciudades.

Proporcione métodos que doten a la clase de funcionalidad para resolver el problema que se detalla a continuación.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

Construya un programa que lea el valor de n y los valores de la matriz C , así como la ciudad inicial. Luego calculará el recorrido con el algoritmo descrito anteriormente y posteriormente su coste. Finalmente mostrará ambos resultados.

Por ejemplo, supongamos un problema con $n = 4$ cuya matriz de costes es:

$$C = \begin{bmatrix} - & 10 & 15 & 9 \\ 11 & - & 13 & 8 \\ 17 & 21 & - & 15 \\ 26 & 7 & 14 & - \end{bmatrix}$$

Por simplicidad, asumimos que los nombres de las ciudades son $1, \dots, n$. Si partimos de la ciudad 1 (índice 0), entonces la solución encontrada es: 1, 4, 2 y 3 con un coste de $9 + 7 + 13 + 17 = 46$. Si el viaje comienza por la ciudad 3 (índice 2), el recorrido final sería 3, 4, 2 y 1 con un coste de $15 + 7 + 11 + 15 = 48$.

TABLA DENTADA USANDO UNA MATRIZ

53. Escribir para la clase `ClaseTablaDentadaCaracteres` (representación con matriz clásica) los métodos descritos en la figura 50 (uno de ellos fue pregunta de examen).

TablaDentadaCaracteres	
- const int	NUM_FILS
- const int	NUM_COLS
- char	matriz_privada[NUM_FILS][NUM_COLS]
- int	filas_utilizadas
- int	num_cols_utilizadas[NUM_FILS]
+	TablaDentadaCaracteres()
+	TablaDentadaCaracteres(SecuenciaCaracteres primera_fila)
+ int	CapacidadFilas()
+ int	CapacidadColumnas()
+ int	FilasUtilizadas()
+ int	ColumnasUtilizadas(int fila)
+ char	Elemento(int fila, int columna)
+ SecuenciaCaracteres	Fila(int indice_fila)
+ void	Añade(SecuenciaCaracteres fila_nueva)
+ void	Inserta (int num_fila, SecuenciaCaracteres fila_nueva)
+ void	Elimina (int num_fila)
+ void	EliminaTodos (void)
+ bool	EstaVacía (void)
+ bool	EsIgual (TablaDentadaCaracteres otra_tabla)

Figura 50: La clase `ClaseTablaDentadaCaracteres` (con matriz clásica)

(Examen Febrero 2011) Añadid un método a la clase `TablaDentadaCaracteres` (almacenamiento con matriz clásica, de dos índices) para **insertar** una fila completa en una posición (de fila) determinada. La fila a insertar se proporciona como un objeto de la clase `SecuenciaCaracteres`. Tras la inserción, la nueva fila ocupará la posición indicada, y la que ocupaba esa posición, y todas las superiores, se habrán desplazado una posición. Por ejemplo,

$$\begin{pmatrix} c & b & a & l & ? \\ f & g & h & ? & ? \\ ? & ? & ? & ? & ? \end{pmatrix} \xrightarrow[\text{en la fila 1}]{\text{Insertar } (d, d, e, e)} \begin{pmatrix} c & b & a & l & ? \\ d & d & e & ? & ? \\ f & g & h & ? & ? \end{pmatrix}$$

dónde ? representa un valor no utilizado.

Finalidad: Pasar a un método de una clase un parámetro de otra clase. Dificultad Baja.

54. Se desea crear una matriz dentada de valores reales leyendo sus valores de la entrada estándar y realizar algunos cálculos con ella.

Todos los valores necesarios (número de grupos, número de datos de cada grupo y los valores de cada grupo) se introducen en una sola línea.

Por ejemplo, una línea de entrada podría ser la siguiente:

3 2 3.1 0.4 5 1.0 1.0 1.0 1.0 1.0 2 5.2 4.7

Se puede observar que se distinguen *tres* grupos de datos (lo indica el primer número (`int`) de la línea). Cada grupo empieza por un valor (`int`) que indica el número de elementos que lo componen (en el ejemplo, los grupos tienen *dos*, *cinco* y *dos* elementos, respectivamente). A continuación de este valor se encuentran tantos valores (`double`) como indique el valor entero que encabeza cada grupo. Escribir un programa que escriba en `cout` la suma de los valores de cada grupo.

En el ejemplo, el primer grupo contendrá los valores 3.1 y 0.4, el segundo 1.0, 1.0, 1.0, 1.0 y 1.0 y el tercero 5.2 y 4.7. El programa debería escribir: 3.5, 5.0 y 9.9

Implementar la clase `TablaDentadaReales` con una *matriz* de datos `double`.

Finalidad: Practicar con matrices dentadas. Dificultad Baja.

55. Escribir un método de la clase `TablaDentadaReales` (con matriz clásica) que devuelva un objeto `TablaRectangularReales` (con matriz clásica).

El objeto devuelto tendrá tantas filas como tenga el objeto implícito, y tantas columnas como columnas tenga la fila con menos columnas del objeto implícito.

Finalidad: Practicar con matrices dentadas y rectangulares. Dificultad Baja.

56. En el ejercicio 40 de esta Relación de Problemas (ordenación de marcas de atletismo) se propuso el uso de clases para caracterizar las *marcas* sobre las que se trabaja, además de sus componentes complejas: el *tiempo* y la *fecha*.

En ese ejercicio, no obstante, se usaba una *matriz clásica* y un *array* para poder manejar una estructura de matriz dentada que nos permitiera representar y organizar las marcas por cada tipo de prueba. Ahora proponemos crear un tipo de dato específico, `TablaDentadaMarcas`, para gestionar una tabla dentada de datos de tipo `RegistroDeMarca`. La descripción de las clases (una propuesta orientativa) se especifica en la figura 51.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

Reescriba la solución del ejercicio 40 de esta Relación de Problemas usando las clases `SecuenciaMarcas` y `TablaDentadaMarcas`.

	TablaDentadaMarcas
- const int	<u>NUM_FILAS</u>
- const int	<u>NUM_COLS</u>
- RegistroDeMarca	matriz_privada[NUM_FILS][NUM_COLS]
- int	filas_utilizadas
- int	num_cols_utilizadas[NUM_FILS]
+	TablaDentadaMarcas (int num_marcas)
+	TablaDentadaMarcas (SecuenciaMarcas primera_fila)
+ int	CapacidadFilas (void)
+ int	CapacidadColumnas (void)
+ int	FilasUtilizadas (void)
+ int	ColumnasUtilizadas (int fila)
+ RegistroDeMarca	Elemento (int fila, int columna)
+ SecuenciaMarcas	Fila (int fila)
+ void	AniadeFila (SecuenciaMarcas fila_nueva)
+ void	InsertaFila (int fila, SecuenciaMarcas fila_nueva)
+ void	EliminaFila (int fila)
+ void	SustituyeFila (int fila, SecuenciaMarcas fila_nueva)
+ void	AniadeMarca (int fila, RegistroDeMarca marca)
+ void	ModificaMarca (int fila, int columna, RegistroDeMarca marca_nueva)
+ void	EliminaTodos (void)
+ string	ToString (void)

	SecuenciaMarcas
- const int	<u>TAMANIO</u>
- RegistroDeMarca	vector_privado[TAMANIO]
- int	total_utilizados
+	SecuenciaMarcas (void)
+ int	TotalUtilizados (void)
+ int	Capacidad (void)
+ RegistroDeMarca	Elemento (int indice)
+ void	Aniade (RegistroDeMarca nuevo)
+ bool	Inserta (int indice, RegistroDeMarca nuevo)
+ void	Elimina (int indice)
+ void	Modifica (int indice, RegistroDeMarca nuevo)
+ void	EliminaTodos (void)
+ string	ToString (void)

Figura 51: Las clases `TablaDentadaMarcas` y `SecuenciaMarcas`

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

57. Hemos construido la clase `Empleado` usando un campo de clase `Fecha` (ejercicio 23), hemos ordenado un array de objetos `Empleado` (ejercicio 24) y hemos construido la clase `SecuenciaEmpleados` (ejercicio 25). Todos los ejercicios mencionados pertenecen a esta misma Relación de Problemas. Seguimos trabajando en este mismo problema.

Se desea organizar los datos de los empleados de una empresa agrupados por departamentos. No todos los departamentos tienen el mismo número de empleados, por lo que se escoge una *tabla dentada* como estructura de datos para guardar los datos de los empleados de la empresa.

Escriba un programa que lea los datos de los empleados de la empresa y los guarde en una tabla dentada. Después (ver ejemplo):

- Muestre un listado de todos los empleados, por orden de lectura, sin distinción de departamentos.
- Muestre un listado de todos los empleados, agrupados por departamentos indicando el número de empleados en cada departamento.
- Calcular y mostrar los empleados de menor y mayor sueldo, indicando su departamento.
- Calcular y mostrar el sueldo medio, y a continuación, un listado de los empleados con un salario mayor que el sueldo medio y otro con los empleados con salario menor. Estos dos listados estarán ordenados por *apellidos*.

LISTADO DE EMPLEADOS

Num. total de empleados = 12

Martinez Lopez	Juan Carlos	10 Dic 2000	2345.55
Mendez Morales	Paula	04 Jun 1998	1234.59
Abad y Paz de Molina	Maria del Carmen	01 Ene 1999	3002.88
Pi Garcia	Jose	05 May 1987	1987.44
Martin Martinez	Patricia	04 Jun 1998	1234.56
Santa Claus	Noel	24 Dic 1990	2142.55
Ferrero Roche	Bombon	25 Dic 1991	1123.66
Gaspar y Baltasar	Melchor	06 Ene 1985	2222.22
Martin Berlanga	Joaquin	10 Oct 1990	1145.33
Bernal Bernardilla	Bernarda	02 Oct 1986	1356.00
Belmez de Jaen	Cesar Augusto	01 Ene 1998	999.99
Joaquina Melgarejo	Maria Luisa	10 Oct 1984	1345.66

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

LISTADO DE EMPLEADOS POR DEPARTAMENTO

Num. Departamentos = 3

Departamento 1 (5 empleados)

Martinez Lopez	Juan Carlos	10 Dic 2000	2345.55
Mendez Morales	Paula	04 Jun 1998	1234.59
Abad y Paz de Molina	Maria del Carmen	01 Ene 1999	3002.88
Pi Garcia	Jose	05 May 1987	1987.44
Martin Martinez	Patricia	04 Jun 1998	1234.56

Departamento 2 (3 empleados)

Santa Claus	Noel	24 Dic 1990	2142.55
Ferrero Roche	Bombon	25 Dic 1991	1123.66
Gaspar y Baltasar	Melchor	06 Ene 1985	2222.22

Departamento 3 (4 empleados)

Martin Berlanga	Joaquin	10 Oct 1990	1145.33
Bernal Bernardilla	Bernarda	02 Oct 1986	1356.00
Belmez de Jaen	Cesar Augusto	01 Ene 1998	999.99
Joaquina Melgarejo	Maria Luisa	10 Oct 1984	1345.66

Sueldo minimo = 999.99

Belmez de Jaen	Cesar Augusto	01 Ene 1998	999.99
----------------	---------------	-------------	--------

Departamento = 3

Sueldo maximo = 3002.88

Abad y Paz de Molina	Maria del Carmen	01 Ene 1999	3002.88
----------------------	------------------	-------------	---------

Departamento = 1

Sueldo medio = 1678.37

Empleados con salario mayor que el sueldo medio:

Abad y Paz de Molina	Maria del Carmen	01 Ene 1999	3002.88
Gaspar y Baltasar	Melchor	06 Ene 1985	2222.22

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

Martinez Lopez	Juan Carlos	10 Dic 2000	2345.55
Pi Garcia	Jose	05 May 1987	1987.44
Santa Claus	Noel	24 Dic 1990	2142.55

Empleados con salario menor que el sueldo medio:

Belmez de Jaen	Cesar Augusto	01 Ene 1998	999.99
Bernal Bernardilla	Bernarda	02 Oct 1986	1356.00
Ferrero Roche	Bombon	25 Dic 1991	1123.66
Joaquina Melgarejo	Maria Luisa	10 Oct 1984	1345.66
Martin Berlanga	Joaquin	10 Oct 1990	1145.33
Martin Martinez	Patricia	04 Jun 1998	1234.56
Mendez Morales	Paula	04 Jun 1998	1234.59

Deberá usar las clases:

- Empleado y Fecha (ejercicio 23 de esta Relación de Problemas).
- SecuenciaEmpleados (ejercicio 25 de esta Relación de Problemas).
- TablaDentadaEmpleados.
- ParejaEnteros.

Sobre la clase TablaDentadaEmpleados le proponemos la siguiente representación para los datos:

```
class TablaDentadaEmpleados {  
  
private:  
  
    static const int NUM_FILS = 50; // Filas disponibles  
    static const int NUM_COLS = 40; // Columnas disponibles  
  
    Empleado matriz_privada[NUM_FILS][NUM_COLS];  
  
    // PRE: 0 <= filas_utilizadas <= NUM_FILS  
  
    int filas_utilizadas;  
  
    // PRE: 0 <= num_cols_utilizadas[i] <= NUM_COLS  
    //      para i=0,1,...,filas_utilizadas-1  
    // Número de columnas ocupadas en cada fila  
  
    int num_cols_utilizadas[NUM_FILS];  
}
```

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

Además de los métodos mínimos para la gestión de la tabla dentada podría ser útil implementar los **métodos**:

```
ParejaEnteros LocalizacionSalarioMenor (void);
ParejaEnteros LocalizacionSalarioMayor (void);

double SalarioMedio (void);
SecuenciaEmpleados EmpleadosSalarioMenor (double valor);
SecuenciaEmpleados EmpleadosSalarioMayor (double valor);
```

La clase ParejaEnteros sirve para gestionar una pareja de valores int. En este caso nos interesa para obtener la localización (*fila*, *columna*) de un empleado. Una implementación sencilla y operativa sería:

```
class ParejaEnteros {

private:
    int primero, segundo;

public:
    ParejaEnteros (int el_primero, int el_segundo)
        : primero (el_primero), segundo (el_segundo) { }

    void SetPrimero (int el_primero) { primero = el_primero; }
    void SetSegundo (int el_segundo) { segundo = el_segundo; }

    int GetPrimero (void) { return primero; }
    int GetSegundo (void) { return segundo; }

    string ToString () {
        return "{" + to_string(primero) + ", " +
            to_string(segundo) + "}";
    }
};
```

Tablas (representación: secuencia de secuencias)

La representación basada en una *secuencia de secuencias* quiere decir que la estructura de datos que se empleará para almacenar los datos será un *vector clásico de objetos de tipo secuencia*. En definitiva, se trata de una secuencia de secuencias.

Por ejemplo, si se trata de una **tabla rectangular** de caracteres (todas las filas tienen el mismo número de columnas *ocupadas*):

```
static const int NUM_FILS = 50; // "filas" disponibles
static const int NUM_COLS = 40;
```

```
SecuenciaEnteros vector_privado[NUM_FILS];
```

```
// PRE: 0 <= filas_utilizadas <= NUM_FILS
// PRE: 0 <= cols_utilizadas <= NUM_COLS
```

```
int filas_utilizadas;
int cols_utilizadas;
```

Al ser una tabla rectangular, deberemos asegurarnos de que todas las secuencias (las “filas”) tengan el mismo número de elementos, lo que implica un **control estricto** en las inserciones/adiciones de filas y en no permitir borrados de elementos individuales.

Si se trata de una **tabla dentada** de caracteres, las “filas” (cada secuencia guardada `vector_privado`) podrán tener diferente número de casillas ocupadas, por lo que no tiene sentido emplear `cols_utilizadas`.

El control acerca del número de elementos que tendrá cada “fila” se realizará en la clase `SecuenciaCaracteres` ya que es ahí dónde está definida `TAMANIO` (la constante que hace el papel de `NUM_COLS`).

```
static const int TAMANIO = 100; // "filas" disponibles
```

```
SecuenciaCaracteres vector_privado[TAMANIO];
```

```
// PRE: 0 <= secuencias_utilizadas <= TAMANIO
```

```
int secuencias_utilizadas; // Número de "filas" ocupadas
```

TABLA RECTANGULAR USANDO UNA SECUENCIA DE SECUENCIAS

58. Cread la clase `TablaRectangularEnteros`. Hacedlo usando una *secuencia de secuencias de enteros*, con la restricción de que **todas las secuencias tienen el mismo tamaño**. Escribir los métodos descritos en la figura 52.

TablaRectangularEnteros	
- const int	<u>NUM_FILS</u>
- const int	<u>NUM_COLS</u>
- SecuenciaEnteros	vector_privado [NUM_FILS]
- int	filas_utilizadas
- int	cols_utilizadas
+	TablaRectangularEnteros(void)
+	TablaRectangularEnteros(int num_columnas)
+	TablaRectangularEnteros(SecuenciaEnteros primera_fila)
+ int	CapacidadFilas (void)
+ int	CapacidadColumnas (void)
+ int	FilasUtilizadas (void)
+ int	ColumnasUtilizadas (void)
+ int	Elemento(int fila, int columna)
+ SecuenciaEnteros	Fila(int indice_fila)
+ SecuenciaEnteros	Columna(int indice_fila)
+ void	Añade(SecuenciaEnteros fila_nueva)
+ void	Inserta (int num_fila, SecuenciaEnteros fila_nueva)
+ void	Elimina (int num_fila)
+ void	EliminaTodos (void)
+ bool	EstaVacía (void)
+ bool	EsIgual (TablaRectangularEnteros otra_tabla)
+ TablaRectangularEnteros	Traspuesta(void)
+ bool	EsSimétrica (void)
+ bool	Contiene (int buscado)
+ PosicionEnMatriz	PrimeraOcurrencia (int buscado)

Figura 52: La clase `TablaRectangularEnteros`

Probad la funcionalidad de la clase empleando exactamente la misma función `main` que se usó para el problema 46 (calcular la posición de aquel elemento que sea el

mayor de entre los mínimos de cada fila).

Si no ha tenido que cambiar nada, la clase está bien construida.

Finalidad: Usar clases que tienen componentes que son objetos de otras clases. Dificultad Media.

59. Reescribir el ejercicio 48 (*MaxiMin*) usando la clase `TablaRectangularEnteros` que emplea una *secuencia de secuencias de enteros* como dato privado.

Emplear la misma función `main` que se usó para la solución propuesta usando la clase que emplea una matriz clásica como dato privado. Si no ha tenido que cambiar nada, la clase está bien construida.

Finalidad: Practicar con la ocultación de información. Dificultad Baja.

60. Reescribir el ejercicio 51 (fila más cercana de un conjunto de filas preseleccionadas) usando la clase `TablaRectangularEnteros` que emplea una *secuencia de secuencias de enteros* como dato privado.

Emplear la misma función `main` que se usó para la solución propuesta usando la clase que emplea una matriz clásica como dato privado. Si no ha tenido que cambiar nada, la clase está bien construida.

Finalidad: Practicar con la ocultación de información. Dificultad Baja.

TABLA DENTADA USANDO SECUENCIA DE SECUENCIAS

61. Reescribir el ejercicio 53 para la clase `ClaseTablaDentadaCaracteres` (representación con secuencia de secuencias).

62. Un *texto* puede verse como una secuencia de frases, algunas de ellas vacías (*líneas en blanco*). Además, cada línea puede tener un número indeterminado de palabras, y el número de palabras de una línea a otra puede variar. En definitiva, un *texto* puede representarse con una *secuencia de frases*, y dado que una frase es una *secuencia de palabras*, un texto puede verse como *una matriz dentada de palabras*.

Usar las clases `Frase` y `Palabra` sobre las que se ha trabajado en el ejercicio 36 para escribir la clase `Texto` y dotarla de funcionalidad.

Escriba un programa que lea una serie indeterminada de cadenas, finalizando cuando se detecta el *fin de fichero*. Cada cadena formará una frase y todas ellas un texto.

El programa mostrará cuantas frases, palabras y caracteres componen el texto.

A continuación “comprimirá” el texto, eliminando las líneas innecesarias en blanco (si hay dos o más consecutivas, dejará una).

Muy importante: Esta tarea la realizará en la función `main` (no escribirá ningún método específico de la clase `Texto` para comprimir el texto).

Finalidad: Usar clases que tienen componentes que son objetos de otras clases. Practicar con matrices dentadas de objetos de otra clases. Dificultad Media.

63. Reescribir el ejercicio 62 incorporando a la clase `Texto` el método `Comprime`, que elimina de un objeto `Texto` las líneas innecesarias en blanco: cuando hay dos o más líneas en blanco consecutivas sólo deja una.

Finalidad: Profundizar en la modularidad. Aumentar la funcionalidad de una clase.

Dificultad Baja.

64. Incorporar a la clase `Texto` los métodos:

```
void Reemplazar (Palabra buscada, Palabra nueva, int linea);
```

Sustituye todas las ocurrencias de la palabra buscada por la palabra nueva en la línea número `linea`. Las dos palabras deben ser *exactamente* iguales.

```
void ReemplazarTodo (Palabra buscada, Palabra nueva);
```

Sustituye todas las ocurrencias de la palabra buscada por la palabra nueva en todas las líneas del texto.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

y realizar diversas pruebas sobre estos métodos usando el texto comprimido.

Finalidad: Usar clases que tienen componentes que son objetos de otras clases. Practicar con matrices dentadas de objetos de otra clases. Dificultad Media.

65. Reescribir el ejercicio 54 de esta misma Relación de Problemas, usando como estructura de datos para almacenar la tabla dentada *una secuencia de secuencias de datos double*. Escribir los métodos descritos en la figura 53.

TablaDentadaReales	
- const int	<u>TAMANIO</u>
- SecuenciaReales	vector_privado[TAMANIO]
- int	total_utilizados
+	TablaDentadaReales()
+	TablaDentadaReales(SecuenciaReales primera_fila)
+ int	CapacidadFilas()
+ int	CapacidadColumnas()
+ int	FilasUtilizadas()
+ int	ColumnasUtilizadas(int indice_fila)
+ double	Elemento(int fila, int columna)
+ SecuenciaReales	Fila(int indice_fila)
+ void	Aniade(SecuenciaReales fila_nueva)
+ void	Inserta (int num_fila, SecuenciaReales fila_nueva)
+ void	Elimina (int num_fila)
+ void	EliminaTodos (void)
+ bool	EstaVacía (void)
+ bool	EsIgual (TablaDentadaReales otra)

Figura 53: La clase ClaseTablaDentadaReales

Finalidad: Usar clases que tienen componentes que son objetos de otras clases. Practicar con matrices dentadas. Dificultad Baja.

66. Escribir un método de la clase `TablaDentadaReales` (con secuencia de secuencias) que devuelva un objeto `TablaRectangularReales` (con secuencia de secuencias).

El objeto devuelto tendrá tantas filas como tenga el objeto implícito, y tantas columnas como columnas tenga la fila con menos columnas del objeto implícito.

Finalidad: Practicar con matrices dentadas y rectangulares. Dificultad Baja.

67. Reescriba la solución del ejercicio 56 de esta Relación de Problemas usando como estructura de datos para almacenar la tabla dentada *una secuencia de secuencias de datos de tipo* `RegistroDeMarca` (secuencia de `SecuenciaMarcas`).

Escribir los métodos descritos en la figura 51.

68. En el ejercicio 57 de esta Relación de Problemas tuvo que escribir un programa para la gestión de los empleados de una empresa que se organiza en diferentes departamentos. En ese programa implementó una tabla dentada que usaba una matriz clásica.

Reescriba el programa modificando *únicamente* la clase `TablaDentadaEmpleados` para que se base en una secuencia de secuencias de datos `Empleado`. Por ejemplo:

```
class TablaDentadaEmpleados {  
  
private:  
  
    static const int TAMANIO = 50; // "Filas" disponibles  
    SecuenciaEmpleados vector_privado[TAMANIO];  
  
    //PRE: 0 <= total_utilizados <= TAMANIO  
  
    int total_utilizados; // Número de casillas ocupadas
```

Tan sólo debería modificar la clase `TablaDentadaEmpleados`. Las demás clases y, especialmente, la función `main` deberían permanecer sin cambios, como en la versión anterior.

EJERCICIOS VARIADOS SOBRE TABLAS

69. (Basado en el Examen Extraordinario de 2020) Considere la clase `SecuenciaCaracteres` y las clases `TablaRectangularCaracteres` (figura 54) y `TablaDentadaCaracteres` (figura 55).

TablarectangularCaracteres	
-	<code>const int NUM_FILS = 100</code>
-	<code>const int NUM_COLS = 100</code>
-	<code>char matriz_privada[NUM_FILS][NUM_COLS]</code>
-	<code>int filas_utilizadas</code>
-	<code>int cols_utilizadas</code>
+	<code>TablaRectangularCaracteres(int cols)</code>
+	<code>int CapacidadFilas()</code>
+	<code>int CapacidadColumnas()</code>
+	<code>int FilasUtilizadas()</code>
+	<code>int ColumnasUtilizadas()</code>
+	<code>int Elemento(int fil, int col)</code>
+	<code>SecuenciaCaracteres Fila(int fil)</code>
+	<code>void Aniade(SecuenciaCaracteres nueva)</code>

Figura 54: La clase `TablaRectangularCaracteres`

- a) Implemente el siguiente constructor:

```
TablaRectangularCaracteres (SecuenciaCaracteres sec,  
                             int num_filas);
```

crea una tabla rectangular de caracteres con `num_filas` filas a partir de la secuencia de caracteres `sec`.

La tabla rectangular no tendrá nunca más valores que la secuencia `sec`. El número de columnas de la tabla rectangular deberá calcularse adecuadamente para que el número de caracteres *sobrantes* (que no pueden formar una fila completa) sea mínimo. Estos caracteres se descartarán. Además, en el caso de que el tamaño de la secuencia `sec` sea menor que `num_filas` se creará una tabla rectangular vacía.

RELACIÓN DE PROBLEMAS VI. Clases (Segunda parte)

Por ejemplo, dada la siguiente secuencia de caracteres de la izquierda y considerando que `num_filas=3`, entonces el constructor creará la tabla rectangular de la derecha (se han descartado 2 caracteres):

Secuencia		Tabla Rectangular
a b v i l a q z g y n m q w y z d	→	a b v i l a q z g y n m q w y

Otro ejemplo, en el que se considera que `num_filas=3`, es el siguiente (en este caso, no se descartan caracteres):

Secuencia		Tabla Rectangular
c g v z f j p l g y n m	→	c g v z f j p l g y n m

b) Sobre la clase `TablaDentadaCaracteres`:

TablaDentadaCaracteres	
-	const int NUM_FILS = 100
-	const int NUM_COLS = 100
-	char matriz_privada[NUM_FILS][NUM_COLS]
-	int filas_utilizadas
-	int cols_utilizadas[NUM_FILS]
+	TablaDentadaCaracteres()
+	int CapacidadFilas()
+	int CapacidadColumnas()
+	int FilasUtilizadas()
+	int ColumnasUtilizadas(int fil)
+	int Elemento(int fil, int col)
+	SecuenciaCaracteres Fila(int fil)
+	void Aniade(SecuenciaCaracteres nueva)

Figura 55: La clase `TablaDentadaCaracteres`

implemente el siguiente método:

`TablaRectangularCaracteres`

`ExtraeTablaRectangularMaximoProvecho (void);`

el método construye y devuelve un objeto de la clase `TablaRectangularCaracteres` con el mismo número de filas que la tabla dentada original y que contiene el máximo número de sus caracteres

Por ejemplo:

Tabla Dentada								Tabla Rectangular					
a	b	v	i	l	a		→	a	b	v	i	l	
q	z	g	y					a	q	z	g	y	
n	m	q	w	y	z	d		n	m	q	w	y	

70. *(Basado en el Examen Ordinario de 2020)* Un tablero es un cuadrado de dimensiones $N \times N$ donde se ubican valores enteros. Se dice que un tablero es **califragilístico** si sus filas, columnas y sus dos diagonales tienen sumas *diferentes*. Además, si un tablero es *califragilístico* entonces puede cumplir una propiedad adicional: ser **supercalifragilístico**. Para ello, se necesita que con los valores de las sumas en filas, columnas y diagonales se pueda formar una secuencia creciente completa. Es decir, dada la secuencia $S = \{s_1, s_2, \dots, s_k\}$, decimos que es creciente y completa si $s_{i+1} = s_i + 1$.

A continuación se muestran dos ejemplos. A la izquierda, un tablero *califragilístico*. Podemos ver que las filas, columnas y diagonales dan lugar a sumas diferentes. A la derecha, un tablero *supercalifragilístico*. Todas las sumas son diferentes (es *califragilístico*) y además dan lugar a una secuencia creciente completa de valores $\{30, 31, 32, \dots, 39\}$.

$\left[\begin{array}{cccccc} & 9 & 8 & 7 & \rightarrow & 24 \\ & 2 & 1 & 6 & \rightarrow & 9 \\ & 3 & 4 & 5 & \rightarrow & 12 \\ \swarrow & \downarrow & \downarrow & \downarrow & \searrow & \\ 11 & 14 & 13 & 18 & 15 & \end{array} \right]$	$\rightarrow$	$\left[\begin{array}{cccccc} & 15 & 2 & 12 & 4 & \rightarrow & 33 \\ & 1 & 14 & 10 & 5 & \rightarrow & 30 \\ & 8 & 9 & 3 & 16 & \rightarrow & 36 \\ & 11 & 13 & 6 & 7 & \rightarrow & 37 \\ \swarrow & \downarrow & \downarrow & \downarrow & \downarrow & \searrow & \\ 34 & 35 & 38 & 31 & 32 & 39 & \end{array} \right]$

Dado un tablero, se quiere implementar un programa para saber si es *califragilístico* y *supercalifragilístico*. Para ello use la clase `TableroCuadrado` (figura 56) y `SecuenciaEnteros`.

TableroCuadrado	
-	const int MAX = 15
-	int matriz_privada[MAX] [MAX]
-	int dimension
+	TableroCuadrado (void)
+	TableroCuadrado (int dimension)
+	int GetDimension (void)
+	int Elemento (int fil, int col)
+	SecuenciaCaracteres Fila (int fil)
+	SecuenciaCaracteres Columna (int col)

Figura 56: La clase TableroCuadrado

Implemente los siguientes métodos:

- EsCalifragilistico que comprueba si el tablero cumple la propiedad de ser *califragilístico*
- EsSuperCalifragilistico que comprueba si el tablero cumple la propiedad de ser *supercalifragilístico*.
- En todos los métodos que escriba debe explicar claramente qué hacen, utilizando comentarios. Indique el tipo del resultado, así como los parámetros de entrada necesarios (si los hubiera). **Puede añadir todos los métodos que considere oportunos.** Justifique la necesidad y el ámbito (público, privado) de los mismos.

71. (*Basado en el Examen Ordinario de 2020*) Un tablero T es un cuadrado de dimensiones $N \times N$ donde se ubican valores enteros (positivos y negativos). Se dice que T es un tablero *heterogéneo* si sus columnas tienen sumas **diferentes**. Además, si un tablero es *heterogéneo* entonces puede cumplir una propiedad adicional: ser *heterogéneo completo*. Para ello, se necesita que con los valores ordenados de las sumas de las columnas se pueda formar una secuencia creciente completa. Es decir, dada la secuencia $S = \{s_1, s_2, \dots, s_k\}$, decimos que es creciente y completa si $s_{i+1} = s_i + 1$. A continuación se muestran dos ejemplos. A la izquierda, un tablero *heterogéneo*. Podemos ver que las columnas dan lugar a sumas diferentes. A la derecha, un tablero *heterogéneo completo*. Todas las sumas son diferentes (es *heterogéneo*) y además, al ordenarlas obtenemos una secuencia creciente completa de valores $\{13, 14, 15\}$.

$$\begin{bmatrix} 9 & 8 & 7 \\ 2 & 1 & 6 \\ 3 & 4 & 5 \\ \downarrow & \downarrow & \downarrow \\ 14 & 13 & 18 \end{bmatrix}$$

$$\begin{bmatrix} 9 & 8 & 7 \\ 2 & 1 & 6 \\ 3 & 4 & 2 \\ \downarrow & \downarrow & \downarrow \\ 14 & 13 & 15 \end{bmatrix}$$

Dado un tablero T , se quiere implementar un programa para saber si es *heterogéneo* y *heterogéneo completo*. Para ello use la clase `TableroCuadrado` (figura 56) y `SecuenciaEnteros`.

Implemente los siguientes métodos:

- a) Implemente un método para obtener la diagonal inversa de un tablero como un objeto de la clase `SecuenciaEnterosLong`. Con cualquiera de los tableros de la figura de arriba, la secuencia resultante sería {7, 1, 3}
- b) Implemente el método `Heterogeneo` que comprueba si el tablero cumple la propiedad de ser *heterogéneo*
- c) Implemente el método `HeterogeneoCompleto` que comprueba si el tablero cumple la propiedad de ser *heterogéneo completo*.

En todos los métodos debe considerar que deben ser lo más **EFICIENTES** posible, en términos de la velocidad de ejecución. Considere que no hay restricciones de memoria. Para ambos métodos indique la cabecera completa, así como los datos locales. Explique brevemente cómo ha resuelto las tareas pedidas.

Puede añadir todos los métodos que considere oportunos. Deberá implementarlos completamente, indicando también el ámbito (público, privado) de los mismos.

RELACIÓN DE PROBLEMAS VII. Recursividad

Problemas Básicos

1. Definir una función recursiva para sumar los dígitos de un entero positivo.
2. Cread sendas funciones recursivas para:
 - a) Calcular la división entera entre dos números enteros positivos.
 - b) Calcular el resto de la división entera de dos números enteros positivos.

Pasad como parámetros a cada función los dos enteros, y suponed que en la llamada siempre pasarán números positivos (es la precondition de la función)

En la definición de las funciones **no** pueden utilizarse los operadores de división, multiplicación ni módulo.

3. *(Examen Febrero 2012)*

Sobre la clase `SecuenciaCaracteres`, definir un método recursivo que nos diga si es un palíndromo, es decir, que se lee igual de izquierda a derecha que de derecha a izquierda. Por ejemplo, `{ 'a', 'b', 'b', 'a' }` sería un palíndromo.

Pasad al método como parámetros las posiciones izquierda y derecha sobre las que trabajará.

4. Definir una función recursiva a la que se le pasen dos valores enteros n y k y devuelva como resultado el valor k -ésimo comenzando por la derecha del número n . Por ejemplo para $n = 427$ y $k = 3$ el resultado sería 4.
Si k es mayor que el número de dígitos de n la función devolverá el valor 0. Por ejemplo, para $n = 23$ y $k = 5$ el resultado es 0.
5. Definid una función recursiva con prototipo

```
int MCD(int un_entero, int otro_entero)
```

para que calcule el máximo común divisor entre dos enteros, aplicando el algoritmo de Euclides, que nos dice lo siguiente:

$$\begin{aligned} \text{MCD}(a, 0) &= a \\ \text{MCD}(a, b) &= \text{MCD}(b, a \% b) \end{aligned}$$

dónde $a \% b$ es el resto de la división entera entre a y b . Implementad el algoritmo suponiendo que a es mayor que b . A continuación, comprobad que también funciona en el caso contrario. ¿Por qué?

RELACIÓN DE PROBLEMAS VII. Recursividad

6. Cread una función recursiva con la siguiente cabecera

```
bool ContieneDesdeInicio(string cadena_grande, string cadena_peque)
```

que compruebe si la cadena `cadena_peque` se encuentra al inicio de la cadena `cadena_grande` (desde la posición cero)

7. Sobre la clase `SecuenciaCaracteres`, definir un método recursivo que construya otro vector de caracteres con las mismas componentes pero en sentido inverso, de forma que si el vector contenía los caracteres {'a', 'g', 't', 'b', 'i', 'o'}, el método construirá el vector {'o', 'i', 'b', 't', 'g', 'a'}.

Pasad al método como parámetros las posiciones izquierda y derecha sobre las que trabajará.

Analizad la eficiencia de esta solución.

8. Averiguar qué hace y qué devuelve la siguiente función recursiva:

```
int f (int x) {  
    if (x > 100)  
        return (x - 10);  
    else  
        return(f(f(x+11)));  
}
```

9. Escribir una función recursiva a la que se le pasa un número entero y devuelva una cadena que contenga la representación textual de ese número separando cada tres cifras por un punto. La función tendrá la cabecera:

```
string PonerPuntos (int numero);
```

Por ejemplo, si `numero` vale 9912789, la función devolverá la cadena 9.912.789

10. Sea a un número positivo y real. Definamos la secuencia de valores reales x_i como

$$x_i = \begin{cases} 1 & \text{si } i = 0 \\ \frac{1}{2} \left(x_{i-1} + \frac{a}{x_{i-1}} \right) & \text{si } i > 0 \end{cases}$$

Se demuestra que $x_i \rightarrow \sqrt{a}$ si $i \rightarrow \infty$. Escriba una función recursiva que use este algoritmo para calcular la raíz cuadrada de un número. a esta función debemos proporcionarle el número del que vamos a calcular la raíz cuadrada aproximada y el número de términos de la sucesión que se emplearán para la aproximación.

11. Implemente una función recursiva que permita calcular el término n -ésimo de la serie de los números de Catalan (*Eugène Charles Catalan, 1814-1894*) empleando la siguiente expresión:

$$C_n = \begin{cases} 1 & \text{si } n = 0 \\ \frac{2(2n-1)}{n+1} C_{n-1} & \text{si } n > 0 \end{cases}$$

Nota: Los primeros términos de la serie son: 1, 1, 2, 5, 14, 42, 132, ...

12. Escriba una función recursiva que reciba n y k , dos valores `int`, y devuelva la suma de los k dígitos menos significativos de n .

Por ejemplo, para $n = 61427$ y $k = 3$ el resultado será $7 + 2 + 4 = 13$ y para $n = 23$ y $k = 4$ el resultado será $3 + 2 = 5$

13. Escriba una función recursiva que para dos valores enteros positivos n y k calcule cuántos divisores propios positivos tiene n que sean estrictamente menores que k .

Un número d es divisor propio de m si divide a m y es distinto de m (el 1 es divisor propio de cualquier número distinto de 1).

La cabecera de la función será:

```
int DivisoresMenores (int n, int k);
```

Por ejemplo, la llamada `DivisoresMenores (10, 6)` debe devolver 3 ya que 1, 2 y 5 son los divisores propios positivos de 10 estrictamente menores que 6.

Observe que para cualquier número p mayor que 1 el resultado devuelto por `DivisoresMenores (p, p)` debe ser 1 si p es primo.

Problemas de Ampliación

14. Cread una función recursiva con la siguiente cabecera

```
bool Contiene(string cadena_grande, string cadena_peque)
```

que compruebe si la cadena `cadena_peque` se encuentra dentro de la cadena `cadena_grande`. En la definición de esta función, se puede llamar a la función `ContieneDesdeInicio` del ejercicio 6 de esta misma Relación de Problemas.

15. Escribir una función recursiva a la que se le pasa un número entero y devuelva una secuencia de caracteres que contenga la representación textual de ese número separando cada tres cifras por un punto. La función tendrá la cabecera:

```
SecuenciaCaracteres PonerPuntos (int numero);
```

Por ejemplo, si `numero` vale 9912789, la función devolverá la secuencia `{'9','.', '9','1','2','.', '7','8','9'}`

16. Definir una función recursiva para construir una secuencia de enteros compuesto por 0 y 1 con la representación en binario de un entero.

El bit más significativo (el de la mayor potencia de 2) debe ir en la posición 0, mientras que el menos significativo (el correspondiente a 2^0) debe ir en la última posición.

La función debe tener la siguiente cabecera:

```
SecuenciaEnteros PasaBinario(int n)
```

Por ejemplo, para $n = 28$ la secuencia devuelta será `{1, 1, 1, 0, 0}`.

17. El método de *bisección* usado para calcular de forma aproximada el punto de corte de una función f con el eje de abscisas (la raíz de la función) se puede enunciar como sigue:

Sea $f(x)$ una función real, estrictamente monótona en $[i, d]$, donde $f(i)$ y $f(d)$ tienen distinto signo y sea ϵ una constante real pequeña (del orden de 10^{-4} , por ejemplo).

- Calcular m , el punto medio entre i y d ,
- Si $|i - d| < \epsilon$, terminar.
- Si $f(m)$ tiene igual signo que $f(i)$, repetir considerando el intervalo $[m, d]$
- Si $f(m)$ tiene igual signo que $f(d)$, repetir considerando el intervalo $[i, m]$

Implemente una función recursiva que reciba como datos de entrada los extremos i, d del intervalo, así como el valor de ϵ y devuelva la raíz de la función.

Se supone que la función $f(x)$ ya está definida con anterioridad.

18. Sobre la clase `SecuenciaCaracteres`, definir un método recursivo que devuelva la diferencia entre el número de letras mayúsculas y el número de letras minúsculas que hay entre dos posiciones del vector. Por ejemplo, si el vector tuviese los caracteres `Hay UnAS LeTRas DESpuEs`, la diferencia entre mayúsculas y minúsculas entre las posiciones 4 y 15 sería $6 - 4 = 2$.

19. (*Examen Septiembre 2012*)

Construir una función recursiva que muestre en la salida estándar los m mayores números pares que sean menores o iguales que un número n . El prototipo sería el siguiente:

```
void MayoresPares(int m, int n);
```

Por ejemplo:

- `MayoresPares(3,6)`: mostrará 6, 4, 2
- `MayoresPares(4,3)`: mostrará 2, 0, -2, -4
- `MayoresPares(0,8)`: no mostrará nada

20. Construir la función

`SecuenciaEnteros Aproximacion (int valor, SecuenciaEnteros sec)`
para calcular una aproximación a `valor` usando elementos de la secuencia `sec`.

La aproximación será una *secuencia*, formada por elementos de `sec`, de manera que si `s` es la secuencia resultante:

a) Sea $n = s.TotalUtilizados()$

b) $n \leq sec.TotalUtilizados()$

c) $\left[\sum_{i=0}^n s.Elemento(i) \right] \leq valor$

d) $s.Elemento(0) \leq s.Elemento(1) \leq \dots \leq s.Elemento(n)$

Por ejemplo, dada la secuencia { 1, 5, 7, 0, 9 } la aproximación al valor 14 sería la secuencia { 0, 1, 5, 7 } porque $0 + 1 + 5 + 7 = 13 \leq 14$.

Para la misma secuencia, la aproximación al valor 6 sería la secuencia { 0, 1, 5 } porque $0 + 1 + 5 = 6 \leq 6$.

Nota: La función propuesta llamará a la *función recursiva*

```
SecuenciaEnteros AproximacionConTerminos (int valor,  
                                           int num_terminos, SecuenciaEnteros sec);
```

para calcular la aproximación a `valor` usando `num_terminos` elementos de la secuencia `sec` (empezad por 1).

21. Realizar una función recursiva para multiplicar dos enteros según el método de multiplicación rusa (ver ejercicio 48 de la *Relación de Problemas II*).
22. Escribir una función recursiva que calcule la descomposición factorial de un número entero. El resultado (los factores primos calculados) los guardará en una secuencia de enteros. La cabecera de la función será:

```
SecuenciaEnteros FacroresPrimos (int valor);
```

Por ejemplo, para calcular la descomposición factorial de 360 ($360 = 2^3 \cdot 3^2 \cdot 5$) desde la función `main`, p.e. se escribirá:

```
SecuenciaEnteros factores = FacroresPrimos (360);
```

23. Implemente una función recursiva para *dibujar*, en dos dimensiones, el fractal conocido como *esponja de Menguer* (figura 57).

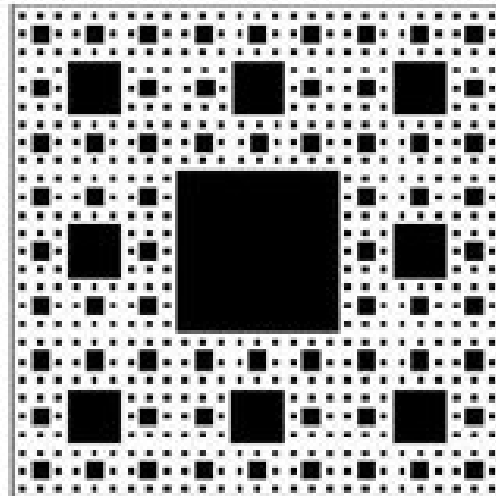


Figura 57: Esponja de Menguer

El prototipo de la función debe ser el siguiente:

```
void Fractal_Menguer (double x, double y, int tam);
```

siendo (x, y) las coordenadas de la esquina superior izquierda y `tam` el tamaño del cuadrado.

Nota: Suponga que dispone de la función `DibujaCuadrado (x, y, tam)` que *dibuja* un cuadrado relleno de color negro cuya esquina superior izquierda tiene coordenadas (x, y) y cuyo ancho es `tam`.

24. Implemente una función recursiva para *dibujar*, en dos dimensiones, el fractal conocido como *Triángulo de Sierpinski* (figura 58).

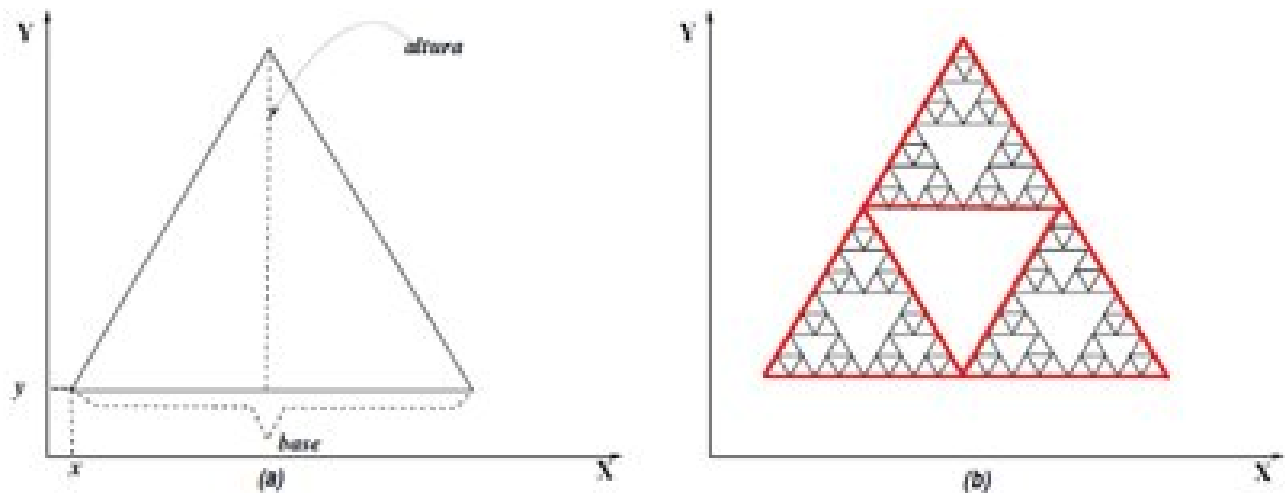


Figura 58: Triángulo de Sierpinski

El prototipo de la función debe ser el siguiente:

```
void Fractal_Sierpinski (double x, double y, double base,
                        double altura, int n);
```

que dibuje n veces un triángulo de la siguiente manera:

- Dibuja un triángulo cuya base se inicia en la coordenada (x, y) con una longitud base y con altura altura (figura 58.A)
 - Dibuja en cada esquina un nuevo triángulo de base $base/2$ y altura $altura/2$ internos al triángulo dibujado anteriormente (triángulos rojos *exteriores* de la figura 58.B)
25. El método de ordenación por mezcla (*mergesort*) puede explicarse fácilmente. Dado un vector a ordenar, 1) se divide en dos mitades de igual tamaño, 2) se ordenan separadamente, y 3) se *mezclan* para obtener un vector ordenado.
- Escriba una función recursiva que implemente este método de ordenación para ordenar una secuencia de enteros. Emplee un método simple para ordenar un subvector de tamaño menor que TAM.
- Implemente este problema sobre la clase `SecuenciaEnteros` con un método `void` para ordenar los valores comprendidos entre las casillas `izda` y `dcha`: La cabecera será:

```
void OrdenaPorMezcla (int izda, int dcha);
```

26. Implemente una función recursiva con la cabecera siguiente:

```
void Reglar (double izda, double dcha, int min, int max, int n);
```

que *dibuja* marcas de una regla. Los parámetros indican:

- *izda* y *dcha* son las posiciones *horizontales* entre las que se dibujan las marcas.
- *min* y *max* indican la *altura* de las marcas más alta y más baja, erspectivamente.
- *n* indica el número de marcas de cada altura.

Por ejemplo, la llamada a `Reglar (1, 17, 1, 3, 3)` generaría el dibujo mostrado en la figura 59.

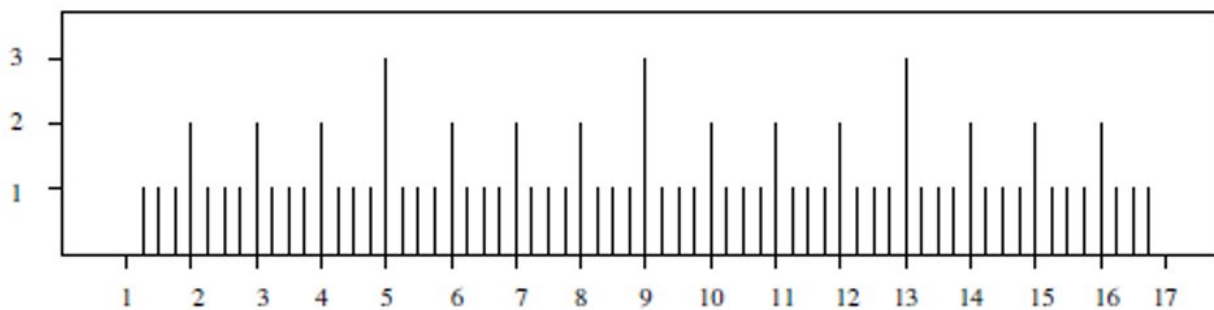


Figura 59: Resultado de `Reglar (1, 17, 1, 3, 3)`

En la figura 59 se dibujan marcas entre las posiciones 1 y 17. Dibuja 3 marcas (*n*) de altura 3 (*max*) en el intervalo 1 (*izda*) y 17 (*dcha*). Después dibuja 3 marcas (*n*) de altura 2 (*max-1*) en cada uno de los cuatro intervalos que han determinado las marcas de altura *max*. Este proceso se repite hasta que se dibujan marcas de altura 1 (*min*).

Suponga que dispone de la función

```
void DibujarMarca (double posicion, int altura);
```

que *dibuja* una línea de altura *altura* en la posición *posicion*. Por ejemplo, la línea central de la figura 59 se ha dibujado con la llamada `DibujarMarca (9, 3)`.